Qualcomm Technologies, Inc.

QCA_Networking_2021.SPF.11.5 CSU1

Release Notes

80-19559-4 Rev. AD

July 19, 2022

For additional information or to submit technical questions, go to: https://createpoint.qti.qualcomm.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Confidential Distribution: Use or distribution of this item, in whole or in part, is prohibited except as expressly permitted by written agreement(s) and/or terms with Qualcomm Incorporated and/or its subsidiaries.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Qualcomm ChipCode and Krait are trademarks or registered trademarks of Qualcomm Incorporated. Bluetopia is a trademark or registered trademark of Qualcomm Technologies, Inc. HY-FI is a trademark or registered trademark of Qualcomm Atheros, Inc. CSR and CSRmesh are trademarks or registered trademarks of Qualcomm Technologies International, Ltd. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2022 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	January 2022	Initial release
AB	January 2022	Updated section 1.2.3
AC	June 2022	Minor update in section 5.2
AD	July 2022	Updated restrictions on software while using it for testing for QCN90xx in section 2.3

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Contents

1 Introduction 7

1.1 Purpose 8

1.2 Related documentation 10

1.2.1 Reference documents for IPQ807x 10

1.2.2 Reference documents for IPQ60xx 12

1.2.3 Reference documents for QCN90xx 13

1.2.4 Related documentation for IPQ50xx 13

1.2.5 Reference documents for IPQ806x 15

1.2.6 Reference documents for IPQ4019 16

1.2.7 Reference documents for QCN61xx 17

2 IPQ8074.ILQ.11.5 CSU1 19

2.1 Supported features 19

2.2 Frequency spectrum and regulatory domain support 20

2.3 Restrictions on software while using it for testing 22

2.4 Supported hardware for this SP 23

2.5 Build and load the image for IPQ8074.ILQ.11.5 CSU1 24

2.5.1 Download packages available through Qualcomm Chip Code 25

2.5.2 Download packages from external websites 25

2.5.3 Generate the firmware for IPQ8074.ILQ.11.5 CSU1 28

2.5.4 Generate a complete firmware image 38

2.5.5 Flash the complete default software image 39

2.5.6 Create customized IPQ807x Wi-Fi firmware images 44

2.5.7 Create customized IPQ807x + QCN90xx Wi-Fi firmware images 46

2.6 U-Boot device tree optimization 47

3 IPQ6018.ILQ.11.5 CSU1 48

3.1 Supported features 48

3.2 Frequency spectrum and regulatory domain support 49

3.3 Restrictions on software while using it for testing 50

3.4 Supported hardware for this SP 51

3.5 Build and load the image for IPQ6018.ILQ.11.5 CSU1 51

3.5.1 Download packages available through Qualcomm ChipCode Portal 51

3.5.2 Download packages from external websites 51

3.5.3 Generate the firmware for IPQ6018.ILQ.11.5 CSU1 52

3.5.4 Generate a complete firmware image 61

3.5.5 Flash the complete default software image 63

3.5.6 Create customized IPQ60x8 Wi-Fi firmware images 65

3.5.7 Create customized IPQ60x8 + QCN90xx Wi-Fi firmware images 66
3.6 Flash Wi-Fi firmware image only 67
3.7 Test the GATT profile with sample applications with onboard CSR8811 Bluetooth on AP.CP01 68

4 IPQ8064.ILQ.11.5 CSU1 71
4.1 Supported hardware for this SP 71
4.2 Restrictions on software while using it for testing 72
4.3 Power management 73
4.4 Build and load the image for IPQ8064.ILQ.11.5 CSU1 75
4.4.1 Download packages available through Qualcomm ChipCode 75
4.4.2 Download packages from external websites 75
4.4.3 Generate the firmware for IPQ8064.ILQ.11.5 CSU1 75
4.4.4 Generate a complete firmware image 83
4.4.5 Flash the complete default software image 85

5 IPQ4019.ILQ.11.5 CSU1 88
5.1 Supported hardware for this SP 88
5.2 Restrictions on software while using it for testing 89
5.3 Power management 90
5.4 Build and load the image for IPQ4019.ILQ.11.5 CSU1 91
5.4.1 Download packages available through Qualcomm ChipCode 91
5.4.2 Download packages from external websites 92
5.4.3 Generate the firmware for IPQ4019.ILQ.11.5 CSU1 92
5.4.4 Generate a complete firmware image 99
5.4.5 Flash the complete default software image 101
5.5 Updates for PLC Wi-Fi SON 105
5.5.1 ESS port mapping in REH172 105
5.5.2 Enable PLC on REH172 for Wi-Fi SON and non-SON configurations 107
5.5.3 Included PIBs 107
5.5.4 Map the PLC Wi-Fi SON interface 108
5.6 Test the GATT profile with sample applications with onboard CSR8811 Bluetooth on AP.DK07 108

6 IPQ5018.ILQ.11.5 CSU1 112
6.1 Supported features 112
6.2 Frequency spectrum and regulatory domain support 112
6.3 Supported hardware for this SP 113
6.4 Build and load the image for IPQ5018.ILQ.11.5 CSU1 114
6.4.1 Download packages available through Qualcomm ChipCode Portal 114
6.4.2 Download packages from external websites 114
6.4.3 Generate the firmware for IPQ5018.ILQ.11.5 CSU1 114
6.4.4 Generate a complete firmware image 126
6.4.5 Flash the complete default software image 128
6.4.6 Create customized IPQ50xx Wi-Fi firmware images 133
6.4.7 Create customized IPQ50xx + QCN90xx Wi-Fi firmware images 134

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.4.8 Create customized IPQ50xx + QCN61xx Wi-Fi firmware images 135
6.5 Flash Wi-Fi firmware image only 136
6.6 Generate secure boot image 137
6.7 Test the serial port profile (SPP) over generic access profile (GAP) (BR/EDR) profile with sample applications with onboard Bluetooth on AP.MP03.1 137
6.7.1 Audio TDM driver support 141
6.8 Low memory profile – feature portfolio 142
6.8.1 List of features 142
6.8.2 NSS features – 256 MB DDR/16 MB flash retail profiles 144

7 Features list 146
7.1 New features 146

8 QDART 147
8.1 QDART reference documents 147
8.2 QDART requirements for QCN90xx with IPQ807x/IPQ50xx/IPQ60xx and QN61xx with IPQ50xx 148
8.3 QDART bringup 149
8.4 EEPROM, Flash commit 149
8.5 Managing BDF when EEPROM, Flash are not used 150
8.6 Fusing board ID to OTP in QCN61xx 152
8.7 XTAL CAL for QCN61xx 153
8.8 Enable DPD in the default BDF for FTM mode 154
8.8.1 Modify the BDF 154
8.8.2 Changes to load the modified BDF in DUT 154
8.9 Verification test tree 155
8.10 FTM calibration and verification KPI 163
8.10.1 Factors impacting KPI 163
8.10.2 RDP386 164
8.10.3 RDP413 166
8.10.4 RDP420 168
8.10.5 RDP403 172

9 BDF updates 175
9.1 BDF updates (IPQ8074) 175
9.2 BDF updates (IPQ5018/QCN61xx) 175
9.2.1 BDF support (IPQ5018) 176
9.3 BDF updates (IPQ6018) 176
9.4 BDF updates (QCN90xx) 176

10 Known issues 177
10.1 Functional, performance, and other issues for SPF11.5 CSU1 177
10.2 Stability issues for SPF11.5 CSU1 177

11 Fixed Issues 180

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

12 Limitations 182
12.1 Limitations for SPF11.5 182

13 Workaround to pass WFA certification 187

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

1 Introduction

This document provides details on the QCA_Networking_2021.SPF.11.5 CSU1 software release.

Whether downloaded from the Qualcomm ChipCode™ portal, the Qualcomm® CreatePoint site, or embedded on Equipment received from Qualcomm Atheros, Inc. ("QCA") or its affiliates, the QCA_Networking_2021.SPF.11.5 CSU1 software release (the "SW Package") shall be considered (in order of priority): (i) Evaluation Technology under the terms of the product kit license agreement accompanying the release (the "PKLA"), (ii) Deliverables under the terms of your Limited Use Agreement (the "LUA"), or (iii) Licensed Technology under the terms of your Technology License Agreement (the "TLA"), each with QCA or its affiliate (the LUA, PKLA, or TLA, as applicable, the "Agreement"). The applicable period for which the SW Package is licensed (the "Use Period") starts on the Effective Date of your Agreement or the date you received the SW Package, whichever is later, and expires on January 25, 2023 (unless a different Use Period for the SW Package is specified in the Agreement, in which case the Use Period in the Agreement shall prevail). By receiving and/or using the SW Package, you acknowledge and agree that your use of the SW Package is subject to the terms and conditions of the Agreement. If you do not agree to the terms of the Agreement, have not accepted any such Agreement, or your agreement with QCA or its affiliate does not include Deliverables, Evaluation Technology, or Licensed Technology, you shall immediately delete the SW Package from all storage media and destroy any and all copies made.

Information published by QCA or its affiliates regarding any third-party information does not constitute a license to use such information or endorsement thereof. QCA or its affiliates provides any such third party information as-is, without any representation, warranty, or indemnity, either express or implied. Use of such information may require a license from a third party under the intellectual property rights of such third party, or a license from QCA or its affiliates under the intellectual property rights of QCA or its affiliates. Users assume all risk of any use of such third party information.

1.1 Purpose

This document describes new and changed features, download and installation procedures, and known and resolved problems in the hardware and software. This product includes software developed by the University of California, Berkeley, and its contributors.

This release aggregates these SPs:
IPQ8074.ILQ.11.5
IPQ5018.ILQ.11.5
IPQ8064.ILQ.11.5
IPQ4019.ILQ.11.5
IPQ6018.ILQ.11.5

This release is:	**QCA_Networking_2021.SPF.11.5 CSU1**
The release version is:	**IPQ8074.ILQ.11.5-00299-P-1** **IPQ5018.ILQ.11.5-00299-P-1** **IPQ8064.ILQ.11.5-00299-P-1** **IPQ4019.ILQ.11.5-00299-P-1** **IPQ6018.ILQ.11.5-00299-P-1**
The Linux Foundation hosted open source label (the CAF_TAG) that corresponds to this release is:	**caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml**
Qualcomm ChipCode distribution tag (Use this tag to check out the code from git repository)	**r11.5_00003.0**

- This CSU1 release for QCA_Networking_2021.SPF.11.5 supports the IPQ401x, IPQ806x, IPQ807x/IPQ807xA/IPQ817x, IPQ50xx, IPQ60xx, QCN90xx/QCN602x, and QCN61xx chipsets.
- The Factory Test Mode (FTM) Wi-Fi functionality is fully functional in this release.

Software product and software image names included with this release:

Software product build	Included software image builds
IPQ8074.ILQ.11.5-00299-P-1	NHSS.QSDK.11.5-07876-P-1
	WLAN.HK.2.6-02451-QCAHKSWPL_SILICONZ-1
	NSS.FW.11.5-00557-ALL-1
	RPM.BF.2.4.1-00108-IPQ8074AAAAANAZR-1
	CNSS.PS.3.16-00018-S-1
	WLAN.BL.3.16-00035-S-1
	TZ.BF.4.0.8-00224-IPQ807xSANAANAZT-1
	BOOT.BF.3.3.1-00163-IPQ8074HAASANAA-1
IPQ5018.ILQ.11.5-00299-P-1	NHSS.QSDK.11.5-07876-P-1
	WLAN.HK.2.6-02451-QCAHKSWPL_SILICONZ-1
	NSS.FW.11.5-00557-ALL-1
	CNSS.PS.3.16-00018-S-1
	WLAN.BL.3.16-00035-S-1
	TZ.WNS.4.0-00107-IPQ5018SANAANAZT-1

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Software product build	Included software image builds
	BTFW.MAPLE.1.0.1-00013-MPL_ROM_PATCHZ-1
	BOOT.BF.3.3.1.1-00067-IPQ5018HAASANAA-1
IPQ8064.ILQ.11.5-00299-P-1	NHSS.QSDK.11.5-07876-P-1
	NSS.FW.11.5-00557-ALL-1
	CNSS.PS.3.16-00018-S-1
	WLAN.BL.3.16-00035-S-1
	RPM.AK.1.0-00171-C-2
	BOOT.AK.1.0-00244-C-1
	TZ.AK.1.0-00191-C-1
IPQ4019.ILQ.11.5-00299-P-1	NHSS.QSDK.11.5-07876-P-1
	CNSS.PS.3.16-00018-S-1
	WLAN.BL.3.16-00035-S-1
	BOOT.BF.3.1.1-00138-QCA961xAAAAANAAB-1
	TZ.BF.2.7-00147-QCA961xMAZAANAZT-1
IPQ6018.ILQ.11.5-00299-P-1	NHSS.QSDK.11.5-07876-P-1
	WLAN.HK.2.6-02451-QCAHKSWPL_SILICONZ-1
	NSS.FW.11.5-00557-ALL-1
	CNSS.PS.3.16-00018-S-1
	WLAN.BL.3.16-00035-S-1
	RPM.BF.2.4.1-00108-IPQ8074AAAAANAZR-1
	TZ.WNS.5.1-00143-IPQ60xxAAAAANAZT-1
	BOOT.XF.0.3-00102-IPQ60xxLZB-1

Software product and software image names included with QCN90xx release:

Software product name	Included software image names
IPQ8074.ILQ.11.5-00299-P-1	NHSS.QSDK.11.5-07876-P-1
	WLAN.HK.2.6-02451-QCAHKSWPL_SILICONZ-1
	NSS.FW.11.5-00557-ALL-1
	RPM.BF.2.4.1-00108-IPQ8074AAAAANAZR-1
	CNSS.PS.3.16-00018-S-1
	WLAN.BL.3.16-00035-S-1
	TZ.BF.4.0.8-00224-IPQ807xSANAANAZT-1
	BOOT.BF.3.3.1-00163-IPQ8074HAASANAA-1
IPQ6018.ILQ.11.5-00299-P-1	NHSS.QSDK.11.5-07876-P-1
	WLAN.HK.2.6-02451-QCAHKSWPL_SILICONZ-1
	NSS.FW.11.5-00557-ALL-1
	CNSS.PS.3.16-00018-S-1
	WLAN.BL.3.16-00035-S-1

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Software product name	Included software image names
	RPM.BF.2.4.1-00108-IPQ8074AAAAANAZR-1
	TZ.WNS.5.1-00143-IPQ60xxAAAAANAZT-1
	BOOT.XF.0.3-00102-IPQ60xxLZB-1

Software product and software image names included with QCN907x/QCN602x/QCN61xx release:

Software product build	Included software image builds
IPQ5018.ILQ.11.5-00299-P-1	NHSS.QSDK.11.5-07876-P-1
	WLAN.HK.2.6-02451-QCAHKSWPL_SILICONZ-1
	NSS.FW.11.5-00557-ALL-1
	CNSS.PS.3.16-00018-S-1
	WLAN.BL.3.16-00035-S-1
	TZ.WNS.4.0-00107-IPQ5018SANAANAZT-1
	BTFW.MAPLE.1.0.1-00013-MPL_ROM_PATCHZ-1
	BOOT.BF.3.3.1.1-00067-IPQ5018HAASANAA-1

1.2 Related documentation

1.2.1 Reference documents for IPQ807x

DCN	Document Title
80-YC865-1	*IPQ8072A WI-FI ACCESS POINT SOC DEVICE SPECIFICATION*
80-YC865-2	*IPQ8074A WI-FI ACCESS POINT SOC DEVICE SPECIFICATION*
80-YC865-3	*IPQ8076A WI-FI ACCESS POINT SOC DEVICE SPECIFICATION*
80-YC865-4	*IPQ8078A WI-FI ACCESS POINT SOC DEVICE SPECIFICATION*
80-YA727-1	*IPQ8074 AP.HK01 HARDWARE REFERENCE*
80-YA727-2	*IPQ8074 AP.HK01 SETUP GUIDE*
80-YA727-16	*IPQ8072A AP.HK09 HARDWARE REFERENCE*
80-YA727-17	*IPQ8072A AP.HK09 SETUP GUIDE*
80-YB198-1	*QCN5024 2.4 GHZ WLAN RFIC DEVICE SPECIFICATION*
80-YB198-2	*QCN5054 5 GHZ WLAN RFIC DEVICE SPECIFICATION*
80-YB198-4	*QCN5124 2.4 GHZ WLAN RFIC DEVICE SPECIFICATION*
80-YB198-5	*QCN5154 5 GHZ WLAN RFIC DEVICE SPECIFICATION*
80-YC820-1	*IPQ8070 WI-FI ACCESS POINT SOC DEVICE SPECIFICATION*
80-YC820-4	*IPQ8070 DEVICE REVISION SPECIFICATION*
80-YC820-2	*IPQ8071 WI-FI ACCESS POINT SOC DEVICE SPECIFICATION*
80-YC820-5	*IPQ8070 AP.AC01 HARDWARE REFERENCE*
80-YC820-6	*IPQ8070 AP.AC01 SETUP GUIDE*
80-YC812-1	*QCA8081 MD.NP01 HARDWARE REFERENCE*
80-Y0615-1	*QCA8081 DEVICE SPECIFICATION*
80-Y0615-4	*QCA8081 DEVICE REVISION SPECIFICATION*

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

DCN	Document Title
80-WL502-1	*QCA4024 BLE AND IEEE 802.15.4 DEVICE SPECIFICATION*
80-WL502-4	*QCA4024 DEVICE REVISION GUIDE*
80-YA728-4	*IPQ807X SOC SOFTWARE USER GUIDE*
80-Y8050-1	*QCA99XX QDART CONNECTIVITY USER GUIDE*
80-YB215-1	*FTM RF TEST FOR 802.11AX WLAN AP CHIPSETS USER GUIDE*
80-Y9005-20	*QCN90XX FTM RF TEST APPLICATION NOTE*

Reference documents for IPQ8070/IPQ8074/IPQ8070A/IPQ8074A/IPQ8174

DCN	Document Title
80-YC865-2	*IPQ8074A WI-FI ACCESS POINT SOC DEVICE SPECIFICATION*
80-YC865-5	*IPQ8072A/IPQ8074A/IPQ8076A/IPQ8078A DEVICE REVISION SPECIFICATION*
80-Y0137-392B	*PRODUCT MATERIAL DECLARATION - IPQ8074A*
80-YC865-6	*IPQ8072A/IPQ8074A/IPQ8076A/IPQ8078A POWER CONSUMPTION APPLICATION NOTE*
80-Y9000-14	*IPQ8074A AP.HK10.2 HARDWARE REFERENCE*
80-Y9000-17	*IPQ8074A AP.HK10.2 SETUP GUIDE*

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

1.2.2 Reference documents for IPQ60xx

DCN	Document Title
80-YC609-2	*IPQ60x8 WLAN FIRMWARE DEBUG STATS APPLICATION NOTE*
80-YC609-5	*IPQ60xx WI-FI HOST SOFTWARE FEATURE SPECIFICATION*
80-YC609-7	*IPQ60xx QSDK USER GUIDE*
80-YC609-8	*IPQ60xx SOC SOFTWARE USER GUIDE*
80-YB726-3	*IPQ6000 WI-FI ACCESS POINT SOC DATA SHEET*
80-YB726-1	*IPQ6018 WI-FI ACCESS POINT SOC DATA SHEET*
80-YB726-2	*IPQ6028 WI-FI ACCESS POINT SOC DATA SHEET*
80-YB726-4	*IPQ6010 WI-FI ACCESS POINT SOC DATA SHEET*
80-YB726-5	*IPQ6000/IPQ6018/IPQ6028 DEVICE REVISION GUIDE*
80-YD032-1	*QCN5052 5 GHZ WLAN RFIC DATA SHEET*
80-YD032-2	*QCN5022 2.4 GHZ WLAN RFIC DATA SHEET*
80-YD032-3	*QCN5021 2.4 GHZ WLAN RFIC DATA SHEET*
80-YD032-6	*QCN5152 5 GHZ WLAN RFIC DATA SHEET*
80-YD032-7	*QCN5122 2.4 GHZ WLAN RFIC DATA SHEET*
80-YD032-8	*QCN5121 2.4 GHZ WLAN RFIC DATA SHEET*
80-YD032-10	*QCN5021/QCN5022/QCN5052/QCN5121/QCN5122/QCN5152 DEVICE REVISION SPECIFICATION*
80-Y9112-1	*QCA8075 DEVICE SPECIFICATION*
80-Y9112-3	*QCA8075 DEVICE REVISION SPECIFICATION*
80-Y0615-1	*QCA8081 DEVICE SPECIFICATION*
80-Y0615-4	*QCA8081 DEVICE REVISION SPECIFICATION*
80-YC769-1	*QCN5550 5 GHZ HIGH-POWER WLAN FRONT-END MODULE DATA SHEET*
80-YC769-10	*QCN5550 DEVICE REVISION SPECIFICATION*
80-YB772-1	*IPQ6018 AP.CP01 HARDWARE REFERENCE*
80-YB772-2	*IPQ6018 AP.CP01 SETUP GUIDE*
80-Y9000-20	*IPQ6010 AP.CP01.7 SETUP GUIDE*
80-Y9000-22	*IPQ6010 AP.CP01.7 HARDWARE REFERENCE GUIDE*

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

1.2.3 Reference documents for QCN90xx

DCN	Document Title
80-Y9000-1	*QCN9074-0 2.4/5/6 GHZ 4 X 4 MIMO 802.11AX/AC/A/N/P WLAN AP RFIC DATA SHEET*
80-Y9000-12	*QCN90XX PN01.1/PN02.1/PN02.7 HARDWARE REFERENCE*
80-Y9005-19	*QCN90XX MANUFACTURING TEST RECOMMENDATION APPLICATION NOTE*
80-Y9005-20	*QCN90XX FTM RF TEST APPLICATION NOTE*
80-Y9005-21	*QCN90XX QMSL APIS FOR FTM USER GUIDE*
80-Y9005-22	*QCN90XX WLAN TLV2 COMMANDS FOR FTM DIAGNOSTICS REFERENCE GUIDE*
80-Y9005-23	*QCN90XX BOARD DATA FILE UTILITY USER GUIDE*
80-YB215-8	*CTL ASSISTANT TOOL FOR 802.11AX CHIPSETS USER GUIDE*
80-Y9000-16	*QCN90XX 6 GHZ WDS NETWORK SETUP APPLICATION NOTE*
80-Y9006-10	*QCN90XX PN03 HARDWARE REFERENCE*
80-Y9004-13	*RDP0404: IPQ5018.AP.MP03.1.QCN9000.M.2.PN02.1 SETUP GUIDE*
80-Y9001-13	*RDP0415: IPQ5018.AP.MP03.3.QCN9100.M2.SPC01.1.QCN9000.M.2.PN02.6 SETUP GUIDE*
80-Y9000-12	*QCN90XX PN01/PN02 HARDWARE REFERENCE*
80-Y9004-10	*RDP0419: IPQ8076A WITH PN02.6 (6 GHZ) HARDWARE REFERENCE*
80-Y9006-10	*QCN90XX PN03 HARDWARE REFERENCE*
80-Y9005-10	*QCN90XX APPROVED VENDORS LIST*
80-Y9000-1	*QCN9074 2.4/5/6 GHZ 4 X 4 MIMO 802.11AX/AC/A/B/G/N WLAN AP RFIC DATA SHEET*
80-Y9017-1	*QCN9072-0 2.4/5/6 GHZ WLAN 2 X 2 MIMO 802.11AX/AC/A/B/G/N WLAN AP RFIC DATA SHEET*
80-Y9018-1	*QCN9070-0 2.4/5/6 GHZ WLAN 2 X 2 MIMO 802.11AX/AC/A/B/G/N WLAN AP RFIC DATA SHEET*
80-Y9001-1	*QCN9024 2.4/5/6 GHZ 4 X 4 MIMO 802.11AX/AC/A/B/G/N WLAN AP RFIC DATA SHEET*
80-Y9002-1	*QCN9022 2.4 GHZ 4 X 4 MIMO 802.11AX/AC/A/B/G/N WLAN AP RFIC DATA SHEET*
80-Y9003-1	*QCN6024-0 2.4/5 GHZ 4 X 4 MIMO 802.11AX/AC/A/B/G/N WLAN AP RFIC DATA SHEET*
80-Y9010-1	*QCN6023-0 2.4/5 GHZ 3 X 3 MIMO 802.11AX/AC/A/B/G/N WLAN AP RFIC DATA SHEET*
80-Y9000-5	*QCN9074-0 2.4/5/6 GHZ WLAN 4 X 4 MIMO 802.11AX/AC/A/N/P WLAN AP RFIC DEVICE REVISION GUIDE*
80-Y9005-26	*QCN90XX WLAN FIRMWARE PROGRAMMING GUIDE (QCN90XX ONLY)*
80-Y9005-27	*801.11AX WLAN FIRMWARE DEBUG STATS APPLICATION NOTE*
80-YA728-4	*IPQ807X/IPQ807XA/IPQ817X/QCN90XX SOC SOFTWARE USER GUIDE*
80-Y9000-19	*QCN90XX 6 GHZ WDS NETWORK SETUP APPLICATION NOTE*
80-Y9000-30	*QCN90XX WI-FI HOST SOFTWARE USER GUIDE*
80-Y9005-37	*QCN90XX WITH THIRD-PARTY PLATFORM PORTING GUIDE*

Software support for QCN90xx

RDP	NSS Offload	NSS Re-direct	Enterprise profile
Test Coverage	CST CS cycle tested		
RDP413(IPQ8074 + QCN90xx (2) + QCN90xx (5))	P(LM512M/1G)	P(LM512M/1G)	P(LM512M/1G)
RDP419(IPQ8076A (2) + IPQ8076A (5) + QCN90xx (6))	P(LM512M/1G)	P(LM512M/1G)	P(LM512M/1G)
RDP420(IPQ6010 (2) + IPQ6010 (5) + QCN90xx(6))	P(LM512M/1G)	P(LM512M/1G)	P(LM512M/1G)

NOTE: 64-bit is only supported for 1 GB DDR.

1.2.4 Related documentation for IPQ50xx

DCN	Document Title
80-10650-31	*IPQ5018 AP.MP03.1 HARDWARE REFERENCE*
80-10650-32	*IPQ5018 AP.MP03.1 SETUP GUIDE*

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

DCN	Document Title
80-10651-4	*IPQ5028 DEVICE REVISION GUIDE*
80-Y0615-4	*QCA8081 DEVICE REVISION SPECIFICATION*
80-Y0615-2	*QCA8081 PROGRAMMER'S MANUAL*
80-Y0615-3	*QCA8081 HARDWARE REGISTER DESCRIPTION*
80-YA728-5	*WIRELESS LAN ACCESS POINT (DRIVER VERSION 11.0) COMMAND REFERENCE GUIDE*
80-YA728-6	*WIRELESS LAN DRIVER VERSION 11.0 FOR ACCESS POINT PROGRAMMER GUIDE*
80-16052-15	*IPQ50XX WI-FI HOST SOFTWARE USER GUIDE*
80-YB478-3	*WLAN DRIVER CFG80211 SUPPORT APPLICATION NOTE*
80-YA728-10	*LXC AND DOCKER CONTAINERS FOR 802.11AX WLAN AP CHIPSETS USER GUIDE*
80-16052-16	*IPQ50XX QSDK USER GUIDE*
80-16052-17	*IPQ50XX SOC SOFTWARE USER GUIDE*
80-YA728-11	*ENABLE USB DEBUG CONSOLE FOR 802.11AC AND 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YA809-1	*FAIL SAFE BOOT FOR 802.11AC AND 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YA728-8	*CRASH DUMP COLLECTION IN U-BOOT FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-16052-18	*IPQ50XX SECURE BOOT ENABLEMENT USER GUIDE*
80-YB478-7	*PROFILING WITH PERF TOOL FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB714-1	*CDT DEFINITION AND MEMORY CONFIGURATION FOR 802.11AX WLAN AP CHIPSETS USER GUIDE*
80-YB478-5	*LINUX MODULE SIGNED KERNEL SUPPORT APPLICATION NOTE*
80-Y8045-3	*THERMAL MANAGEMENT ALGORITHM FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-Y8045-4	*COLD TEMPERATURE BOOT MITIGATION FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB478-10	*NOISE FLOOR CALIBRATION FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YC610-1	*QCA8081 SDK API REFERENCE MANUAL*
80-YC610-2	*QCA8081 SDK DIAGNOSTIC SHELL USER GUIDE*
80-YC610-3	*QCA8081 STANDALONE PHY DRIVER BUILD GUIDE*
80-YC610-5	*QCA8081 MACSEC DRIVER API REFERENCE*
80-YC610-6	*QCA8081 MACSEC DRIVER COMMAND LINE INTERFACE GUIDE*
80-YC610-7	*QCA8081 IEEE 1588 PTP DRIVER API REFERENCE*
80-YC610-8	*QCA8081 IEEE 1588 PTP DRIVER SHELL COMMAND REFERENCE*
80-YC610-9	*QCA8081 IEEE 1588 PTP APPLICATION NOTE*
80-YC610-11	*QCA8081 MACSEC DRIVER PORTING GUIDE*
80-Y9700-29	*WATCHDOG IMPLEMENTATION OR 802.11AC AND 802.11AX WLAP AP CHIPSETS APPLICATION NOTE*
80-YB215-2	*MANUFACTURING TEST RECOMMENDATIONS FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB215-1	*FTM RF TEST FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB215-4	*BOARD DATA FILE UTILITY FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB215-9	*BOARD ID FUSING FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-16052-31	*BLUETOPIA INSTALLATION AND OPERATION FOR IPQ50XX BLUETOOTH APPLICATION NOTE*

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

1.2.5 Reference documents for IPQ806x

DCN	DOCUMENT TITLE
80-Y7866-1	*IPQ806X DDR TUNING AND FLASH PARTITION CONFIGURATION APPLICATION NOTE*
80-Y9446-1	*IPQ806X LITTLE KERNEL BOOT LOADER APPLICATION NOTE*
80-Y8954-4	*ENABLING SECURE BOOT FOR IPQ806X APPLICATION NOTE*
80-Y9427-1	*QCA9984 DUAL-BAND 4X4 WITH 4 SS MIMO 802.11AC /ABGN WLAN SOC DEVICE SPECIFICATION*
80-Y9427-2	*QCA9994 DUAL-BAND 4X4 WITH 4 SS MIMO 802.11AC /ABGN WLAN SOC DEVICE SPECIFICATION*
80-Y9427-25	*QCA9984/QCA9994 DEVICE REVISION GUIDE*
80-Y5080-6	*IPQ8065 NETWORK PROCESSOR DEVICE SPECIFICATION*
80-Y5080-9	*IPQ8069 NETWORK PROCESSOR DEVICE SPECIFICATION*
80-Y7930-1	*QCA9990 DUAL-BAND 4X4 WITH MIMO 802.11AC/ABGN WLAN SOC DEVICE SPEC*
80-Y2314-1	*QCA9980 DUAL-BAND 4X4 WITH MIMO 802.11AC/ABGN WLAN SOC DEVICE SPEC*
80-Y8021-6	*QCA9984/QCA9994 CS.CAS01.1/1.3 HARDWARE REFERENCE GUIDE*
80-Y8021-5	*QCA9984/QCA9994 CUS239.5/7 HARDWARE REFERENCE GUIDE*
80-Y8022-5	*QCA9984/QCA9994 CUS260.5/7 HARDWARE REFERENCE GUIDE*
80-Y8672-3	*QCA9984/QCA9994 CUS238.7 HARDWARE REFERENCE GUIDE*
80-Y8672-4	*QCA9984/QCA9994 CUS240.7 HARDWARE REFERENCE GUIDE*
80-Y8021-1	*QCA9980/QCA9990 CUS239.1/2/3/4 HARDWARE REFERENCE GUIDE*
80-Y8022-1	*QCA9980/QCA9990 CUS260.1/2/3/4 HARDWARE REFERENCE GUIDE*
80-Y8022-2	*QCA9980/QCA9990 CS.BL01.1/2 HARDWARE REFERENCE GUIDE*
80-Y8672-2	*QCA9980/QCA9990 CUS238.1/3 HARDWARE REFERENCE GUIDE*
80-Y8672-5	*QCA9980/QCA9990 CUS240.1/3 HARDWARE REFERENCE GUIDE*
80-Y8133-2	*IPQ806X AP148 HARDWARE REFERENCE GUIDE*
80-Y9104-9	*IPQ8065 AP161 HARDWARE REFERENCE GUIDE*
80-Y9104-5	*AP160-02X (IPQ8068 + CUS239/CUS260 + XB242 AND VI807) HARDWARE REFERENCE GUIDE*
80-Y9122-1	*XB242 HARDWARE REFERENCE GUIDE*
80-Y9104-5	*IPQ8065.AP161.1.QCA9984.CS.CAS01.1.QCA9984.CS.CAS01.3.QCA9984.CUS260.5 SETUP GUIDE*
80-Y9104-8	*IPQ8068.AP148.5.QCA9994.CUS239.8.QCA9992.CS.BL01.1 SETUP GUIDE*
80-Y8023-5	*IPQ8064.AP148.6.QCA9980.CUS238.1.QCA9380.XB112.1 SETUP GUIDE*
80-Y9104-6	*IPQ8068.AP148.5.QCA9994.CUS239.7.CUS260.7 SETUP GUIDE*
80-Y9104-1	*IPQ8065 AP161 CUS239 CUS260 SETUP GUIDE*
80-Y9104-4	*IPQ8065 AP161 CUS238 CUS240 SETUP GUIDE*
80-Y9103-12	*AP160 IPQ8069+QCA9994 CUS239/260+XB242 AND VI807 SETUP GUIDE*
80-Y8023-6	*IPQ8068.AP148.5.QCA9994.CUS239.8.QCA9992.CS.BL01.2 SETUP GUIDE*
80-Y8023-7	*QCA9558.AP.SCP01.1.QCA9984.CUS239.5 SETUP GUIDE*
80-Y8023-1	*AP148 (IPQ8064/8068+ QCA8337N+ QCA9980/QCA9990 CUS239+CUS260) SETUP GUIDE*
80-Y8023-1	*AP148 (IPQ8064/8068+ QCA8337N +QCA9980/QCA9990 CUS238/240) SETUP GUIDE*
80-Y9103-7	*IPQ8068.AP160.1.QCA9990.CUS239.3.QCA9990.CUS260.3.QCA9889.XB242.2 SETUP GUIDE*
80-Y9103-6	*AP160 (IPQ8068+CUS239/CUS260+XB242 AND VI807) SETUP GUIDE*
80-Y9103-9	*IPQ8064.AP160.3.QCA9980.CUS239.1.QCA9980.CUS260.1.QCA6320.HB.SP02.1 SETUP GUIDE*

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

1.2.6 Reference documents for IPQ4019

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

DCN	DOCUMENT TITLE
80-Y9571-2	*IPQ4018/IPQ4019/IPQ4028/IPQ4029 PROFILING WITH PERF APPLICATION NOTE*
80-Y9571-4	*IPQ40X8 AND IPQ40X9 SOFTWARE CONFIGURATION GUIDE*
80-Y9571-5	*IPQ4018/IPQ4019/IPQ4028/IPQ4029 LAUTERBACH APPLICATION NOTE*
80-Y9571-6	*IPQ4018/IPQ4019/IPQ4028/IPQ4029 PERF MANAGEMENT APP NOTE*
80-Y8950-19	*IPQ40XX CDT AND FLASH PARTITION CONFIGURATION GUIDE*
80-Y8950-22	*IPQ40XX SECURITY DESIGN USER GUIDE*
80-Y8951-2	*IPQ40XX EMERGENCY DOWNLOAD APPLICATION NOTE*
80-YA809-1	*IPQ40XX FAIL SAFE BOOT DESIGN*
80-Y9700-30	*IPQ40XX PULSE WIDTH MODULATION (PWM) APPLICATION NOTE*
80-Y9347-18	*IPQ4018 ACCESS POINT SOC DEVICE SPECIFICATION*
80-Y9347-28	*IPQ4028 ACCESS POINT SOC DEVICE SPECIFICATION*
80-Y9348-28	*IPQ40X8 DEVICE REVISION GUIDE*
80-Y9347-19	*IPQ4019 ACCESS POINT SOC DEVICE SPECIFICATION*
80-Y9347-29	*IPQ4029 ACCESS POINT SOC DEVICE SPECIFICATION*
80-Y9348-29	*IPQ40X9 DEVICE REVISION GUIDE*
80-Y9112-1	*QCA8075 FIVE-PORT 10/100/1000 MBPS ETHERNET TRANSCEIVER DEVICE SPECIFICATION*
80-Y9112-2	*QCA8075 5-PORT 10/100/1000 MBPS TRANSCEIVER HARDWARE PROGRAMMING REFERENCE*
80-Y9112-3	*QCA8075 DEVICE REVISION GUIDE*
80-Y9112-4	*QCA8072 DEVICE REVISION GUIDE*
80-Y9112-5	*QCA8075 QCA8072 CO-LAYOUT DESIGN APPLICATION NOTE*
80-Y9700-3	*IPQ4018/IPQ4028/IPQ4019/IPQ4029 RF TEST USER GUIDE*
80-Y9700-4	*IPQ4018/IPQ4028 THERMAL DESIGN GUIDELINES APP NOTE*
80-Y9700-7	*IPQ4019/IPQ4029 THERMAL DESIGN GUIDELINES*
80-Y9700-5	*IPQ4019/4029 AP.DK04 HARDWARE REFERENCE GUIDE*
80-Y9700-6	*IPQ4019/4029 AP.DK04 SETUP GUIDE*
80-Y9798-1	*WLAN AP TEST AND CALIBRATION USER GUIDE*
80-Y9571-7	*IPQ4018/IPQ4019/IPQ4028/IPQ4029 HOME SWITCH SDK USER GUIDE*
80-Y9571-8	*IPQ4018/IPQ4019/IPQ4028/IPQ4029 SWITCH SDK REFERENCE MANUAL*
80-Y9571-9	*IPQ4018/IPQ4019/IPQ4028/IPQ4029 SSDK DIAGNOSTIC SHELL USER GUIDE*
80-Y9571-10	*IPQ4018/IPQ4019/IPQ4028/IPQ4029 SWITCH UCI COMMAND USER GUIDE*
80-Y6399-3	*IPQ40XX QSDK SETUP USER GUIDE*
80-Y9700-1	*IPQ4018/4028 AP.DK01 HARDWARE REFERENCE GUIDE*
80-Y9700-2	*IPQ4018/4028 AP.DK01 SETUP GUIDE*
80-Y9700-24	*IPQ4019/4029 AP.DK07 REFERENCE DESIGN HARDWARE REFERENCE GUIDE*
80-Y9700-25	*IPQ4019/4029 AP.DK07 REFERENCE DESIGN SETUP GUIDE*
80-YA758-1	*HARDWARE REWORK FOR INTEROPERATION OF DK07 AND SILICON LABS MODULES FOR ZIGBEE BRINGUP GUIDE*
80-Y7866-2	*5G CHANNEL SUPPORT IN THREE RADIO DESIGN USING SINGLE BAND SIMULTANEOUS FILTER APPLICATION NOTE*

1.2.7 Reference documents for QCN61xx

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

DCN	Title
80-10651-4	*IPQ5028 DEVICE REVISION GUIDE*
80-Y0615-4	*QCA8081 DEVICE REVISION SPECIFICATION*
80-Y0615-2	*QCA8081 PROGRAMMER'S MANUAL*
80-Y0615-3	*QCA8081 HARDWARE REGISTER DESCRIPTION*
80-YA728-5	*WIRELESS LAN ACCESS POINT (DRIVER VERSION 11.0) COMMAND REFERENCE GUIDE*
80-YA728-6	*WIRELESS LAN DRIVER VERSION 11.0 FOR ACCESS POINT PROGRAMMER GUIDE*
80-16052-15	*IPQ50XX WI-FI HOST SOFTWARE USER GUIDE*
80-YB478-3	*WLAN DRIVER CFG80211 SUPPORT APPLICATION NOTE*
80-YA728-10	*LXC AND DOCKER CONTAINERS FOR 802.11AX WLAN AP CHIPSETS USER GUIDE*
80-16052-16	*IPQ50XX QSDK USER GUIDE*
80-16052-17	*IPQ50XX SOC SOFTWARE USER GUIDE*
80-YA728-11	*ENABLE USB DEBUG CONSOLE FOR 802.11AC AND 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YA809-1	*FAIL SAFE BOOT FOR 802.11AC AND 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YA728-8	*CRASH DUMP COLLECTION IN U-BOOT FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-16052-18	*IPQ50XX SECURE BOOT ENABLEMENT USER GUIDE*
80-YB478-7	*PROFILING WITH PERF TOOL FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB714-1	*CDT DEFINITION AND MEMORY CONFIGURATION FOR 802.11AX WLAN AP CHIPSETS USER GUIDE*
80-YB478-5	*LINUX MODULE SIGNED KERNEL SUPPORT APPLICATION NOTE*
80-Y8045-3	*THERMAL MANAGEMENT ALGORITHM FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-Y8045-4	*COLD TEMPERATURE BOOT MITIGATION FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB478-10	*NOISE FLOOR CALIBRATION FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YC610-1	*QCA8081 SDK API REFERENCE MANUAL*
80-YC610-2	*QCA8081 SDK DIAGNOSTIC SHELL USER GUIDE*
80-YC610-3	*QCA8081 STANDALONE PHY DRIVER BUILD GUIDE*
80-YC610-5	*QCA8081 MACSEC DRIVER API REFERENCE*
80-YC610-6	*QCA8081 MACSEC DRIVER COMMAND LINE INTERFACE GUIDE*
80-YC610-7	*QCA8081 IEEE 1588 PTP DRIVER API REFERENCE*
80-YC610-8	*QCA8081 IEEE 1588 PTP DRIVER SHELL COMMAND REFERENCE*
80-YC610-9	*QCA8081 IEEE 1588 PTP APPLICATION NOTE*
80-YC610-11	*QCA8081 MACSEC DRIVER PORTING GUIDE*
80-Y9700-29	*WATCHDOG IMPLEMENTATION OR 802.11AC AND 802.11AX WLAP AP CHIPSETS APPLICATION NOTE*
80-YB215-2	*MANUFACTURING TEST RECOMMENDATIONS FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB215-1	*FTM RF TEST FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB215-4	*BOARD DATA FILE UTILITY FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-YB215-9	*BOARD ID FUSING FOR 802.11AX WLAN AP CHIPSETS APPLICATION NOTE*
80-Y0631-1	*QCN6122 5/6 GHZ 2 X 2 802.11AX/AC/A/N WLAN AP DATA SHEET*

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2 IPQ8074.ILQ.11.5 CSU1

2.1 Supported features

- CPU/platform
  - Quad ARM Cortex A53 at 2.2 GHz, 64-bit ISA v8 instruction set, 20 K DMIPS
  - 32 KB/32 KB I$/D$ and 512 KB L2$
  - Floating point and NEON SIMD DSP for each core
  - DDR4/DDR3L, USB3, PCIe, all serial interfaces
- Wireless connectivity
  - For IPQ807x/IPQ817x: 5 GHz antenna configuration
    - 8 x 8/8S-80 MHz or 4x4/4S-160 MHz
    - 2x2/2S-160 MHz
    - 4 x 4/4S-80 MHz
    - 4 x 4/4S-80 MHz + 4 x 4/4S-80 MHz (SBS)
    - 4 x 4/4S-80 MHz + 2 x 2/2S-80 MHz (SBS)
  - 2.4 GHz antenna configuration
    - 4 x 4/4S-40 MHz
- Networking
  - Dual-core multi-threaded network accelerator (NPU)
  - Advanced classification, policing, queuing at wire speed (PPE)
  - USXGMII/XFI, SGMII, SGMII+, PSGMII
  - IPv4/IPv6 passthrough feature
  - Fixed wireless access (FWA) support

    This release contains FWA features. Enable these features by following the instructions shown in the *IPQ8074.ILQ.11.5/IPQ5018.ILQ.11.5 CS Fixed Wireless Access (FWA) (80-11333-8).*
- Standard programming model
  - Linux 4.4 (32-bit and 64-bit)
  - QSDK based on OpenWrt CC

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

QCN90xx chipset features

Platform features

- DC 12 V, 5-A power adapter
- Two x16 DDR3 devices, 1866MT/s 32-bit interface, Total 1 GB (512 MB each)
- SPI NOR
- 8-bit parallel NAND flash
- Two PCIe interfaces
- JTAG
- UART
- One 10G MAC and four 1G MAC
- 1 SFP port

QCN90xx chipset features

- 4x4/160 MHz 11ax PCIe Radio
- 2.4 GHz, 5 GHz, 6 GHz full band support
- Dual Lane PCIe Gen 3
- Package: 11.1 x 12 FCBGA, 0.65 mm ball pitch
- WLAN
  - Dual-synthesizer WLAN radio up to 160 MHz band width, support aSA, ADFS
  - Supports 20/40 MHz in 2.4 GHz
  - Supports 20/40/80/160 MHz in 5 GHz
  - Supports 20/40/80/160 MHz in 6 GHz
  - Supports up to 1024 QAM (4 Spatial stream (SS)) and 4096 QAM (2 SS)
- Supported standards
  - IEEE802.11a/b/g/n/ac/ax
  - IEEE 802.11d/e/h/i/j/k/r/u/v/w

2.2 Frequency spectrum and regulatory domain support

QCA_Networking_2021.SPF.11.5 release implements the regulatory versions, 34 through 36.For the AFC FR testing support FCC SP rules alone are added from Regulatory version 37. For details about the regulatory domains and mapping country codes, see *Regulatory Configuration for QCA WLAN (80-Y6630-2).*

NOTE: Please contact customer engineering team to refer the 80-Y6630-2 document revision corresponding to the Regulatory data base versions supported in this release.

Contact Qualcomm Customer Engineering for guidance to enable 6 GHz operation for desired countries for evaluation or customer release. See the *WLAN Conformance Testing and Compliant Power Configuration for 802.11ax Chipsets Information to Share with Test Lab* application note

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

(80-YB952-3) and the *Wireless LAN Driver Version 11.0 for Access Point Programming Guide* (80-YA728-6) for full description of 6 GHz support on WLAN AP chipsets.

The Regulatory database updates which have been maintained in FW (2.4GHz & 5GHz) is deprecated from 11.4 release. There is no FW regulatory database maintained for 6GHz and it is maintained only in BDF.

The Regulatory database updates are done only in BDF for all 11ax chipsets, and the FW database is not updated from 11.1 release onwards.

The Regulatory database will be picked up from BDF by default.

New 6 GHz rules and regulatory domain support will change frequently during 2021-2022. This status is accurate as of the date of publication of these *Release Notes*:
■ Supported in this release:

- U.S.: LPI AP
- U.S: SP_AP (only for testing purpose)
- UK: LPI and VLP
- European Community (Draft Requirements): LPI and VLP
- Chile and UAE: LPI
- Guatemala: LPI and VLP
- Korea and Brazil: LPI and VLP

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.3 Restrictions on software while using it for testing

DL OFDMA Test Tones for FCC Conformance Testing	WLAN Conformance Testing And Compliant Power Configuration for 802.11ax Chipsets Information to Share With Test Lab (80-YB952-3) have been updated with FCC conformance test guidance for DL OFDM transmissions. Test tone XML files to support the FCC testing using QSPR and QRCT are available for the WLAN AP software release at these paths: ▪ In the FW package, the file is available at "/bdfUtil". ▪ In the WLAN source, the file is available at: wlan_proc\wlan\phyrf_svc\tools\bdfExcel\qca8074\CTL_installer_python_scripts\FCC_interim_Guidance_Test_Tones.zip
NSS	▪ Only IPv4/IPv6 + TCP/UDP flows are accelerated through fast path ▪ LAG: Linux bonding driver configuration supported in fast path through the bonding driver sysfs IF: ▫ Mode: 802.3ad (4): xmit_hash_policy: layer2 (0), layer2+3 (2) ▫ Mode: Balance-xor (2): xmit_hash_policy: layer2 (0), layer3+4 (1), layer2+3 (2) ▪ Flows that require ALG support are not accelerated by NSS (except for TFTP (port 69)) ▪ Interfaces configured with NSS qdiscs must have a leaf node configured as the default node for enqueue (using the set_default qdisc parameter); if qdisc structures are created without a default, no packets are transmitted; this includes management packets such as ARP ▪ Tunnel support (6RD, DS-Lite) expects at least the initial few packets in the LAN > WAN direction to push the base fast path rule; further flows for these tunnels can be initiated on either WAN or LAN ▪ PPPoE-relay packets in NSS is not supported ▪ IGMPv3 and MLDv2 traffic over PPPoE WAN interface is not accelerated ▪ DHCP relay over PPPoE is not supported ▪ SAMBA is not turned on by default in this release and must be enabled manually ▪ Qualcomm provides a software implementation for IGMP/MLD snooping ▪ Inline IPsec flows deviate from current NSS behavior for upstream flows; during NSS IPsec acceleration, fragmentation (when required) is performed pre-encapsulation while inline IPsec (hardware) acceleration fragmentation is done post encapsulation ▪ EDMA can fail to queue the packets to eth_rx; the eth_rx drop counter increments even though EDMA holds on to these packets and tries queueing later ▪ NSS connection manager expects to see at least one packet in outgoing direction to push tunnel 3/5 Tuple offload rule ▪ L2TP tunnel must brought up and down explicitly using ifup and if down commands ▪ IPSec anti-replay must be disabled in KLIPS by using a module parameter ▪ After IPSec restart, if flows are already accelerated by NSS, an ECM restart is required to reestablish flows (known limitation with IPSec) ▪ If LCP timeout or AVP packets timeout occurs between link plug out and plugin, L2TP tunnel goes down and must be brought up using ifup command. ▪ Only IPv4 encapsulation is supported in L2TP for outer encapsulation. However, IPv4/IPv6 is supported as inner payloads ▪ L2TPv2 supported for acceleration. L2TPv3 not supported for acceleration ▪ WLAN multicast to unicast conversion functionality supports only up to 16 groups.
QCN90xx	▪ Low memory profiles are supported only for 32-bit. Premium/Enterprise profiles supported on both 32-bit & 64-bit.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.4 Supported hardware for this SP

RDP	RD	Chipset part number
RDP0258	AP.HK01	IPQ8078A QCN5124 QCN5154 PMP8074 QCA8075 Third-party chipset AQR107 (10 Gbps)
RDP0385	AP.HK01-SBS	IPQ8074A QCN5024 QCN5054 PMP8074 QCA8075 Third-party chipset AQR107 (10 Gbps)
RDP0255	AP.HK01 v1.2	IPQ8074 QCN5024 QCN5054 PMP8074
RDP0386	AP.HK09	IPQ8076A QCN5124 QCN5154 PMP8074 QCA8081
RDP0379	AP.AC01	IPQ8070 QCA8075 QCA8081 PMP8074
RDP0389	AP.AC01-v2	IPQ8070A QCN5124 QCN5154 PMP8074 QCA8081
RDP0380	AP.AC02	IPQ8071 QCA8075 QCA8081 PMP8074
RDP0390	AP.AC02-v2	IPQ8071A QCN5124 QCN5154 PMP8074 QCA8081
RDP0400	AP.AC03.1	IPQ8070A QCN5124 QCN5154 PMP8074 QCA8075
RDP0401	AP.AC04.1	IPQ8071A QCN5124 QCN5154 PMP8074 QCA8075

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

RDP	RD	Chipset part number
RDP0393	AP. OAK03	IPQ8174 QCN5024 QCN5054 PMP8074 QCA8075
RDP0409	AP.HK01.5	IPQ8074A QCN5024 QCN5054 PMP8074 QCA8075 SDX55
RDP413	AP.V2HK10.2	IPQ8074A QCN9074/QCN90xx QCA8075 PMP8074
RDP414	AP.V2HK10.2	IPQ8074A QCN9074/QCN90xx QCA8075 PMP8074
RDP419	AP.V2HK014.1	IPQ8076A QCN9074 QCA8075 PMP8074 QCN5124 QCN5154
RDP430	AP.V2HK10.1	IPQ8078A QCN5024 QCN5054 PMP8074 QCN90xx
RDP0438	AP.HK01.5	IPQ8074A.AP.HK01.5.RDP

2.5 Build and load the image for IPQ8074.ILQ.11.5 CSU1

1. Download the Qualcomm Technologies, Inc. (QTI) proprietary code from Qualcomm Chip Code (see Section 2.5.1).

   This release contains FWA features. Enable these features by following the instructions shown in *the IPQ8074.ILQ.11.5/IPQ5018.ILQ.11.5 CS Fixed Wireless Access (FWA) (*80-11333-8).

2. Download other components from external websites for QSDK while building the default configuration (see Section 2.5.2).
3. Generate the firmware:
   a. Re-assemble the code (see Section 2.5.3.2).
   b. Create the QSDK build (see Section 2.5.4).
   c. Generate a complete firmware image (see Section 2.5.3)
4. Install the image in the device flash memory and boot using the image from the flash (see Section 2.5.5).

Users must be familiar with directory structures that contain SP images for the different subsystems before downloading the code and building the images for loading. For each SP included in an SPF, SP binary files are generated from the SI binary files of only a subset of the included SIs. In an SPF, some SIs may support multiple SPs while others may only support one SP.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.5.1 Download packages available through Qualcomm Chip Code

QTI proprietary code is available from Qualcomm Chip Code. A web/GUI interface and a secure git server both allow access to this code.

- Browse available packages and obtain the download URL at: https://chipcode.qti.qualcomm.com/
- For more information on cloning the code, see: https://chipcode.qti.qualcomm.com/helpki/cloning-code-from-a-repository
- For more information on installation and configuration of the correct version of git and OpenSSL on both Windows and Linux platforms, see: https://chipcode.qti.qualcomm.com/helpki

  These versions are required to support the authentication methods used by Qualcomm Chip Code.

2.5.2 Download packages from external websites

QSDK downloads these components while building the default profile configuration. Further, customize QSDK to download more components; this table lists only the components that are necessary for at least one of the QSDK 2.0 default profiles. This list does not include the packages obtained from Qualcomm ChipCode.

Table 2-1 IPQ8074.ILQ.11.5 packages available from external sites

Package
1.0.4.3.arm
accel-pptp-0.8.5.tar.bz2
acl-20140812.tar.gz
alsa-lib-1.0.28.tar.bz2
alsa-utils-1.0.28.tar.bz2
argp-standalone-1.3.tar.gz
arptables-v0.0.4.tar.gz
attr-20150220.tar.gz
autoconf-2.69.tar.xz
automake-1.15.tar.xz
bash-5.0.tar.gz
bc-1.06.95.tar.bz2
binutils-2.24.tar.bz2
binutils-linaro-2.24.0-2014.09.tar.xz
bison-3.0.2.tar.xz
breakpad-0.1-c46151db0ffd1a8dae914e45f1212ef427f61ed3.tar.gz
bridge-utils-1.5.tar.gz
busybox-1.30.1.tar.bz2
bzip2-1.0.6.tar.gz
ca-certificates_20190110.tar.xz
c-ares-1.10.0.tar.gz

Package
cmake-3.3.2.tar.gz
coreutils-8.23.tar.xz
cryptsetup-1.6.7.tar.xz
curl-7.66.0.tar.xz
db-4.7.25.NC.tar.gz
dhcp-4.3.6.tar.gz
dnsmasq-2.80.tar.xz
dosfstools-3.0.28.tar.gz
dropbear-2019.78.tar.bz2
e2fsprogs-1.42.12.tar.gz
e2fsprogs-1.42.8.tar.gz
easycwmp-1.6.0.tar.gz
ebtables-v2.0.10-4.tar.gz
elfutils-0.161.tar.bz2
ethtool-3.18.tar.xz
expat-2.2.4.tar.bz2
file-5.25.tar.gz
findutils-4.4.2.tar.gz
firewall-2015-07-27-980b7859bbd1db1e5e46422fccccbce38f9809ab.tar.gz
flex-2.5.39.tar.bz2

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Package
fstools-2016-01-10-96415afecef35766332067f4205ef3b2c7561d21.tar.gz
gcc-5.2.0.tar.bz2
gdb-7.8.tar.xz
gdb-linaro-7.6-2013.05.tar.bz2
gdbm-1.11.tar.gz
gengetopt-2.22.6.tar.gz
gmp-5.1.3.tar.xz
gmp-6.0.0a.tar.xz
go1.16.6.linux-amd64.tar.gz
i2c-tools-3.1.2.tar.bz2
iozone3_420.tar
iperf-2.0.5.tar.gz
iperf-3.0.11.tar.gz
iproute2-4.0.0.tar.xz
iptables-1.4.21.tar.bz2
iputils-s20101006.tar.bz2
iw-5.9.tar.xz
jansson-2.7.tar.gz
json-c-0.12.tar.gz
jsonfilter-2014-06-19-cdc760c58077f44fc40adbbe41e1556a67c1b9a9.tar.gz
kmod-20.tar.xz
libaio_0.3.110.orig.tar.gz
libcap-2.24.tar.xz
libelf-0.8.13.tar.gz
libevent-2.1.11-stable.tar.gz
libffi-3.0.13.tar.gz
libgcrypt-1.6.6.tar.bz2
libgpg-error-1.12.tar.bz2
libiwinfo-2018-12-25-dd508af4.tar.xz
libmicroxml-2015-03-18-caa8d3e6887f5c70e54df555dd78e4e45cfa74cc.zip
libnl-3.2.21.tar.gz
libpcap-1.8.1.tar.gz
libroxml-2.3.0.tar.gz
librpc-2015-11-04-a921e3ded051746f9f7cd5e5a312fb6771716aac.tar.gz
libtool-2.4.tar.gz
libubox-2015-11-08-10429bccd0dc5d204635e110a7a8fae7b80d16cb.tar.gz
libusb-1.0.19.tar.bz2

Package
libwebsockets-1.3-chrome37-firefox30.tar.xz
libxml2-2.9.9.tar.gz
linux-atm-2.5.2.tar.gz
linuxptp-2.0.tgz
lldpd-0.7.15.tar.gz
lmbench-3.0-a9.tgz
lrzsz-0.12.20.tar.gz
lss-1.0.tar.gz
ltp-20170929.tar.gz
lua-5.1.5.tar.gz
luafilesystem-1.6.2.tar.gz
LuaSrcDiet-0.12.1.tar.bz2
LVM2.2.02.119.tgz
lxc-2.1.1.tar.gz
lzma-4.65.tar.bz2
lzo-2.10.tar.gz
m4-1.4.17.tar.xz
make-ext4fs-2015-05-01.tar.gz
mbedtls-1.3.17-gpl.tgz
mcproxy-2014-12-31-b7bd2d0809a0d1f177181c361b9a6c83e193b79a.tar.bz2
mdadm-3.2.5.tar.xz
memtester-4.1.3.tar.gz
minicom-2.7.tar.gz
miniupnpd-1.9.20150609.tar.gz
mklibs_0.1.35.tar.gz
mm-common-0.9.7.tar.xz
mosquitto-1.4.10.tar.gz
mpc-1.0.2.tar.gz
mpfr-3.1.2.tar.bz2
mtd-utils-1.5.1-92686f212c9a4e16891c6a3c57629cbf4f0f8360.tar.gz
musl-1.1.16.tar.gz
nat46-6.tar.xz
ncurses-6.1.tar.gz
netifd-2015-12-16-245527193e90906451be35c2b8e972b8712ea6ab.tar.gz
nghttp2-1.29.0.tar.xz
ntfs-3g_ntfsprogs-2014.2.15.tgz
odhcp6c-2016-02-08.tar.bz2

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Package
odhcpd-2016-10-09.tar.bz2
openssl-1.1.1g.tar.gz
openswan-2.6.51.3.tar.gz
openvpn-2.4.7.tar.gz
openvpn-easy-rsa-2013-01-30-ff5bfd1dd8e548cb24d302742af3894f893ef92f.tar.gz
openvswitch-2.11.1.tar.gz
opkg-9c97d5ecd795709c8584e972bfdf3aee3a5b846d.tar.gz
p910nd-0.97.tar.bz2
patch-2.7.5.tar.xz
patchelf-0.8.tar.bz2
pciutils-3.3.1.tar.gz
pkg-config-0.29.tar.gz
pm-utils-1.4.1.tar.gz
popt-1.16.tar.gz
ppp-2.4.7.git-2019-05-25.tar.xz
procd-2015-10-29.1-d5fddd91b966424bb63e943e789704d52382cc18.tar.gz
procps-3.2.8.tar.gz
Python-2.7.13.tar.xz
quagga-1.2.4.tar.gz
quilt-0.63.tar.gz
readline-8.0.tar.gz
remserial-1.4.tar.gz
rng-tools-5.tar.gz
rpcd-2016-06-30-23417e94d25570e6d62542bac46edd51e8e0243a.tar.gz
rp-pppoe-3.11.tar.gz
rstp-2011-10-11-434d24bae108dbb21461a13a4abcf014afa8b029.tar.gz
rtl8712u.bin
samba-3.6.25.tar.gz
sed-4.2.2.tar.bz2
shadow-4.2.1.tar.xz
six-1.10.0.tar.gz
sqlite-autoconf-3260000.tar.gz
squashfs4.2.tar.gz
strace-4.8.tar.xz
stress-1.0.4.tar.gz

Package
sysfsutils-2.1.0.tar.gz
sysstat-11.0.4.tar.xz
tayga-0.9.2.tar.bz2
tcpdump-4.9.2.tar.gz
ubi-utils-1.5.1.tar.gz
u-boot-2014.10.tar.bz2
ubox-2015-11-22-c086167a0154745c677f8730a336ea9cf7d71031.tar.gz
ubus-2015-05-25-f361bfa5fcb2daadf3b160583ce665024f8d108e.tar.gz
uci-2019-09-01-415f9e48.tar.xz
uClibc++-0.2.4.tar.bz2
uhttpd-2015-11-08-fe01ef3f52adae9da38ef47926cd50974af5d6b7.tar.gz
unbound-1.9.6.tar.gz
upx-3.96-amd64_linux.tar.xz
usb.ids.2013-01-16.gz
usbutils-007.tar.bz2
usign-2015-05-08-cf8dcdb8a4e874c77f3e9a8e9b643e8c17b19131.tar.gz
ustream-ssl-2015-07-09-c2d73c22618e8ee444e8d346695eca908ecb72d3.tar.gz
util-linux-2.25.2.tar.xz
v1.01.zip
valgrind-3.13.0.tar.bz2
wireless_tools.29.tar.gz
wolfssl-3.8.0.zip
xl2tpd-devel-20150930.tar.gz
xtables-addons-2.10.tar.xz
xz-5.2.1.tar.bz2
yaffs2_android-2008-12-18.tar.bz2
zlib-1.2.11.tar.xz

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.5.3 Generate the firmware for IPQ8074.ILQ.11.5 CSU1

The firmware image can be generated either using a script or manually.

2.5.3.1 Script to generate complete firmware image

Meta generation script for SPF 11.5 (meta_generation_script.py) aims to simplify the release notes instructions required to be executed by the customers.

Steps to follow:

```
$ mkdir $BUILD_WS
(Assume BUILD_WS is workspace directory)
$ cd $BUILD_WS
$ git clone <OEM distro chipcode link>
<Clone required deliverable distro in the path where OEM distro is cloned
using Git clone commands and checkout(git checkout) to release tag on each
deliverable distro>
$ cd <OEM Chipcode directory>
$ git checkout r11.5_00003.0
$ rm -rf BOOT.AK.1.0 BOOT.BF.3.1.1 IPQ4019.ILQ.11.* IPQ8064.ILQ.11.*
RPM.AK.* TZ.AK.* TZ.BF.2.7 WIGIG.TLN* IPQ6018.ILQ.11.* TZ.WNS.5.1
BOOT.XF.0.3 BOOT.BF.3.3.1.1 TZ.WNS.4.0 IPQ5018.ILQ.11.* BTFW.MAPLE.*
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
--repo-url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable
If this command does not work, please use below repo init command
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
$ repo sync -j8 --no-tags -qc
$ cd common/build
$ python meta_generation_script.py -c r11.5_00003.0 -s IPQ8074.ILQ.11.5 -p
<Profile> -b <32/64> -d <DELIVERABLE_COMBO> -m [MESH_OPTIONS] --path
$BUILD_WS
Eg : python meta_generation_script.py -c r11.5_00003.0 -s IPQ8074.ILQ.11.5
-p P -b 64 -d HYFI,SIGMA-DUT -m SON --path /local/mnt/workspace/Build
```

Software Product	Profile and bit config	Supported Deliverables Combo	Mesh options
	P – 32,64	HYFI,WHC,WAPID,SIGMA-DUT,FWA,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH,SON,EZMESH_FULL,EZMESH_COLOCATED
	E – 32,64	SIGMA-DUT,OEM	NA

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Software Product	Profile and bit config	Supported Deliverables Combo	Mesh options
IPQ8074.ILQ.11.5	LM512 – 32	HYFI,WHC,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH, SON, EZMESH_FULL, EZMESH_COLOCATED
	LM256 – 32	HYFI,WHC,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH, SON,EZMESH_FULL,EZMESH_COLOCATED
	LM512E – 32	SIGMA-DUT,OEM	NA

NOTE: If there are no additional deliverables, use -d OEM while triggering meta_generation_script.

NOTE: If there is no Mesh options, please remove -m <Mesh option> while triggering meta_generation_script.

Single images will be present in below directory;

```
32 Bit : $BUILD_WS/IPQ8074.ILQ.11.5/<Profile>_Timestamp/<OEM distro>/common/build/bin
64 Bit : $BUILD_WS/IPQ8074.ILQ.11.5/<Profile>_Timestamp/ipq_8074*64*/<OEM distro>/common/build/bin
```

2.5.3.2 Manually generate complete firmware image

This example assumes that all packages listed in sections 2.5.1 and 2.5.2 are obtained using the **git clone** command and placed in the top-level directory.

NOTE: 64-bit single image generation requires one of the elf generated by compiling the 32-bit build. Complete the 32-bit compilation before following the 64-bit build generation steps.

1. Enter these commands to re-assemble the code and generate the QSDK framework:

```
$ git clone <Chipcode Directory>
$ cd <Chipcode Directory>
$ git checkout r11.5_00003.0
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

After the copy of the existing Git repository is completed, change the directories where the files are present as described in this table before running the repo command:

Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
qsdk-qca-wifi qsdk-qca-wlan qsdk-qca-art qsdk-ieee1905-security qsdk-whc qsdk-whcpy	NHSS.QSDK.11.5\apss_proc\out\proprietary\Wifi
meta-tools common-tools qsdk-qca-nss qca-lib qca-mcs-apps qca-nss-userspace qca-time-services qca-qmi-framework gpio-debug qca-diag qca-cnss-daemon athtestcmd fw-qca-stats qca-IOT/qca-IOT.tar.bz2 qca-nss-fw-eip-hk btdaemon minidump	NHSS.QSDK.11.5\apss_proc\out\proprietary\QSDK-Base
qca-bluetopia	NHSS.QSDK.11.5\apss_proc\out\proprietary\BLUETOPIA
qca-wifi-fw-src-component-cmn-WLAN.HK.*.tgz qca-wifi-fw-QCA8074_v1.0-WLAN.HK.*.tar.bz2 qca-wifi-fw-QCA8074_v2.0-WLAN.HK.*.tar.bz2	WLAN.HK.2.6\wlan_proc\src\components\QCA8074_v2.0 WLAN.HK.2.6\wlan_proc\pkg\wlan_proc\bin\QCA8074_v1.0 WLAN.HK.2.6\wlan_proc\pkg\wlan_proc\bin\QCA8074_v2.0
qca-wifi-fw-src-component-cmn-WLAN.BL.*.tgz qca-wifi-fw-src-component-halphy_tools-WLAN.BL.*.tgz qca-wifi-fw-QCA9888_hw_2-WLAN.BL.*.tar.bz2 qca-wifi-fw-AR900B_hw_2-WLAN.BL.*.tar.bz2 qca-wifi-fw-QCA9984_hw_1-WLAN.BL.*.tar.bz2 qca-wifi-fw-IPQ4019_hw_1-WLAN.BL.*.tar.bz2	WLAN.BL.3.16\cnss_proc\src\components WLAN.BL.3.16\cnss_proc\bin
qca-wifi-fw-AR9887_hw_1-CNSS.PS.*.tar.bz2 qca-wifi-fw-AR9888_hw_2-CNSS.PS.*.tar.bz2	CNSS.PS.3.16
BIN-NSS.FW.*.tar.bz2	NSS.FW.11.5\nss_proc\out\proprietary
qca-afc-bin	NHSS.QSDK.11.5\apss_proc\out\proprietary\RBIN-AFCAgent

```
$ rm -rf BOOT.AK.1.0 BOOT.BF.3.1.1 IPQ4019.ILQ.11.* IPQ8064.ILQ.11.* RPM.AK.* TZ.AK.*
 TZ.BF.2.7 WIGIG.TLN* IPQ6018.ILQ.11.* TZ.WNS.5.1 BOOT.XF.0.3 BOOT.BF.3.3.1.1
 TZ.WNS.4.0 IPQ5018.ILQ.11.* BTFW.MAPLE.*
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b release -m
 caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
```

If this command does not work, please use below repo init command:

```
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b release -
 m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml --repo-
 url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable
$ repo sync -j8 --no-tags -qc
$ mkdir -p qsdk/dl
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-ieee1905-security/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-art/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wifi/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wlan/* qsdk
$ cp -rf wlan_proc/src/components/QCA8074_v2.0/qca-wifi-fw-src-component-cmn-WLAN.HK.*
 qsdk/dl/
$ cp -rf wlan_proc/pkg/wlan_proc/bin/QCA8074_v1.0/qca-wifi-fw-QCA8074_v1.0-
 WLAN.HK.*.tar.bz2 qsdk/dl/
$ cp -rf wlan_proc/pkg/wlan_proc/bin/QCA8074_v2.0/qca-wifi-fw-QCA8074_v2.0-
 WLAN.HK.*.tar.bz2 qsdk/dl/
$ tar xvf cnss_proc/src/components/qca-wifi-fw-src-component-cmn-WLAN.BL.*.tgz -C
 qsdk/dl
$ tar xvf cnss_proc/src/components/qca-wifi-fw-src-component-halphy_tools-
 WLAN.BL.*.tgz -C qsdk/dl
$ cp -rf cnss_proc/src/components/* qsdk/dl
$ cp -rf cnss_proc/bin/QCA9888/hw.2/* qsdk/dl
$ cp -rf cnss_proc/bin/AR900B/hw.2/* qsdk/dl
$ cp -rf cnss_proc/bin/QCA9984/hw.1/* qsdk/dl
$ cp -rf cnss_proc/bin/IPQ4019/hw.1/* qsdk/dl
$ cp -rf qca-wifi-fw-AR988* qsdk/dl
$ cp -rf apss_proc/out/proprietary/QSDK-Base/meta-tools/ .
$ cp -rf apss_proc/out/proprietary/QSDK-Base/common-tools/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qsdk-qca-nss/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-lib/* qsdk/
$ cp -rf apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-mcs-apps/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-nss-userspace/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-time-services/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-qmi-framework/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/gpio-debug/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-diag/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-cnss-daemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/athtestcmd/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/fw-qca-stats/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/btdaemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/minidump/* qsdk
$ tar xjvf apss_proc/out/proprietary/QSDK-Base/qca-IOT/qca-IOT.tar.bz2 -C qsdk
$ sed -i '/QCAHKSWPL_SILICONZ/c\PKG_VERSION:=WLAN.HK.2.6-02451-QCAHKSWPL_SILICONZ-1'
 qsdk/qca/feeds/qca_hk/net/qca-hk/Makefile
$ cp apss_proc/out/proprietary/QSDK-Base/qca-nss-fw-eip-hk/BIN-EIP*.HK.* qsdk/dl/
$ cp nss_proc/out/proprietary/* qsdk/dl
$ cp -rf apss_proc/out/proprietary/RBIN-AFCAgent/qca-afc-bin/* qsdk
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

32-bit :

```
$ mkdir -p qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/usr/lib/
```

Premium 32-bit	$ cd qsdk/prebuilt/ipq807x/ipq_premium
Enterprise 32-bit	$ cd qsdk/prebuilt/ipq807x/ipq_enterprise
LM512 32-bit	$ cd qsdk/prebuilt/ipq807x/ipq_512
LM512E 32-bit	$ cd qsdk/prebuilt/ipq807x/ipq_512enterprise
LM256 32-bit	$ cd qsdk/prebuilt/ipq807x/ipq_256

```
$ tar zxpvf qti-mfg-provision_1.0-1_ipq.ipk
$ tar zxpvf data.tar.gz
$ cp -rf ./usr/lib/libprovision.so ../../../staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/usr/lib/
$ cd <ChipCode Directory>
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

```
$ mkdir -p qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/pkginfo/
$ touch qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/pkginfo/qti-mfg-provision.provides
$ echo libprovision.so > qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/pkginfo/qti-mfg-provision.provides
```

64 bit:

```
$  mkdir -p qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/usr/lib/
```

Premium 64-bit	$ cd qsdk/prebuilt/ipq807x_64/ipq_premium
Enterprise 64-bit	$ cd qsdk/prebuilt/ipq807x_64/ipq_enterprise

```
$ tar zxpvf qti-mfg-provision_1.0-1_ipq.ipk
$ tar zxpvf data.tar.gz
$ cp -rf ./usr/lib/libprovision.so ../../../staging_dir/target-aarch64_cortex-a53_musl-1.1.16/usr/lib/
$ cd <ChipCode Directory>
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

```
$ mkdir -p qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/pkginfo/
$ touch qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/pkginfo/qti-mfg-provision.provides
$ echo libprovision.so > qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/pkginfo/qti-mfg-provision.provides
```

NOTE: Prebuilt ipk images (such as Qualcomm® Bluetopia™ or qca-mcs-apps, qca-hyd-init, qca-hyd-son, qca-hyd-map) are in their respective folders:

<Chipcode Directory>/qsdk/prebuilt/ipq807x_64/ipq_<profile_name>/	Contains all 64-bit ipk images
<Chipcode Directory>/qsdk/prebuilt/ipq807x/ipq_<profile_name>/	Contains all 32-bit ipk images

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Where ipq_<profile_name> is ipq_premium, ipq_enterprise, ipq_256, ipq_512 and ipq_512enterprise

2. Customers with Qualcomm® HY-FI™, WHC, WAPid, or FWA packages:

Follow enterprise instructions for LM512E.

	These files are fetched from ChipCode and copied to the working QSDK top-level directory:		
Premium/LM	**HY-FI and WHC**	Hyfi qsdk-whc qsdk-whcpy	apss_proc\out\proprietary\Hyfi apss_proc\out\proprietary\Wifi\qsdk-whc apss_proc\out\proprietary\Wifi\qsdk-whcpy
Premium	**WAPid**	qsdk-wapid	apss_proc\out\proprietary\Wapid\ qsdk-wapid
Premium/ Enterprise	**Sigma-DUT**	sigma-dut.tar.bz2	apss_proc\out\proprietary\sigma-dut\sigma-dut.tar.bz2
Premium	**FWA (5G)**	lte-data-prop modem-monitor-prop	apss_proc\out\proprietary\RSRC-LTE-DATA-PROP\lte-data-prop apss_proc\out\proprietary\RSRC-MODEM-MONITOR-PROP\modem-monitor-prop
Premium/LM	**EZMESH-SRC**	qsdk-ezmesh-src	apss_proc\out\proprietary\RSRC-EZMESH\qsdk-ezmesh-src
Premium/LM	**EZMESH-BIN**	qsdk-ezmesh-bin	apss_proc\out\proprietary\RBIN-EZMESH\qsdk-ezmesh-bin
Premium/LM	**EZMESH-ALG**	qsdk-ezmesh-alg-bin	apss_proc\out\proprietary\RBIN-EZMESH-ALG\qsdk-ezmesh-alg-bin
Premium/LM	**EZ-ALG-SRC**	qsdk-ez-alg-src	apss_proc\out\proprietary\RSRC-EZMESH-ALG\qsdk-ezmesh-alg-src

Then run the additional code:

Premium/LM	**HY-FI and WHC**	`$ cp -rf apss_proc/out/proprietary/Hyfi/hyfi/* qsdk`
		`$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whc/* qsdk`
		`$ mkdir qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug`
		`$ mv qsdk/qca/feeds/qca-son-mem-debug/Makefile qsdk/qca/feeds/qca-son-mem-debug/Config.in qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug`
		`$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whcpy/* qsdk`
		`$ sed -i "s/@PACKAGE_whc-son/@PACKAGE_whc-map/g" qsdk/qca/feeds/qca-lib/qca-wifison-ext-lib/Makefile`

Premium	**WAPid**	`git clone <wapid ChipCode Directory>`
		`$ cd <wapid ChipCode Directory>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ cp -rf <wapid ChipCode Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/Wapid/qsdk-wapid/* qsdk`
		Where wapid ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_wapid-src for this release.

Premium/ Enterprise	**Sigma-DUT:**	`$ git clone <sigma-dut ChipCode Directory>`
		`$ cd <sigma-dut ChipCode directory>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ tar xjvf <sigma-dut ChipCode Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/sigma-dut/sigma-dut.tar.bz2 -C qsdk`
		Where sigma-dut ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_sigma-dut for this release.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Premium	**FWA (5G):**	git clone <*FWA ChipCode Directory*>
		$ cd <*FWA ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -r <*FWA ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-LTE-DATA-PROP/lte-data-prop/* qsdk
		$ cp -r <FWA ChipCode directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-MODEM-MONITOR-PROP/modem-monitor-prop/* qsdk
		Where FWA ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_fwa for this release.

Premium/LM	**EZMESH-SRC**	#Only for Ezmesh customer
		
		$ git clone <*ezmesh ChipCode Directory*>
		$ cd <*ezmesh ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH/qsdk-ezmesh-src/* qsdk
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-src for this release
		
		#Below sed commands in EZMESH-SRC should be skipped if customer is using **EZ-ALG-SRC** also (i.e. both ezmesh-src and ezmesh-alg-src)
		
		$ sed -i 's/HYD_MODULE_STRATEGY=y/HYD_MODULE_STRATEGY=n/g' qsdk/qca/src/qca-ezmesh/ezmeshConfig.defs
		$ sed -i '0,/ifeq/{/ifeq/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
		$ sed -i '0,/endif/{/endif/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
		$ sed -i '/libezmeshalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
		$ sed -i '/libezmeshagentalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
		$ sed -i '/DUMP/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile

Premium/LM	**EZMESH-BIN**	#Only for Ezmesh customer
		
		$ git clone <*ezmesh ChipCode Directory*>
		$ cd < *ezmesh ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH/qsdk-ezmesh-bin/* qsdk
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-bin for this release

Premium/LM	**EZMESH-ALG**	#Only for Ezmesh customer
		
		$ git clone <*ezmesh ChipCode Directory*>
		$ cd < *ezmesh ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH-ALG/qsdk-ezmesh-alg-bin/* qsdk

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-alg for this release

Premium/LM	**EZ-ALG-SRC**	`#Only for Ezmesh customer`
		`$ git clone <ezmesh ChipCode Directory>`
		`$ cd < ezmesh ChipCode Directory>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ cp -rf <ezmesh ChipCode Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH-ALG/qsdk-ezmesh-alg-src/* qsdk`
		Where ezmesh ChipCode Direectory is qca-networking-2021-spf-11-5_qca_oem_ez-alg-src for this release

2.5.3.3 Setup the QSDK build environment (one time)

QSDK supports the **premium, enterprise, LM512, LM512E, and LM256** profiles for Linux kernel 4.4.60**.** The QSDK framework is developed using Ubuntu (from version 16.04 to version 18.04) and Debian. However, QSDK framework regenerates critical tools required to compile firmware at build time. Thus, the framework is independent from the host environment. Although it is developed using the listed distributions, it is expected to work on others such as Red Hat, Mint, or Fedora.

The required tools can be installed manually (see Section 2.4.3.2.1 for using a script that is available in CAF (see Section 2.4.3.2.2)

2.4.3.2.1 Set up the build environment manually

This command is for Ubuntu 16.04 (for older/32-bit Debian/Ubuntu releases; customize it for other distributions):

```
$ sudo apt-get install gcc g++ binutils patch bzip2 flex make \
gettext pkg-config unzip zlib1g-dev libc6-dev subversion libncurses5-dev
   gawk sharutils \
curl libxml-parser-perl python-yaml ocaml-nox ocaml ocaml-findlib libssl-
   dev libfdt-dev
$ sudo apt-get install device-tree-compiler u-boot-tools
```

For Ubuntu 18.04 build hosts additionally, install:

```
$ sudo apt-get install libssl1.0-dev
```

2.4.3.2.2 Set up the build environment using script

Follow these steps to get the required tools installed in Ubuntu host for QSDK compilation. Verified on Ubuntu 16 and Ubuntu 18.

1. Assuming the repo init and sync to the release AU from CAF (CodeAurora Forum) is done successfully, the setup_qsdk.sh script should be available in qsdk/scripts/setup_qsdk.sh
2. The script assumes that repo and git are already installed and aborts the execution otherwise.
3. If the repo init is not done, follow the steps:

   ```
   $ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
   release -m
   caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
   ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ repo sync -j8 --no-tags -qc
```

4. The user running the script should have admin privileges. If not, the script will abort upon unsuccessful verification of sudo/admin access.
5. Run the script:

```
$sudo bash -x setup_qsdk.sh
```

2.5.3.4 Create the QSDK build

Because the framework automatically downloads the open source components needed during the build/make processing, ensure that an Internet connection is active on the build host when creating the build

1. Install the different feeds in the build framework:

```
$ cd qsdk
$ ./scripts/feeds update -a
$ ./scripts/feeds install -a -f
```

2. Copy the base configuration to use for the build. Choose **a** profile and use it to build with Linux 4.4.60 support.
3. Regenerate a complete configuration file and start the build:

Premium 32-bit	`$ cp qca/configs/qsdk/ipq_premium.config .config`
	`$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq807x/g" .config`
	`$ mv prebuilt/ipq807x/ipq_premium/* prebuilt/ipq807x/`

Premium 64-bit	`$ cp qca/configs/qsdk/ipq_premium.config .config`
	`$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq807x_64/g" .config`
	`$ mv prebuilt/ipq807x_64/ipq_premium/* prebuilt/ipq807x_64/`

Enterprise 32-bit	`$ cp qca/configs/qsdk/ipq_enterprise.config .config`
	`$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq807x/g" .config`
	`$ mv prebuilt/ipq807x/ipq_enterprise/* prebuilt/ipq807x/`

Enterprise 64-bit	`$ cp qca/configs/qsdk/ipq_enterprise.config .config`
	`$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq807x_64/g" .config`
	`$ mv prebuilt/ipq807x_64/ipq_enterprise/* prebuilt/ipq807x_64/`

LM512 32-bit	`$ cp qca/configs/qsdk/ipq_512.config .config` `$ mv prebuilt/ipq807x/ipq_512/* prebuilt/ipq807x/`
LM512E 32-bit	`$ cp qca/configs/qsdk/ipq_512enterprise.config .config` `$ mv prebuilt/ipq807x/ipq_512enterprise/* prebuilt/ipq807x/`
LM256 32-bit	`$ cp qca/configs/qsdk/ipq_256.config .config` `$ mv prebuilt/ipq807x/ipq_256/* prebuilt/ipq807x/`

Hy-Fi, WHC customers only:

- Starting with QCA_Networking_2020.SPF.11.3 release, SON and EasyMesh Features are compiled as separate build. SON build supports only SON features and EZMESH build supports only EasyMesh features.
- Starting with QCA_Networking_2021.SPF.11.4 release, MAP package qca-hyd-map is replaced with qca-ezmesh. Enable additional configuration to enable ezmesh packages.
- Use the SON package commands in this table for a SON features enabled build. Use the EZMESH package commands for an EasyMesh features enabled build.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Enable SON or EZMESH configuration:

SON package	$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_whc-son=y" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-son=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config

EZMESH package (Full Mode)	$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
	Additional configuration to enable ezmesh packages:
	$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config

EZMESH package (Co-located mode)	$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
	Additional configuration to enable ezmesh packages:
	$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config

- Use the commands in this table to enable both SON and EasyMesh Features.

Enable SON and EZMESH (full mode) configuration:

SON + EZMESH package (full mode)	$ echo "CONFIG_PACKAGE_whc-mesh=y" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-mesh=y" >> .config
	Additional configuration to enable ezmesh packages:
	$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config

NOTE: Enabling both SON and EasyMesh Features in a build will increase image size and consume extra memory in Flash storage.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ make defconfig
$ make V=s -j5
```

4. For 64-bit firmware, once the premium/enterprise 32-bit compilation completes, save the file: **openwrt-ipq807x-u-boot*.elf, openwrt-ipq807x-lkboot*.elf, and openwrt-ipq807x_tiny-u-boot*.elf f**rom the directory: **qsdk/bin/ipq**

   Use this file to generate the complete 64-bit firmware image.

5. Download the required packages for the corresponding profile and create the image. Once the build is complete, these files are in the **qsdk/bin/ipq** directory:

32-bit	▪ openwrt-ipq807x-u-boot.elf (ELF) ▪ openwrt-ipq-ipq807x-squashfs-root.img (SquashFS) ▪ openwrt-ipq-ipq807x-qcom-ipq807x-hkxx-fit-uImage.itb (ITB) ▪ openwrt-ipq-ipq807x-ubi-root.img (IMG)
64-bit	▪ openwrt-ipq-ipq807x_64-squashfs-root.img (SquashFS) ▪ openwrt-ipq-ipq807x_64-qcom-ipq807x-hkxx-fit-uImage.itb (ITB) ▪ openwrt-ipq-ipq807x_64-ubi-root.img (IMG)

2.5.4 Generate a complete firmware image

IPQ807x requires multiple images to be flashed for Bootup, including SBL1, SBL2, SBL3, RPM, TZ, CDT, MIBIB, NSS Images, Kernel, Filesystem, and so on. To simplify device loading using the usage from flash memory, the images are combined into a single Flattened Image Tree (FIT) image. The FIT can be flashed into the respective partition based on user configuration. More tools required on the Ubuntu 16.04 64-bit machine include:

1. Install DTC:
   ```
   $sudo apt-get install device-tree-compiler
   ```
2. Install Python 2.7; Install tools required to run packing single image:
   $sudo apt-get install libc6-i386
   $sudo apt-get install libgl1-mesa-dri:i386
3. Switch to the Qualcomm ChipCode directory:
   $ cd <Chipcode Directory>
4. Copy the flash config files to **common/build/ipq**
5. Find the single images under **common/build/bin** directory.

32-bit

```
$ cd <Chipcode Directory>
$ mkdir -p common/build/ipq
$ mkdir -p apss_proc/out/meta-scripts
$ cp qsdk/qca/src/u-boot-2016/tools/pack.py apss_proc/out/meta-scripts/pack_hk.py
$ sed -i 's#</linux_root_path>#/</linux_root_path>#' contents.xml
$ sed -i 's#</windows_root_path>#\\</windows_root_path>#' contents.xml
$ sed -i 's/WLAN.BL.3.16//g' contents.xml
$ sed -i 's/CNSS.PS.3.16//g' contents.xml
$ cp qsdk/bin/ipq/openwrt* common/build/ipq
$ cp -r apss_proc/out/proprietary/QSDK-Base/meta-tools apss_proc/out/
$ cp -rf qsdk/bin/ipq/dtbs/* common/build/ipq/
$ cp -rf skales/* common/build/ipq/
$ cp qsdk/staging_dir/host/bin/ubinize common/build/ipq/
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ cd common/build
$ sed
 '/debug/d;/packages/d;/"ipq807x_64"/d;/t32/d;/ret_prep_64image/d;/skales/d;/os.s
 ystem(cmd)/d;/os.chdir(ipq_dir)/d;/ret_pack_64image/d;/ret_pack_apps_64image/d;/
 64image/d' -i update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
(Where <profile-name> is E, P, LM256 or LM512. Use profile-name LM512 for both
LM512 and LM512E)
```

64-bit

```
$ cd <Chipcode Directory>
$ mkdir -p common/build/ipq_x64
$ mkdir -p apss_proc/out/meta-scripts
$ cp qsdk/qca/src/u-boot-2016/tools/pack.py apss_proc/out/meta-scripts/pack_hk.py
$ sed -i 's#</linux_root_path>#/</linux_root_path>#' contents.xml
$ sed -i 's#</windows_root_path>#\\</windows_root_path>#' contents.xml
$ sed -i 's/WLAN.BL.3.16//g' contents.xml
$ sed -i 's/CNSS.PS.3.16//g' contents.xml
$ cp qsdk/bin/ipq/openwrt* common/build/ipq_x64
```

Copy the saved openwrt-ipq807x-u-boot*.elf, openwrt-ipq807x-lkboot*.elf, and openwrt-ipq807x_tiny-u-boot*.elf from the 32-bit build to: common/build/ipq_x64

```
$ cp -r apss_proc/out/proprietary/QSDK-Base/meta-tools apss_proc/out/
$ cp -rf qsdk/bin/ipq/dtbs/* common/build/ipq_x64/
$ cp -rf skales/* common/build/ipq_x64/
$ cp qsdk/staging_dir/host/bin/ubinize common/build/ipq_x64/
$ cd common/build
$ sed -i 's/\.\/ipq/\.\/ipq_x64/g' update_common_info.py
$ sed -i 's/\.\/ipq_x64_x64/\.\/ipq_x64/g' update_common_info.py
$ sed
 '/debug/d;/packages/d;/"ipq807x"/d;/t32/d;/ret_prep_32image/d;/Required/d;/lk/d;
 /os.system(cmd)/d;/skales/d;/os.chdir(ipq_dir)/d;/ret_pack_32image/d;/ret_pack_a
 pps_32image/d;/32image/d' -i update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
```

(Where <*profile-name*> is E or P).

2.5.5 Flash the complete default software image

2.5.5.1 Set up the flash environment

1. Ensure that the board console port is connected to the PC using these RS232 parameters:
   - 115200 bps
   - 8N1
2. Confirm that the PC is connected to the board using one of the Ethernet ports. The PC must have a TFTP server launched and listening on the interface to which the board is connected. The interface must have an IP address configured manually. At this stage power up the board and after a few seconds, press any key during the countdown.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.5.5.2 Standard board configuration

1. Copy the **xxxx-ipq807x-single.img** to the TFTP server root directory.
2. Check hardware jumper configuration according to reference board and default memory configuration (see the appropriate board setup guide for more information) to verify which flash memory the board is booting from.
3. Confirm the machine ID, Meta version, profile and single image.
4. Set the IP address and server IP using the TFTP process:

   ```
   set ipaddr 192.168.1.11
   set serverip 192.168.1.xx (TFTP server address)
   ping ${serverip}
   ```

   (set bootargs console=ttyMSM0,115200n8)

   tftpboot 0x44000000 xxxx-ipq807x-single.img
5. If using a NOR flash, execute this command:

   ```
   sf probe
   ```
6. Flash the image:

   ```
   imgaddr=0x44000000 && source $imgaddr:script
   ```
7. Power cycle the board after step 5 has completed, as indicated by printing of the U-boot prompt (may be hundreds of seconds depending on image size and memory technology).
8. For an Arithmetic exception issue in the fw_printenv/fw_setenv commands in eMMC boot, run these commands:

   ```
   cat /etc/fw_env.config |  awk '{$NF="";sub(/[ \t]+$/,"")}1'
    >/tmp/fw_env.config
   mv /tmp/fw_env.config /etc/fw_env.config
   ```
9. Once the single image has booted, transfer this package to the AP /tmp directory from ./**qsdk/prebuilt/ipq807x/** or ./**qsdk/prebuilt/ipq807x_64/**, depending on whether it is a 32-bit or 64-bit image.

   ```
   bluetopia_4.2-1_ipq807x.ipk
   ```
10. Install these packages from the /tmp directory:

    ```
    opkg install bluetopia_4.2-1_ipq807x.ipk
    ```

NOTE: For Qualcomm® Wi-Fi SON, perform steps 11, 12, and 13.

11. Once the single image has booted, transfer the SON packages for SON features or EZMESH packages for EasyMesh features to the AP /tmp directory from: **./qsdk/prebuilt/ipq80xx/** or **./qsdk/prebuilt/ ipq80xx _64/**, depending on whether the booted image is 32-bit or 64-bit.

SON package (32-bit)	qsdk/prebuilt/ipq807x/qca-hyd-init_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x/qca-hyd-son_ge474d2e-1_ipq.ipk
SON package (64-bit)	qsdk/prebuilt/ipq807x_64/qca-hyd-init_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x_64/qca-hyd-son_ge474d2e-1_ipq.ipk
EZMESH package (32-bit)	**To support Full mode:** qsdk/prebuilt/ipq807x/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x/qca-ezmesh_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x/qca-ezmesh-alg_ge474d2e-1_ipq.ipk

EZMESH package (64-bit)	**To support Co-located mode:** qsdk/prebuilt/ipq807x/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x/qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x/qca-ezmesh-agent_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x/qca-ezmesh-alg_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x/qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk
	To support Full mode: qsdk/prebuilt/ipq807x_64/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x_64/qca-ezmesh_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x_64/qca-ezmesh-alg_ge474d2e-1_ipq.ipk **To support Co-located mode:** qsdk/prebuilt/ipq807x_64/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x_64/qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x_64/qca-ezmesh-agent_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x_64/qca-ezmesh-alg_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq807x_64/qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk

NOTE: ge474d2e is git commit ID, which may also be changed.

12. Install these packages from the /tmp directory: Choose the SON package to enable SON features, or choose EZMESH packages to enable MAP features:

SON package	opkg install qca-hyd-init_ge474d2e-1_ipq.ipk opkg install qca-hyd-son_ge474d2e-1_ipq.ipk
EZMESH package	**To support Full mode:** opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk opkg install qca-ezmesh_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk **To support Co-located mode:** opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-agent_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk

NOTE: ge474d2e is the git commit ID, which may also be changed.

13. The HYD daemon application binary name is changed from hyd to hyd-son for the SON package. Similarly, the WSPLCD daemon application binary name is changed from wsplcd to wsplcd-son and wsplcd-map for the SON and EZMESH packages, respectively. The symbolic link to the older binary name must be created after package installation to ensure that scripts and tools work with the new binary name. Because application binary name has changed, a crash dump is created based on new binary name. Customers must take care of updating any scripts and tools that use or depend on the older binary name.

    Execute these commands to create a symbolic softlink to the older binary name:

SON package	ln -s /usr/sbin/wsplcd-son /usr/sbin/wsplcd ln -s /usr/sbin/hyd-son /usr/sbin/hyd
EZMESH package	ln -s /usr/sbin/wsplcd-map /usr/sbin/wsplcd

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.5.5.3 Upgrade the firmware

This release can upgrade board images without the need for a TFTP server. After using U-Boot to flash an initial image and booting the device, use the web interface for future upgrades. After using U-Boot to flash an initial image and boot the device, future upgrades can be done from either the OpenWRT web interface or serial console. Upgrade the existing flash image using either the appropriate single image file or the apps image file generated by **update_common_info.py**.

Upgrading image files via the web interface takes several minutes to complete depending on factors such as memory technology, vendor, image size, browser connectivity, and network load. Completion of the flash upgrade process is signaled by the refresh of the OpenWRT login page, or lack of further messages on the serial console.

- An invalid image remains in the flash memory if the upgrade process is interrupted, such as if system power is lost during upgrade or a key-press event is detected on the serial console port.
- If a board is configured for failsafe boot, and the U-boot environment variable *bootargs* contains any reference to the rootfs or other partitions managed by the failsafe scheme. In this case, the sysupgrade process may only be partially successful: the board may or may not boot subsequently, and the image may only be partly updated.
- Using the **sysupgrade** command from the Linux console requires the sysupgrade image to be visible:
  - Within the Linux filesystem supplied using development host SSH or TFTPD server, OR
  - On an NFS server that is mounted to the local file system, OR
  - Transferred using a USB storage device
- The sysupgrade feature can only be used to update the image in the memory the board boots from and is used by the running kernel. If the board supports multiple memory technologies, use U-Boot to change the image in memories that the board did not boot from.

See the *IPQ807x/IPQ807xA/IPQ817x SoC Software User Guide* (80-YA728-4) for more information on Sysupgrade support.

See the *QCN90xx Firmware Build Application Note* (80-Y9005-29) for more information.

2.5.5.4 eMMC boot GPT backup partition handling

The location of the GPT backup partition in the final flash image is defined in the file:
./NHSS.QSDK.11.5/apss_proc/out/proprietary/QSDK-Base/meta-tools/ipq807x/config.xml

In the lines:

```
<data type="EMMC_PARAMETER">
<total_block>7634944</total_block>
<partition_mbn>gpt_main0.bin</partition_mbn>
<partition_mbn_backup>gpt_backup0.bin</partition_mbn_backup>
```

This file is copied to the **./common/build/ipq** or **./common/build/ipq_x64** directory for 32-bit and 64-bit image builds, respectively. It assumes an eMMC device size of 4 GB; therefore, the GPT backup partition will be incorrectly located for boards that are provisioned with different size eMMC device; for example, HK01.

The configuration file can be corrected by an additional sed command after the ChipCode repository has been checked out: for example, for the 8 GB device provisioned on the HK01 board, use:

```
$sed
's#<total_block>7634944</total_block>#<total_block>15269888</total_block>#'
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
-i ./NHSS.QSDK.11.5/apss_proc/out/proprietary/QSDK-Base/meta-
tools/ipq807x/config.xml
```

Sysupgrade behavior is not guaranteed when the GPT backup partition is not correctly located at the end of the eMMC image.

2.5.5.5 Bring up 802.11ay interface on IPQ8074A platforms

1. After flashing the image, copy the ipk files on the AP to the /root dir and install the IPKs in this sequence using the correct ipk names:
   opkg install wigig-firmware_WIGIG.TLN.*-*-WIGIGTLNZ-1_ipq.ipk
   opkg install wigig-mac80211_1_ipq.ipk
   opkg install kmod-qca-wigig-driver_*.ipk
   opkg install qca-wigig-debug-tools_*.ipk
   opkg install qca-fst-manager_*.ipk
2. Reboot the device.
3. Enable 802.11ay using the UCI commands:
   ```
   $uci set wireless.radio0.disabled=0
   $uci set wireless.radio0.edmg_channel=10
   $uci set wireless.@wifi-iface[0].ssid="xxxxxx"
   $uci commit wireless
   ```
4. Reboot the device.
5. Split WiGig tasks among CPU cores ():
   ```
   $uci set wireless.mac80211=mac80211
   $uci set wireless.mac80211.enable_smp_affinity=1
   $uci commit wireless
   /etc/init.d/network restart
   ```
6. Verify the changes.
   a. Run this shell command to find all wigig N_MSI:
      ```
      cat /proc/interrupts |grep wil6210
      ```
      For example:
      ```
      118: 0 0 0 0 GIC 448 Edge wil6210_tx
      119: 0 0 0 0 GIC 449 Edge wil6210_rx
      ```
   b. Validate the changes using the command:
      ```
      root@OpenWrt:~# cat /proc/irq/xxx/smp_affinity
      8 (core 3)
      root@OpenWrt:~# cat /proc/irq/yyy/smp_affinity
      4 (core 2)
      ```
7. Check CPU use:
   ```
   mpstat -P ALL 1
   ```

2.5.5.5.1 Enable FST on IPQ8074A platforms

1. Enable FST using the UCI commands:
   ```
   $uci set wireless.wifi0.disabled=0
   ```
   For this instruction, assume by default ath0/wifi0 is used; change accordingly if other Wi-Fi interfaces used.
   ```
   $uci set wireless.@wifi-iface[1].ssid="xxxxxx" – same as 802.11ay SSID
   $uci commit wireless
   ```

```
/etc/init.d/network restart
```

2. Get MAC address of the Wi-Fi interface:
```
cat /sys/class/net/ath0/address
```
3. Enable FST manager using UCI commands:
```
# uci set fst.config.interface1=ath0 <by default, configuration uses
 ath0; change accordingly if other wifi interfaces used>
uci set fst.config.interface2_mac=<MAC address from point 11>
uci set fst.config.disabled=0
uci commit fst
```
4. Reboot the device.
5. Check if the FST manager is enabled (the assumption is ath0 is used for FST configuration).
   a. Check the command **brctl show** lists bond1 as part of the bridge interface. Both ath0 and wlan0 are no longer part of br-lan.
   b. Check if both ath0 and wlan0 are listed as slave interfaces in **/proc/net/bonding/bond1**.
   c. Check if **ps | grep fst** shows "/usr/sbin/fstman /var/run/hostapd/global -s ath0".

2.5.6 Create customized IPQ807x Wi-Fi firmware images

Flash memory layout includes a dedicated partition for the Wi-Fi firmware and BDF information. The release includes squashfs and ubifs images in the final image created by the last steps of section 2.5.5. Use these steps to update or recreate the squashfs/ubifs Wi-Fi firmware image and the complete final image.

NOTE: These steps are not needed if the BDF and PIL files are not customized.

1. Install these tools in the workstation:
   - mtd-utils: sudo apt-get install mtd-utils
   - mksquashfs4 binary from:
     < *ChipCode Directory*>IPQ8074.ILQ.11.5/common/build to /usr/bin or /usr/sbin or any location included in $PATH
2. Download the WLAN IPQ807x firmware BIN tar ball from the corresponding SP release distro.
3. Download the WLAN SquashFS image wifi_fw_ipq8074_qcn9000_squashfs_v2.img from WLAN.HK.2.3/wlan_proc/build/ms/bin/8074.wlanfw.eval_v2/FW_IMAGES/
4. Extract the firmware binary tarball:
```
mkdir -p <Chipcode Directory>/fwtemp
cp -rfv wlan_proc/pkg/wlan_proc/bin/QCA8074_v2.0/qca-wififw-
 QCA8074_v2.0-WLAN.HK.*-*  fwtemp
cd fwtemp
tar -xvf qca-wifi-fw-QCA8074_v2.0-WLAN.HK.*-*

mkdir -p fwtemp
cp <download path>/wifi_fw_ipq8074_qcn9000_squashfs_v2.img ./fwtemp
cd fwtemp
unsquashfs wifi_fw_ipq8074_qcn9000_squashfs_v2.img
```
5. Create a staging directory to hold the BD and PIL files.
```
mkdir staging_dir
cp -rfv qca-wifi-fw-WLAN.HK.*/PIL_IMAGES/* staging_dir
```

```
cp -rfv qca-wifi-fw-WLAN.HK.*/bdwlan* staging_dir
```

(Replace the BDF file with the custom versions as needed.)

6. To generate the firmware squash image for **NOR/eMMC** (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):

```
mksquashfs4 staging_dir/ wifi_fw.squashfs -nopad -noappend -root-owned -
 comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -
 processors 1
dd if=wifi_fw.squashfs of=wifi_fw_squashfs.img bs=2k conv=sync
```

7. To generate the firmware UBI image for **NAND** (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):

```
mksquashfs4 staging_dir/ wifi_fw.squashfs -nopad -noappend -root-owned -
 comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -
 processors 1
```

NOTE: If the squashfs image is already generated as in the case of NOR/eMMC, ignore these steps and directly ubinize the image:

8. Create the ubinize image with the ubinize utility, which is part of mtd-util.
9. Create a **ubi-wifi.cfg** file with these contents:

```
[wifi_fw]
# Volume mode (other option is static)
mode=ubi
# Source image
image=wifi_fw.squashfs
# Volume ID in UBI image
vol_id=1
# Allow for dynamic resize
vol_type=dynamic
# Volume name
vol_name=wifi_fw
# vol_flags=autoresize
```

Then run these commands:

```
ubinize -m 2048 -p 128KiB -o wifi_fw.ubi ubi-wifi.cfg
dd if=wifi_fw.ubi of=wifi_fw_ubi.img bs=2k conv=sync
```

10. To create the final single image with the updated or recreated firmware image:

```
cp <Chipcode Directory>/fwtemp/Wi-Fi_fw_squashfs.img <Chipcode
Directory>/wlan_proc/build/ms/bin/FW_IMAGES/
cp <Chipcode Directory>/fwtemp/Wi-Fi_fw_ubi.img <Chipcode
Directory>/wlan_proc/build/ms/bin/FW_IMAGES/
```

11. Run this command again:

```
export BLD_ENV_BUILD_ID=<profile-name>
python update_common_info.py
```

Where <*profile-name*> is E, P.

For instructions on how to flash the firmware images separately, see the *IPQ807x/IPQ807xA/IPQ817x SoC Software User Guide* (80-YA728-4).

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.5.7 Create customized IPQ807x + QCN90xx Wi-Fi firmware images

Flash memory layout includes a dedicated partition for the Wi-Fi firmware and BDF information. The release includes squashfs and ubifs images in the final image created by the last steps of Section 2.5.5. Use these steps to update or recreate the squashfs/ubifs Wi-Fi firmware image and the complete final image.

NOTE: These steps are not needed if the BDF and PIL files are not customized.

1. Install these tools in the workstation:
   - mtd-utils: sudo apt-get install mtd-utils
   - mksquashfs4 binary from:
     <C*hipCode Directory*>IPQ8074.ILQ.11.1/common/build to /usr/bin or /usr/sbin
     or any location included in $PATH
2. Download the WLAN SquashFS image wifi_fw_ipq8074_qcn9000_squashfs_v2.img from WLAN.HK.2.3/wlan_proc/build/ms/bin/8074.wlanfw.eval_v2/FW_IMAGES/
3. Extract the squashfs image in a temp directory
   ```
   mkdir -p fwtemp
   cp <download path>/wifi_fw_ipq8074_qcn9000_squashfs_v2.img ./fwtemp
   cd fwtemp
   unsquashfs wifi_fw_ipq8074_qcn9000_squashfs_v2.img
   ```
   All extracted files will be in ./squashfs-root folder.
4. Copy/modify any files in *./squashfs-root* folder.
5. To generate the firmware SquashFS image for **NOR/eMMC** (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):
   ```
   mksquashfs4 squashfs-root/ wifi_fw.squashfs -nopad -noappend -root-owned
      -comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -
      processors 1
   dd if=wifi_fw.squashfs of=wifi_fw_squashfs.img bs=2k conv=sync
   ```
6. To generate the firmware UBI image for **NAND** (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):
   ```
   mksquashfs4 squashfs-root/ wifi_fw.squashfs -nopad -noappend -root-owned
      -comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -
      processors 1
   ```

NOTE: If the squashfs image is already generated as in the case of NOR/eMMC, ignore these steps and directly ubinize the image:

7. Create the ubinize image with the ubinize utility, which is part of mtd-util.
8. Create a **ubi-wifi.cfg** file with these contents:
   ```
   [wifi_fw]
   # Volume mode (other option is static)
   mode=ubi
   # Source image
   image=wifi_fw.squashfs
   # Volume ID in UBI image
   vol_id=1
   # Allow for dynamic resize
   vol_type=dynamic
   # Volume name
   vol_name=wifi_fw
   ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
# vol_flags=autoresize
```

Then run these commands:

```
ubinize -m 2048 -p 128KiB -o wifi_fw.ubi ubi-wifi.cfg
dd if=wifi_fw.ubi of=wifi_fw_ubi.img bs=2k conv=sync
```

9. To create the final single image with the updated or recreated firmware image:

```
cp <Chipcode Directory>/fwtemp/Wi-Fi_fw_squashfs.img <Chipcode
   Directory>/wlan_proc/build/ms/bin/FW_IMAGES/
cp <Chipcode Directory>/fwtemp/Wi-Fi_fw_ubi.img <Chipcode
   Directory>/wlan_proc/build/ms/bin/FW_IMAGES/
```

10. Run this command again:

```
export BLD_ENV_BUILD_ID=<profile-name>
python update_common_info.py
```

Where <*profile-name*> is E, P.

For instructions on how to flash the firmware images separately, see the *IPQ807x/IPQ807xA/IPQ817x SoC Software User Guide* (80-YA728-4).

2.6 U-Boot device tree optimization

In order to save flash space and also remove redundant information. Some of the device tree in U-Boot has been removed/reused.

- HK01.C3 now uses HK01 device tree
- AC02 now uses AC01 device tree
- OAK03 now uses HK01 device tree

If this re-use feature is not needed, revert the following commits:

```
f2b16c5a1e825f59cadd9c0982adf5b71b7d6c07
c8365407da56a1fc67ddb389da12649bd3c2c99b
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3 IPQ6018.ILQ.11.5 CSU1

3.1 Supported features

- CPU/platform
  - Quad ARM Cortex A53 at 1.8 GHz, 64-bit ISA v8 instruction set
  - 32 KB/32 KB I$/D$ and 512 KB L2$
  - Floating point and NEON SIMD DSP for each core
  - DDR4/DDR3L, USB3, PCIe, all serial interfaces
- Wireless connectivity
  - For IPQ601x: 5 GHz antenna configuration
    - 2 x 2/2S-80 MHz
  - 2.4 GHz antenna configuration
    - 2 x 2/2S-40 MHz
- Networking
  - Single-core multithreaded network accelerator (NPU)
  - Advanced classification, policing, queuing, forwarding at wire speed (PPE)
  - Hardware crypto offload engine
  - USXGMII/XFI, SGMII, SGMII+, PSGMII
- Standard programming model
  - Linux 4.4 (32-bit and 64-bit)
  - QSDK based on OpenWRT CC

QCN90xx chipset features

- 4x4/160 MHz 11ax PCIe Radio
- 2.4 GHz, 5 GHz, 6 GHz full band support
- Dual Lane PCIe Gen 3
- Package: 11.1 x 12 FCBGA, 0.65 mm ball pitch
- WLAN
  - Dual-synthesizer WLAN radio up to 160 MHz band width, support aSA, ADFS
  - Supports 20/40 MHz in 2.4 GHz
  - Supports 20/40/80/160 MHz in 5 GHz
  - Supports 20/40/80/160 MHz in 6 GHz

- Supports up to 1024 QAM (4 Spatial stream (SS)) and 4096 QAM (2 SS)

- Supported standards
  - IEEE802.11a/b/g/n/ac/ax
  - IEEE 802.11d/e/h/i/j/k/r/u/v/w

3.2 Frequency spectrum and regulatory domain support

QCA_Networking_2021.SPF.11.5 release implements the regulatory versions, 34 through 36.For the AFC FR testing support FCC SP rules alone are added from Regulatory version 37. For details about the regulatory domains and mapping country codes, see *Regulatory Configuration for QCA WLAN (80-Y6630-2).*

NOTE: Please contact customer engineering team to refer the 80-Y6630-2 document revision corresponding to the Regulatory data base versions supported in this release.

Contact Qualcomm Customer Engineering for guidance to enable 6 GHz operation for desired countries for evaluation or customer release. See the *WLAN Conformance Testing and Compliant Power Configuration for 802.11ax Chipsets Information to Share with Test Lab* application note (80-YB952-3) and the *Wireless LAN Driver Version 11.0 for Access Point Programming Guide* (80-YA728-6) for full description of 6 GHz support on WLAN AP chipsets.

The Regulatory database updates which has been maintained in FW (2.4GHz & 5GHz) is deprecated from 11.4 release. There is no FW regulatory database maintained for 6GHz and it is maintained only in BDF.

The Regulatory database updates are done only in BDF for all 11ax chipsets and the FW database is not updated from 11.1 release onwards.

The Regulatory database will be picked up from BDF by default.

New 6 GHz rules and regulatory domain support will change frequently during 2021-2022. This status is accurate as of the date of publication of these *Release Notes*:

- Supported in this release:
  - U.S.: LPI AP
  - US: SP_AP(only for testing purpose)
  - UK: LPI and VLP
  - European Community (Draft Requirements): LPI and VLP
  - Chile and UAE: LPI
  - Guatemala: LPI and VLP
  - Korea and Brazil : LPI and VLP

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.3 Restrictions on software while using it for testing

DL OFDMA Test Tones for FCC Conformance Testing	*WLAN Conformance Testing and Compliant Power Configuration for 802.11ax Chipsets Information to Share with Test Lab* (80-YB952-3) have been updated with FCC conformance test guidance for DL OFDM transmissions. Test tone XML files to support the FCC testing using QSPR and QRCT are available for the WLAN AP software release at these paths: ▪ In the firmware package, the file is available at "/bdfUtil". ▪ In the WLAN source, the file is available at wlan_proc\wlan\phyrf_svc\tools\bdfExcel\qca8074\CTL_installer_python_scripts\FCC_interim_Guidance_Test_Tones.zip
NSS	▪ Only IPv4/IPv6 + TCP/UDP flows are accelerated through fast path ▪ LAG: Linux bonding driver configuration supported in fast path through the bonding driver sysfs IF: ▫ Mode: 802.3ad (4): xmit_hash_policy: layer2 (0), layer2+3 (2) ▫ Mode: Balance-xor (2): xmit_hash_policy: layer2 (0), layer3+4 (1), layer2+3 (2) ▪ Flows that require ALG support are not accelerated by NSS (except for TFTP (port 69)) ▪ Interfaces configured with NSS qdiscs must have a leaf node configured as the default node for enqueue (using the set_default qdisc parameter); if qdisc structures are created without a default, no packets are transmitted; this includes management packets such as ARP ▪ Tunnel support (6RD, DS-Lite) expects at least the initial few packets in the LAN > WAN direction to push the base fast path rule; further flows for these tunnels can be initiated on either the WAN or LAN network ▪ PPPoE-relay packets in NSS is not supported ▪ IGMPv3 and MLDv2 traffic over PPPoE WAN interface is not accelerated ▪ DHCP relay over PPPoE is not supported ▪ SAMBA is not turned on by default in this release and must be enabled manually ▪ Qualcomm provides a software implementation for IGMP/MLD snooping ▪ Inline IPsec flows deviate from current NSS behavior for upstream flows; during NSS IPsec acceleration, fragmentation (when required) is performed pre-encapsulation while inline IPsec (hardware) acceleration fragmentation is done post encapsulation ▪ EDMA can fail to queue the packets to eth_rx; the eth_rx drop counter increments even though EDMA holds on to these packets and tries queueing later ▪ NSS connection manager expects to see at least one packet in outgoing direction to push tunnel 3/5-Tuple offload rule ▪ L2TP tunnel must brought up and down explicitly using **ifup** and **ifdown** commands ▪ IPSec anti-replay must be disabled in KLIPS by using a module parameter ▪ After IPSec restart, if flows are already accelerated by NSS, an ECM restart is required to re-establish flows (known limitation with IPSec) ▪ If LCP timeout or AVP packets timeout occurs between link plugout and plugin, L2TP tunnel will go down and must be brought up using **ifup** command. ▪ Only IPv4 encapsulation is supported in L2TP for outer encapsulation. However, IPv4/IPv6 is supported as inner payloads ▪ L2TPv2 supported for acceleration. L2TPv3 not supported for acceleration ▪ WLAN multicast to unicast conversion functionality supports only upto 16 groups.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.4 Supported hardware for this SP

RD	RDP	Chipset part number
CP01	RDP361 RDP352	IPQ6028 IPQ6018
CP03	RDP365	IPQ6018
CP04	RDP421	IPQ6018
AP.CP01.7	RDP420	IPQ6010

3.5 Build and load the image for IPQ6018.ILQ.11.5 CSU1

1. Download the QTI proprietary code from Qualcomm ChipCode Portal (see Section 3.5.1).
2. Download other components from external websites for QSDK while building the default configuration (see Section 3.5.2).
3. Generate the firmware:Kernel
   a. Re-assemble the code (see Section 3.5.3.2).
   b. Create the QSDK build (see Section 3.5.3.2).
   c. Generate a complete firmware image (see Section 3.5.4)
4. Install the image in the device flash memory and boot using the image from the flash (see Section 3.5.7).

Users should be familiar with directory structures that contain SP images for the different subsystems before downloading the code and building the images for loading. For each SP included in an SPF, SP binary files are generated from the SI binary files of only a subset of the included Sis. In an SPF, some Sis may support multiple SPs while others may only support one SP.

3.5.1 Download packages available through Qualcomm ChipCode Portal

Qualcomm Technologies proprietary code is available from Qualcomm ChipCode Portal. A web/GUI interface and a secure git server both allow access to this code.

- Browse available packages and obtain the download URL at:
  https://chipcode.qti.qualcomm.com/
- For more information on cloning the code, see:
  https://chipcode.qti.qualcomm.com/helpki/cloning-code-from-a-repository
- For more information on installation and configuration of the correct version of git and OpenSSL on both Windows and Linux platforms, see:
  https://chipcode.qti.qualcomm.com/helpki

  These versions are required to support the authentication methods used by Qualcomm ChipCode Portal.

3.5.2 Download packages from external websites

For details on downloading packages from external sites, see Section 2.5.2.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.5.3 Generate the firmware for IPQ6018.ILQ.11.5 CSU1

The firmware image can be generated either using a script or manually.

3.5.3.1 Script to generate complete firmware image

Meta generation script for SPF 11.5 (meta_generation_script.py) aims to simplify the release notes instructions required to be executed by the customers.

Follow these steps:

```
$ mkdir $BUILD_WS
(Assume BUILD_WS as workspace directory)
$ cd $BUILD_WS
$ git clone <OEM distro chipcode link>
<Clone required deliverable distro in the path where OEM distro is cloned
using Git clone commands and checkout(git checkout) to release tag on each
deliverable distro>
$ cd <OEM Chipcode directory>
$ git checkout r11.5_00003.0
$ rm -rf BOOT.AK.1.0 BOOT.BF.3.* IPQ8064.ILQ* IPQ4019.ILQ* IPQ8074.ILQ*
RPM.AK.1.0 TZ.AK.1.0 TZ.BF.2.7 TZ.BF.4.0.8 WIGIG.TLN* BOOT.BF.3.3.1.1
TZ.WNS.4.0 IPQ5018.ILQ.11.* BTFW.MAPLE.* WIGIG.TLN.7.5
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
--repo-url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable
If this command does not work, please use below repo init command
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
$ repo sync -j8 --no-tags -qc
$ cd common/build
$ python meta_generation_script.py -c r11.5_00003.0 -s IPQ6018.ILQ.11.5 -p
<Profile> -b <32/64> -d <DELIVERABLE_COMBO> -m [MESH_OPTIONS] --path
$BUILD_WS
Eg : python meta_generation_script.py -c r11.5_00003.0 -s IPQ6018.ILQ.11.5
-p P -b 64 -d HYFI,SIGMA-DUT -m SON --path /local/mnt/workspace/Build
```

Software Product	Profile and bit config	Supported Deliverables Combo	Mesh options
IPQ6018.ILQ.11.5	P- 32,64	HYFI,WHC,WAPID,SIGMA-DUT,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH,SON,EZMESH_FULL,EZMESH_COLOCATED
	E - 32,64	SIGMA-DUT,OEM	NA

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Software Product	Profile and bit config	Supported Deliverables Combo	Mesh options
	LM512 - 32	HYFI,WHC,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH,SON,EZMESH_FULL, EZMESH_COLOCATED
	LM256 - 32	HYFI,WHC,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH,SON,EZMESH_FULL, EZMESH_COLOCATED

NOTE: If there are no additional deliverables, use -d OEM while triggering meta_generation_script.

NOTE: If there is no Mesh options, please remove -m <Mesh option> while triggering meta_generation_script.

Single images will be present in below directory:

```
32 Bit : $BUILD_WS/IPQ6018.ILQ.11.5/<Profile>_Timestamp/<OEM distro>/common/build/bin
64 Bit : $BUILD_WS/IPQ6018.ILQ.11.5/<Profile>_Timestamp/ipq_6018*64*/<OEM distro>/common/build/bin
```

3.5.3.2 Manually generate complete firmware image

This example assumes that all packages listed in sections 3.5.1 and 3.5.2 are obtained using the **git clone** command and placed in the top-level directory.

NOTE: 64-bit single image generation requires the U-boot elf files generated by compiling the 32-bit build. Complete the 32-bit compilation before following the 64-bit build generation steps.

1. Re-assemble the code and generate the QSDK framework:

```
$ git clone <Chipcode Directory>
$ cd <Chipcode Directory>
$ git checkout r11.5_00003.0
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

After the copy of the existing Git repository is completed, change the directories where the files are present as described in this table before running the Repo command:

Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
qsdk-qca-wifi qsdk-qca-wlan qsdk-qca-art qsdk-ieee1905-security qsdk-whc qsdk-whcpy	NHSS.QSDK.11.5\apss_proc\out\proprietary\Wifi

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
qca-lib qca-mcs-apps qca-nss-userspace qca-time-services qca-qmi-framework gpio-debug qca-diag athtestcmd qca-IOT/qca-IOT.tar.bz2 qca-cnss-daemon fw-qca-stats meta-tools common-tools qsdk-qca-nss qca-nss-fw-eip-cp btdaemon minidump	NHSS.QSDK.11.5\apss_proc\out\proprietary\QSDK-Base
qca-bluetopia	NHSS.QSDK.11.5\apss_proc\out\proprietary\BLUETOPIA
qca-wifi-fw-src-component-cmn-WLAN.HK.*.tgz qca-wifi-fw-QCA6018_v1.0-WLAN.HK.*.tar.bz2	WLAN.HK.2.6\wlan_proc\src\components WLAN.HK.2.6\wlan_proc\pkg\wlan_proc\bin\QCA6018
qca-wifi-fw-src-component-cmn-WLAN.BL.*.tgz qca-wifi-fw-src-component-halphy_tools-WLAN.BL.*.tgz qca-wifi-fw-QCA9888_hw_2-WLAN.BL.*.tar.bz2 qca-wifi-fw-AR900B_hw_2-WLAN.BL.*.tar.bz2 qca-wifi-fw-QCA9984_hw_1-WLAN.BL.*.tar.bz2 qca-wifi-fw-IPQ4019_hw_1-WLAN.BL.*.tar.bz2	WLAN.BL.3.16\cnss_proc\src\components WLAN.BL.3.16\cnss_proc\bin
qca-wifi-fw-AR9887_hw_1-CNSS.PS.*.tar.bz2 qca-wifi-fw-AR9888_hw_2-CNSS.PS.*.tar.bz2	CNSS.PS.3.16
BIN-NSS.FW.*.tar.bz2	NSS.FW.11.5\nss_proc\out\proprietary
qca-afc-bin	NHSS.QSDK.11.5\apss_proc\out\proprietary\RBIN-AFCAgent

2. After copying the files to the appropriate directories, continue generating the QSDK framework:

```
$ rm -rf BOOT.AK.1.0 BOOT.BF.3.* IPQ8064.ILQ* IPQ4019.ILQ* IPQ8074.ILQ* RPM.AK.1.0
   TZ.AK.1.0 TZ.BF.2.7 TZ.BF.4.0.8 WIGIG.TLN* BOOT.BF.3.3.1.1 TZ.WNS.4.0
   IPQ5018.ILQ.11.* BTFW.MAPLE.* WIGIG.TLN.7.5
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b release -
   m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
```

If this command does not work, please use below repo init command:

```
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b release -
 m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml --repo-
 url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable

$ repo sync -j8 --no-tags -qc
$ mkdir -p qsdk/dl
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-ieee1905-security/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-art/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wifi/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wlan/* qsdk
$ cp -rf wlan_proc/src/components/qca-wifi-fw-src-component-cmn-WLAN.HK.* qsdk/dl/
```

```
$ cp -rf wlan_proc/pkg/wlan_proc/bin/QCA6018/qca-wifi-fw-QCA6018_v1.0-WLAN.HK.*
   qsdk/dl/
$ tar xvf cnss_proc/src/components/qca-wifi-fw-src-component-cmn-WLAN.BL.*.tgz -C
   qsdk/dl
$ tar xvf cnss_proc/src/components/qca-wifi-fw-src-component-halphy_tools-
   WLAN.BL.*.tgz -C qsdk/dl
$ cp -rf cnss_proc/src/components/* qsdk/dl
$ cp -rf cnss_proc/bin/QCA9888/hw.2/* qsdk/dl
$ cp -rf cnss_proc/bin/AR900B/hw.2/* qsdk/dl
$ cp -rf cnss_proc/bin/QCA9984/hw.1/* qsdk/dl
$ cp -rf cnss_proc/bin/IPQ4019/hw.1/* qsdk/dl
$ cp -rf qca-wifi-fw-AR988* qsdk/dl
$ cp -rf apss_proc/out/proprietary/QSDK-Base/meta-tools/ qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/common-tools/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qsdk-qca-nss/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-lib/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-mcs-apps/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-nss-userspace/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-time-services/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-qmi-framework/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/gpio-debug/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-diag/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/athtestcmd/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/btdaemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/minidump/* qsdk
$ tar xjvf apss_proc/out/proprietary/QSDK-Base/qca-IOT/qca-IOT.tar.bz2 -C qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-cnss-daemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/fw-qca-stats/* qsdk
$ cp apss_proc/out/proprietary/QSDK-Base/qca-nss-fw-eip-cp/BIN-EIP197.CP.*
   qsdk/dl/
$ sed -i '/QCAHKSWPL_SILICONZ/c\PKG_VERSION:=WLAN.HK.2.6-02451-QCAHKSWPL_SILICONZ-
   1' qsdk/qca/feeds/qca-cp/net/qca-cyp/Makefile
$ cp -rf apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/* qsdk
$ cp nss_proc/out/proprietary/* qsdk/dl
$ cp -rf apss_proc/out/proprietary/RBIN-AFCAgent/qca-afc-bin/* qsdk
```

32 bit:

```
$ mkdir -p qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/usr/lib/
```

Premium 32-bit	$ cd qsdk/prebuilt/ipq60xx/ipq_premium
Enterprise 32-bit	$ cd qsdk/prebuilt/ipq60xx/ipq_enterprise
LM512 32-bit	$ cd qsdk/prebuilt/ipq60xx/ipq_512
LM256 32-bit	$ cd qsdk/prebuilt/ipq60xx/ipq_256

```
$ tar zxpvf qti-mfg-provision_1.0-1_ipq.ipk
$ tar zxpvf data.tar.gz
$ cp -rf ./usr/lib/libprovision.so ../../../staging_dir/target-arm_cortex-
a7_musl-1.1.16_eabi/usr/lib/
$ cd <ChipCode Directory>
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

```
$ mkdir -p qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/pkginfo/
$ touch qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/pkginfo/qti-
mfg-provision.provides
$ echo libprovision.so > qsdk/staging_dir/target-arm_cortex-a7_musl-
1.1.16_eabi/pkginfo/qti-mfg-provision.provides
```

64 bit:

```
$  mkdir -p qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/usr/lib/
```

Premium 64-bit	`$ cd qsdk/prebuilt/ipq60xx_64/ipq_premium`
Enterprise 64bit	`$ cd qsdk/prebuilt/ipq60xx_64/ipq_enterprise`

```
$ tar zxpvf qti-mfg-provision_1.0-1_ipq.ipk
$ tar zxpvf data.tar.gz
$ cp -rf ./usr/lib/libprovision.so ../../../staging_dir/target-
aarch64_cortex-a53_musl-1.1.16/usr/lib/
$ cd <ChipCode Directory>
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

```
$ mkdir -p qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/pkginfo/
$ touch qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/pkginfo/qti-
mfg-provision.provides
$ echo libprovision.so > qsdk/staging_dir/target-aarch64_cortex-a53_musl-
1.1.16/pkginfo/qti-mfg-provision.provides
```

NOTE: Prebuilt ipk images (such as Qualcomm Bluetopia or qca-mcs-apps, qca-hyd-init, qca-hyd-son, qca-hyd-map) are in their respective folders:

`<ChipCode Directory>/qsdk/prebuilt/ipq60xx_64/ipq_<profile_name>/`	Contains all 64-bit ipk images
`<ChipCode Directory>/qsdk/prebuilt/ipq60xx/ipq_<profile_name>/`	Contains all 32-bit ipk images

Where ipq_<profile_name> is ipq_premium, ipq_enterprise, ipq_256, and ipq_512.

3. Customers with Qualcomm HY-FI, WHC, WAPid, or Sigma DUT packages:

	These files are fetched from ChipCode and copied to the working QSDK top-level directory:		
Premium/LM	**HY-FI and WHC:**	Hyfi qsdk-whc qsdk-whcpy	apss_proc\out\proprietary\Hyfi apss_proc\out\proprietary\Wifi\qsdk-whc apss_proc\out\proprietary\Wifi\qsdk-whcpy
Premium	**WAPid:**	qsdk-wapid	apss_proc\out\proprietary\Wapid\qsdk-wapid
Premium/ Enterprise	**Sigma-DUT:**	sigma-dut.tar.bz2	apss_proc\out\proprietary\sigma-dut\sigma-dut.tar.bz2
Premium/LM	**EZMESH-SRC**	qsdk-ezmesh-src	apss_proc\out\proprietary\RSRC-EZMESH\qsdk-ezmesh-src
Premium/LM	**EZMESH-BIN**	qsdk-ezmesh-bin	apss_proc\out\proprietary\RBIN-EZMESH\qsdk-ezmesh-bin
Premium/LM	**EZMESH-ALG**	qsdk-ezmesh-alg-bin	apss_proc\out\proprietary\RBIN-EZMESH-ALG\qsdk-ezmesh-alg-bin

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Premium/LM	**EZ-ALG-SRC**	qsdk-ezmesh-alg-src	apss_proc\out\proprietary\RSRC-EZMESH-ALG\qsdk-ezmesh-alg-src

Then run the additional code:

Premium/ LM	**HY-FI and WHC**	`$ cp -rf apss_proc/out/proprietary/Hyfi/hyfi/* qsdk`
		`$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whc/* qsdk`
		`$ mkdir qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug`
		`$ mv qsdk/qca/feeds/qca-son-mem-debug/Makefile qsdk/qca/feeds/qca-son-mem-debug/Config.in qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug`
		`$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whcpy/* qsdk`
		`$ sed -i "s/@PACKAGE_whc-son/@PACKAGE_whc-map/g" qsdk/qca/feeds/qca-lib/qca-wifison-ext-lib/Makefile`

Premium	**WAPid**	`git clone <wapid ChipCode directory>`
		`$ cd <wapid ChipCode directory>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ cp -rf <wapid ChipCode directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/Wapid/qsdk-wapid/* qsdk`
		Where wapid ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_wapid-src for this release.

Premium/ Enterprise	**Sigma-DUT**	`git clone <sigma-dut ChipCode Directory>`
		`$ cd <sigma-dut ChipCode Directory>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ tar xjvf <sigma-dut ChipCode Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/sigma-dut/sigma-dut.tar.bz2 -C qsdk`
		Where sigma-dut ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_sigma-dut for this release.

Premium/LM	**EZMESH-SRC**	`#Only for Ezmesh customer`
		
		`$ git clone <ezmesh ChipCode Directory>`
		`$ cd <ezmesh ChipCode Directory>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ cp -rf <ezmesh ChipCode Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH/qsdk-ezmesh-src/* qsdk`
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-src for this release
		
		`#Below sed commands in EZMESH-SRC should be skipped if customer is using` **`EZ-ALG-SRC`** `also (i.e. both ezmesh-src and ezmesh-alg-src)`
		
		`$ sed -i 's/HYD_MODULE_STRATEGY=y/HYD_MODULE_STRATEGY=n/g' qsdk/qca/src/qca-ezmesh/ezmeshConfig.defs`
		`$ sed -i '0,/ifeq/{/ifeq/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile`
		`$ sed -i '0,/endif/{/endif/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile`
		`$ sed -i '/libezmeshalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile`

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

		$ sed -i '/libezmeshagentalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile $ sed -i '/DUMP/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile

Premium/LM	**EZMESH-BIN**	#Only for Ezmesh customer
		
		$ git clone <ezmesh *ChipCode Directory*>
		$ cd <*ezmesh ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH/qsdk-ezmesh-bin/* qsdk
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-bin for this release

Premium/LM	**EZMESH-ALG**	#Only for Ezmesh customer
		
		$ git clone <*ezmesh ChipCode Directory*>
		$ cd < *ezmesh ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH-ALG/qsdk-ezmesh-alg-bin/* qsdk
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-alg for this release

Premium/LM	**EZ-ALG-SRC**	#Only for Ezmesh customer
		
		$ git clone <*ezmesh ChipCode Directory*>
		$ cd < *ezmesh ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH-ALG/qsdk-ezmesh-alg-src/* qsdk
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ez-alg-src for this release

Create the 32-bit and 64-bit QSDK build

QSDK supports the **premium, enterprise, LM512, and LM256** profiles for Linux kernel 4.4.60 support**.** The QSDK framework is developed using Ubuntu (from version 16.04 to version 18.04) and Debian. However, QSDK framework regenerates critical tools required to compile firmware at build time. Thus, the framework is independent from the host environment. Although it is developed using the listed distributions, it is expected to work on others such as Red Hat, Mint, or Fedora.

See Section 2.4.3.2.1 , to install the pre-requisite tools manually (or)

See Section 2.4.3.2.2, to use the script to get the pre-requisites installed in one go.

Because framework automatically downloads the open-source components needed during the build/ make process, ensure that an Internet connection is active on the build host when creating the build.

1. Install the different feeds in the build framework:

```
$ cd qsdk
$ ./scripts/feeds update -a
$ ./scripts/feeds install -a -f
```

2. Copy the base configuration to use for the build. Choose a profile and use it to build with Linux 4.4.60 support.
3. Regenerate a complete configuration file and start the build:

Premium 32-bit	`$ cp qca/configs/qsdk/ipq_premium.config .config`
	`$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq60xx/g" .config`
	`$ sed -i "s/CONFIG_PACKAGE_qal/#CONFIG_PACKAGE_qal/g" .config`
	`$ mv prebuilt/ipq60xx/ipq_premium/* prebuilt/ipq60xx/`

Premium 64-bit	`$ cp qca/configs/qsdk/ipq_premium.config .config`
	`$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq60xx_64/g" .config`
	`$ sed -i "s/CONFIG_PACKAGE_qal/#CONFIG_PACKAGE_qal/g" .config`
	`$ mv prebuilt/ipq60xx_64/ipq_premium/* prebuilt/ipq60xx_64/`

Enterprise 32-bit	`$ cp qca/configs/qsdk/ipq_enterprise.config .config`
	`$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq60xx/g" .config`
	`$ mv prebuilt/ipq60xx/ipq_enterprise/* prebuilt/ipq60xx/`

Enterprise 64-bit	`$ cp qca/configs/qsdk/ipq_enterprise.config .config`
	`$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq60xx_64/g" .config`
	`$ mv prebuilt/ipq60xx_64/ipq_enterprise/* prebuilt/ipq60xx_64/`

LM512 32-bit	`$ cp qca/configs/qsdk/ipq_512.config .config`
	`$ sed -i "s/TARGET_ipq_ipq807x/TARGET_ipq_ipq60xx/g" .config`
	`$ mv prebuilt/ipq60xx/ipq_512/* prebuilt/ipq60xx/`

LM256 32-bit	`$ cp qca/configs/qsdk/ipq_256.config .config`
	`$ sed -i "s/TARGET_ipq_ipq807x/TARGET_ipq_ipq60xx/g" .config`
	`$ mv prebuilt/ipq60xx/ipq_256/* prebuilt/ipq60xx/`

Hy-Fi, WHC customers only:

- Starting with QCA_Networking_2020.SPF.11.3 release, SON and EasyMesh Features are compiled as separate build. SON build supports only SON features and EZMESH build supports only EasyMesh features.
- Starting with QCA_Networking_2021.SPF.11.4 release, MAP package qca-hyd-map is replaced with qca-ezmesh. Enable additional configuration to enable ezmesh packages.
- Use the SON package commands in this table for a SON features enabled build.
  Use the EZMESH package commands for an EasyMesh features enabled build.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Enable SON or EZMESH configuration:

SON package	$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_whc-son=y" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-son=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config

EZMESH package (Full Mode)	$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
	Additional configuration to enable ezmesh packages:
	$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config

EZMESH package (Co-located mode)	$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
	$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
	Additional configuration to enable ezmesh packages:
	$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config

- Use the commands in this table to enable both SON and EasyMesh Features.

Enable SON and EZMESH (full mode) configuration:

SON + EZMESH package (full mode)	$ echo "CONFIG_PACKAGE_whc-mesh=y" >> .config
	$ echo "CONFIG_PACKAGE_hyfi-mesh=y" >> .config
	Additional configuration to enable ezmesh packages:
	$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
	$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config

NOTE: Enabling both SON and EasyMesh Features in a build will increase image size and consume extra memory in Flash storage.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ make defconfig
$ make V=s -j5
```

4. For 64-bit firmware, once the 32-bit compilation completes, save all the U-boot executable files **openwrt-ipq6018*.elf** from the directory: **qsdk/bin/ipq.** These files are required to generate all 14 variants of the complete 64-bit firmware image.
5. Download the required packages for the corresponding profile and create the image. Once the build is complete, these files are in the **qsdk/bin/ipq** directory:

32-bit	▪ openwrt-ipq6018*.elf (ELF files for U-Boot variants) ▪ openwrt-ipq-ipq60xx-squashfs-root.img (SquashFS) ▪ openwrt-ipq-ipq60xx-qcom-ipq60xx-cpxx-fit-uImage.itb (ITB)
64-bit	▪ openwrt-ipq-ipq60xx_64-squashfs-root.img (SquashFS) ▪ openwrt-ipq-ipq60xx_64-qcom-ipq60xx-cpxx-fit-uImage.itb (ITB)

NOTE: The openwrt-ipq-ipq60xx-ubi-root.img generated in qsdk/bin/ipq folder **will no longer be used** for IPQ6018 because the Wi-Fi firmware image must be added as a ubi volume to the UBI image.

3.5.4 Generate a complete firmware image

IPQ60x8 requires flashing multiple images for Bootup, including xBL, RPM, TZ, CDT, MIBIB, NSS Images, Kernel, Filesystem, and so on. To simplify both image flashing and device boot, individual images are combined into a single Flattened Image Tree (FIT) image. FIT image components can be flashed into the respective partition based on user configuration. More tools required on the Ubuntu 16.04 64-bit machine include:

1. Install DTC:
   ```
   $sudo apt-get install device-tree-compiler
   ```
2. Install libfdt:
   ```
   $sudo apt-get install libfdt-dev
   ```
3. Install Python 2.7.
4. Install tools required to run packing single image:
   ```
   $sudo apt-get install libc6-i386
   $sudo apt-get install libgl1-mesa-dri:i386
   ```
5. Switch to the Qualcomm ChipCode Portal directory:
   ```
   $ cd <Chipcode Directory>
   ```
6. Copy the flash config files to **common/build/ipq**
7. Find the single images in the **common/build/bin** directory. Images are generated for default image content. Fewer images indicate an issue with one of the partition sizes, or that not all required files are present.

32-bit image

```
$ cd <Chipcode Directory>
$ mkdir -p common/build/ipq
$ mkdir -p apss_proc/out/meta-scripts
$ cp qsdk/qca/src/u-boot-2016/tools/pack.py apss_proc/out/meta-
scripts/pack_hk.py
$ sed -i 's#</linux_root_path>#/</linux_root_path>#' contents.xml
$ sed -i 's#</windows_root_path>#\\</windows_root_path>#' contents.xml
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ cp qsdk/bin/ipq/openwrt* common/build/ipq
$ cp -r apss_proc/out/proprietary/QSDK-Base/meta-tools apss_proc/out/
$ cp -rf qsdk/bin/ipq/dtbs/* common/build/ipq/
$ cp -rf skales/* common/build/ipq/
$ cp -rf wlan_proc/build/ms/bin/FW_IMAGES/* common/build/ipq/
$ cp qsdk/staging_dir/host/bin/ubinize common/build/ipq/
$ cd common/build
$ sed
'/debug/d;/packages/d;/"ipq6018_64"/d;/t32/d;/ret_prep_64image/d;/Required/
d;/skales/d;
/nosmmu/d;/os.system(cmd)/d;/os.chdir(ipq_dir)/d;/atfdir/d;/noac/d;/single-
atf/d;/bl31.mbn/d;/ret_pack_64image/d;/ret_pack_apps_64image/d;/64image/d'
-i update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
```

(Where <profile-name> is E, P, LM256, or LM512)

64-bit image

```
$ cd <Chipcode Directory>
$ mkdir -p common/build/ipq_x64
$ mkdir -p apss_proc/out/meta-scripts
$ cp qsdk/qca/src/u-boot-2016/tools/pack.py apss_proc/out/meta-scripts/pack_hk.py
$ sed -i 's#</linux_root_path>#/</linux_root_path>#' contents.xml
$ sed -i 's#</windows_root_path>#\\</windows_root_path>#' contents.xml
$ cp qsdk/bin/ipq/openwrt* common/build/ipq_x64
copy the saved openwrt-ipq6018-u-boot*.elf from the 32-bit build to:
   common/build/ipq_x64
$ cp -r apss_proc/out/proprietary/QSDK-Base/meta-tools apss_proc/out/
$ cp -rf qsdk/bin/ipq/dtbs/* common/build/ipq_x64/
$ cp -rf skales/* common/build/ipq_x64/
$ cp -rf wlan_proc/build/ms/bin/FW_IMAGES/* common/build/ipq_x64
$ cp qsdk/staging_dir/host/bin/ubinize common/build/ipq_x64
$ cd common/build
$ sed -i 's/\.\/ipq/\.\/ipq_x64/g' update_common_info.py
$ sed -i 's/\.\/ipq_x64_x64/\.\/ipq_x64/g' update_common_info.py
$ sed
   '/debug/d;/packages/d;/"ipq6018"/d;/t32/d;/ret_prep_32image/d;/Required/d;/nos
   mmu/d
   ;/os.system(cmd)/d;/skales/d;/os.chdir(ipq_dir)/d;/atfdir/d;/noac/d;/single-
   atf/d;/bl31.mbn/d;/ret_pack_32image/d;/ret_pack_apps_32image/d;/32image/d' -i
   update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
```

(Where <profile-name> is E or P)

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.5.5 Flash the complete default software image

3.5.5.1 Set up the flash environment

1. Ensure that the board console port is connected to the PC using these RS232 parameters:
   - 115200 bps
   - 8N1
2. Confirm that the PC is connected to the board using one of the Ethernet ports. The PC must have a TFTP server launched and listening on the interface to which the board is connected. The interface must have an IP address configured manually. At this stage powerup the board and after a few seconds, press any key during the countdown.

3.5.5.2 Standard board configuration

Load the image in flash and boot the platform:

1. Copy the intended **xxxx-ipq6018-single.img** (or 64-bit equivalent file) to the TFTP server root directory.
2. Check hardware jumper configuration according to reference board and default memory configuration (see the appropriate board setup guide for more information) to verify which flash memory the board is booting from.
3. Confirm the machine ID, Meta version, profile, and selected single image to match the memory type that the board boots from.
4. Set the IP address and server IP using the TFTP process:
   ```
   setenv ipaddr 192.168.1.11
   setenv serverip 192.168.1.xx (TFTP server address)
   ```

For IPQ6018+QCN90xx

```
ping ${serverip}
set bootargs console=ttyMSM0,115200n8
tftpboot xxxx-ipq6018-single.img
```

1. Once the single image has booted, transfer SON packages for SON features or MAP packages for EasyMesh features to the AP /tmp directory from **./qsdk/prebuilt/ipq60xx/** or **./qsdk/prebuilt/ ipq60xx _64/**, depending on whether the booted image is 32-bit or 64-bit.
   ```
   set bootargs console=ttyMSM0,115200n8
   tftpboot xxxx-ipq6018-single.img
   ```
2. Flash the image:
   ```
   imgaddr=$fileaddr && source $imgaddr:script
   ```
3. Power cycle the board after step 5 has completed, as indicated, by the printing of the U-boot prompt. It may be hundreds of seconds depending on image size and memory technology.

NOTE: For Bluetopia, perform Steps 7 and 8.

4. Once the single image has booted, transfer these packages to the AP /tmp directory from ./**qsdk/prebuilt/ipq60xx/** or **./qsdk/prebuilt/ ipq60xx _64/**, depending on whether the booted image is 32-bit or 64-bit.
   ```
   qsdk/prebuilt/ipq60xx/bluetopia_4.2.1.c1_13-1_ipq.ipk
   qsdk/prebuilt/ipq60xx_64/bluetopia_4.2.1.c1_13-1_ipq.ipk
   ```

5. Install these packages from the /tmp directory:

```
opkg install bluetopia_4.2.1.c1_13-1_ipq.ipk
```

NOTE: For Wi-Fi SON, perform steps 10, 11, and 12:

6. Once the single image has booted, transfer SON packages for SON Features or EZMESH Packages for EasyMesh Features to the AP /tmp directory from ./**qsdk/prebuilt/ipq60xx/** or ./**qsdk/prebuilt/ ipq60xx _64/**, depending on whether the booted image is 32-bit or 64-bit.

SON package (32-bit)	qsdk/prebuilt/ipq60xx/qca-hyd-init_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx/qca-hyd-son_ge474d2e-1_ipq.ipk
SON package (64-bit)	qsdk/prebuilt/ipq60xx_64/qca-hyd-init_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx_64/qca-hyd-son_ge474d2e-1_ipq.ipk
EZMESH package (32-bit)	**To support Full mode:** qsdk/prebuilt/ipq60xx/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx/qca-ezmesh_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx/qca-ezmesh-alg_ge474d2e-1_ipq.ipk **To support Co-located mode:** qsdk/prebuilt/ipq60xx/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx/qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx/qca-ezmesh-agent_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx/qca-ezmesh-alg_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx/qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk
EZMESH package (64-bit)	**To support Full mode:** qsdk/prebuilt/ipq60xx_64/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx_64/qca-ezmesh_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx_64/qca-ezmesh-alg_ge474d2e-1_ipq.ipk **To support Co-located mode:** qsdk/prebuilt/ipq60xx_64/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx_64/qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx_64/qca-ezmesh-agent_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx_64/qca-ezmesh-alg_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq60xx_64/qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk

NOTE: ge474d2e is the git commit ID, which may also be changed.

7. Install these packages from the /tmp directory: Choose SON package to enable SON features, or choose MAP package to enable MAP features:

SON package	opkg install qca-hyd-init_ge474d2e-1_ipq.ipk opkg install qca-hyd-son_ge474d2e-1_ipq.ipk
EZMESH package	**To support Full mode:** opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk opkg install qca-ezmesh_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk **To support Co-located mode:** opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-agent_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk

NOTE: ge474d2e is the git commit ID, which may also be changed.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The HYD daemon application binary name is changed from hyd to hyd-son for the SON package. Similarly, the WSPLCD daemon application binary name is changed from wsplcd to wsplcd-son and wsplcd-map for the SON and EZMESH packages, respectively. The symbolic link to the older binary name must be create after package installation to ensure that scripts and tools using work with the new binary name. Because application binary name has changed, a crash dump is created based on new binary name. Customers must take care of updating any scripts and tools that use or depend on the older binary name.

Execute these commands to create a symbolic softlink to the older binary name:

SON package	`ln -s /usr/sbin/wsplcd-son /usr/sbin/wsplcd` `ln -s /usr/sbin/hyd-son /usr/sbin/hyd`
EZMESH package	`ln -s /usr/sbin/wsplcd-map /usr/sbin/wsplcd`

3.5.5.3 Upgrade the firmware

This release can upgrade board images without a TFTP server. After using U-Boot to flash an initial image and booting the device, use the web interface for future upgrades. These future upgrades can take place from either the OpenWRT web interface or serial console. Upgrade the existing flash image using either the appropriate single image file or the apps image file generated by **update_common_info.py**.

Upgrading image files via the web interface takes several minutes to complete depending on factors such as memory technology, vendor, image size, browser connectivity, and network load. Completion of the flash upgrade process is signaled by the refresh of the OpenWRT login page, or lack of further messages on the serial console.

- An invalid image remains in the flash memory if the upgrade process is interrupted, such as if system power is lost during upgrade or a keypress event is detected on the serial console port.
- If a board is configured for failsafe boot, and the U-boot environment variable *bootargs* contains any reference to the rootfs or other partitions managed by the failsafe scheme. In this case, the sysupgrade process may only be partially successful: the board may or may not boot later, and the image may only be partly updated.
- To use the **sysupgrade** command from the Linux console, sysupgrade image is required to be visible:
  - Within the Linux file system supplied using development host ssh or tftpd server, OR
  - On an NFS server that is mounted to the local file system, OR
  - Transferred using a USB storage device
- The sysupgrade feature can only be used to update the image in the memory the board boots from and is used by the running kernel. If the board supports multiple memory technologies, use U-Boot to change the image in memories that the board did not boot from.

See the *IPQ60x8 SoC Software User Guide* (80-YC609-8) for more information on sysupgrade support.

3.5.6 Create customized IPQ60x8 Wi-Fi firmware images

Flash memory layout includes a UBI volume for the Wi-Fi firmware and BDF information. The release includes squashfs images in the final image created by the last steps of section 2.5.5. Use these steps to update or recreate the squashfs Wi-Fi firmware image and the complete final image.

NOTE: These steps are not needed if the BDF and PIL files are not customized.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

1. Install these tools in the workstation:
   - mtd-utils: sudo apt-get install mtd-utils
   - mksquashfs4 binary from **<Chipcode Directory> /common/build** to /usr/bin or /usr/sbin or any location included in $PATH
2. Download the WLAN IPQ6018 firmware BIN tar ball from the corresponding SP release distro.
3. Extract the firmware binary tarball:
```
mkdir -p <Chipcode Directory>/fwtemp
cp -rfv <Chipcode Directory>/wlan_proc/pkg/wlan_proc/bin/QCA6018/qca-wifi-fw-QCA6018_v1.0-WLAN.HK.*.tar.bz2 fwtemp
cd fwtemp
tar -xvf qca-wifi-fw-QCA6018_v1.0-WLAN.HK.*.tar.bz2
```
4. Create a staging directory to hold the BD and PIL files.
```
mkdir staging_dir
cp -rfv qca-wifi-fw-WLAN.HK.*/PIL_IMAGES/* staging_dir
cp -rfv qca-wifi-fw-WLAN.HK.*/bdwlan* staging_dir
```
   (Replace the BDF file with the custom versions as needed.)
5. To generate the firmware squash image for NAND/NOR/EMMC (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):
```
mksquashfs4 staging_dir/ wifi_fw.squashfs -nopad -noappend -root-owned -comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -processors 1
dd if=wifi_fw.squashfs of=wifi_fw_squashfs.img bs=2k conv=sync
```
   UBI firmware image for NAND will no longer be required as Wi-Fi image will be added as UBI volume and will be ubinized along with Kernel and rootfs

For instructions on how to generate ubinized image and flash the firmware images separately, see the *IPQ6018 SoC Software User Guide* (80-YC609-8).

3.5.7 Create customized IPQ60x8 + QCN90xx Wi-Fi firmware images

Flash memory layout includes a UBI volume for the Wi-Fi firmware and BDF information. The release includes squashfs images in the final image created by the last steps of section 2.5.5. Use these steps to update or recreate the squashfs Wi-Fi firmware image and the complete final image.

NOTE: These steps are not needed if the BDF and PIL files are not customized.

1. Install these tools in the workstation:
   - mtd-utils: sudo apt-get install mtd-utils
   - mksquashfs4 binary from:
     < *ChipCode Directory*> /common/build to /usr/bin or /usr/sbin or any location included in $PATH
2. Download the WLAN SquashFS image wifi_fw_ipq6018_qcn9000_squashfs.img from WLAN.HK.2.3/wlan_proc/build/ms/bin/FW_IMAGES/
3. Extract the squashfs image in a temp directory.
```
mkdir -p fwtemp
cp <download path>/wifi_fw_ipq6018_qcn9000_squashfs.img  ./fwtemp
cd fwtemp
unsquashfs wifi_fw_ipq6018_qcn9000_squashfs.img
```

All extracted files will be in ./squashfs-root folder.

4. Copy/modify any files in *./squashfs-root* folder.
5. To generate the firmware SquashFS image for NAND/NOR/EMMC (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):

```
mksquashfs4 squashfs-root/ wifi_fw.squashfs -nopad -noappend -root-owned
   -comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -
   processors 1
dd if=wifi_fw.squashfs of=wifi_fw_squashfs.img bs=2k conv=sync
```

6. UBI firmware image for NAND will no longer be required as WiFi image will be added as UBI volume and will be ubinized along with Kernel and rootfs.

For instructions on how to generate ubinized image and flash the firmware images separately, see the *IPQ6018 SoC Software User Guide* (80-YC609-8).

3.6 Flash Wi-Fi firmware image only

The Wi-Fi firmware/BDF information stored in the flash image can be updated without having to rewrite the whole flash image as shown in this section. Regardless of the memory type the system is booting from, follow the same basic process of using tftpboot to load the new Wi-Fi firmware image to DDR, then identify the correct location in the flash and write the Wi-Fi firmware image to that location.

NAND boot

- Copy the image (wifi_fw_squashsfs.img):

```
# tftpboot wifi_fw_squashsfs.img
```

For IPQ6018+QCN90xx

```
# tftpb wifi_fw_ipq6018_qcn9000_squashfs.img
```

- Flash the partition using the flash command:

```
# flash wifi_fw
```

Where wifi_fw is the UBI volume name and should be used in the flash command.

NOR boot / eMMC boot

- Copy the image (wifi_fw_squashsfs.img):

```
# tftpboot wifi_fw_squashfs.img
```

For IPQ6018+QCN90xx

```
# tftpb wifi_fw_ipq6018_qcn9000_squashfs.img
```

- Flash the partition using the flash command:

```
# flash 0:WIFIFW
```

Where 0:WIFIFW is the Wi-Fi partition name. This name is used for flash command.

UBI partition changes

Separate partition for Wi-Fi firmware image is removed from the NAND flash layout and the Wi-Fi firmware image is added as a UBI volume, along with Kernel and rootfs, to save flash space. This feature will be supported in this release. See *IPQ6018 SoC Software User Guide* (80-YC609-8) for more details.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.7 Test the GATT profile with sample applications with onboard CSR8811 Bluetooth on AP.CP01

This release contains a preloaded Bluetooth sample application. To test GATT profile containing sample applications:

1. Launch the Bluetooth sample application from the AP after the Premium profile is built. It is available at:
   apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/prebuilt
2. After the complete firmware image is built, launch the Bluetooth stack to test the GATT profile with these LinuxSPPLE sample applications:
   - The LinuxSPPLE application is intended to demonstrate the usage of the Qualcomm Bluetopia Serial Port Profile Low Energy API and relevant Bluetopia Core APIs. The application supports issuing all the basic commands used by the Serial Port Profile Low Energy.
   - Launching the BT sample APP, which enables BT and download the PSKeys to the BT module.
3. Wi-Fi/BT coex PSKeys are enabled after the sample APP is launched.
4. Export the environment variable before launching the sample application.

Platform/Module	Environment variable
IPQ6018 platform CP01 or QFN BT module	BTHOST_8311_SOC_TYPE=cp01
custom SOC GPIO settings	BTHOST_8311_SOC_TYPE=custom

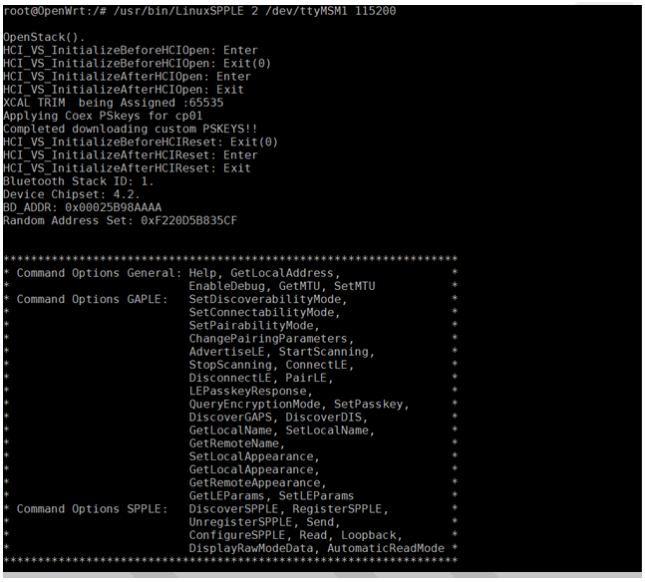
PSKeys will be read from the text file located in **/usr/bin as PS_KEY_CSR8811.txt.** Contents of the text file must be in this format:

```
# PSKEY_COEX_SCHEME
\x06\xC2\x02\x00\x09\x00\x34\x00\x03\x70\x00\x00\x80\x24\x01\x00\x00\x00\x03\x00
# PSKEY_COEX_PIO_UNITY_3_BT_ACTIVE (BT_active)
\x05\xC2\x02\x00\x0A\x00\x35\x00\x03\x70\x00\x00\x83\x24\x02\x00\x00\x00\x05\x00\x
   01\x00
# PSKEY_COEX_PIO_UNITY_3_BT_STATUS (BT_STATUS)
\x05\xC2\x02\x00\x0A\x00\x36\x00\x03\x70\x00\x00\x84\x24\x02\x00\x00\x00\xff\xff\x
   01\x00
# PSKEY_COEX_PIO_UNITY_3_BT_DENY (WLAN Deny)
\x05\xC2\x02\x00\x0A\x00\x37\x00\x03\x70\x00\x00\x85\x24\x02\x00\x00\x00\x04\x00\x
   01\x00
# WARM RESET
\x00\xC2\x02\x00\x09\x00\x47\x00\x02\x40\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
```

root@openwrt:cd /usr/bin
root@openwrt: ./LinuxSPPLE 2 /dev/ttyMSM1 115200

5. Note the Bluetooth address of the module on the AP.
   - Bluetooth Stack ID: 1
   - Device Chipset: 4.2.
     BD_ADDR: 0x00025B98AAAA

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION



6. Ignore the HCI_VS print statements; the Bluetooth stack application will launch with the prompt.
7. Run these commands to ensure that the radio is attached to the BT connector is good and can scan surrounding devices:
   - StartScanning – Starts scanning all Bluetooth devices with their device ID and Bluetooth address
   - StopScanning – Stops scanning devices. This command can be run during scanning.
8. Use this command to register a SPPLE service.

   ```
   SPPLE>RegisterSPPLE
   Successfully registered SPPLE Service.
   SPPLE>
   ```

   This function returns zero on successful execution and a negative value on errors. This command takes no parameters.
9. Use this command to turn on LE advertising on the device:

   ```
   AdvertiseLE 1
   GAP_LE_Advertising_Enable success.
   SPPLE>
   ```

   The only parameter required is the advertising flag which is 0 for disable and 1 for enable. This registers the device as SPPLE and clients can connect to this one.
   - Bluetooth clients
     - Bluetooth clients can use a custom App or Light Blue App on an AP.CP01 device with Bluetooth or with the CSR™ USB dongle 8510. LinuxSPPLE must show up in the scanning. Connect to a Bluetooth server with the Bluetooth address and send data.
     - Connecting using Bluetopia stack from another CP01 with On-board CSR 8811

     ```
     SPPLE>ConnectLE 00025B98AAAA 0.
     ```
10. This command performs a SPPLE service discovery operation. This function returns zero on successful execution and a negative value on errors. This command takes no parameters.

    Client side:

    ```
    SPPLE>DiscoverSPPLE
    ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
GATT_Start_Service_Discovery_Handle_Range() success.
SPPLE>
Service 0x001E - 0x0028, UUID: 14839AC47D7E415C9A42167340CF2339.
SPPLE>
Service Discovery Operation Complete, Status 0x00.
Valid SPPLE Service Found.
```

11. This command configures a SPPLE service on a remote device. This function returns zero on successful execution and a negative value on errors. This command takes no parameters.

    This function enables notifications of the proper characteristics based on a specified handle. Depending on whether the device role for SPPLE is server or client, either a GATT_Handle_Value_Notification or a GATT_Write_Without_Response_Request API function is called. The called function notifies the receiving credit characteristic or sends a write without response packet to the transmission credit characteristic respectively.

```
SPPLE>ConfigureSPPLE
SPPLE Service found on remote device, attempting to read Transmit
  Credits, and configured CCCDs.

SPPLE>
Write Response.
Connection ID:   1.
```

HSUART node name

Board	Kernel version	Device node name
IPQ6018	4.4	/dev/ttyMSM1

4 IPQ8064.ILQ.11.5 CSU1

4.1 Supported hardware for this SP

Board	Comments
AP148.5	IPQ8068 for Skyworks (Enterprise)
AP148.3	IPQ8068 for Skyworks (Qualcomm Connected Smart Home)
AP148.6	IPQ8068 for CUS238/CUS240 (Connected Smart Home)
AP148.7	IPQ8068 for CUS238/CUS240 (Enterprise)
AP160.1	IPQ8069 design for 3 radios (Enterprise)
AP160.2	IPQ8069 design for 3 radios + 2.5G Ethernet (Enterprise)
AP161.1	IPQ8069 design for 3 radios
CS.BL01.1	QCA9992-based 802.11n 2 GHz; 3x3 configuration + Skyworks 2.4 GHz (Enterprise)
CS.BL01.1	QCA9992-based 802.11n 2 GHz; 2x2 configuration + Skyworks 2.4 GHz (Enterprise
CS.CAS01.1	QCA9994-based 802.11ac 5 GHz; 4x4 configuration + Qorvo 5 GHz (lower band)
CS.CAS01.3	QCA9994-based 802.11ac 5 GHz; 4x4 configuration + Qorvo 5 GHz (higher band)
CUS238.1	QCA9990 4x4 configuration + QFE1952-based 802.11n 5 GHz (Connected Smart Home)
CUS238.3	QCA9990 4x4 configuration + QFE1952-based 802.11n 5 GHz (Enterprise)
CUS238.7	QCA9994-based 802.11ac 5 GHz 4x4 configuration + QFE1952 5 GHz (Enterprise)
CUS239.1	QCA9990-based 802.11n 5 GHz; 4x4 configuration + Skyworks 5 GHz (Connected Smart Home)
CUS239.2	QCA9990-based 802.11n 5 GHz; 3x3 configuration + Skyworks 5 GHz (Connected Smart Home)
CUS239.3	QCA9990-based 802.11n 5 GHz; 4x4 configuration + Skyworks 5 GHz (Enterprise)
CUS239.4	QCA9990-based 802.11n 5 GHz; 3x3 configuration + Skyworks 5 GHz (Enterprise)
CUS239.5	QCA9994-based 802.11ac 5 GHz; 4x4 configuration + Qorvo 5 GHz (Connected Smart Home)
CUS239.7	QCA9994-based 802.11ac 5 GHz; 4x4 configuration + Qorvo 5 GHz (Enterprise)
CUS240.1	QCA9990 4x4 configuration + QFE1952-based 802.11n 2.4 GHz (Connected Smart Home)
CUS240.3	QCA9990 4x4 configuration + QFE1952-based 802.11n 2.4 GHz (Enterprise)
CUS240.7	QCA9994-based 802.11ac 2.4 GHz 4x4 configuration + QFE1922 2.4 GHz (Enterprise)
CUS260.1	QCA9990-based 802.11n 2 GHz; 4x4 configuration + Skyworks 2.4 GHz (Connected Smart Home)
CUS260.2	QCA9990-based 802.11n 2 GHz; 3x3 configuration + Skyworks 2.4 GHz (Connected Smart Home)
CUS260.3	QCA9990-based 802.11n 2 GHz; 4x4 configuration + Skyworks 2.4 GHz (Enterprise)
CUS260.4	QCA9990-based 802.11n 2 GHz; 3x3 configuration + Skyworks 2.4 GHz (Enterprise)
CUS260.5	QCA9994-based 802.11n 2.4 GHz; 4x4 configuration + Skyworks 2.4 GHz (Connected Smart Home)
CUS260.7	QCA9994-based 802.11n 2.4 GHz; 4x4 configuration + Skyworks 2.4 GHz (Enterprise)
RDP0317	IPQ8069.AP161.2.QCA9994.CUS238.7.QCA9994.CUS240.8.QCA9889.XB242.2.RDP IPQ8069 AP161 (IPQ806x 3.0 3 PCIe Ent 4 layer) + QCA9994 (CUS238 (4x4 5 GHz, MP) + CUS240 (3x3 2.4 GHz MP) + QCA9889 XB242(1x1 2.4/5 GHz 11ac DB iPA PCIe)
RDP0329	IPQ8065.AP161.1.AR9287.HB97.1.QCA9886.XB.BSR01.1.RDP

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Board	Comments
RDP0330	IPQ8065.AP161.1.QCA9984.CUS238.5.QCA9985.CUS240.9.RDP
XB242.2	QCA9889 based 802.11n 2.4 GHz/5 GHz scan radio (Enterprise)
RDP0108	AP161.1, AK2.0, 4X4, CAS1.0, CUS 239.5, CUS260.5

4.2 Restrictions on software while using it for testing

<table>
<tr><td rowspan="8">Channel Conditions</td><td colspan="3">For peak performance results, the AP, STA, and channel conditions must be configured to:</td></tr>
<tr><td>Attribute</td><td colspan="2">Value</td></tr>
<tr><td>Image</td><td colspan="2">See Section 1.1 for the release version</td></tr>
<tr><td>AP</td><td>AP160 + CUS239</td><td>AP160 + CUS260</td></tr>
<tr><td>STA</td><td>AP160 + CUS239 in WDS mode</td><td>AP160 + CUS260 in WDS mode</td></tr>
<tr><td>RSSI of AP at STA</td><td>0x43</td><td>0x40</td></tr>
<tr><td>RSSI of STA at AP</td><td>0x43</td><td>0x40</td></tr>
<tr><td>Attenuation</td><td>32 dB per chain</td><td>40 dB per chain</td></tr>
<tr><td>WPS</td><td colspan="3">▪ Qualcomm provides an Independent Hardware Vendor (IHV) mechanism which bypasses the Windows connection manager (and internal supplicant) and allows alternative security negotiation.
▪ The IHV mechanism enables a secured connection, but the Windows APIs provided supporting WPS do not work properly. Therefore, WPS operation for establishing a secure connection is not functional.
▪ Qualcomm is pursuing this issue with Microsoft.</td></tr>
<tr><td>DBDC repeater and TBTC</td><td colspan="3">▪ Dynamically changing primary radio configuration and always primary configuration during runtime is not supported.
▪ If the platform is tri-radio and supports NSS Wi-Fi offload, disable NSS Wi-Fi offload for the TBTC repeater feature to work. These UCI configuration commands disable NSS Wi-Fi offload mode:
<code>uci set wireless.qcawifi=qcawifi</code>
<code>uci set wireless.qcawifi.nss_wifi_olcfg=0</code>
<code>uci commit</code>
▪ By default, wifi0 is set as primary radio. On tri-radio board, if wifi0 is disabled, and if wifi1 and wifi2 are configured as DBDC Repeater mode, then user must explicitly set wifi1 or wifi2 as primary radio.</td></tr>
</table>

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

<table>
<tr><td rowspan="8">Channel Conditions</td><td colspan="3">For peak performance results, the AP, STA, and channel conditions must be configured to:</td></tr>
<tr><td>Attribute</td><td colspan="2">Value</td></tr>
<tr><td>Image</td><td colspan="2">See Section 1.1 for the release version</td></tr>
<tr><td>AP</td><td>AP160 + CUS239</td><td>AP160 + CUS260</td></tr>
<tr><td>STA</td><td>AP160 + CUS239 in WDS mode</td><td>AP160 + CUS260 in WDS mode</td></tr>
<tr><td>RSSI of AP at STA</td><td>0x43</td><td>0x40</td></tr>
<tr><td>RSSI of STA at AP</td><td>0x43</td><td>0x40</td></tr>
<tr><td>Attenuation</td><td>32 dB per chain</td><td>40 dB per chain</td></tr>
<tr><td>NSS</td><td colspan="3">▪ Only IPv4/IPv6 + TCP/UDP flows are accelerated through fast path.
▪ LAG: Linux bonding driver configuration supported in fast path through the bonding driver sysfs IF:
▪ Mode: 802.3ad (4): xmit_hash_policy: layer2 (0), layer2+3 (2)
▪ Mode: Balance-xor (2): xmit_hash_policy: layer2 (0), layer3+4 (1), layer2+3 (2)
▪ Flows that require ALG support are not accelerated by NSS (except for TFTP (port 69))
▪ Interfaces configured with NSS qdiscs must have a leaf node configured as the default node for enqueue (using the set_default qdisc parameter); if qdisc structures are created without a default, no packets are transmitted; this includes management packets such as ARP
▪ Tunnel support (6RD, DS-Lite) expects at least the initial few packets in the LAN > WAN direction to push the base fast path rule; further flows for these tunnels can be initiated on either the WAN or LAN network
▪ IPv4 fragmented packets acceleration is not supported
▪ PPPoE-relay packets in NSS is not supported
▪ ICMPv6 is not supported
▪ IGMPv3 and MLDv2 traffic over PPPoE WAN interface is not accelerated
▪ DHCP relay over PPPoE is not supported
▪ SAMBA is not turned on by default in this release and must be enabled manually
▪ IPsec does not work with fragmentation
▪ The software is designed to work with the default configuration; thus, software configures GMAC0/GMAC1 to operate only in 1000 M mode. If the board is reconfigured to connect GMAC0/GMAC1 to a PHY instead of the switch, then software must be altered to allow speed-switching to lower Ethernet speeds. PHY ID mapping used by software must also be changed.
▪ IGMP/MLD snooping for QCA8337N does not support MLDv1 completely. Qualcomm provides a software implementation for IGMP/MLD snooping.
▪ It is recommended to disable link detection for eth0 in case eth0 is used as part of static LAG bond group between the IPQ806x and QCA8337 chips. Link detection can be controlled with ethtool commands:
▪ ethtool --set-priv-flags eth0 linkpoll off
▪ To check status: ethtool --show-priv-flags eth0</td></tr>
</table>

4.3 Power management

These power management features are enabled by default in this build:

Qualcomm® Krait™ frequency and voltage scaling on both cores	Krait frequency and voltage are scaled up and down based on CPU load. ▪ Performance governor is turned ON during the boot process ▪ On-demand governor is turned ON post-boot ▪ Voltage/frequency settings used vary by process variations ▪ Values for fast, typical, and slow parts are taken from the Krait PVS tables ▪ L2 frequencies and voltages are scaled in synch with the Krait frequencies and voltages; they also are selected based on process variations ▪ The lowest frequency Krait0 can go to is 800 MHz (which solves a problem on some test cases where the power management code does not get enough bandwidth to pull itself from the low-frequency mode); this limitation has little impact on power dissipation in the idle states
Krait clock gating is enabled	The software detects the inactivity in parts of the Krait and shuts down the clock to these parts accordingly

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Qualcomm® Krait™ frequency and voltage scaling on both cores	Krait frequency and voltage are scaled up and down based on CPU load. ▪ Performance governor is turned ON during the boot process ▪ On-demand governor is turned ON post-boot ▪ Voltage/frequency settings used vary by process variations ▪ Values for fast, typical, and slow parts are taken from the Krait PVS tables ▪ L2 frequencies and voltages are scaled in synch with the Krait frequencies and voltages; they also are selected based on process variations ▪ The lowest frequency Krait0 can go to is 800 MHz (which solves a problem on some test cases where the power management code does not get enough bandwidth to pull itself from the low-frequency mode); this limitation has little impact on power dissipation in the idle states
Frequency and voltage scaling of the multithreaded network accelerator engines processors (UBI32)	Core frequencies are scaled up and down based on load between 110 and 800 MHz; voltage is scaled between 1.05 and 1.15 V
Frequency scaling of the FABRICs	FABRICS frequencies are scaled up and down based on the load detected by the CPU. Only the APPS and NSS FABRICs are scaled. The rest of the FABRICS are left at their nominal frequency settings set at boot. VDD_CX rail to which the FABRICs are connected is scaled between 1.1 and 1.15 V accordingly. Details of the frequency values used in the scaling for each of the FABRICs are provided in the *IPQ806x Power Management Application Note.*

- VDD_CX and multi-threaded network accelerator engines processors voltage rails are combined on the AP reference design

Many power management features can be enabled and disabled with user commands (see the *IPQ806x Power Management Application Note (*80-Y6477-2)). These guidelines apply:

- Auto-scaling can be turned ON and OFF on multi-threaded network accelerator engine processors and Kraits by user commands. All features are turned ON by default.
- Details on power management in general, including frequencies and voltages used in the scaling, and information on user commands to enable and disable power management features and power configuration settings and default values.

For performance-related tests such as RFC2544, Qualcomm Technologies recommends disabling auto scaling. While several improvements have been made in auto-scaling algorithms, the RFC2544 0% stipulation is still quite demanding, especially when starting traffic burst. To disable auto scaling, execute these commands at the prompt:

```
echo “userspace” > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
echo “userspace” > /sys/devices/system/cpu/cpu1/cpufreq/scaling_governor
echo “1725000” > /sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
echo “1725000” > /sys/devices/system/cpu/cpu1/cpufreq/scaling_setspeed
echo 0 > /proc/sys/dev/nss/clock/auto_scale
echo 800000000 > /proc/sys/dev/nss/clock/current_freq
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4 Build and load the image for IPQ8064.ILQ.11.5 CSU1

1. Download the QTI proprietary code from ChipCode (see Section 4.4.1).
2. Download other components from external websites by QSDK while building the default configuration (see Section 4.4.2).
3. Generate the firmware:
   a. Reassemble the code (see Section 4.4.3.2).
   b. Create the QSDK build (see Section 4.4.3.3).
   c. Build a complete firmware image (see Section 4.4.4).
4. Install the image in the flash memory of the device and boot using the image from the flash (see Section 4.4.5).

Users should be familiar with directory structures that contain SP images for the different subsystems before downloading the code and building the images for loading. For each SP included in an SPF, SP binary files are generated from the SI binary files of only a subset of the included Sis. In an SPF, some Sis may support multiple SPs while others may only support one SP.

NOTE: The WiGig image is not part of this release.

4.4.1 Download packages available through Qualcomm ChipCode

QTI proprietary code is available from Qualcomm ChipCode. A web/GUI interface and a secure git server both allow access to this code.

- Browse available packages and obtain the download URL at: https://chipcode.qti.qualcomm.com/
- For more information on cloning the code, see: https://chipcode.qti.qualcomm.com/helpki/cloning-code-from-a-repository
- For more information on installation and configuration of the correct version of git and OpenSSL on both Windows and Linux platforms, see: https://chipcode.qti.qualcomm.com/helpki

These versions are required to support the authentication methods used by Qualcomm ChipCode.

4.4.2 Download packages from external websites

For details on downloading packages from external sites, see Section 2.5.2

4.4.3 Generate the firmware for IPQ8064.ILQ.11.5 CSU1

The firmware image can be generated either using a script or manually.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4.3.1 Script to generate complete firmware image

Meta generation script for SPF 11.5 (meta_generation_script.py) aims to simplify the release notes instructions required to be executed by the customers.

Steps to follow:

```
$ mkdir $BUILD_WS
(Assume BUILD_WS as workspace directory)
$ cd $BUILD_WS
$ git clone <OEM distro chipcode link>
<Clone required deliverable distro in the path where OEM distro is cloned
using Git clone commands and checkout(git checkout) to release tag on each
deliverable distro>
$ cd <OEM Chipcode directory>
$ git checkout r11.5_00003.0
$ rm -rf BOOT.BF.* RPM.BF.* TZ.BF.* IPQ4019.ILQ* IPQ8074.ILQ* WIGIG.TLN*
IPQ6018.ILQ* BOOT.XF* TZ.WNS* BOOT.BF.3.3.1.1 TZ.WNS.4.0 WIGIG.TLN.7.5
IPQ5018.ILQ.11.* BTFW.MAPLE.*
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
--repo-url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable
If this command does not work, please use below repo init command
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
$ repo sync -j8 --no-tags -qc
$ cd common/build
$ python meta_generation_script.py -c r11.5_00003.0 -s IPQ8064.ILQ.11.5  -p
<Profile> -b 32 -d <DELIVERABLE_COMBO> -m [MESH_OPTIONS] --path $BUILD_WS
Eg : python meta_generation_script.py -c r11.5_00003.0 -s IPQ8064.ILQ.11.5
-p P -b 32 -d HYFI,SIGMA-DUT -m SON --path /local/mnt/workspace/Build
```

Software Product	Profile and bit config	Supported Deliverables Combo	Mesh options
IPQ8064.ILQ.11.5	P- 32	HYFI,WHC,WAPID,SIGMA-DUT,EZMESH_SRC,EZMESH_BIN, EZMESH_ALG,EZ_ALG_SRC,OEM	MESH,SON, EZMESH_FULL,EZMESH_COLOCATED
	E – 32	SIGMA-DUT,OEM	NA

NOTE: If there are no additional deliverables, Please use -d OEM while triggering meta_generation_script.

NOTE: If there is no Mesh options, please remove -m <Mesh option> while triggering meta_generation_script.

Single images will be present in below directory:

```
32 Bit : $BUILD_WS/IPQ8064.ILQ.11.5/<Profile>_Timestamp/<OEM
distro>/common/build/bin
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4.3.2 Manually generate complete firmware image

This section describes the procedure to reassemble the code, create the QSDK build, and generate the single image. This example assumes that all packages listed in sections 4.4.1 and 4.4.2 are obtained using the **git clone** command and placed in the top-level directory.

1. Reassemble the code and generate the QSDK framework:

```
$ git clone <Chipcode Directory>
$ cd <Chipcode Directory>
$ git checkout r11.5_00003.0
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

   a. After the copy of the existing Git repository is completed, change the directories where the files are present as described in this table before running the repo command:

Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
qsdk-qca-wifi qsdk-qca-wlan qsdk-ieee1905-security qsdk-qca-art qsdk-whc qsdk-whcpy	NHSS.QSDK.11.5\apss_proc\out\proprietary\Wifi
qca-lib qca-mcs-apps qsdk-qca-nss qca-nss-userspace common-tools qca-time-services qca-qmi-framework gpio-debug qca-diag qca-cnss-daemon athtestcmd qsdk-qca-athdiag fw-qca-stats btdaemon minidump	NHSS.QSDK.11.5\apss_proc\out\proprietary\QSDK-Base
qca-bluetopia	NHSS.QSDK.11.5\apss_proc\out\proprietary\BLUETOPIA
qca-wifi-fw-QCA9984_hw_1-WLAN.BL.*.tar.bz2 qca-wifi-fw-AR900B_hw_2-WLAN.BL.*.tar.bz2	WLAN.BL.3.16\cnss_proc\bin\hw.1 WLAN.BL.3.16\cnss_proc\bin\hw.2
qca-wifi-fw-src-component-cmn-WLAN.BL.*.tgz qca-wifi-fw-src-component-halphy_tools-WLAN.BL.*.tgz	WLAN.BL.3.16\cnss_proc\src\components
qca-wifi-fw-AR9887_hw_1-CNSS.PS.*.tar.bz2 qca-wifi-fw-AR9888_hw_2-CNSS.PS.*.tar.bz2	CNSS.PS.3.16
qca-wifi-fw-QCA8074_v1.0-WLAN.HK.*.tar.bz2 qca-wifi-fw-QCA8074_v2.0-WLAN.HK.*.tar.bz2	WLAN.HK.2.6\wlan_proc\pkg\wlan_proc\bin\QCA8074_v1.0 WLAN.HK.2.6\wlan_proc\pkg\wlan_proc\bin\QCA8074_v2.0
qca-wifi-fw-src-component-cmn-WLAN.HK.*.tgz	WLAN.HK.2.6\wlan_proc\src\components\QCA8074_v2.0
BIN-NSS.FW.*.tar.bz2	NSS.FW.11.5\nss_proc\out\proprietary

2. After copying the necessary files to the appropriate directories, enter these commands to continue generating the QSDK framework:

```
$ rm -rf BOOT.BF.* RPM.BF.* TZ.BF.* IPQ4019.ILQ* IPQ8074.ILQ* WIGIG.TLN*
IPQ6018.ILQ* BOOT.XF* TZ.WNS* BOOT.BF.3.3.1.1 TZ.WNS.4.0 WIGIG.TLN.7.5
IPQ5018.ILQ.11.* BTFW.MAPLE.*
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
```

If this command does not work, please use below repo init command:

```
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
--repo-url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable

$ repo sync -j8 --no-tags -qc
$ mkdir -p qsdk/dl
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wifi/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wlan/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-ieee1905-security/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-art/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/common-tools/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-time-services/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-qmi-framework/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/gpio-debug/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-diag/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-cnss-daemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/athtestcmd/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qsdk-qca-athdiag/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-lib/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-mcs-apps/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qsdk-qca-nss/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-nss-userspace/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/fw-qca-stats/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/btdaemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/minidump/* qsdk
$ cp -rf apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/* qsdk
$ cp cnss_proc/bin/hw.1/* qsdk/dl
$ cp cnss_proc/bin/hw.2/* qsdk/dl
$ cp -rf wlan_proc/pkg/wlan_proc/bin/QCA8074_v1.0/qca-wifi-fw-QCA8074*-
WLAN.HK.* qsdk/dl/
$ cp -rf wlan_proc/pkg/wlan_proc/bin/QCA8074_v2.0/qca-wifi-fw-QCA8074*-
WLAN.HK.* qsdk/dl/
$ cp -rf wlan_proc/src/components/QCA8074_v2.0/qca-wifi-fw-src-component-
cmn-WLAN.HK.* qsdk/dl
$ sed -i '/QCAHKSWPL_SILICONZ/c\PKG_VERSION:=WLAN.HK.2.6-02451-
QCAHKSWPL_SILICONZ-1' qsdk/qca/feeds/qca_hk/net/qca-hk/Makefile
$ cp cnss_proc/src/components/* qsdk/dl
$ cp qca-wifi-fw-AR988* qsdk/dl/
$ cp nss_proc/out/proprietary/* qsdk/dl
```

NOTE: Prebuilt ipk images (such as Qualcomm Bluetopia or qca-mcs-apps, qca-hyd-init, qca-hyd-son, qca-hyd-map) are in their respective folders:

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

<ChipCode Directory>/qsdk/prebuilt/ipq806x_64/ipq_<profile_name>/	Contains all 64-bit ipk images
<ChipCode Directory>/qsdk/prebuilt/ipq806x/ipq_<profile_name>/	Contains all 32-bit ipk images

where ipq_<profile_name> is ipq_premium and ipq_enterprise

3. Customers with Qualcomm Hy-Fi, WHC, WAPid, or Sigma-DUT packages:

	These files are fetched from ChipCode and copied to the working QSDK top-level directory:	
HY-FI and WHC:	hyfi-ipq qsdk-whc qsdk-whcpy	NHSS.QSDK.11.5\apss_proc\out\proprietary\Hyfi NHSS.QSDK.11.5\apss_proc\out\proprietary\Wifi
WAPid:	qsdk-wapid	NHSS.QSDK.11.5\apss_proc\out\proprietary\Wapid
Sigma-DUT:	sigma-dut.tar.bz2	NHSS.QSDK.11.5\apss_proc\out\proprietary\sigma-dut
EZMESH-SRC	qsdk-ezmesh-src	NHSS.QSDK.11.5\apss_proc\out\proprietary\RSRC-EZMESH\qsdk-ezmesh-src
EZMESH-BIN	qsdk-ezmesh-bin	NHSS.QSDK.11.5\apss_proc\out\proprietary\RBIN-EZMESH\qsdk-ezmesh-bin
EZMESH-ALG	qsdk-ezmesh-alg-bin	NHSS.QSDK.11.5\apss_proc\out\proprietary\RBIN-EZMESH\qsdk-ezmesh-alg-bin
EZ-ALG-SRC	qsdk-ezmesh-alg-src	NHSS.QSDK.11.5\apss_proc\out\proprietary\RSRC-EZMESH-ALG\qsdk-ezmesh-alg-src

Then run the additional code:

HY-FI and WHC (Premium):

```
$ cp -rf apss_proc/out/proprietary/Hyfi/hyfi/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whc/* qsdk
$ mkdir qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug
$ mv qsdk/qca/feeds/qca-son-mem-debug/Makefile qsdk/qca/feeds/qca-son-mem-
   debug/Config.in qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whcpy/* qsdk
$ sed -i "s/@PACKAGE_whc-son/@PACKAGE_whc-map/g" qsdk/qca/feeds/qca-lib/qca-
   wifison-ext-lib/Makefile
```

WAPid (Premium):

```
$ git clone <wapid ChipCode Directory>
$ cd <wapid ChipCode Directory>
$ git checkout r11.5_00003.0
$ cd ..
$ cp -rf <wapid ChipCode
   Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/Wapid/qsdk-wapid/* qsdk
```

Where *wapid ChipCode Directory* is *qca-networking-2021-spf-11-5_qca_oem_wapid-src* for this release.

Sigma-DUT (Premium/Enterprise):

```
$ git clone <sigma-dut ChipCode Directory>
$ cd <sigma-dut ChipCode Directory>
$ git checkout r11.5_00003.0
$ cd ..
$ tar -xjvf <sigma-dut ChipCode
   Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/sigma-dut/sigma-
   dut.tar.bz2 -C qsdk
```

Where *sigma-dut ChipCode Directory* is *qca-networking-2021-spf-11-5_qca_oem_sigma-dut* for this release.

EZMESH-SRC (Premium):

```
#Only for Ezmesh customer

$ git clone <ezmesh ChipCode Directory>
$ cd <ezmesh ChipCode Directory>
$ git checkout r11.5_00003.0
$ cd ..
$ cp -rf <ezmesh ChipCode
  Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH/qsdk-ezmesh-
  src/* qsdk
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

	Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-src for this release
	#Below sed commands in EZMESH-SRC should be skipped if customer is using **EZ-ALG-SRC** also (i.e. both ezmesh-src and ezmesh-alg-src)
	
	$ sed -i 's/HYD_MODULE_STRATEGY=y/HYD_MODULE_STRATEGY=n/g' qsdk/qca/src/qca-ezmesh/ezmeshConfig.defs
	$ sed -i '0,/ifeq/{/ifeq/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
	$ sed -i '0,/endif/{/endif/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
	$ sed -i '/libezmeshalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
	$ sed -i '/libezmeshagentalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
	$ sed -i '/DUMP/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile

EZMESH-BIN(Premium):	#Only for Ezmesh customer
	
	$git clone <ezmesh ChipCode Directory>
	$ cd < ezmesh ChipCode Directory>
	$ git checkout r11.5_00003.0
	$ cd ..
	$ cp -rf <ezmesh ChipCode Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH/qsdk-ezmesh-bin/* qsdk
	
	Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-bin for this release

EZMESH-ALG(Premium):	#Only for Ezmesh customer
	
	$ git clone <*ezmesh ChipCode Directory*>
	$ cd < *ezmesh ChipCode Directory*>
	$ git checkout r11.5_00003.0
	$ cd ..
	$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH-ALG/qsdk-ezmesh-alg-bin/* qsdk
	
	Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-alg for this release

EZ-ALG-SRC(Premium):	#Only for Ezmesh customer
	
	$ git clone <*ezmesh ChipCode Directory*>
	$ cd < *ezmesh ChipCode Directory*>
	$ git checkout r11.5_00003.0
	$ cd ..
	$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH-ALG/qsdk-ezmesh-alg-src/* qsdk
	
	Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ez-alg-src for this release

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4.3.3 Create the QSDK build

The QSDK framework has been developed using Ubuntu (from version 16.04 to version 18.04), and Debian. However, QSDK framework regenerates critical tools required to compile firmware at build-time. In that sense, the framework is independent from the host environment; although it is developed using the distributions above, it is expected to work on others such as RedHat, Mint, or Fedora.

See Section 2.4.3.2.1 , to install the pre-requisite tools manually (or)

See Section 2.4.3.2.2, to use the script to get the pre-requisites installed in one go.

Because the framework automatically downloads the open-source components, make sure an internet connection is active on the build host while creating the build.

1. Install the different feeds in the build framework:

```
$ cd qsdk
$ ./scripts/feeds update -a
$ ./scripts/feeds install -a -f
```

2. Copy the base configuration to use for the build. Choose either **premium or enterprise** profile.

Premium (and Wi-Fi Base)	`$ cp qca/configs/qsdk/ipq_premium.config .config` `$ mv prebuilt/ipq806x/ipq_premium/* prebuilt/ipq806x/`
Enterprise	`$ cp qca/configs/qsdk/ipq_enterprise.config .config` `$ mv prebuilt/ipq806x/ipq_enterprise/* prebuilt/ipq806x/`

3. Regenerate a complete configuration file and start the build:

```
$ sed '/CONFIG_PACKAGE_kmod-qca-wigig-driver=m/d' -i .config
$ sed '/CONFIG_PACKAGE_qca-fst-manager=m/d' -i .config
$ sed '/CONFIG_PACKAGE_wigig-firmware=m/d' -i .config
$ sed '/CONFIG_PACKAGE_qca-wigig-tools=m/d' -i .config
$ sed '/CONFIG_PACKAGE_qca-wigig-debug-tools=m/d' -i .config
$ sed '/CONFIG_PACKAGE_qca-wigig-wfa-sigma=m/d' -i .config
```

Hy-Fi and WHC customers only:

- Starting with QCA_Networking_2020.SPF.11.3 release, SON and EasyMesh Features are compiled as separate build. SON build supports only SON features and EZMESH build supports only EasyMesh features.
- Starting with QCA_Networking_2021.SPF.11.4 release, MAP package qca-hyd-map is replaced with qca-ezmesh. Enable additional configuration to enable ezmesh packages.
- Use the SON package commands in this table for a SON features enabled build.
  Use the EZMESH package commands for an EasyMesh features enabled build.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Enable SON or EZMESH configuration:

SON package

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-son=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-son=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```

EZMESH package (Full Mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```

EZMESH package (Co-located mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config
```

- Use the commands in this table to enable both SON and EasyMesh Features.

Enable SON and EZMESH (full mode) configuration:

SON + EZMESH package (full mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

NOTE: Enabling both SON and EasyMesh Features in a build will increase image size and consume extra memory in Flash storage.

```
$ make defconfig
$ make V=s -j5
```

These instructions download the packages required for the corresponding profile and create the image. Once the build is complete, these files must be available in the **qsdk/bin/ipq** directory:

Bootloader	openwrt-ipq806x-u-boot.elf
SquashFS	openwrt-ipq806x-qcom-ipq8064-${board_name}-fit-uImage.itb (Kernel + dtb)openwrt-ipq806x-squashfs-root.img
UBIFS	openwrt-ipq806x-qcom-ipq8064-${board_name}-ubi-root.img

Where board_name in the image names can be any board from ipq806x family.

4.4.4 Generate a complete firmware image

IPQ806x requires multiple images to be flashed for Bootup, including SBL1, SBL2, SBL3, RPM, TZ, CDT, MIBIB, NSS Images, Kernel, Filesystem, and so on. To simplify the loading of the device using the usage from flash memory, the images are combined into a single Flattened Image Tree (FIT) image, which can be flashed into the respective partition based on user configuration. Additional tools required on the Ubuntu 16.04 64-bit machine are:

1. Install mkimage for Ubuntu 16.04:
   ```
   $ sudo apt-get install u-boot-tools
   ```
2. Install DTC:
   ```
   $ sudo apt-get install device-tree-compiler
   ```
3. Install libfdt:
   ```
   $ sudo apt-get install libfdt-dev
   ```
4. Ubuntu 16.04 requires installation of the mtd-utils package:
   ```
   $ sudo apt-get install mtd-utils
   ```
5. Install Python 2.7.
6. Install tools required to run packing single image:
   ```
   $sudo apt-get install libc6-i386
   $sudo apt-get install libgl1-mesa-dri:i386
   ```
7. Switch to the Qualcomm ChipCode directory:
   ```
   $ cd <ChipCode Directory
   ```
8. Create a dummy directory cnss_proc_ps:
   ```
   $ mkdir cnss_proc_ps
   ```
9. Copy the flash config files to **common/build/ipq**:

Premium and Enterprise

```
$ cp meta-scripts/ipq806x_standard/* common/build/ipq
```

10. Copy pack.py to the apss_proc/out/meta-scripts/ directory:
    ```
    $ mkdir -p apss_proc/out/meta-scripts
    ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ cp qsdk/qca/src/u-boot-2016/tools/pack_legacy.py apss_proc/out/meta-
scripts/pack.py
```

11. Copy the **openwrt*** images built to the **common/build/ipq** folder.
12. Run these commands to create a single image:

```
$ cp qsdk/bin/ipq/openwrt* common/build/ipq
$ cp -rf qsdk/bin/ipq/dtbs/* common/build/ipq/
$ cp -rf skales/* common/build/ipq/
$ cd common/build
$ sed '/debug/d' -i update_common_info.py
$ sed '/gcc/d' -i update_common_info.py
$ sed '/allconf/d' -i update_common_info.py
$ sed '/skales/d' -i update_common_info.py
$ sed '/lkboot/d' -i update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
```

(Where <*profile-name*> is P and E)

After these steps complete, a single image can be created in the bin/ folder.

The binary images are copied to the **ipq** directory either by the user or by the command **update_common_info.py** to create the FIT image:

```
AP145-cdt.mbn
AP145_1XX-cdt.mbn
AP148-cdt.mbn
AP148_1XX-cdt.mbn
AP160-cdt.mbn
AP160_2XX-cdt.mbn
AP161-cdt.mbn
gpt_backup0.bin
gpt_main0.bin
nor-system-partition.bin
nand-system-partition.bin
nand_bootconfig.bin
nand_bootconfig1.bin
nand_sbl1.mbn
nand_sbl2.mbn
nand_sbl3.mbn
nor_sbl1.mbn
nor_sbl2.mbn
nor_sbl3.mbn
sdcc_sbl1.mbn
sdcc_sbl2.mbn
sdcc_sbl3.mbn
ssd.mbn
tz.mbn
rpm.mbn
openwrt-ipq-ipq806x-qcom-ipq806x-akxx-fit-uImage.itb
openwrt-ipq-ipq806x-squashfs-root.img
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
openwrt-ipq-ipq806x-ubi-root.img
openwrt-ipq806x-u-boot-2016.mbn
openwrt-ipq806x_standard-u-boot-2016.mbn
```

4.4.5 Flash the complete default software image

Recalibration of the Wi-Fi transceivers is necessary for images that are flashed on the boards that were previously running QCA_Networking_2016.SPF.3.0 release or earlier. Exercise caution while performing this task. Contact Qualcomm technical support contact for more details and assistance.

4.4.5.1 Set up the flash environment

1. Ensure that the board console port is connected to the PC using these RS232 parameters:
   - 115200 bps
   - 8N1
2. Confirm that the PC is connected to the board using one of the Ethernet ports. The PC must have a TFTP server launched and listening on the interface to which the board is connected. At this stage power up the board and, after a few seconds, press any key during the countdown.

4.4.5.2 Flash the image

The **xxxx-ipq806x-single.img** is already a packed image and does not need any further packing. Start by copying the xxxx-ipq806x-single.img to the TFTP server root directory.Commands for the upgrade process:

```
set ipaddr 192.168.1.1
set serverip 192.168.1.xx (This must be the address of the TFTP server)
set ethaddr 00:aa:bb:cc:dd:ee
set bootargs console=ttyMSM0,115200n8
saveenv
ping ${serverip}
tftpboot 0x44000000 xxxx-ipq806x-single.img
```

1. If using a NOR flash, use this command:
   ```
   sf probe
   ```
2. Flash the image:
   ```
   imgaddr=0x44000000 && source $imgaddr:script
   ```
3. Power cycle the board after step 3 has completed, as indicated by printing of the U-boot prompt (may be 10s of seconds depending on image size and memory technology).

NOTE: For Bluetopia, perform step 4 and 5.

4. After the single image is loaded, TFTP this package to the AP /tmp dir from **qsdk/prebuilt/ipq806x/**:
   - bluetopia_4.2-1_ipq806x.ipk
5. Install this package from the /tmp dir:
   - opkg bluetopia_4.2-1_ipq806x.ipk

NOTE: For Qualcomm Wi-Fi SON, perform these steps 6, 7, 8.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6. Once the single image has booted, transfer SON packages for SON features or EZMESH packages for EasyMesh features to the AP /tmp directory from ./**qsdk/prebuilt/ipq806x/**.

SON package
```
qsdk/prebuilt/ipq806x/qca-hyd-init_ge474d2e-1_ipq806x.ipk
qsdk/prebuilt/ipq806x/qca-hyd-son_ge474d2e-1_ipq806x.ipk
```

EZMESH package
```
To support Full mode:
qsdk/prebuilt/ipq806x/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq806x/qca-ezmesh_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq806x/qca-ezmesh-alg_ge474d2e-1_ipq.ipk
To support Co-located mode:
qsdk/prebuilt/ipq806x/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq806x/qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq806x/qca-ezmesh-agent_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq806x/qca-ezmesh-alg_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq806x/qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk
```

NOTE: ge474d2e is the git commit ID, which may also be changed.

7. Install these packages from the /tmp directory: Choose SON Package to enable SON features, or choose EZMESH packages to enable MAP features:

SON package
```
opkg install qca-hyd-init_ge474d2e-1_ipq806x.ipk
opkg install qca-hyd-son_ge474d2e-1_ipq806x.ipk
```

EZMESH package
```
To support Full mode:
opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk
To support Co-located mode:
opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-agent_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk
```

NOTE: ge474d2e is the git commit ID, which may also be changed.

8. The HYD daemon application binary name is changed from hyd to hyd-son for the SON packages. Similarly, the WSPLCD daemon application binary name is changed from wsplcd to wsplcd-son and wsplcd-map for the SON and EZMESH packages, respectively. The symbolic link to the older binary name must be create after package installation to ensure that scripts and tools using work with the new binary name. Because application binary name has changed, a crash dump is created based on new binary name. Customers must take care of updating any scripts and tools that use or depend on the older binary name.

   Execute these commands to create a symbolic softlink to the older binary name:

SON package
```
ln -s /usr/sbin/wsplcd-son /usr/sbin/wsplcd
ln -s /usr/sbin/hyd-son /usr/sbin/hyd
```

EZMESH package
```
ln -s /usr/sbin/wsplcd-map /usr/sbin/wsplcd
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4.5.3 Upgrade the firmware

This release can upgrade images via the OpenWRT web interface without the need for a TFTP server. After using U-Boot to flash an initial image and booting the device, use the web interface for future upgrades. Upgrade the complete flash image using the image file generated by **update_common_info.py**.

Upgrading image files via the web interface takes several minutes to complete depending on factors such as memory technology, vendor, image size, browser connectivity, and network load.

- An invalid image remains in the flash memory if the upgrade process is interrupted, such as if system power is lost during upgrade or a key-press event is detected on the serial console port. In this case, a refresh of the OpenWRT login page signals flash upgrade process completion.
- If a board is configured for failsafe boot, and the U-boot environment variable *bootargs* contains any reference to the rootfs or other partitions managed by the failsafe scheme. In this case, the sysupgrade process may only be partially successful: the board may or may not boot subsequently, and the image may only be partly updated.
- To use the **sysupgrade** command from the Linux console requires the sysupgrade image to be visible:
  - Within the Linux filesystem supplied using development host ssh or tftpd server, OR
  - On an NFS server that is mounted to the local file system, OR
  - Transferred using a USB storage device
- The sysupgrade feature can only be used to update the image in the memory the board boots from and is used by the running kernel. If the board supports multiple memory technologies, use U-Boot to change the image in memories that the board did not boot from.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5 IPQ4019.ILQ.11.5 CSU1

5.1 Supported hardware for this SP

Board	Comments
AP.DK01.1	IPQ4018-based, Retail
AP.DK01.2	IPQ4028-based, Enterprise
AP.DK03.1	IPQ4018-based, Retail
AP.DK03.2	IPQ4028-based, Enterprise
AP.DK04.1	IPQ4019-based, Retail
AP.DK04.2	IPQ4029-based, Enterprise
REH172 (AP.DK05.1)	IPQ4018-based, Retail
AP.DK07.1	IPQ4019-based, Retail with CSR8811 (BT module)
AP.DK07.2	IPQ4029-based, Enterprise with CSR8811 (BT module)
AP.DK07.5	IPQ4019-SBS, Retail
AP.DK07.6	IPQ4019, Retail
RDP0321	IPQ4019.AP.DK07.1.QCA9888.XB.BSR02.1.QCA0000.SiLabs.QCA0000.MFI.1.RDP
RDP0278	IPQ4019-based, Retail

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5.2 Restrictions on software while using it for testing

<table>
<tr><td rowspan="7">Channel Conditions</td><td colspan="2">For peak performance results, the AP, STA, and channel conditions must be configured to:</td></tr>
<tr><td>Attribute</td><td>Value</td></tr>
<tr><td>Image</td><td>See Section 1.1 for the release version</td></tr>
<tr><td>AP</td><td>AP-DK04</td></tr>
<tr><td>RSSI of AP at STA</td><td>43 – 53</td></tr>
<tr><td>RSSI of STA at AP</td><td>43 – 53</td></tr>
<tr><td>Attenuation</td><td>32 dB per chain</td></tr>
<tr><td>NSS</td><td colspan="2">▪ IPQ40xx Ethernet subsystem (ESS) LAN and WAN groups to be tagged with different VLAN IDs. Two VLAN IDs (1 and 2) are reserved for the LAN and WAN groups, respectively. Different VLAN IDs (such as X and Y) can be used through this configuration:
<code>echo X > /proc/sys/net/edma/edma_default_wtag</code> (for WAN)
<code>echo Y > /proc/sys/net/edma/edma_default_ltag</code> (for LAN)
▪ Ethernet full-sized jumbo frames are not supported
▪ Enhanced DMA (EDMA) driver supports S/G through paged array of fragments (nr_frags); it does not support fraglist
▪ Auto scaling must be disabled and CPU operating at highest frequency for performance KPI measurements; in bidirectional KPI case, two flows must have two different RSS hash and different core mapping to run in multiple cores
▪ GRO on GMAC interfaces needs to be disabled for performance KPI measurements. (It is done through a script automatically)
▪ IGMP snooper supports IGMPv3/MLDv2 but it has the report suppression issue when it works in MLDv1, and it does not support the IGMP server in the LAN side
▪ Shortcut forwarding engine (SFE) can only accelerate UDP IPSEC downlink case</td></tr>
<tr><td>ESS</td><td colspan="2">▪ IPQ40xx Ethernet subsystem (ESS) LAN and WAN groups to be tagged with different VLAN IDs. Two VLAN IDs (1 and 2) are reserved for the LAN and WAN groups, respectively. Different VLAN IDs (such as X and Y) can be used through the following configuration:
<code>echo X > /proc/sys/net/edma/edma_default_wtag</code> (for WAN)
<code>echo Y > /proc/sys/net/edma/edma_default_ltag</code> (for LAN)
▪ Ethernet full-sized jumbo frames are not supported.
▪ Enhanced DMA (EDMA) driver supports S/G through paged array of fragments (nr_frags). It does not support fraglist.
▪ Auto scaling needs to be disabled and CPU operating at the highest frequency for performance KPI measurements. In bidirectional KPI case, two flows must have two different RSS hash and different core mapping to run in multiple cores.
▪ GRO on GMAC interfaces must be disabled for performance KPI measurements. (It is done through a script automatically)
▪ IGMP snooper supports IGMPv3/MLDv2 but it has the report suppression issue when it works in MLDv1. And it does not support the IGMP server on the LAN side.
▪ Shortcut Forwarding Engine (SFE) can only accelerate UDP IPSEC downlink case.</td></tr>
</table>

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

<table>
<tr><td rowspan="7">Channel Conditions</td><td colspan="2">For peak performance results, the AP, STA, and channel conditions must be configured to:</td></tr>
<tr><td>Attribute</td><td>Value</td></tr>
<tr><td>Image</td><td>See Section 1.1 for the release version</td></tr>
<tr><td>AP</td><td>AP-DK04</td></tr>
<tr><td>RSSI of AP at STA</td><td>43 – 53</td></tr>
<tr><td>RSSI of STA at AP</td><td>43 – 53</td></tr>
<tr><td>Attenuation</td><td>32 dB per chain</td></tr>
<tr><td>Wi-Fi</td><td colspan="2">Qualcomm Wireless Repeater AP (QWRAP) design limitations
▪ Supported security modes: Open, WPA2-PSK, AES, TKIP
▫ Other modes such as WDS are not supported when QWRAP/Proxy mode is enabled.
▫ Dynamic switching of modes between Proxy STA and other modes is not supported.
▪ Enterprise authentication modes are not supported.
▪ Roaming supports STA roams among WRAPs, but not between WRAP and Root AP.
▪ For a list of protocols supported in payload when MAT is used, see the AP 10.4 Programmers Guide – Wireless LAN.
▪ Cascading QWRAP is supported only with a unique MAC address defined for each proxy station in each QWRAP AP level. More than one QWRAP level is not extensively deployed and tested.
▪ QWRAP + WAPI encryption mode is not supported.
DBDC repeater and TBTC
▪ Dynamically changing primary radio configuration and <code>always primary</code> configuration during runtime is not supported.
▪ By default, wifi0 is set as primary radio. On tri-radio board, if wifi0 is disabled, and if wif1 and wifi2 are configured in DBDC Repeater mode, then the user must explicitly set wifi1 or wifi2 as primary radio.
▪ mBSSID mode not supported along with QWRAP
▪ Isolation between qwrap clients is supported only in single radio mode</td></tr>
<tr><td>PLC</td><td colspan="2">▪ PLC SON and Ethernet backhaul/loop prevention cannot co-exist. PLC SON and Ethernet backhaul are mutually exclusive.
▪ VLAN ID configuration for PLC interface is not supported in PLC SON.
▪ Avitar/EDM tool is not supported in PLC SON.
▪ Guest and private networks are not supported in PLC SON.
▪ If QCA9531 AP152 + QCA7500 (or QCA7550 or QCA7420) platform runs with 90-100% CPU utilization in SON network, there might be topology discovery entry of PLC changing frequently due to CPU busy handling traffic. The topology discovery packet is a keep-alive message between CAP and repeater.</td></tr>
</table>

5.3 Power management

ARM A7 CPU Frequency Scaling	CPU frequency can be scaled up and down based on CPU load. ▪ OnDemand governor is the default CPU frequency governor ▪ This release supports these CPU frequencies: ▫ 716 MHz ▫ 500 MHz ▫ 200 MHz ▫ 48 MHz
Dynamic clock gating is enabled	The software detects the inactivity in parts of the system and shuts down the clock to these parts accordingly

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

ARM A7 CPU Frequency Scaling	CPU frequency can be scaled up and down based on CPU load. ▪ OnDemand governor is the default CPU frequency governor ▪ This release supports these CPU frequencies: ▫ 716 MHz ▫ 500 MHz ▫ 200 MHz ▫ 48 MHz
DDR and NOC	▪ DDR runs at 537 MHz or 672 MHz depending on IP40xx package ▪ SNOC at 200 MHz ▪ PCNOC at 100 MHz ▪ Frequency scaling is not supported for DDR and NOC

For example, use these commands to switch governor and CPU frequency:

```
echo userspace > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
echo 716000 > /sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

5.4 Build and load the image for IPQ4019.ILQ.11.5 CSU1

1. Download the QTI proprietary code from ChipCode (see Section 5.4.1).
2. Download other components from external websites by QSDK while building the default configuration (see Section 5.4.2)
3. Generate the firmware:
   a. Reassemble the code (see Section 5.4.3.2)
   b. Create the QSDK build (see Section 5.4.3.3)
   c. Generate a complete firmware image (see Section 5.4.4)
4. Install the image in the device flash memory and boot using the image from the flash (see Section 5.4.5).

Users should be familiar with directory structures that contain SP images for the different subsystems before downloading the code and building the images for loading. For each SP included in an SPF, SP binary files are generated from the SI binary files of only a subset of the included Sis. In an SPF, some Sis may support multiple SPs while others may only support one SP.

5.4.1 Download packages available through Qualcomm ChipCode

Qualcomm Technologies proprietary code is available from Qualcomm ChipCode. A web/GUI interface and a secure git server both allow access to this code.

- Browse available packages and obtain the download URL at: https://chipcode.qti.qualcomm.com/
- For more information on cloning the code, see: https://chipcode.qti.qualcomm.com/helpki/cloning-code-from-a-repository
- For more information on installation and configuration of the correct version of git and OpenSSL on both Windows and Linux platforms, see: https://chipcode.qti.qualcomm.com/helpki

These versions are required to support the authentication methods used by Qualcomm ChipCode.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5.4.2 Download packages from external websites

For details on downloading packages from external sites, see Section 2.5.2.

5.4.3 Generate the firmware for IPQ4019.ILQ.11.5 CSU1

The firmware image can be generated either using a script or manually.

5.4.3.1 Script to generate complete firmware image

Meta generation script for SPF 11.5 (meta_generation_script.py) aims to simplify the release notes instructions required to be executed by the customers.

Follow these steps:

```
$ mkdir $BUILD_WS
(Assume BUILD_WS as workspace directory)
$ cd $BUILD_WS
$ git clone <OEM distro chipcode link>
<Clone required deliverable distro in the path where OEM distro is cloned
using Git clone commands and checkout(git checkout) to release tag on each
deliverable distro>
$ cd <OEM Chipcode directory>
$ git checkout r11.5_00003.0
$ rm -rf BOOT.AK.* BOOT.BF.3.3.1 RPM.* TZ.AK.* TZ.BF.4.0.8 IPQ80*
WIGIG.TLN* IPQ6018.ILQ* BOOT.XF* TZ.WNS* BOOT.BF.3.3.1.1 TZ.WNS.4.0
IPQ5018.ILQ.11.* BTFW.MAPLE.* WIGIG.TLN.7.5
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
--repo-url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable
If this command does not work, please use below repo init command
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
$ repo sync -j8 --no-tags -qc
$ cd common/build
$ python meta_generation_script.py -c r11.5_00003.0 -s IPQ4019.ILQ.11.5 -p
P -b 32 -d <DELIVERABLE_COMBO> -m [MESH_OPTIONS] --path $BUILD_WS
Eg : python meta_generation_script.py -c r11.5_00003.0 -s IPQ4019.ILQ.11.5
-p P -b 32 -d HYFI,SIGMA-DUT -m SON --path /local/mnt/workspace/Build
```

Software Product	Profile and bit config	Supported Deliverables Combo	Mesh options
IPQ4019.ILQ.11.5	P-32	HYFI,WHC,WAPID,PLC,SIGMA-DUT,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH,SON,EZMESH_FULL,EZMESH_COLOCATED

NOTE: If there are no additional deliverables, please use -d OEM while triggering meta_generation_script.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

NOTE: If there are no additional Mesh options, please remove -m <Mesh option> while triggering meta_generation_script.

Single images will be present in below directory:

```
32 Bit : $BUILD_WS/IPQ4019.ILQ.11.5/<Profile>_Timestamp/<OEM
distro>/common/build/bin
```

5.4.3.2 Manually generate complete firmware image

This section describes the procedure to reassemble the code, create the QSDK build, and generate a single image. This example assumes that all packages listed in sections 5.4.1 and 5.4.2 are obtained using the **git clone** command and placed in the top-level directory.

1. Reassemble the code and generate the QSDK framework:
```
$ git clone <Chipcode Directory>
$ cd <Chipcode Directory>
$ git checkout r11.5_00003.0
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

   a. After the copy of the existing Git repository is completed, change the directories where the files are present as described in this table before running the repo command:

Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
qsdk-qca-wifi qsdk-qca-wlan qsdk-ieee1905-security qsdk-qca-art qsdk-whc qsdk-whcpy	NHSS.QSDK.11.5\apss_proc\out\proprietary\Wifi
Athtestcmd qsdk-qca-athdiag qca-lib qca-mcs-apps common-tools qca-time-services qca-qmi-framework gpio-debug qca-diag qca-cnss-daemon fw-qca-stats btdaemon minidump	NHSS.QSDK.11.5\apss_proc\out\proprietary\QSDK-Base
qca-bluetopia	NHSS.QSDK.11.5\apss_proc\out\proprietary\BLUETOPIA
qca-wifi-fw-IPQ4019_hw_1- WLAN.BL.*.tar.bz2 qca-wifi-fw-QCA9888_hw_2- WLAN.BL.*.tar.bz2 qca-wifi-fw-QCA9984_hw_1- WLAN.BL.*.tar.bz2	WLAN.BL.3.16\cnss_proc\bin\IPQ4019\hw.1 WLAN.BL.3.16\cnss_proc\bin\QCA9888\hw.2 WLAN.BL.3.16\cnss_proc\bin\QCA9984\hw.1
qca-wifi-fw-src-component-cmn- WLAN.BL.*.tgz qca-wifi-fw-src-component-halphy_tools-WLAN.BL.*.tgz	WLAN.BL.3.16\cnss_proc\src\components
qca-wifi-fw-AR9887_hw_1-CNSS.PS.*.tar.bz2 qca-wifi-fw-AR9888_hw_2-CNSS.PS.*.tar.bz2	CNSS.PS.3.16

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
qca-wifi-fw-QCA8074_v1.0-WLAN.HK.*.tar.bz2 qca-wifi-fw-QCA8074_v2.0-WLAN.HK.*.tar.bz2	WLAN.HK.2.6\wlan_proc\pkg\wlan_proc\bin\QCA8074_v1.0 WLAN.HK.2.6\wlan_proc\pkg\wlan_proc\bin\QCA8074_v2.0
qca-wifi-fw-src-component-cmn-WLAN.HK.*.tgz	WLAN.HK.2.6\wlan_proc\src\components\QCA8074_v2.0

2. Enter these commands to continue generating the QSDK framework:

```
$ rm -rf BOOT.AK.* BOOT.BF.3.3.1 RPM.* TZ.AK.* TZ.BF.4.0.8 IPQ80* WIGIG.TLN*
  IPQ6018.ILQ* BOOT.XF* TZ.WNS* BOOT.BF.3.3.1.1 TZ.WNS.4.0 IPQ5018.ILQ.11.*
  BTFW.MAPLE.* WIGIG.TLN.7.5
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
  release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
```

If this command does not work, please use below repo init command:

```
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml -
-repo-url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable
```

```
$ repo sync -j8 --no-tags -qc
$ mkdir -p qsdk/dl
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wifi/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wlan/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-ieee1905-security/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-art/* qsdk
$ cp -rf apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/athtestcmd/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qsdk-qca-athdiag/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-lib/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-mcs-apps/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/common-tools/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-time-services/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-qmi-framework/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/gpio-debug/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-diag/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-cnss-daemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/fw-qca-stats/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/btdaemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/minidump/* qsdk
$ cp -rf wlan_proc/pkg/wlan_proc/bin/QCA8074_v1.0/qca-wifi-fw-QCA8074*-
WLAN.HK.* qsdk/dl/
$ cp -rf wlan_proc/pkg/wlan_proc/bin/QCA8074_v2.0/qca-wifi-fw-QCA8074*-
WLAN.HK.* qsdk/dl/
$ cp -rf wlan_proc/src/components/QCA8074_v2.0/qca-wifi-fw-src-
component-cmn-WLAN.HK.* qsdk/dl
$ sed -i '/QCAHKSWPL_SILICONZ/c\PKG_VERSION:=WLAN.HK.2.6-02451-
QCAHKSWPL_SILICONZ-1'  qsdk/qca/feeds/qca_hk/net/qca-hk/Makefile
$ cp cnss_proc/bin/IPQ4019/hw.1/* qsdk/dl
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ cp cnss_proc/bin/QCA9888/hw.2/* qsdk/dl
$ cp cnss_proc/bin/QCA9984/hw.1/* qsdk/dl
$ cp -rf cnss_proc/src/components/* qsdk/dl
$ cp qca-wifi-fw-AR988* qsdk/dl
```

NOTE: Prebuilt ipk images (such as Qualcomm Bluetopia or qca-mcs-apps, qca-hyd-init, qca-hyd-son, qca-hyd-map) are in their respective folders:

<ChipCode Directory>/qsdk/prebuilt/ipq40xx/ipq_<profile_name>/	Contains all 32-bit ipk images

where ipq_<profile_name> is ipq_premium,

3. (Optional) This step applies only for customers with Qualcomm HY-FI, WHC, WAPid, PLC, or Sigma-DUT:

	Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
HY-FI and WHC:	hyfi-ipq qsdk-whc qsdk-whcpy	NHSS.QSDK.11.5\apss_proc\out\proprietary\Hyfi NHSS.QSDK.11.5\apss_proc\out\proprietary\Wifi
WAPid:	qsdk-wapid	NHSS.QSDK.11.5\apss_proc\out\proprietary\Wapid
PLC:	qca-plc-ipq	NHSS.QSDK.11.5\apss_proc\out\proprietary\PLC
Sigma-DUT:	sigma-dut.tar.bz2	NHSS.QSDK.11.5\apss_proc\out\proprietary\sigma-dut
EZMESH-SRC	qsdk-ezmesh-src	NHSS.QSDK.11.5\apss_proc\out\proprietary\RSRC-EZMESH\qsdk-ezmesh-src
EZMESH-BIN	qsdk-ezmesh-bin	NHSS.QSDK.11.5\apss_proc\out\proprietary\RBIN-EZMESH\qsdk-ezmesh-bin
EZMESH-ALG	qsdk-ezmesh-alg-bin	NHSS.QSDK.11.5\apss_proc\out\proprietary\RBIN-EZMESH\qsdk-ezmesh-alg-bin
EZ-ALG-SRC	qsdk-ezmesh-alg-src	NHSS.QSDK.11.5\apss_proc\out\proprietary\RSRC-EZMESH-ALG\qsdk-ezmesh-alg-src

Run the additional code:

HY-FI and WHC:

```
$ cp -rf apss_proc/out/proprietary/Hyfi/hyfi/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whc/* qsdk
$ mkdir qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug
$ mv qsdk/qca/feeds/qca-son-mem-debug/Makefile qsdk/qca/feeds/qca-son-mem-
  debug/Config.in qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whcpy/* qsdk
$ sed -i "s/@PACKAGE_whc-son/@PACKAGE_whc-map/g" qsdk/qca/feeds/qca-lib/qca-
  wifison-ext-lib/Makefile
```

WAPid:

```
$ git clone <wapid ChipCode Directory>
$ cd <wapid ChipCode Directory>
$ git checkout r11.5_00003.0
$ cd ..
$ cp -rf <wapid ChipCode
  Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/Wapid/qsdk-wapid/* qsdk
```

Where *wapid ChipCode Directory* is *qca-networking-2021-spf-11-5_qca_oem_wapid-src* for this release.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

PLC:

```
$ git clone <PLC ChipCode Directory>
$ cd <PLC ChipCode Directory>
$ git checkout r11.5_00003.0
$ cd ..
$ cp -rf <PLC ChipCode
  Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/PLC/qca-plc-ipq/* qsdk
```

To make PLC prebuilt IPK part of build, comment these lines out in **qsdk/qca/feeds/qca-plc/qca-plc-serv/Makefile**:

```
# define Build/InstallDev
# $(INSTALL_DIR) $(1)/usr/include/plcserv
# $(foreach header_file,$(QCA_PLCSERV_HEADERS), $(CP) $(header_file)
  (1)/usr/include/plcserv;)
# endef
```

(Or) use this command:

```
$ sed -i '/InstallDev/,+3d' qsdk/qca/feeds/qca-plc/qca-plc-serv/Makefile
```

Where plc distro name is qca-networking-2021-spf-11-5_qca_oem_bin-plc for this release.

Sigma-DUT:

```
$ git clone <sigma-dut ChipCode Directory>
$ cd <sigma-dut ChipCode Directory>
$ git checkout  r11.5_00003.0
$ cd ..
$ tar -xjvf <sigma-dut ChipCode
   Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/sigma-dut/sigma-
   dut.tar.bz2 -C qsdk
```

Where *sigma-dut ChipCode Directory* is *qca-networking-2021-spf-11-5_qca_oem_sigma-dut* for this release.

EZMESH-SRC

```
#Only for Ezmesh customer

$ git clone <ezmesh ChipCode Directory>
$ cd <ezmesh ChipCode Directory>
$ git checkout r11.5_00003.0
$ cd ..
$ cp -rf <ezmesh ChipCode
  Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH/qsdk-ezmesh-
  src/* qsdk
```

Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-src for this release.

```
#Below sed commands in EZMESH-SRC should be skipped if customer is using EZ-ALG-
  SRC also (i.e. both ezmesh-src and ezmesh-alg-src)

$ sed -i 's/HYD_MODULE_STRATEGY=y/HYD_MODULE_STRATEGY=n/g' qsdk/qca/src/qca-
  ezmesh/ezmeshConfig.defs
$ sed -i '0,/ifeq/{/ifeq/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
$ sed -i '0,/endif/{/endif/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
$ sed -i '/libezmeshalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
$ sed -i '/libezmeshagentalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
$ sed -i '/DUMP/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
```

EZMESH-BIN

```
git clone <ezmesh ChipCode Directory>
$ cd < ezmesh ChipCode Directory>
$ git checkout r11.5_00003.0
$ cd ..
$ cp -rf <ezmesh ChipCode
Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH/qsdk-ezmesh-bin/*
   qsdk
```

Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-bin for this release

EZMESH-ALG

```
#Only for Ezmesh customer

$ git clone <ezmesh ChipCode Directory>
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

	$ cd < *ezmesh ChipCode Directory*>
	$ git checkout r11.5_00003.0
	$ cd ..
	$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH-ALG/qsdk-ezmesh-alg-bin/* qsdk
	Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-alg for this release

EZ-ALG-SRC	#Only for Ezmesh customer
	
	$ git clone <*ezmesh ChipCode Directory*>
	$ cd < *ezmesh ChipCode Directory*>
	$ git checkout r11.5_00003.0
	$ cd ..
	$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH-ALG/qsdk-ezmesh-alg-src/* qsdk
	Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ez-alg-src for this release

5.4.3.3 Create the QSDK build

QSDK supports one profile: **Premium.** The premium profile is targeted for 32 Mbytes flash and 256 Mbytes DDR configuration. This release distributes the Premium distro.

QSDK framework has been developed using Ubuntu (from version 16.04 to version 18.04), and Debian. However, QSDK framework regenerates critical tools required to compile firmware at build-time. In that sense, the framework is independent from the host environment; although it is developed using the distributions above, it is expected to work on others such as RedHat, Mint, or Fedora.

See 2.4.3.2.1, to install the pre-requisite tools manually (or)

See 2.4.3.2.2, to use the script to get the pre-requisites installed in one go.

Because the framework automatically downloads the open-source components, make sure an internet connection is active on the build host while creating the build.

1. Move to the qsdk dir:
   ```
   $ cd qsdk
   ```
2. Install the different feeds in the build framework:
   ```
   $ ./scripts/feeds update -a
   $ ./scripts/feeds install -a -f
   ```
3. Copy the base configuration to use for the build. Choose the premium profile.

Premium	$ cp qca/configs/qsdk/ipq_premium.config .config $ mv prebuilt/ipq40xx/ipq_premium/* prebuilt/ipq40xx/

4. Regenerate a complete configuration file:
   ```
   $ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq40xx/g" .config
   ```
5. (Optional) This step applies only for PLC customers. Enter the following commands to include PLC IPKs as part of single image:
   ```
   $ sed 's/CONFIG_PACKAGE_qca-plc-fw=m/CONFIG_PACKAGE_qca-plc-fw=y/g' -i
   .config
   $ sed 's/CONFIG_PACKAGE_qca-plc-serv=m/CONFIG_PACKAGE_qca-plc-serv=y/g'
   -i .config
   $ sed '/CONFIG_PACKAGE_kmod-qca-wigig-driver=m/d' -i .config
   $ sed '/CONFIG_PACKAGE_qca-fst-manager=m/d' -i .config
   $ sed '/CONFIG_PACKAGE_wigig-firmware=m/d' -i .config
   ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ sed '/CONFIG_PACKAGE_qca-wigig-tools=m/d' -i .config
$ sed '/CONFIG_PACKAGE_qca-wigig-debug-tools=m/d' -i .config
$ sed '/CONFIG_PACKAGE_qca-wigig-wfa-sigma=m/d' -i .config
```

Hy-Fi, WHC customers only:

- Starting with QCA_Networking_2020.SPF.11.3 release, SON and EasyMesh Features are compiled as separate build. SON build supports only SON features and EZMESH build supports only EasyMesh features.
- Starting with QCA_Networking_2021.SPF.11.4 release, MAP package qca-hyd-map is replaced with qca-ezmesh. Enable additional configuration to enable ezmesh packages.
- Use the SON package commands in this table for a SON features enabled build. Use the EZMESH package commands for an EasyMesh features enabled build.

Enable SON or EZMESH configuration:

SON package

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-son=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-son=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```

EZMESH package (Full Mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```

EZMESH package (Co-located mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config
```

- Use the commands in this table to enable both SON and EasyMesh Features.

Enable SON and EZMESH (full mode) configuration:

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

SON + EZMESH package (full mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```

NOTE: Enabling both SON and EasyMesh Features in a build will increase image size and consume extra memory in Flash storage.

```
$ make defconfig
$ make V=s -j5
```

The preceding instructions download the packages required for the corresponding profile and create the image. After the build is complete, these files are available in the **qsdk/bin/ipq** directory:

- openwrt-ipq40xx-u-boot-stripped.elf (Bootloader)
- openwrt-ipq-ipq40xx-qcom-ipq40xx-ap.dkxx-fit-uImage.itb (Kernel + dtb)
- openwrt-ipq-ipq40xx-squashfs-root.img (SquashFS)
- openwrt-ipq-ipq40xx-ubi-root.img (UBIFS)

5.4.4 Generate a complete firmware image

The IPQ40xx flash image includes multiple components. These include DTB, SBL1, TZ, CDT, DDR, MIBIB, Kernel, Filesystem, etc. To simplify the loading of images from flash memory and booting of devices, they have been combined into a single Flattened Image Tree (FIT) image. The FIT image can be flashed into the respective partition based on user configuration information. Additional tools are required on the Ubuntu 16.04 64-bit machine. To build a single image:

1. Install mkimage for Ubuntu 16.04:
   ```
   $sudo apt-get install u-boot-tools
   ```
2. Install DTC:
   ```
   $sudo apt-get install device-tree-compiler
   ```
3. Install libfdt:
   ```
   $sudo apt-get install libfdt-dev
   ```
4. Ubuntu 16.04 build hosts also require the mtd-utils package:
   ```
   $sudo apt-get install mtd-utils
   ```
5. Install Python 2.7.
6. Install tools required to run packing single image:
   ```
   $sudo apt-get install libc6-i386
   $sudo apt-get install libgl1-mesa-dri:i386
   ```
7. Switch to the Qualcomm ChipCode directory:
   ```
   $ cd <Chipcode Directory>
   ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8. Copy the root folders in top level directory:

```
$ mkdir cnss_proc_ps
```

9. Copy the flash config files to **common/build/ipq**; choose the premium profile:

Premium

```
$ cp meta-scripts/ipq40xx_premium/* common/build/ipq
```

10. Copy pack.py to the apss_proc/out/ directory:

```
$ cp ./qsdk/qca/src/u-boot-2016/tools/pack_legacy.py
apss_proc/out/pack.py
```

11. Copy TrustZone files to common/build/ipq

```
$ cp -rf trustzone_images/build/ms/bin/MAZAANAA/* common/build/ipq
```

12. Copy the openwrt* images built to the common/build/ipq folder:

```
$ cp -rf qsdk/bin/ipq/openwrt* common/build/ipq
```

13. Copy the **boardconfig*** files to the **common/build/ipq** folder; choose premium:

Premium

```
$ cp boot_images/build/ms/bin/40xx/misc/tools/config/boardconfig_premium
 common/build/ipq
$ cp boot_images/build/ms/bin/40xx/misc/tools/config/appsboardconfig_premium
 common/build/ipq
$ cp -rf qsdk/bin/ipq/dtbs/* common/build/ipq/
$ cp -rf skales/* common/build/ipq/
$ cd common/build
$ sed "s/'''$/\n'''/g" -i update_common_info.py
$ sed '/debug/d' -i update_common_info.py
$ sed '/skales/d' -i update_common_info.py
$ sed '/lkboot/d' -i update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
(Where <profile-name> is  P)
```

The commands create nor-ipq40xx-single.img, nand-ipq40xx-single.img, nornand-ipq40xx-single.img, emmc-ipq40xx-single.img and norplusemmc-single.img single images in the bin folder. The binary images are copied to the ipq directory either by the user or by the update_common_info.py command to create the FIT image:

```
cdt-AP.DK01.1-C1.bin
cdt-AP.DK01.1-C2.bin
cdt-AP.DK01.1-S1.bin
cdt-AP.DK04.1-C1.bin
cdt-AP.DK04.1-C2.bin
cdt-AP.DK04.1-C3.bin
cdt-AP.DK04.1-C5.bin
cdt-AP.DK04.1-S1.bin
cdt-AP.DK05.1-C1.bin
cdt-AP.DK06.1-C1.bin
cdt-AP.DK07.1-C1.bin
gpt_backup0.bin
gpt_main0.bin
nor-system-partition-ipq40xx-s.bin
nand-system-partition-ipq40xx.bin
nor-system-partition-ipq40xx.bin
norplusnand-system-partition-ipq40xx.bin
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
norplusemmc-system-partition-ipq40xx.bin
openwrt-ipq40xx-u-boot-2016-stripped.elf
openwrt-ipq40xx_standard-u-boot-2016-stripped.elf
openwrt-ipq-ipq40xx-ubi-root.img
openwrt-ipq-ipq40xx-ubi-root-m4096-p512KiB.img
openwrt-ipq-ipq40xx-qcom-ipq40xx-ap.dkxx-fit-uImage.itb
openwrt-ipq-ipq40xx-squashfs-root.img
sbl1_emmc.mbn
sbl1_nand.mbn
sbl1_nor.mbn
tz.mbn
nand-flash.conf
nor-flash.conf
emmc-flash.conf
norplusnand-flash.conf
norplusemmc-flash.conf
packages
```

Where:

Flash type	Single image
SPI NOR (32 Mbytes)	nor-ipq40xx-single.img
SPI NOR + NAND	nornand-ipq40xx-single.img
ONFI NAND	nand-ipq40xx-single.img
eMMC	emmc-ipq40xx-single.img
SPI NOR (16 Mbytes) + eMMC	norplusemmc-ipq40xx-single.img

5.4.5 Flash the complete default software image

Recalibration of the Wi-Fi transceivers is necessary for images that are flashed on the boards that were previously running QCA_Networking_2016.SPF.3.0 release or earlier. Exercise caution while performing this task. Contact Qualcomm technical support contact for details.

5.4.5.1 Set up the flash environment

1. Ensure that the board console port is connected to the PC using these RS232 parameters:
   - 115200 bps
   - 8N1
2. Confirm that the PC is connected to the board using one of the Ethernet ports. The PC must have a TFTP server launched and listening on the interface to which the board is connected. At this stage power up the board and, after a few seconds, press any key during the countdown.

The xxxx-ipq40xx-single.img is already a packed image and does not need any further packing.

5.4.5.2 Standard board configuration

load the image in flash and boot the platform as follows:

1. Copy **the xxxx-ipq40xx-single.img** to the TFTP server root directory.
2. Check hardware jumper configuration according to reference board and default memory configuration (see the appropriate board setup guide for more information).

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3. Confirm the machine ID, Meta version, profile, and single image.

Board/ Configuration	Machine ID	Flash	Meta/ Profile Support	Image Name
AP.DK01.1-C1	8010000	NOR 32 Mbytes	Premium profile	nor-ipq40xx-single.img
AP.DK01.1-C2	8010100	NOR (SO8) + SPI NAND (128 Mbytes)	Premium profile	nornand-ipq40xx-single.img
AP.DK04.1-C1	8010001	NOR 32 Mbytes	Premium profile	nor-ipq40xx-single.img
AP.DK04.1-C1	8010001	QPIC NAND (128 Mbytes)	Premium profile	nand-ipq40xx-single.img
AP.DK04.1-C1	8010001	eMMC	Premium profile	emmc-ipq40xx-single.img
AP.DK04.1-C1	8010001	NOR (SO8) + QPIC NAND (128 Mbytes)	Premium profile	nornand-ipq40xx-single.img
AP.DK04.1-C2	8010101	NOR 32 Mbytes – Audio	Premium profile	nor-ipq40xx-single.img
AP.DK04.1-C3	8010201	NOR (SO8) + eMMC	Premium profile	norplusemmc-ipq40xx–single.img
AP.DK04.1-C5	8010401	NOR (SO8) + SPI NAND (128 Mbytes)	Premium profile	nornand-ipq40xx-single.img
AP.DK05.1-C1	8010007	NOR (SO8) + SPI NAND (128 Mbytes)	Premium profile	nornand-ipq40xx-single.img
AP.DK07.1-C1	8010006	QPIC NAND (128 Mbytes)	Premium profile	nand-ipq40xx-single.img
AP.DK07.1-C2	8010106	QPIC NAND (128 Mbytes)	Premium profile	nand-ipq40xx-single.img
AP.DK07.1-C3	8010206	QPIC NAND (128 Mbytes)	Premium profile	nand-ipq40xx-single.img
AP.DK07.1-C4	8010306	QPIC NAND (128 Mbytes)	Premium profile	nand-ipq40xx-single.img

4. Commands for the TFTP process:

```
set ipaddr 192.168.1.11
set serverip 192.168.1.xx (The address of the TFTP server)
tftpboot 0x84000000 xxxx-ipq40xx-single.img
```

5. Flash the image:

```
imgaddr=0x84000000 && source $imgaddr:script
```

6. Power cycle the board after step 3 has completed, as indicated by printing of the U-boot prompt (may be many seconds depending on image size and memory technology).

NOTE: For Bluetopia, perform steps 7 and 8. CSRmesh™ support has been removed, complete these steps to ensure the extra packages are transferred to and installed on the target system.

7. When the new image boots, tftp these packages to the AP /tmp dir from **qsdk/prebuilt/ipq40xx/**:

```
bluetopia_4.2.1.c1_13-1_ipq.ipk
```

NOTE: 37f09b7 is git commit ID, which may also be changed.

8. Install these packages from the /tmp dir:

```
opkg install bluetopia_4.2.1.c1_13-1_ipq.ipk
```

NOTE: 37f09b7 is git commit ID, which may also be changed.

NOTE: For Wi-Fi SON, perform steps 9, 10, and 11

9. Once the single image has booted, transfer SON packages for SON features or EZMESH Packages for EasyMesh features to the AP /tmp directory from ./**qsdk/prebuilt/ipq40xx/**.

SON package
```
qsdk/prebuilt/ipq40xx/qca-hyd-init_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq40xx/qca-hyd-son_ge474d2e-1_ipq.ipk
```

EZMESH package
```
To support Full mode:
qsdk/prebuilt/ipq40xx/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq40xx/qca-ezmesh_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq40xx/qca-ezmesh-alg_ge474d2e-1_ipq.ipk
To support Co-located mode:
qsdk/prebuilt/ipq40xx/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq40xx/qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq40xx/qca-ezmesh-agent_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq40xx/qca-ezmesh-alg_ge474d2e-1_ipq.ipk
qsdk/prebuilt/ipq40xx/qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk
```

NOTE: 37f09b7 is git commit ID, which may also be changed.

Install these packages from the /tmp directory: Choose SON package to SON features or choose EZMESH packages to enable MAP features.

SON package
```
opkg install qca-hyd-init_ge474d2e-1_ipq.ipk
opkg install qca-hyd-son_ge474d2e-1_ipq.ipk
```

EZMESH package
```
To support Full mode:
opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk
To support Co-located mode:
opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-ctrl_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-agent_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk
opkg install qca-ezmesh-agentalg_ge474d2e-1_ipq.ipk
```

NOTE: ge474d2e is git commit ID, which may also be changed.

10. The HYD daemon application binary name is changed from hyd to hyd-son for the SON package. Similarly, the WSPLCD daemon application binary name is changed from wsplcd to wsplcd-son and wsplcd-map for the SON and EZMESH packages, respectively. The symbolic link to the older binary name must be create after package installation to ensure that scripts and tools using work with the new binary name. Because application binary name has changed, a crash dump is created based on new binary name. Customers must take care of updating any scripts and tools that use or depend on the older binary name.

    Execute these commands to create a symbolic softlink to the older binary name:

SON package
```
ln -s /usr/sbin/wsplcd-son /usr/sbin/wsplcd
ln -s /usr/sbin/hyd-son /usr/sbin/hyd
```

EZMESH package
```
ln -s /usr/sbin/wsplcd-map /usr/sbin/wsplcd
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Non-standard board configuration programming

The procedures for changing machine ID in U-Boot are complex and require more steps. For example: changing AP.DK01.1-C1 to AP.DK01.1-C2 on a DK01 board. AP.DK01 includes a 32 Mbytes NOR flash and a 2 Mbytes NOR flash. For a new AP.DK01 board, the 2 Mbytes NOR flash is empty.

1. To load the SPI NOR+NAND singe image from flash and boot the device:
   a. Check the hardware jumper configuration. Because the 2M NOR flash is empty, the jumper settings for AP.DK01.1-C2 cannot be used to bring up the board. Use the jumper settings for AP.DK01.1-C1 to bring up to u-boot shell. Ensure the machine ID is 0x8010000.
   b. Confirm machine ID, meta version, profile, and single image name of image to be flashed into the 2M NOR + NAND flash.
   c. Commands for setting machine ID
      ```
      setenv machid ABCDEYZ
      ```
      Example for AP.DK01.1-C1
      ```
      setenv machid 8010000
      ```
   d. Set the machine ID to 0x8010100 (AP.DK01.1-C2).
   e. Upload nornand-ipq40xx-single.img by TFTP.
   f. Flash the image.
      i Make sure the jumper settings for AP.DK01.1-C2 is used. Otherwise the 2 Mbytes NOR and NAND cannot be accessed.
      ii imgaddr=0x84000000 && source $imgaddr:script
      iii An error occurs with the log:
      ```
      "NAND erase: Size exceeds partition or device limit"
      ```
      Root cause of failure: If the running u-boot uses machine ID 0x801000, it cannot access NAND. In this case, reset the board and redo steps 4 and 5 to write the second part of the image to NAND.
      iv After board reset, new software is read and booted from the 2 Mbytes NOR flash. The running u-boot uses machine ID 0x8010100. No need to redo Step 3.
      v Once done, Step 5 will be successful.
2. To load the image to the NOR flash and boot, enter the following command:
   ```
   sf probe && imgaddr=0x84000000 && source $imgaddr:script
   ```

5.4.5.3 Upgrade the firmware

This release can upgrade images via the OpenWRT web interface without the need for a TFTP server. After using U-Boot to flash an initial image and booting the device, use the web interface for future upgrades. Upgrade the complete flash image using the image file generated by **update_common_info.py**.

Upgrading image files via the web interface takes several minutes to complete depending on factors such as memory technology, vendor, image size, browser connectivity, and network load.

- An invalid image remains in the flash memory if the upgrade process is interrupted, such as if system power is lost during upgrade or a key-press event is detected on the serial console port. In this case, a refresh of the OpenWRT login page signals flash upgrade process completion.
- If a board is configured for failsafe boot, and the U-boot environment variable *bootargs* contains any reference to the rootfs or other partitions managed by the failsafe scheme. In

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

this case, the sysupgrade process may only be partially successful: the board may or may not boot subsequently, and the image may only be partly updated.

- To use the **sysupgrade** command from the Linux console requires the sysupgrade image to be visible:
  - Within the Linux filesystem supplied using development host ssh or tftpd server, OR
  - On an NFS server that is mounted to the local file system, OR
  - Transferred using a USB storage device
- The sysupgrade feature can only be used to update the image in the memory the board boots from and is used by the running kernel. If the board supports multiple memory technologies, use U-Boot to change the image in memories that the board did not boot from.

5.4.5.4 Load RAMDisk image

1. Halt the auto-boot process at u-boot prompt by typing during the auto-boot count-down phase.
2. Execute set fdt_high 0x87000000.
3. Download ramdisk image to address 0x88000000 based on board type.
   `Tftpb 0x88000000 openwrt-ipq806x-qcom-ipq40xx-<xxx>-fit-uImage.itb`
4. Execute bootm 0x88000000.

Board/Configuration	Machine ID	Image Name
AP.DK01.1-C1	8010000	openwrt-ipq806x-qcom-ipq40xx-ap.dk01.1-c1-fit-uImage-initramfs.itb
AP.DK01.1-C2	8010100	openwrt-ipq806x-qcom-ipq40xx-ap.dk01.1-c2-fit-uImage-initramfs.itb
AP.DK04.1-C1	8010001	openwrt-ipq806x-qcom-ipq40xx-ap.dk04.1-c1-fit-uImage-initramfs.itb
AP.DK04.1-C2	8010101	openwrt-ipq806x-qcom-ipq40xx-ap.dk04.1-c2-fit-uImage-initramfs.itb
AP.DK04.1-C3	8010201	openwrt-ipq806x-qcom-ipq40xx-ap.dk04.1-c3-fit-uImage-initramfs.itb
AP.DK04.1-C5	8010401	openwrt-ipq806x-qcom-ipq40xx-ap.dk04.1-c5-fit-uImage-initramfs.itb
AP.DK05.1-C1	8010007	openwrt-ipq806x-qcom-ipq40xx-ap.dk05.1-c1-fit-uImage-initramfs.itb
AP.DK06.1-C1	8010005	openwrt-ipq806x-qcom-ipq40xx-ap.dk06.1-c1-fit-uImage-initramfs.itb
AP.DK07.1-C1	8010006	openwrt-ipq806x-qcom-ipq40xx-ap.dk07.1-c1-fit-uImage-initramfs.itb
AP.DK07.1-C2	8010106	openwrt-ipq806x-qcom-ipq40xx-ap.dk07.1-c2-fit-uImage-initramfs.itb
AP.DK07.1-C3	8010206	openwrt-ipq806x-qcom-ipq40xx-ap.dk07.1-c3-fit-uImage-initramfs.itb
AP.DK07.1-C4	8010306	openwrt-ipq806x-qcom-ipq40xx-ap.dk07.1-c1-fit-uImage-initramfs.itb

5.5 Updates for PLC Wi-Fi SON

5.5.1 ESS port mapping in REH172

On REH172, the machine ID of the device must be set to 8010007. By default, REH172 runs in Qualcomm Wi-Fi SON mode. ESS controls the physical port mappings.

Non-SON mode: ESS port mappings in REH172		
Interface	**Connectivity**	**ESS/Physical port numbers**
Eth0 On REH172 platforms for non-SON mode, only one Linux interface (eth0) is shared for both physical ports.	PLC	0, 4

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Non-SON mode: ESS port mappings in REH172		
Eth0	External LAN Port	0,5

SON mode: ESS port mappings in REH172		
Interface	**Connectivity**	**Ports connection**
Eth1	LAN	0,1,2,3,4
Eth0	WAN	0,5

Configure Wi-Fi SON mode for router mode REH172

1. Configure Wi-Fi SON mode, so that REH172 can be configured in in the router mode:
```
cd /proc/sys/net/edma/
echo 0x20 > default_group1_bmp
echo 0x1E > default_group2_bmp
echo 2 > default_group1_vlan_tag
echo 1 > default_group2_vlan_tag
cd /* go back to root dir,*/
```
2. Edit **/etc/config/network** to ensure that the ports are in the right group.
```
<snip>
config switch
        option name 'switch0'
        option reset '1'
        option enable_vlan '1'
config switch_vlan
        option device 'switch0'
        option vlan '1'
        option ports '0t 1 2 3 4'
config switch_vlan
        option device 'switch0'
        option vlan '2'
        option ports '0t 5'
<snip>
```
3. Add eth0 in the bridge using one of these commands:
   - brctl addif br-lan eth0
   - Or: uci set network.lan.ifname='eth1 eth0'
4. Save the configuration with the command: **uci commit**
5. Reboot the system after it is verified.

Configure REH172 in non-SON mode (bridge mode)

By default, REH172 runs in non-SON mode, so these commands are listed for reference. If the file **/etc/rc.local** contains other EDMA commands (such as those for SON mode), delete them from the file.

1. Configure non-SON mode:
```
cd /proc/sys/net/edma/
echo 0x3e > default_group1_bmp
echo 0x0 > default_group2_bmp
echo 0 > default_group1_vlan_tag
echo 0 > default_group2_vlan_tag
cd /* go back to root dir,*/
```
2. Edit **/etc/config/network** to ensure that the ports are in the right group.
```
<snip>
```

```
config interface 'lan'
        option ifname 'eth0'
<snip>
config switch
        option name 'switch0'
        option reset '1'
        option enable_vlan '0'
config switch_vlan
        option device 'switch0'
        option vlan '0'
        option ports '0 1 2 3 4 5'
#config switch_vlan
#option device 'switch0'
#option vlan '2'
#option ports '0t 5'
```

3. Or do a factory reset to revert to default (non-SON) mode:

```
jffs2reset
This erases all settings and removes installed packages. Are you sure?
[N/y]
Y
```

5.5.2 Enable PLC on REH172 for Wi-Fi SON and non-SON configurations

1. Install these IPKs:

```
opkg install qca-plc-fw_00041_ipq.ipk
opkg install qca-plc-fw-7500_00041_ipq.ipk
opkg install qca-plc-serv_g5a1c62a-1_ipq.ipk –force-depends
```

2. Enable PLC:

```
uci set plc.config.Enabled='1'
uci set plc.config.FwPib_Download='1'
uci set plc.config.PibPath='/lib/firmware/plc/7500/QCA7500-
 REH172_EN50561-1.pib'
uci commit plc
```

5.5.3 Included PIBs

In REH172, these PIBs are in **qca-plc-fw-7500_00041_ipq.ipk** (under **/lib/firmware/plc/7500/**) for QCA7500/QCA7520/QCA7550 support:

- QCA7550-REH172_HomePlugAV_NorthAmerica.pib
- QCA7520-REH172_HomePlugAV_NorthAmerica.pib
- QCA7500-REH172_HomePlugAV_NorthAmerica.pib
- QCA7550-REH172_EN50561-1.pib
- QCA7520-REH172_EN50561-1.pib
- QCA7500-REH172_EN50561-1.pib

To change the PIB:

1. Run the commands:

```
uci set plc.config.PibPath='/lib/firmware/plc/7500/QCA7550-
REH172_EN50561-1.pib'
uci commit
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2. Remove all the files under **/etc/plc/** directory.
3. Reboot the system.

5.5.4 Map the PLC Wi-Fi SON interface

1. Before configuring PLC for Wi-Fi SON, set the PLC interface name on CAP and RE side:

   ```
   uci set plc.config.PlcIfname='eth1'
   uci commit
   ```

   eth1 is used for PLC Interface for PLC backhaul connectivity between CAP and RE.
2. In the repeater, add PLC and one Ethernet port in the bridge networks along with STA and AP VAPs:

   ```
   uci set network.lan.ifname ='eth1 eth0'
   uci commit
   ```
3. When only PLC is used as backhaul between CAP and RE, use this command on RE for auto configuration cloning thru PLC interface:

   ```
   uci set repacd.repacd.DefaultREMode='son'
   uci commit
   ```

For more details on the interface mapping, see the *Wireless LAN Access Point (Driver Version 10.4) Programmer Guide* (80-Y8053-1).

5.6 Test the GATT profile with sample applications with onboard CSR8811 Bluetooth on AP.DK07

This release contains a preloaded Bluetooth sample application. To test GATT profile containing sample applications:

1. Launch the Bluetooth sample application from the AP after the Premium profile is built. It is in: **apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/prebuilt**
2. After the complete firmware image is built, launch the Bluetooth stack to test the GATT profile with these LinuxSPPLE sample applications:
   - The LinuxSPPLE application is intended to demonstrate the usage of the Qualcomm Bluetopia Serial Port Profile Low Energy API and relevant Bluetopia Core APIs. The application supports issuing all basic commands used by the Serial Port Profile Low Energy.
   - Launching the BT sample APP enables BT and downloads the PSKeys to the BT module.
3. Wi-Fi/BT coex PSKeys are enabled after the sample APP is launched.
4. Export the environment variable before launching the sample application.

Platform/Module	Environment variable
IPQ4019 platform DK07 or QFN BT module	**BTHOST_8311_SOC_TYPE=onboard**
IPQ8064 platform or CSP module	**BTHOST_8311_SOC_TYPE=sdio**
custom SOC GPIO settings	**BTHOST_8311_SOC_TYPE=custom**

Thus, PSKeys are read from the text file located in **/usr/bin as PS_KEY_CSR8811.txt.** Contents of the text file must be in this format:

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
# PSKEY_COEX_SCHEME
\x06\xC2\x02\x00\x09\x00\x34\x00\x03\x70\x00\x00\x80\x24\x01\x00\x00\x00
\x03\x00
# PSKEY_COEX_PIO_UNITY_3_BT_ACTIVE (BT_active)
\x05\xC2\x02\x00\x0A\x00\x35\x00\x03\x70\x00\x00\x83\x24\x02\x00\x00\x00
\x05\x00\x01\x00
# PSKEY_COEX_PIO_UNITY_3_BT_STATUS (BT_STATUS)
\x05\xC2\x02\x00\x0A\x00\x36\x00\x03\x70\x00\x00\x84\x24\x02\x00\x00\x00
\xff\xff\x01\x00
# PSKEY_COEX_PIO_UNITY_3_BT_DENY (WLAN Deny)
\x05\xC2\x02\x00\x0A\x00\x37\x00\x03\x70\x00\x00\x85\x24\x02\x00\x00\x00
\x04\x00\x01\x00
# WARM RESET
\x00\xC2\x02\x00\x09\x00\x47\x00\x02\x40\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00
```

root@openwrt:cd /usr/bin
root@openwrt: ./LinuxSPPLE 2 /dev/ttyMSM1 115200

5. Note the Bluetooth address of the module on the AP.
   - □ Bluetooth Stack ID: 1.
   - □ Device Chipset: 4.2.
     BD_ADDR: 0x00025B98E115

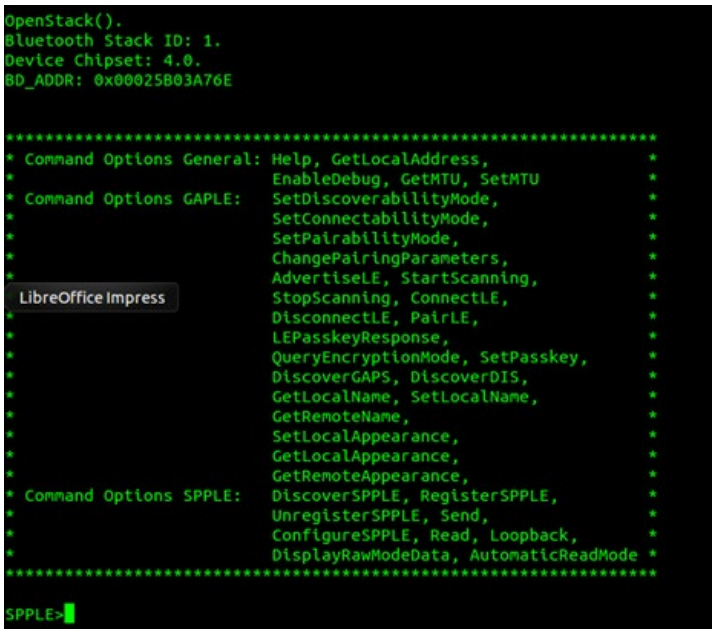
```
root@OpenWrt:/# cd /usr/bin/
root@OpenWrt:/usr/bin# ./LinuxSPPLE 2 /dev/ttyQHS0 115200

OpenStack().
HCI_VS_InitializeBeforeHCIOpen: Enter
HCI_VS_InitializeBeforeHCIOpen: Exit(0)
HCI_VS_InitializeAfterHCIOpen: Enter
HCI_VS_InitializeAfterHCIOpen: Exit
HCI_VS_InitializeBeforeHCIReset: Enter
HCI_VS_InitializeBeforeHCIReset: Exit(0)
HCI_VS_InitializeAfterHCIReset: Enter
HCI_VS_InitializeAfterHCIReset: Exit
Bluetooth Stack ID: 1.
Device Chipset: 4.2.
BD_ADDR: 0x00025B98E115


************************************************************
* Command Options General: Help, GetLocalAddress,          *
```

6. Ignore the HCI_VS print statements; the Bluetooth stack application will launch with the prompt:

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION



7. Run these commands to ensure that the radio is attached to the BT connector is good and can scan surrounding devices:
   - StartScanning – Starts scanning all Bluetooth devices with their device ID and Bluetooth address
   - StopScanning – Stops scanning devices. This command can be run during scanning.
8. Use this command to register a SPPLE service.

   ```
   SPPLE>RegisterSPPLE
   Successfully registered SPPLE Service.
   SPPLE>
   ```

   This function returns zero on successful execution and a negative value on errors. This command takes no parameters.
9. Use this command to turn on LE advertising on the device:

   ```
   AdvertiseLE 1
   GAP_LE_Advertising_Enable success.
   SPPLE>
   ```

   The only parameter required is the advertising flag which is 0 for disable and 1 for enable. This registers the device as SPPLE and clients can connect to this one.
   - Bluetooth clients
     - Bluetooth clients can use a custom App or LightBlue App on an AP.DK07 device with Bluetooth or with the CSR USB dongle 8510. LinuxSPPLE must show up in the scanning. Connect to a Bluetooth server with the Bluetooth address and send data.
     - Connecting using Bluetopia stack from another DK07 with On-board CSR 8811

     ```
     SPPLE>ConnectLE 00025B98E115   0
     ```
10. This command performs a SPPLE service discovery operation. This function returns zero on successful execution and a negative value on errors. This command takes no parameters.

    Client side:

    ```
    SPPLE>DiscoverSPPLE
    GATT_Start_Service_Discovery_Handle_Range() success.
    ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
SPPLE>
Service 0x001E – 0x0028, UUID: 14839AC47D7E415C9A42167340CF2339.

SPPLE>
Service Discovery Operation Complete, Status 0x00.

Valid SPPLE Service Found.
```

11. This command configures a SPPLE service on a remote device. This function returns zero on successful execution and a negative value on errors. This command takes no parameters.

    This function enables notifications of the proper characteristics based on a specified handle. Depending on whether the device role for SPPLE is server or client, either a GATT_Handle_Value_Notification or a GATT_Write_Without_Response_Request API function is called. The called function notifies the receiving credit characteristic or sends a write without response packet to the transmission credit characteristic respectively.

```
SPPLE>ConfigureSPPLE

SPPLE Service found on remote device, attempting to read Transmit
Credits, and configured CCCDs.

SPPLE>
Write Response.
Connection ID:   1.
```

Updates to HSUART node names since QCA_Networking_2017.SPF.5.x

Board	SP	Kernel version	Device node name
IPQ8064	5.x	3.14	/dev/ttyHS0
IPQ8064	5.x	4.4	/dev/ttyMSM1
IPQ4019	5.x	3.14	/dev/ttyQHS0
IPQ4019	5.x	4.4	/dev/ttyMSM1

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6 IPQ5018.ILQ.11.5 CSU1

6.1 Supported features

- CPU/platform
  - Quad Arm Cortex A53 at 1.0 GHz, 64-bit ISA v8 instruction set
  - 32 KB/32 KB I$/D$ and 256 KB L2$
  - Floating point and NEON SIMD DSP for each core
  - DDR3L, USB3, PCIe, all serial interfaces
- Wireless connectivity
  - Wi-Fi radio QCN9000 - 4x4, 5 GHz(RDP404)/6GHz(RDP415) with max bandwidth supported upto 160Mhz
  - Wi-Fi radio QCN61xx - 2x2, 5 GHz/6GHz with max bandwidth supported upto 160Mhz in RDP415, RDP432 and RDP403
  - Wi-Fi internal radio in IPQ5018
    - 2x2, 2.4 Ghz with max bandwidth supported upto 40 MHz in RDP404/RDP415/RDP403/RDP432
- Networking
  - Single-core multithreaded network accelerator
  - Hardware crypto offload using CE5 crypto engine
  - SGMII, SGMII+
- Standard programming model
  - Linux 4.4 (32-bit and 64-bit)
  - QSDK based on OpenWrt Chaos Calmer (CC)

6.2 Frequency spectrum and regulatory domain support

QCA_Networking_2021.SPF.11.5 release implements the regulatory versions, 34 through 36.For the AFC FR testing support FCC SP rules alone are added from Regulatory version 37. For details about the regulatory domains and mapping country codes, see *Regulatory Configuration for QCA WLAN (80-Y6630-2)*.

NOTE: Please contact customer engineering team to refer the 80-Y6630-2 document revision corresponding to the Regulatory data base versions supported in this release.

Contact Qualcomm Customer Engineering for guidance to enable 6 GHz operation for desired countries for evaluation or customer release. See the *WLAN Conformance Testing and Compliant*

Power Configuration for 802.11ax Chipsets Information to Share with Test Lab application note (80-YB952-3) and the *Wireless LAN Driver Version 11.0 for Access Point Programming Guide* (80-YA728-6) for full description of 6 GHz support on WLAN AP chipsets.

The Regulatory database updates which has been maintained in FW (2.4GHz & 5GHz) is deprecated from 11.4 release. There is no FW regulatory database maintained for 6GHz and it is maintained only in BDF.

The Regulatory database updates are done only in BDF for all 11ax chipsets and the FW database is not updated from 11.1 release onwards.

The Regulatory database will be picked up from BDF by default.

New 6 GHz rules and regulatory domain support will change frequently during 2021-2022. This status is accurate as of the date of publication of these *Release Notes*:

- Supported in this release:
  - U.S.: LPI AP
  - US: SP_AP(only for testing purpose)
  - UK: LPI and VLP
  - European Community (Draft Requirements): LPI and VLP
  - Chile and UAE: LPI
  - Guatemala: LPI and VLP
  - Korea and Brazil : LPI and VLP

6.3 Supported hardware for this SP

RD	RDP	Chipset part number
MP03.6	RDP439	IPQ5018.AP.MP03.6.QCN9012.M.2.PN02.1.RDP
MP03.1 (2.4 GHz xFEM (IPQ5018), and 5 GHz iPA (QCN9000))	RDP0404	IPQ5018 QCN9000
MP02.1 (2.4 GHz iPA (IPQ5018), 5 GHz iPA (QCN61xx), and 6 GHz iPA (QCN61xx))	RDP403	IPQ5010 QCN61xx
MP03.3 (2.4 GHz iPA (IPQ5018), 5 GHz xPA (QCN61xx), and 6 GHz xPA (QCN9000)	RDP415	IPQ5018 QCN61xx QCN9000
MP03.5 (2.4 GHz iPA (IPQ5018), 5 GHz xPA (QCN61xx), and 6 GHz xPA (QCN61xx)	RDP432	IPQ5018 QCN61xx QCN61xx

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.4 Build and load the image for IPQ5018.ILQ.11.5 CSU1

1. Download the Qualcomm Technologies, Inc. (QTI) proprietary code from Qualcomm ChipCode Portal (see section 6.4.1).

   This release contains FWA features. Enable these features by following the instructions shown in the *IPQ8074.ILQ.11.5/IPQ5018.ILQ.11.5 CS Fixed Wireless Access (FWA) (80-11333-8).*

2. Download other components from external websites for QSDK while building the default configuration (see section 6.4.1).
3. Generate the firmware:
   a. Re-assemble the code (see Section 6.4.3.1).
   b. Create the QSDK build (see section6.4.3.3).
   c. Generate a complete firmware image (see section 6.4.4)
4. Install the image in the device flash memory and boot using the image from the flash (see section 6.5).

Users should be familiar with directory structures that contain SP images for the different subsystems before downloading the code and building the images for loading. For each SP included in an SPF, SP binary files are generated from the SI binary files as only a subset of the included SIs. In an SPF, some SIs may support multiple SPs while others may only support one SP.

6.4.1 Download packages available through Qualcomm ChipCode Portal

QTI proprietary code is available from Qualcomm ChipCode Portal. A web/GUI interface and a secure git server both allow access to this code.

- Browse available packages and obtain the download URL at: https://chipcode.qti.qualcomm.com/
- For more information on cloning the code, see: https://chipcode.qti.qualcomm.com/helpki/cloning-code-from-a-repository
- For more information on installation and configuration of the correct version of git and OpenSSL on both Windows and Linux platforms, see: https://chipcode.qti.qualcomm.com/helpki

  These versions are required to support the authentication methods used by Qualcomm ChipCode Portal.

6.4.2 Download packages from external websites

For details on downloading packages from external sites, see Section 2.5.2.

6.4.3 Generate the firmware for IPQ5018.ILQ.11.5 CSU1

The firmware image can be generated either using a script or manually.

6.4.3.1 Script to generate complete firmware image:

Meta generation script for SPF 11.5 (meta_generation_script.py) aims to simplify the release notes instructions required to be executed by the customers.

Steps to follow:

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ mkdir $BUILD_WS
(Assume BUILD_WS as workspace directory)
$ cd $BUILD_WS
$ git clone <OEM distro chipcode link>
<Clone required deliverable distro in the path where OEM distro is cloned
using Git clone commands and checkout(git checkout) to release tag on each
deliverable distro>
$ cd <OEM Chipcode directory>
$ git checkout r11.5_00003.0
$ rm -rf BOOT.AK.1.0 BOOT.BF.3.3.1 BOOT.XF.0.3 BOOT.BF.3.1.1 TZ.BF*
TZ.AK.1.0 TZ.WNS.5.1 IPQ4019.ILQ.11.* IPQ80* IPQ6018.ILQ.11.* RPM.* WIGIG.*
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
--repo-url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable
If this command does not work, please use below repo init command
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
release -m caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
$ repo sync -j8 --no-tags -qc
$ cd common/build
$ python meta_generation_script.py -c r11.5_00003.0 -s IPQ5018.ILQ.11.5 -p
<Profile> -b <32/64> -d <DELIVERABLE_COMBO> -m [MESH_OPTIONS] --path
$BUILD_WS
Eg : python meta_generation_script.py -c r11.5_00003.0 -s IPQ5018.ILQ.11.5
-p P -b 64 -d HYFI,SIGMA-DUT -m SON --path /local/mnt/workspace/Build
```

Software Product	Profile and bit config	Supported Deliverables Combo	Mesh options
IPQ5018.ILQ.11.5	P – 32,64	HYFI,WHC,WAPID,SIGMA-DUT,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH, SON, EZMESH_FULL, EZMESH_COLOCATED
	LM512 – 32	HYFI,WHC,WAPID,SIGMA-DUT,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,FWA,EZ_ALG_SRC,OEM	MESH, SON, EZMESH_FULL, EZMESH_COLOCATED
	LM256 – 32	HYFI,WHC,WAPID,SIGMA-DUT,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	MESH, SON, EZMESH_FULL, EZMESH_COLOCATED
	16M – 32	HYFI,WHC,SIGMA-DUT,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	SON, EZMESH_FULL

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Software Product	Profile and bit config	Supported Deliverables Combo	Mesh options
	16M_debug	HYFI,WHC,SIGMA-DUT,EZMESH_SRC,EZMESH_BIN,EZMESH_ALG,EZ_ALG_SRC,OEM	NA

NOTE: If there are no additional deliverables, please use -d OEM while triggering meta_generation_script.

NOTE: If there are no additional Mesh options, please remove -d <Mesh option> while triggering meta_generation_script.

Single images will be present in below directory:

```
32 Bit : $BUILD_WS/IPQ5018.ILQ.11.5/<Profile>_Timestamp/<OEM distro>/common/build/bin
64 Bit : $BUILD_WS/IPQ5018.ILQ.11.5/<Profile>_Timestamp/ipq_5018*64*/<OEM distro>/common/build/bin
```

6.4.3.2 Manually generate complete firmware image

This example assumes that all packages listed in sections 6.4.1 and 6.4.2 are obtained using the **git clone** command and placed in the top-level directory.

NOTE: 64-bit single image generation requires the U-Boot elf files generated by compiling the 32-bit build. Complete the 32-bit compilation before following the 64-bit build generation steps.

1. Re-assemble the code and generate the QSDK framework:

```
$ git clone <Chipcode Directory>
$ cd <Chipcode Directory>
$ git checkout r11.5_00003.0
```

NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for this release.

After the copy of the existing git repository is completed, change the directories where the files are present as described in this table before running the Repo command:

Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
qsdk-qca-wifi qsdk-qca-wlan qsdk-qca-art qsdk-ieee1905-security qsdk-whc qsdk-whcpy	NHSS.QSDK.11.5\apss_proc\out\proprietary\Wifi

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Use git to obtain these files from ChipCode and copy them to the working QSDK top-level directory of the device:	Local directory path to files fetched by git from ChipCode:
qca-lib qca-mcs-apps qsdk-qca-nss qca-nss-userspace athtestcmd fw-qca-stats meta-tools common-tools qca-time-services qca-qmi-framework gpio-debug qca-diag qca-IOT qca-cnss-daemon btdaemon minidump	NHSS.QSDK.11.5\apss_proc\out\proprietary\QSDK-Base
qca-bluetopia	NHSS.QSDK.11.5\apss_proc\out\proprietary\BLUETOPIA
qca-wifi-fw-src-component-cmn-WLAN.HK.*.tgz	WLAN.HK.2.6\wlan_proc\src\components\QCA8074_v1.0
qca-wifi-fw-QCA8074_v1.0-WLAN.HK.*.tar.bz2	WLAN.HK.2.6\wlan_proc\pkg\wlan_proc\bin\QCA8074_v1.0
qca-wifi-fw-src-component-cmn-WLAN.BL.*.tgz qca-wifi-fw-src-component-halphy_tools-WLAN.BL.*.tgz	WLAN.BL.3.16\cnss_proc\src\components
qca-wifi-fw-QCA9888_hw_2-WLAN.BL.*.tar.bz2	WLAN.BL.3.16\cnss_proc\bin\QCA9888\hw.2
qca-wifi-fw-AR900B_hw_2-WLAN.BL.*.tar.bz2	WLAN.BL.3.16\cnss_proc\bin\AR900B\hw.2
qca-wifi-fw-QCA9984_hw_1-WLAN.BL.*.tar.bz2	WLAN.BL.3.16\cnss_proc\bin\QCA9984\hw.1
qca-wifi-fw-IPQ4019_hw_1-WLAN.BL.*.tar.bz2	WLAN.BL.3.16\cnss_proc\bin\IPQ4019\hw.1
qca-wifi-fw-AR9887_hw_1-CNSS.PS.*.tar.bz2 qca-wifi-fw-AR9888_hw_2-CNSS.PS.*.tar.bz2	CNSS.PS.3.16\
BIN-NSS.FW.*.tar.bz2	NSS.FW.11.5\nss_proc\out\proprietary
qca-afc-bin	NHSS.QSDK.11.5\apss_proc\out\proprietary\RBIN-AFCAgent

2. After copying the files to the appropriate directories, continue generating the QSDK framework:

```
$ rm -rf BOOT.AK.1.0 BOOT.BF.3.3.1 BOOT.XF.0.3 BOOT.BF.3.1.1 TZ.BF*
 TZ.AK.1.0 TZ.WNS.5.1 IPQ4019.ILQ.11.* IPQ80* IPQ6018.ILQ.11.* RPM.*
 WIGIG.*
$ cp -rf */* .
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
 release -m
 caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml
```

If this command does not work, please use below repo init command:

```
$ repo init -u git://codeaurora.org/quic/qsdk/releases/manifest/qstak -b
 release -m
 caf_AU_LINUX_QSDK_NHSS.QSDK.11.5_TARGET_ALL.11.5.0.7762.7876.xml --repo-
 url=git://codeaurora.org/tools/repo.git --repo-branch=caf-stable
$ repo sync -j8 --no-tags -qc
$ mkdir -p qsdk/dl
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-art/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wifi/* qsdk
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-qca-wlan/* qsdk
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-ieee1905-security/* qsdk
$ cp -rf wlan_proc/src/components/QCA8074_v1.0/qca-wifi-fw-src-
 component-cmn-* qsdk/dl/
$ cp -rf wlan_proc/pkg/wlan_proc/bin/QCA8074_v1.0/qca-wifi-fw-
 QCA8074_v1.0* qsdk/dl/
$ tar xvf cnss_proc/src/components/qca-wifi-fw-src-component-cmn-
 WLAN.BL.*.tgz -C qsdk/dl
$ tar xvf cnss_proc/src/components/qca-wifi-fw-src-component-
 halphy_tools-WLAN.BL.*.tgz -C qsdk/dl
$ cp -rf cnss_proc/src/components/* qsdk/dl
$ cp -rf cnss_proc/bin/QCA9888/hw.2/* qsdk/dl
$ cp -rf cnss_proc/bin/AR900B/hw.2/* qsdk/dl
$ cp -rf cnss_proc/bin/QCA9984/hw.1/* qsdk/dl
$ cp -rf cnss_proc/bin/IPQ4019/hw.1/* qsdk/dl
$ cp -rf qca-wifi-fw-AR988* qsdk/dl
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qsdk-qca-athdiag/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-lib/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/fw-qca-stats/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/meta-tools/ qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/common-tools/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qsdk-qca-nss/* qsdk/
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-mcs-apps/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-nss-userspace/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-time-services/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-qmi-framework/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/gpio-debug/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-diag/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/athtestcmd/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/btdaemon/* qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/minidump/* qsdk
$ tar xjvf apss_proc/out/proprietary/QSDK-Base/qca-IOT/qca-IOT.tar.bz2 -
 C qsdk
$ cp -rf apss_proc/out/proprietary/QSDK-Base/qca-cnss-daemon/* qsdk
$ sed -i '/QCAHKSWPL_SILICONZ/c\PKG_VERSION:=WLAN.HK.2.6-02451-
 QCAHKSWPL_SILICONZ-1' qsdk/qca/feeds/qca_hk/net/qca-hk/Makefile
$ cp -rf apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/* qsdk
$ cp nss_proc/out/proprietary/* qsdk/dl
$ cp -rf apss_proc/out/proprietary/RBIN-AFCAgent/qca-afc-bin/* qsdk
```

32-bit:

```
$ mkdir -p qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/usr/lib/
```

Premium 32-bit	$ cd qsdk/prebuilt/ipq50xx/ipq_premium
LM512 32-bit	$ cd qsdk/prebuilt/ipq50xx/ipq_512
LM256 32-bit	$ cd qsdk/prebuilt/ipq50xx/ipq_256
16M/16M Debug	*$ cd qsdk/prebuilt/ipq50xx/ipq_256* *$ cp -rf afc* location* qti-* ../ipq_16M/*

```
$ tar zxpvf qti-mfg-provision_1.0-1_ipq.ipk
$ tar zxpvf data.tar.gz
$ cp -rf ./usr/lib/libprovision.so ../../../staging_dir/target-arm_cortex-
a7_musl-1.1.16_eabi/usr/lib/
```

```
$ cd <ChipCode Directory>
NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for
this release.
$ mkdir -p qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/pkginfo/
$ touch qsdk/staging_dir/target-arm_cortex-a7_musl-1.1.16_eabi/pkginfo/qti-
mfg-provision.provides
$ echo libprovision.so > qsdk/staging_dir/target-arm_cortex-a7_musl-
1.1.16_eabi/pkginfo/qti-mfg-provision.provides
```

64 bit:

```
$  mkdir -p qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/usr/lib/
```

Premium 64-bit	`$ cd qsdk/prebuilt/ipq50xx_64/ipq_premium/`

```
$ tar zxpvf qti-mfg-provision_1.0-1_ipq.ipk
$ tar zxpvf data.tar.gz
$ cp -rf ./usr/lib/libprovision.so ../../../staging_dir/target-
aarch64_cortex-a53_musl-1.1.16/usr/lib/
$ cd <ChipCode Directory>
NOTE: Where ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem for
this release.
$ mkdir -p qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/pkginfo/
$ touch qsdk/staging_dir/target-aarch64_cortex-a53_musl-1.1.16/pkginfo/qti-
mfg-provision.provides
$ echo libprovision.so > qsdk/staging_dir/target-aarch64_cortex-a53_musl-
1.1.16/pkginfo/qti-mfg-provision.provides
```

NOTE: Prebuilt ipks (such as Qualcomm Bluetopia or qca-mcs-apps, qca-hyd-son, **qca-ezmesh, qca-ezmesh-cmn, qca-ezmesh-ctrl, qca-ezmesh-agent**) are in their respective folders.

`<ChipCode Directory>/qsdk/prebuilt/ipq50xx_64/ipq_<profile_name>/`	Contains all 64-bit ipk images
`<ChipCode Directory>/qsdk/prebuilt/ipq50xx/ipq_<profile_name>/`	Contains all 32-bit ipk images

where ipq_<profile_name> is ipq_256, ipq_512, ipq_16M and ipq_premium

3. Customers with Qualcomm HY-FI, WHC, WAPid, or Sigma DUT packages:

	These files are fetched from ChipCode and copied to the working QSDK top-level directory:		

Premium/ LM512/LM256/16M	**Hy-Fi**	`Hyfi`
		`qsdk-whc`
		`qsdk-whcpy`
			`apss_proc\out\proprietary\Hyfi`
			`apss_proc\out\proprietary\Wifi\qsdk-whc`
			`apss_proc\out\proprietary\Wifi\qsdk-whcpy`

Premium/LM512/ LM256/	**WAPid**	qsdk-wapid	apss_proc\out\proprietary\Wapid
Premium/LM256/ LM512/16M	**Sigma-DUT**	sigma-dut.tar.bz2	apss_proc\out\proprietary\sigma-dut
LM512	**FWA (5G)**	lte-data-prop modem-monitor-prop	apss_proc\out\proprietary\RSRC-LTE-DATA-PROP\lte-data-prop apss_proc\out\proprietary\RSRC-MODEM-MONITOR-PROP\modem-monitor-prop
Premium/ LM512/LM256/16M	**EZMESH-SRC**	qsdk-ezmesh-src	apss_proc\out\proprietary\RSRC-EZMESH\qsdk-ezmesh-src
Premium/ LM512/LM256/16M	**EZMESH-BIN**	qsdk-ezmesh-bin	apss_proc\out\proprietary\RBIN-EZMESH\qsdk-ezmesh-bin

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Premium/ LM512/LM256/16M	**EZMESH-ALG**	qsdk-ezmesh-alg-bin	apss_proc\out\proprietary\RBIN-EZMESH-ALG\qsdk-ezmesh-bin
Premium/ LM512/LM256/16M	**EZ-ALG-SRC**	qsdk-ez-alg-src	apss_proc\out\proprietary\RSRC-EZMESH-ALG\qsdk-ezmesh-alg-src

Then run the additional code:

Premium/ LM512/LM256/16M	**HyFI**	`$ cp -rf apss_proc/out/proprietary/Hyfi/hyfi/* qsdk`
		`$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whc/* qsdk`
		`$ mkdir qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug`
		`$ mv qsdk/qca/feeds/qca-son-mem-debug/Makefile qsdk/qca/feeds/qca-son-mem-debug/Config.in qsdk/qca/feeds/qca-son-mem-debug/qca-son-mem-debug`
		`$ cp -rf apss_proc/out/proprietary/Wifi/qsdk-whcpy/* qsdk`
		`$ sed -i "s/@PACKAGE_whc-son/@PACKAGE_whc-map/g" qsdk/qca/feeds/qca-lib/qca-wifison-ext-lib/Makefile`

Premium/LM512/ LM256/	**WAPid:**	`$ git clone <wapid ChipCode directory>`
		`$ cd <wapid ChipCode directory>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ cp -rf <wapid ChipCode directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/Wapid/qsdk-wapid/* qsdk`
		Where wapid ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_wapid-src for this release.

Premium/LM256/ LM512/16M	**Sigma-DUT:**	`$ git clone <sigma-dut ChipCode Directory>`
		`$ cd <sigma-dut ChipCode directory>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ tar -xjvf <sigma-dut ChipCode Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/sigma-dut/sigma-dut.tar.bz2 -C qsdk`
		Where sigma-dut ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_sigma-dut for this release.

Premium/LM512 LM256/16M	**EZMESH-SRC**	`#Only for Ezmesh customer`
		
		`$ git clone <`*`ezmesh ChipCode Directory`*`>`
		`$ cd <`*`ezmesh ChipCode Directory`*`>`
		`$ git checkout r11.5_00003.0`
		`$ cd ..`
		`$ cp -rf <`*`ezmesh ChipCode Directory`*`>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH/qsdk-ezmesh-src/* qsdk`
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-src for this release
		
		`#Below sed commands in EZMESH-SRC should be skipped if customer is using` **`EZ-ALG-SRC`** `also (i.e. both ezmesh-src and ezmesh-alg-src)`
		
		`$ sed -i 's/HYD_MODULE_STRATEGY=y/HYD_MODULE_STRATEGY=n/g' qsdk/qca/src/qca-ezmesh/ezmeshConfig.defs`
		`$ sed -i '0,/ifeq/{/ifeq/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile`
		`$ sed -i '0,/endif/{/endif/d;}' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile`

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

		$ sed -i '/libezmeshalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
		$ sed -i '/libezmeshagentalg \\/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile
		$ sed -i '/DUMP/d' qsdk/qca/feeds/qca-ezmesh/qca-ezmesh/Makefile

Premium/LM512 LM256/16M	**EZMESH-BIN**	#Only for Ezmesh customer
		
		$ git clone <*ezmesh ChipCode Directory*>
		$ cd < *ezmesh ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH/qsdk-ezmesh-bin/* qsdk
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-bin for this release

Premium/ LM512/ LM256/16M	**EZMESH-ALG**	#Only for Ezmesh customer
		
		$ git clone <*ezmesh ChipCode Directory*>
		$ cd < *ezmesh ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <*ezmesh ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RBIN-EZMESH-ALG/qsdk-ezmesh-alg-bin/* qsdk
		Where ezmesh ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_ezmesh-alg for this release

Premium/ LM512/ LM256/16M	**EZ-ALG-SRC**	#Only for Ezmesh customer
		
		$ git clone <ezmesh ChipCode Directory>
		$ cd < ezmesh ChipCode Directory>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -rf <ezmesh ChipCode Directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-EZMESH-ALG/qsdk-ezmesh-alg-src/* qsdk
		Where ezmesh ChipCode Direectory is qca-networking-2021-spf-11-5_qca_oem_ez-alg-src for this release

LM512	**FWA (5G):**	git clone <*FWA ChipCode Directory*>
		$ cd <*FWA ChipCode Directory*>
		$ git checkout r11.5_00003.0
		$ cd ..
		$ cp -r <*FWA ChipCode Directory*>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-LTE-DATA-PROP/lte-data-prop/* qsdk
		$ cp -r <FWA ChipCode directory>/NHSS.QSDK.11.5/apss_proc/out/proprietary/RSRC-MODEM-MONITOR-PROP/modem-monitor-prop/* qsdk
		Where FWA ChipCode Directory is qca-networking-2021-spf-11-5_qca_oem_fwa for this release.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.4.3.3 Create the 32-bit and 64-bit QSDK build

QSDK supports **Premium, LM512, LM256 and 16M** profiles for Linux kernel 4.4.60 support**.** The QSDK framework is developed using Ubuntu (from version 16.04 to version 18.04) and Debian. However, QSDK framework regenerates critical tools required to compile firmware at build time. Thus, the framework is independent from the host environment. Although it is developed using the listed distributions, it is expected to work on others such as Red Hat, Mint, or Fedora.

See Section 2.4.3.2.1 , to install the pre-requisite tools manually (or)

See Section 2.4.3.2.2, to use the script to get the pre-requisites installed in one go.

Create the QSDK build

Because framework automatically downloads the open-source components needed during the build/ make process, ensure that an Internet connection is active on the build host when creating the build.

1. Install the different feeds in the build framework:
```
$ cd qsdk
$ ./scripts/feeds update -a
$ ./scripts/feeds install -a -f
```
2. Copy the base configuration and use it to build with Linux 4.4.60 support.
3. Regenerate a complete configuration file and start the build:

Premium 32-bit
```
$ cp qca/configs/qsdk/ipq_premium.config .config
$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq50xx/g" .config
$ mv prebuilt/ipq50xx/ipq_premium/* prebuilt/ipq50xx/
```

Premium 64-bit
```
$ cp qca/configs/qsdk/ipq_premium.config .config
$ sed -i "s/TARGET_ipq_ipq806x/TARGET_ipq_ipq50xx_64/g" .config
$ mv prebuilt/ipq50xx_64/ipq_premium/* prebuilt/ipq50xx_64/
```

LM512 32-bit
```
$ cp qca/configs/qsdk/ipq_512.config .config
$ sed -i "s/TARGET_ipq_ipq807x/TARGET_ipq_ipq50xx/g" .config
$ mv prebuilt/ipq50xx/ipq_512/* prebuilt/ipq50xx/
```

LM256 32-bit
```
$ cp qca/configs/qsdk/ipq_256.config .config
$ sed -i "s/TARGET_ipq_ipq807x/TARGET_ipq_ipq50xx/g" .config
$ mv prebuilt/ipq50xx/ipq_256/* prebuilt/ipq50xx/
```

16M 32-bit
```
$ cp qca/configs/qsdk/ipq_16M.config .config
$ sed -i "s/TARGET_ipq_ipq807x/TARGET_ipq_ipq50xx/g" .config
$ sed -i "s/@PACKAGE_whc-son/@PACKAGE_whc-map/g"
package/feeds/qca_lib/qca-wifison-ext-lib/Makefile
$ mv prebuilt/ipq50xx/ipq_16M/* prebuilt/ipq50xx/
$ sed -i
's/IPQ50XX_16M_ROOTFS_SIZE=.*/IPQ50XX_16M_ROOTFS_SIZE=6656k/'
target/linux/ipq/image/Makefile
# Please change the size 6656K according to your max available size
```

16M Debug 32-bit
```
$ cp qca/configs/qsdk/ipq_16M.config .config
$ cat qca/configs/qsdk/ipq_debug_16M >> .config
$ sed -i "s/TARGET_ipq_ipq807x/TARGET_ipq_ipq50xx/g" .config
$ mv prebuilt/ipq50xx/ipq_16M/* prebuilt/ipq50xx/
```

Only Hy-Fi, WHC customers:

- Starting with QCA_Networking_2020.SPF.11.3 release, SON and EasyMesh features are compiled as separate build. SON build supports only SON features and EZMESH build supports only EasyMesh features.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- Starting with QCA_Networking_2021.SPF.11.4 release, MAP package qca-hyd-map is replaced with qca-ezmesh. Enable additional configuration to enable ezmesh packages.
- Use the SON package commands in this table for a SON features enabled build. Use the EZMESH package commands for an EasyMesh features enabled build.
- Please see section 6.4.3.3.1 to enable additional packages for 16M profile SON/EasyMesh build compilation. For Premium/LM512/LM256 profile use below configurations.

NOTE: EasyMesh specification does not support in 6G band. So EasyMesh is validated in RDP415 with 6 GHz radio disable.

Enable SON or EZMESH configuration:

SON package

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-son=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-son=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```

EZMESH package (Full Mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```

EZMESH package (Co-located mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=n" >> .config
```

- Use the commands in this table to enable both SON and EasyMesh Features.

Enable SON and EZMESH (full mode) configuration:

SON + EZMESH package (full mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=y" >> .config
Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
```

AFC customers

LM256

```
$ echo "CONFIG_PACKAGE_libcurl=m" >> .config
$ echo "CONFIG_PACKAGE_curl=m" >> .config
$ echo "CONFIG_PACKAGE_ca-bundle=m" >> .config
$ echo "CONFIG_PACKAGE_libnghttp2=m" >> .config
```

```
$ make defconfig
$ make V=s -j5
```

4. For 64-bit firmware, once the LM512 32-bit compilation completes, save all the U-Boot executable files **openwrt-ipq5018-u-boot*.elf** from the directory: **qsdk/bin/ipq.** These files are required to generate all variants of the complete 64-bit firmware image.
5. Download the required packages for the corresponding profile and create the image. Once the build is complete, these files are in the **qsdk/bin/ipq** directory:

32-bit	▪ openwrt-ipq5018*.elf (ELF files for U-Boot variants) ▪ openwrt-ipq-ipq50xx-squashfs-root.img (SquashFS) ▪ openwrt-ipq-ipq50xx-qcom-ipq50xx-mpxx-fit-uImage.itb (ITB) ▪ openwrt-ipq-ipq50xx-ubi-root.img (IMG)
64-bit	▪ openwrt-ipq-ipq50xx_64-squashfs-root.img (SquashFS) ▪ openwrt-ipq-ipq50xx_64-qcom-ipq50xx-mpxx-fit-uImage.itb (ITB) ▪ openwrt-ipq-ipq50xx_64-ubi-root.img (IMG)

6.4.3.3.1 Steps to enable additional packages for Wi-Fi SON/EasyMesh in 16M profile:

- For 16M Profile, If both SON and EasyMesh packages needs to be enabled then additional flash memory has to be planned as indicated below,
- For Low memory flash devices (for example: 16M Profile), the SON and EasyMesh Flash memory requirement indicated should be planned if either or both must be enabled:

Feature Enabled	*Additional Flash Memory Requirement (in KB)
EasyMesh (Full Mode)	300.00
SON	150.00

NOTE: The numbers noted are based on QCA_Networking_2021.SPF.11.5 release with additional packages that are mandatory for SON/EasyMesh features to function in 16M Profile 32-bit compilation.

- The following are the additional commands and packages needed for SON and EasyMesh Features,
  - Busybox commands: chown, crond, crontab, date, logger, logread, mkfifo, pgrep, pkill, time, uname, uniq, xargs

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Enable SON/EZMESH configuration using following commands:

SON package

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-son=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-son=y" >> .config
```

EZMESH package (full mode)

```
$ echo "CONFIG_PACKAGE_whc-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-mesh=n" >> .config
$ echo "CONFIG_PACKAGE_whc-son=n" >> .config
$ echo "CONFIG_PACKAGE_hyfi-son=n" >> .config
$ echo "CONFIG_PACKAGE_qca-hyd-map=n" >> .config
$ echo "CONFIG_PACKAGE_whc-map=y" >> .config
$ echo "CONFIG_PACKAGE_hyfi-map=y" >> .config

Additional configuration to enable ezmesh packages:
$ echo "CONFIG_PACKAGE_qca-ezmesh-cmn=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-alg=y" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agentalg=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-agent=n" >> .config
$ echo "CONFIG_PACKAGE_qca-ezmesh-ctrl=n" >> .config
```

Enable Busybox command configuration (SON/EasyMesh):

Busybox package

```
$ echo "CONFIG_BUSYBOX_CONFIG_CHOWN=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_CROND=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_CRONTAB=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_DATE=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_LOGGER=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_LOGREAD=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_MKFIFO=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_PGREP=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_PKILL=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_TIME=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_UNAME=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_UNIQ=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_XARGS=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_MKNOD=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_SORT=y" >> .config
$ echo "CONFIG_BUSYBOX_CONFIG_VCONFIG=y" >> .config
```

For boards without S17C switch, SSDK package is not required, SSDK package can be disable using below command,

Disable SSDK package	`$ echo "CONFIG_PACKAGE_qca-ssdk-shell=n" >> .config`

```
$ make defconfig
$ make V=s -j5
```

Continue from step 4 in Section 6.3.3.2.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.4.4 Generate a complete firmware image

IPQ50xx requires flashing multiple images for Bootup, including Trustzone (TZ), configuration data table (CDT), multi-image boot information block (MIBIB), Kernel, Filesystem, and so on. To simplify both image flashing and device boot, individual images are combined into a single Flattened Image Tree (FIT). FIT image components can be flashed into the respective partition based on user configuration. More tools required on the Ubuntu 16.04 64-bit machine are installed as follows:

1. Install DTC:
   ```
   $sudo apt-get install device-tree-compiler
   ```
2. Install libfdt:
   ```
   $sudo apt-get install libfdt-dev
   ```
3. Install Python 2.7.

To generate the complete firmware image:

4. Switch to the Qualcomm ChipCode Portal directory:
   ```
   $ cd <Chipcode Directory>
   ```
5. Copy the flash config files to **common/build/ipq**
6. Find the single images in the **common/build/bin** (For 16M debug, single image is present in common/build/bin_debug) directory. Images are generated for default image content. Fewer images indicate an issue with one of the partition sizes, or that not all required files are present

32-bit image

```
$ cd <Chipcode Directory>
$ mkdir -p common/build/ipq
$ mkdir -p apss_proc/out/meta-scripts
$ cp qsdk/qca/src/u-boot-2016/tools/pack.py apss_proc/out/meta-
scripts/pack_hk.py
$ sed -i 's#</linux_root_path>#/</linux_root_path>#' contents.xml
$ sed -i 's#</windows_root_path>#\\</windows_root_path>#' contents.xml
$ cp qsdk/bin/ipq/openwrt* common/build/ipq
$ cp -r apss_proc/out/proprietary/QSDK-Base/meta-tools apss_proc/out/
$ cp -rf qsdk/bin/ipq/dtbs/* common/build/ipq/
$ cp -rf skales/* common/build/ipq
$ cp -rf wlan_proc/build/ms/bin/5018.wlanfw.eval/* common/build/ipq/
$ cp -rf wlan_proc/build/ms/bin/5018.wlanfw2.map_spruce_eval/*
common/build/ipq/
$ cp -rf wlan_proc/build/ms/bin/5018.wlanfw2.map_spr_spr_eval_cs/*
common/build/ipq/
```

```
(Please run below command only for 16M profiles)
$ cp -rf wlan_proc/build/ms/bin/5018.wlanfw.eval/LM-16M/* common/build/ipq/
```

```
$ cp -rf btfw_proc/out/IPQ5018/bin/FW_IMAGES/bt_fw_patch_* common/build/ipq
$ cp qsdk/staging_dir/host/bin/ubinize common/build/ipq/
$ cd common/build/
$ sed "s/'''$/\n'''/g" -i update_common_info.py
$ sed -i "s/os.chdir(ipq_dir)//" update_common_info.py
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

P,16M, LM256 and LM512:

```
$ sed -i -e '/update_common_info.py: 16M debug/,+5d' update_common_info.py
$ sed
'/packages/d;/"ipq5018_64"/d;/t32/d;/ret_prep_64image/d;/Required/d;/skales
/d;
/nosmmu/d;/os.system(cmd)/d;/os.chdir(ipq_dir)/d;/atfdir/d;/noac/d;/single-
atf/d;/bl31.mbn/d;/bin_atf/d;/ret_pack_64image/d;/list_out_64_single/d;/lis
t_out_64_apps/d;/dict_64_bit_single/d;/dict_64_bit_apps/d;/list_out_debug/d
' -i update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
```

(Where <profile-name> is P/LM512/LM256/16M)

16M debug profile:

```
$ sed '/ipq_dir,ipq_dir/d;/"tiny-
nor"/d;/packages/d;/"ipq5018_64"/d;/t32/d;/ret_prep_64image/d;/Required/d;/
skales/d;
/nosmmu/d;/os.system(cmd)/d;/os.chdir(ipq_dir)/d;/atfdir/d;/noac/d;/single-
atf/d;/bl31.mbn/d;/bin_atf/d;/ret_pack_64image/d;/list_out_64_single/d;/lis
t_out_64_apps/d;/dict_64_bit_single/d;/dict_64_bit_apps/d' -i
update_common_info.py
$ sed -e '/os.chdir(".\/..")/d' -i update_common_info.py
$ sed -i -e '/for i in fltype: #list_out_32_single/,+15d'
update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
```

(Where <profile-name> is 16M (Use profile-name 16M for both 16M and 16M debug profiles)

64-bit image

```
$ cd <Chipcode directory>
$ mkdir -p common/build/ipq_x64
$ mkdir -p apss_proc/out/meta-scripts
$ cp qsdk/qca/src/u-boot-2016/tools/pack.py apss_proc/out/meta-
scripts/pack_hk.py
$ sed -i 's#</linux_root_path>#/</linux_root_path>#' contents.xml
$ sed -i 's#</windows_root_path>#\\</windows_root_path>#' contents.xml
$ cp qsdk/bin/ipq/openwrt* common/build/ipq_x64
Copy the saved openwrt-ipq5018-u-boot*.elf from the 32-bit build to:
common/build/ipq_x64
$ cp -r apss_proc/out/proprietary/QSDK-Base/meta-tools apss_proc/out/
$ cp -rf qsdk/bin/ipq/dtbs/* common/build/ipq_x64/
$ cp -rf skales/* common/build/ipq_x64/
$ cp -rf wlan_proc/build/ms/bin/5018.wlanfw.eval/* common/build/ipq_x64/
$ cp -rf wlan_proc/build/ms/bin/5018.wlanfw2.map_spruce_eval/*
common/build/ipq_x64/
$ cp -rf wlan_proc/build/ms/bin/5018.wlanfw2.map_spr_spr_eval_cs/*
common/build/ipq_x64/
$ cp -rf btfw_proc/out/IPQ5018/bin/FW_IMAGES/bt_fw_patch_*
common/build/ipq_x64
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
$ cp qsdk/staging_dir/host/bin/ubinize common/build/ipq_x64
$ cd common/build
$ sed "s/'''$/\n'''/g" -i update_common_info.py
$ sed -i "s/os.chdir(ipq_dir)//" update_common_info.py
$ sed
'/debug/d;/packages/d;/"ipq5018"/d;/t32/d;/ret_prep_32image/d;/Required/d;/
nosmmu/d
;/os.system(cmd)/d;/skales/d;/os.chdir(ipq_dir)/d;/atfdir/d;/noac/d;/single
-
atf/d;/bl31.mbn/d;/bin_atf/d;/ret_pack_32image/d;/list_out_32_single/d;/lis
t_out_32_apps/d;/dict_32_bit_single/d;/dict_32_bit_apps/d' -i
update_common_info.py
$ export BLD_ENV_BUILD_ID=<profile-name>
$ python update_common_info.py
```

(Where <profile-name> is P).

6.4.5 Flash the complete default software image

6.4.5.1 Set up the flash environment

1. Ensure that the board console port is connected to the PC using these RS232 parameters:
   - 115200 bps
   - 8N1
2. Confirm that the PC is connected to the board using one of the Ethernet ports. The PC must have a TFTP server launched and listening on the interface to which the board is connected. The interface must have an IP address configured manually. At this stage powerup the board and after a few seconds, press any key during the countdown.

6.4.5.2 Standard board configuration: load the image in flash and boot the platform

1. Copy the intended **xxxx-ipq5018-single.img** (or 64-bit equivalent file) to the TFTP server root directory.
2. Check hardware jumper configuration according to reference board and default memory configuration (see the appropriate board setup guide for more information) to verify which flash memory the board is booting from.
3. Confirm the machine ID, Meta version, profile, and selected single image to match the memory type that the board boots from.
4. Set the IP address and server IP using the TFTP process:
   ```
   setenv ipaddr 192.168.1.11
   setenv serverip 192.168.1.xx (TFTP server address)
   ping ${serverip}
   set bootargs console=ttyMSM0,115200n8
   tftpboot xxxx-ipq5018-single.img
   ```
5. Flash the image:
   ```
   imgaddr=$fileaddr && source $imgaddr:script
   ```
6. Power cycle the board after step 5 has completed, as indicated, by printing of the U-Boot prompt. It may be hundreds of seconds depending on image size and memory technology.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7. For Arithmetic exception issue in the fw_printenv/fw_setenv commands in eMMC boot, run these commands:

```
cat /etc/fw_env.config |  awk '{$NF="";sub(/[ \t]+$/,"")}1'
 >/tmp/fw_env.config
mv /tmp/fw_env.config /etc/fw_env.config
```

NOTE: For Bluetopia, perform steps 8 and 9.

8. Once the single image has booted, transfer these packages to the AP /tmp directory from ./**qsdk/prebuilt/ipq50xx/** or ./**qsdk/prebuilt/ ipq50xx _64/**, depending on whether the booted image is 32-bit or 64-bit.

```
qsdk/prebuilt/ipq50xx/bluetopia_4.2.1.c1_20-1_ipq.ipk
qsdk/prebuilt/ipq50xx_64/bluetopia_4.2.1.c1_20-1_ipq.ipk
```

9. Install these packages from the /tmp directory:

```
opkg install bluetopia_4.2.1.c1_20-1_ipq.ipk
```

NOTE: For Wi-Fi SON, perform these steps 10, 11, 12 (for all profiles) and perform Step 13 (only for 16M profile).

10. Once the single image has booted, transfer SON packages for SON features or transfer EZMESH packages for EasyMesh features to the AP /tmp directory from ./**qsdk/prebuilt/ipq50xx/** or ./**qsdk/prebuilt/ ipq50xx _64/**, depending on whether the booted image is 32-bit or 64-bit.

SON package (32-bit)	qsdk/prebuilt/ipq50xx/qca-hyd-init_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq50xx/qca-hyd-son_ge474d2e-1_ipq.ipk
SON package (64-bit)	qsdk/prebuilt/ipq50xx_64/qca-hyd-init_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq50xx_64/qca-hyd-son_ge474d2e-1_ipq.ipk
EZMESH package (32-bit)	**To support Full mode:** qsdk/prebuilt/ipq50xx/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq50xx/qca-ezmesh_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq50xx/qca-ezmesh-alg_ge474d2e-1_ipq.ipk
EZMESH package (64-bit)	**To support Full mode:** qsdk/prebuilt/ipq50xx_64/qca-ezmesh-cmn_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq50xx_64/qca-ezmesh_ge474d2e-1_ipq.ipk qsdk/prebuilt/ipq50xx_64/qca-ezmesh-alg_ge474d2e-1_ipq.ipk

NOTE: ge474d2e is git commit ID, which may also be changed.

11. Install the packages from the /tmp directory: Choose SON package to enable SON features or choose EZMESH packages for enabling MAP features.

SON package	opkg install qca-hyd-son_ge474d2e-1_ipq.ipk
EZMESH package	**To support Full mode:** opkg install qca-ezmesh-cmn_ge474d2e-1_ipq.ipk opkg install qca-ezmesh_ge474d2e-1_ipq.ipk opkg install qca-ezmesh-alg_ge474d2e-1_ipq.ipk

NOTE: ge474d2e is the git commit ID, which may also be changed.

12. The HYD daemon application binary name is changed from hyd to hyd-son for the SON package. Similarly, the WSPLCD daemon application binary name is changed from wsplcd to wsplcd-son and wsplcd-map for the SON and EZMESH packages, respectively. The symbolic

link to the older binary name must be create after package installation to ensure that scripts and tools using work with the new binary name. Because application binary name has changed, a crash dump is created based on new binary name. Customers must take care of updating any scripts and tools that use or depend on the older binary name.

Execute these commands to create a symbolic softlink to the older binary name:

SON package
```
ln -s /usr/sbin/wsplcd-son /usr/sbin/wsplcd
ln -s /usr/sbin/hyd-son /usr/sbin/hyd
```

EZMESH package
```
ln -s /usr/sbin/wsplcd-map /usr/sbin/wsplcd
```

13. For 16M profile configuration, If SON and/or EZMESH packages are built as loadable modules and packages are not part of the build image. Install the packages present in the build workspace location **qsdk/bin/ipq/packages/**.

    Transfer these packages from **qsdk/bin/ipq/packages/** and install them using **opkg install <ipk>**. Daemon applications are separated into SON and EZMESH packages. SON packages can be installed to enable SON features and EZMESH packages can be installed for EasyMesh features. Init package such as qca-hyd-init is required for SON. Install packages such as qca-wsplcd-init, and qca-whc-repacd-init are required for both SON and EasyMesh packages.

    - HYD (qca-hyd-son, qca-ezmesh)
    - WSPLCD (qca-wsplcd-son, qca-wsplcd-map)
    - REPACD (qca-whc-repacd-son, qca-whc-repacd-map)

NOTE: Installing both SON and EZMESH packages shall make both SON and EasyMesh Features to work in a device, but additional memory must be planned. To switch between SON to EZMESH or EZMESH to SON execution, the symbolic link must be set properly.

SON Packages Installation command:
opkg install qca-ieee1905-init_1_ipq.ipk
opkg install kmod-qca-hyfi-bridge_4.4.60+g9f77ad1-1_ipq.ipk
opkg install qca-libhyfi-bridge_g415362b-1_ipq.ipk
opkg install libieee1905_g143b629-1_ipq.ipk
opkg install qca-hyctl_g576c5f7-1_ipq.ipk
opkg install libwifisoncfg_g1fc9273-1_ipq.ipk
opkg install libwpa2_g2605878-1_ipq.ipk
opkg install qca-whc-init_1_ipq.ipk
opkg install jansson_2.7-1_ipq.ipk
opkg install qca-whc-lbd_gf525639-1_ipq.ipk
opkg install qca-hyd-init_g39cc435-1_ipq.ipk
opkg install qca-hyd-son_g39cc435-1_ipq.ipk
opkg install libstorage_gd8dc4f7-1_ipq.ipk
opkg install qca-wsplcd-init_gf7d67c8-1_ipq.ipk
opkg install qca-wsplcd-son_gf7d67c8-1_ipq.ipk
opkg install qca-ssdk-shell_g5661366-1_ipq.ipk
opkg install libevent2_2.1.11-1_ipq.ipk
opkg install bridge_1.5-3_ipq.ipk --force-overwrite

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

SON Packages Installation command:
opkg install lldpd_0.7.15-1_ipq.ipk
opkg install qca-whc-repacd-son_ge19bc32-1_ipq.ipk
opkg install qca-whc-repacd-init_ge19bc32-1_ipq.ipk

NOTE: Packages needs to be installed in the same order provided in the table above to install as per the dependency order

MAP Packages Installation command:
opkg install libwifisoncfg_g1fc9273-1_ipq.ipk
opkg install libwpa2_g2605878-1_ipq.ipk
opkg install kmod-qca-hyfi-bridge_4.4.60+g9f77ad1-1_ipq.ipk
opkg install qca-libhyfi-bridge_g415362b-1_ipq.ipk
opkg install qca-hyctl_g576c5f7-1_ipq.ipk
opkg install jansson_2.7-1_ipq.ipk
opkg install bridge_1.5-3_ipq.ipk --force-overwrite
opkg install qti-autorole_gba3e2ce-1_ipq.ipk
opkg install qca-ezmesh-cmn_g6b7fbed-1_ipq.ipk
opkg install qca-ezmesh_g6b7fbed-1_ipq.ipk
opkg install qca-ezmesh-alg_g6b7fbed-1_ipq.ipk
opkg install qca-whc-init_1_ipq.ipk
opkg install qca-ieee1905-init_1_ipq.ipk
opkg install libstorage_gd8dc4f7-1_ipq.ipk
opkg install libieee1905_g143b629-1_ipq.ipk
opkg install qca-wsplcd-init_gf7d67c8-1_ipq.ipk
opkg install qca-wsplcd-map_gf7d67c8-1_ipq.ipk
opkg install ethtool_3.18-1_ipq.ipk
opkg install qca-ssdk-shell_g5661366-1_ipq.ipk (only for platform with S17C switch)
opkg install qca-whc-repacd-map_ge19bc32-1_ipq.ipk
opkg install qca-whc-repacd-init_ge19bc32-1_ipq.ipk

NOTE: Packages needs to be installed in the same order provided in the table above to install as per the dependency order

Package type	Package Name	Package path in NHSS
Common packages	kmod-qca-hyfi-bridge	qsdk/bin/ipq/packages/hyfi
	qca-libhyfi-bridge	qsdk/bin/ipq/packages/hyfi
	qca-hyctl	qsdk/bin/ipq/packages/hyfi
	libstorage	qsdk/bin/ipq/packages/ieee1905_security
	libieee1905	qsdk/bin/ipq/packages/ieee1905_security
	libwpa2	qsdk/bin/ipq/packages/ieee1905_security
	qca-ieee1905-init	qsdk/bin/ipq/packages/ieee1905_security

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Package type	Package Name	Package path in NHSS
	qca-wsplcd-init	qsdk/bin/ipq/packages/ieee1905_security
	libwifisoncfg	qsdk/bin/ipq/packages/qca_lib
	qca-whc-init	qsdk/bin/ipq/packages/whc
	qca-whc-repacd-init	qsdk/bin/ipq/packages/whc
	qca-ssdk-shell	qsdk/bin/ipq/packages/ssdk
	jansson	qsdk/bin/ipq/packages/packages
	libevent2	qsdk/bin/ipq/packages/base
	bridge	qsdk/bin/ipq/packages/packages
EasyMesh Packages	ethtool	qsdk/bin/ipq/packages/packages
	qti-autorole	qsdk/bin/ipq/packages/qca_lib
	qca-wsplcd-map	qsdk/bin/ipq/packages/ieee1905_security
	qca-whc-repacd-map	qsdk/bin/ipq/packages/whc
	qca-ezmesh	qsdk/bin/ipq/packages/qca_ezmesh
	qca-ezmesh-cmn	qsdk/bin/ipq/packages/qca_ezmesh
	qca-ezmesh-alg	qsdk/bin/ipq/packages/qca_ezmesh
SON Packages	lldpd	qsdk/bin/ipq/packages/base
	qca-whc-lbd	qsdk/bin/ipq/packages/whc
	qca-hyd-init	qsdk/bin/ipq/packages/hyfi
	qca-hyd-son	qsdk/bin/ipq/packages/hyfi
	qca-wsplcd-son	qsdk/bin/ipq/packages/ieee1905_security
	qca-whc-repacd-son	qsdk/bin/ipq/packages/whc

NOTE: Package version is git commit ID, which may also be changed.

NOTE: Packages need to be installed in the same order, in-order to avoid dependency failure. For example: qca-wsplcd-son package depends on following packages qca-ieee1905-init, libhyficommon, libstorage, libwpa2, libieee1905, libwifisoncfg. So dependency packages need to be installed before qca-wsplcd-son.

6.4.5.3 Upgrade the firmware

This release can upgrade board images without a TFTP server. After using U-Boot to flash an initial image and booting the device, use the web interface for future upgrades. These future upgrades can take place from either the OpenWrt web interface or serial console. Upgrade the existing flash image using either the appropriate single image file or the apps image file generated by **update_common_info.py**.

Upgrading image files via the web interface takes several minutes to complete depending on factors such as memory technology, vendor, image size, browser connectivity, and network load. Completion of the flash upgrade process is signaled by the refresh of the OpenWrt login page, or lack of further messages on the serial console.

- An invalid image remains in the flash memory if the upgrade process is interrupted, such as if system power is lost during upgrade or a keypress event is detected on the serial console port.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- If a board is configured for failsafe boot, and the U-Boot environment variable *bootargs* contains any reference to the rootfs or other partitions managed by the failsafe scheme. In this case, the sysupgrade process may only be partially successful: the board may or may not boot later, and the image may only be partly updated.
- To use the **sysupgrade** command from the Linux console, sysupgrade image is required to be visible:
  - Within the Linux file system supplied using development host ssh or tftpd server, OR
  - On an NFS server that is mounted to the local file system, OR
  - Transferred using a USB storage device
- The sysupgrade feature can only be used to update the image in the memory the board boots from and is used by the running kernel. If the board supports multiple memory technologies, use U-Boot to change the image in memories that the board did not boot from.

See the *IPQ50xx SoC Software User Guide* (80-16052-17) for more information on sysupgrade support.

6.4.6 Create customized IPQ50xx Wi-Fi firmware images

Flash memory layout includes a dedicated partition for the Wi-Fi firmware and BDF information. The release includes squashfs and ubifs images in the final image created by the last steps of Section 2.5.5. Use these steps to update or recreate the squashfs/ubifs Wi-Fi firmware image and the complete final image.

NOTE: These steps are not needed if the BDF and PIL files are not customized.

1. Install these tools in the workstation:
   - mtd-utils: sudo apt-get install mtd-utils
   - mksquashfs4 binary from:
     < *ChipCode Directory*> /common/build to /usr/bin or /usr/sbin or any location included in $PATH
2. Download the WLAN IPQ5018 firmware BIN tar ball from the corresponding SP release distro.
3. Extract the firmware binary tarball:
```
mkdir -p <Chipcode Directory>/fwtemp
cp -rfv <Chipcode Directory>/wlan_proc/pkg/wlan_proc/bin/QCA5018_v1.0/qca-wifi-fw-
 QCA5018_v1.0-WLAN.HK.2.6.r3-xxxx-QCAHKSWPL_SILICONZ-X.tar.bz2 fwtemp
cd fwtemp
tar -xvf qca-wifi-fw-QCA5018_v1.0-WLAN.HK.2.6.r3-xxxx-QCAHKSWPL_SILICONZ-
 X.tar.bz2
```
4. Create a staging directory to hold the BD and PIL files.
```
mkdir staging_dir
cp -rfv qca-wifi-fw-WLAN.HK.2.6.r3-xxxx-QCAHKSWPL_SILICONZ-X/PIL_IMAGES/*
 staging_dir
cp -rfv qca-wifi-fw-WLAN.HK.2.6.r3-xxxx-QCAHKSWPL_SILICONZ-X/bdwlan*
 staging_dir
```
   (Replace the BDF file with the custom versions as needed.)
5. To generate the firmware squash image for NAND/**NOR/EMMC** (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):
```
mksquashfs4 staging_dir/ wifi_fw.squashfs -nopad -noappend -root-owned -comp
   xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -processors 1
dd if=wifi_fw.squashfs of=wifi_fw_squashfs.img bs=2k conv=sync
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6. To create the final single image with the updated or recreated firmware image:
```
cp <Chipcode Directory>/fwtemp/Wi-Fi_fw_squashfs.img <Chipcode
   Directory>/wlan_proc/build/ms/bin/FW_IMAGES/
cp <Chipcode Directory>/fwtemp/Wi-Fi_fw_ubi.img <Chipcode
   Directory>/wlan_proc/build/ms/bin/FW_IMAGES/
```
7. Run this command again:
```
export BLD_ENV_BUILD_ID=<profile-name>
python update_common_info.py
```
Where <*profile-name*> is LM512.

NOTE: For instructions on how to flash the firmware images separately, see the *IPQ50xx SoC Software User Guide* (80-16052-17).

6.4.7 Create customized IPQ50xx + QCN90xx Wi-Fi firmware images

Flash memory layout includes a UBI volume for the Wi-Fi firmware and BDF information. The release includes squashfs images in the final image created by the last steps of Section 2.5.5 Use these steps to update or recreate the squashfs Wi-Fi firmware image and the complete final image.

NOTE: These steps are not needed if the BDF and PIL files are not customized.

1. Install these tools in the workstation:
   - mtd-utils:
```
$sudo apt-get install mtd-utils
```
   - mksquashfs4 binary from:
     < ChipCode Directory> /common/build to /usr/bin or /usr/sbin or any location included in $PATH
2. Download the WLAN SquashFS image wifi_fw_ipq5018_qcn9000_squashfs.img from **<ChipCode Directory>** /wlan_proc/build/ms/bin/FW_IMAGES/
3. Extract the squashfs image in a temp directory.
```
mkdir -p fwtemp
cp <download path>/wifi_fw_ipq5018_qcn9000_squashfs.img ./fwtemp
cd fwtemp
unsquashfs wifi_fw_ipq5018_qcn9000_squashfs.img
```
4. Copy/modify any files in ./squashfs-root folder.
5. To generate the firmware SquashFS image for NAND/NOR/EMMC (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):
```
mksquashfs4 squashfs-root/ wifi_fw.squashfs -nopad -noappend -root-owned
-comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -
processors 1
dd if=wifi_fw.squashfs of= wifi_fw_ipq5018_qcn9000_squashfs.img bs=2k
conv=sync
```
6. UBI firmware image for NAND will no longer be required as WiFi image will be added as UBI volume and will be ubinized along with Kernel and rootfs.

For instructions on how to generate ubinized image and flash the firmware images separately, see the *IPQ50xx SoC Software User Guide* (80-16052-17)

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.4.8 Create customized IPQ50xx + QCN61xx Wi-Fi firmware images

Flash memory layout includes a dedicated partition for the Wi-Fi firmware and BDF information. The release includes squashfs and ubifs images in the final image created by the last steps of section 2.5.5. Use these steps to update or recreate the squashfs/ubifs Wi-Fi firmware image and the complete final image.

NOTE: These steps are not needed if the BDF and PIL files are not customized.

1. Install these tools in the workstation:
   - mtd-utils: sudo apt-get install mtd-utils
   - mksquashfs4 binary from:
     < *ChipCode Directory*> /common/build to /usr/bin or /usr/sbin or any location included in $PATH
2. For **RDP403 and RDP432**, download the WLAN SquashFS image `wifi_fw_ipq5018_qcn6122cs_squashfs.img` from <Chipcode_Dir>/WLAN.HK\wlan_proc\build\ms\bin\5018.wlanfw2.map_spr_spr_eval_cs
3. For **RDP415**, download the WLAN SquashFS image `wifi_fw_ipq5018_qcn9000_qcn6122_squashfs.img` from <Chipcode_Dir>/WLAN.HK\wlan_proc\build\ms\bin\ 5018.wlanfw2.map_spruce_eval
4. Extract the squashfs image in a temp directory.

RDP403 and RDP432:

```
mkdir -p fwtemp
cp <download path>/wifi_fw_ipq5018_qcn6122cs_squashfs.img  ./fwtemp
cd fwtemp
unsquashfs wifi_fw_ipq5018_qcn6122cs_squashfs.img
```

RDP415:

```
mkdir -p fwtemp
cp <download path>/ wifi_fw_ipq5018_qcn9000_qcn6122_squashfs.img
./fwtemp
cd fwtemp
unsquashfs wifi_fw_ipq5018_qcn9000_qcn6122_squashfs.img
```

All extracted files will be in./squashfs-root folder.

5. Copy/modify any files in*./squashfs-root* folder.
6. To generate the firmware SquashFS image for NAND/NOR/EMMC (this example assumes mksquashfs4 is in $PATH; use the exact installed path of mksquashfs4):

RDP403 and RDP432:

```
mksquashfs4 squashfs-root/ wifi_fw.squashfs -nopad -noappend -root-owned
-comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -
processors 1
dd if=wifi_fw.squashfs of=wifi_fw_ipq5018_qcn6122cs_squashfs.img bs=2k
conv=sync
```

RDP415:

```
mksquashfs4 squashfs-root/ wifi_fw.squashfs -nopad -noappend -root-owned
-comp xz -Xpreset 9 -Xe -Xlc 0 -Xlp 2 -Xpb 2 -Xbcj arm -b 256k -
processors 1
```

```
dd if=wifi_fw.squashfs of=wifi_fw_ipq5018_qcn9000_qcn6122_squashfs.img
bs=2k conv=sync
```

NOTE: For instructions on how to flash the firmware images separately, see section 6.5 below.

6.5 Flash Wi-Fi firmware image only

The Wi-Fi firmware/BDF information stored in the flash image can be updated without having to rewrite the whole flash image as shown in this section. Regardless of the memory type the system is booting from, follow the same basic process of using tftpboot to load the new Wi-Fi firmware image to DDR, then identify the correct location in the flash and write the Wi-Fi firmware image to that location.

- wifi_fw_ipq5018_qcn9000_squashfs.img is IPQ5018 Wi-Fi (internal 2 GHz) and QCN90xx integrated firmware image.
- wifi_fw_squashfs.img is IPQ5018 (internal 2 GHz) Wi-Fi firmware only image.

  The IPQ5018 Wi-Fi firmware only image is the squashfs image. The squashfs image generated for RDP403,RDP432 and RDP415 are as follows.
- RDP403/RDP432 - IPQ5018 internal 2G radio and QCN61xx 5/6 GHz external radio
- RDP415 - IPQ5018 internal 2G radio, QCN61xx 5 GHz external radio and QCN90xx 6GHz external radio

NOR+NAND boot/NAND boot

The Wi-Fi firmware image is made as UBI volume, along with kernel, rootfs and BT firmware to save flash space.

- TFTP the image (**wifi_fw_ubi.img**):
  ```
  # tftpboot wifi_fw_ipq5018_qcn9000_squashfs.img
  ```
- Flash the partition using the flash command:
  ```
  # flash wifi_fw
  ```
  Where wifi_fw is the Wi-Fi UBI volume name. This name is used for flash command.

NOR boot/NOR+eMMC boot/eMMC boot

- TFTP the image (wifi_fw_squashsfs.img):
  ```
  # tftpboot wifi_fw_ipq5018_qcn9000_squashfs.img
  ```
- Flash the partition using the flash command:
  ```
  # flash 0:WIFIFW
  ```

NOTE: 0:WIFIFW is the Wi-Fi partition name. This name is used for flash command.

NOR+NAND boot/NAND boot (RDP415 and RDP432)

The Wi-Fi firmware image is created as UBI volume, along with the kernel, rootfs, and BT firmware to save flash space.

NOTE: The Wi-Fi UBI volume name is wifi_fw. This name is used for flash command.

- TFTP the image (wifi_fw_ipq5018_qcn9000_qcn6122_squashfs.img): ***(For RDP415)***
  ```
  # tftpboot wifi_fw_ipq5018_qcn9000_qcn6122_squashfs.img
  ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- TFTP the image (wifi_fw_ipq5018_qcn6122cs_squashfs.img): ***(For RDP432)***
  ```
  # tftpboot wifi_fw_ipq5018_qcn6122cs_squashfs.img
  ```
- Flash the partition using flash command.
  ```
  # flash wifi_fw
  ```

NOR boot/NOR+eMMC boot/eMMC boot (RDP403)

NOTE: The Wi-Fi partition name is 0:WIFIFW. This name is used for flash command.

- TFTP the image (wifi_fw_ipq5018_qcn6122cs_squashfs.img):
  ```
  # tftpboot wifi_fw_ipq5018_qcn6122cs_squashfs.img
  ```
- Flash the partition using flash command.
  ```
  # flash 0:WIFI
  ```

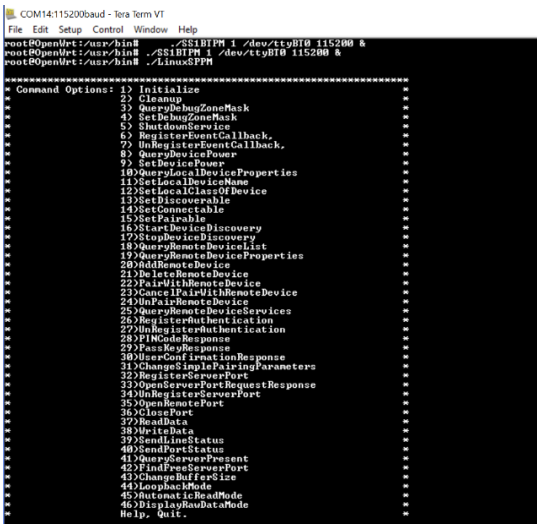
6.6 Generate secure boot image

For more information on how to generate secureboot images, see the *IPQ50xx Secure Boot Enablement User Guide* (80-16052-18).

6.7 Test the serial port profile (SPP) over generic access profile (GAP) (BR/EDR) profile with sample applications with onboard Bluetooth on AP.MP03.1

This release contains a preloaded Bluetooth sample application. To test serial port profile (SPP) profile containing sample applications:

1. Launch the Bluetooth sample application from the AP after the 512M profile is built. It is in:
   32-bit: **apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/prebuilt/ipq50xx**
   64-bit: **apss_proc/out/proprietary/BLUETOPIA/qca-bluetopia/prebuilt/ipq50xx_64**
2. Launch the Bluetooth stack to test the SPP profile with these LinuxSPPM sample applications:
   - The LinuxSPPM application is intended to demonstrate the usage of the Qualcomm Bluetopia Serial Port Profile API and relevant Bluetopia Core APIs. The application supports issuing all the basic commands used by the Serial Port Profile.
   - Launching the BT sample APP, which initializes the BT controller
     ```
     root@openwrt:cd /usr/bin
     root@openwrt: ./SS1BTPM 1 /dev/ttyBT0 115200 &
     root@openwrt: ./LinuxSPPM
     ```



3. Initialize the application with the inter-process connection to the SS1BTPM server application. The parameter determines whether the application will display events.

```
SPPM > Initialize 1
BTPM_Initialize()
Success: 0.
DEVM_RegisterEventCall back()  Success: 5.
```

4. Set the Bluetooth radio power state. The parameter defines the state used to the set the power.

```
SPPM>SetDevicePower 1
DEVM_PowerOnDevice() Success: 0.
```

5. Set the device discoverability. The first parameter is the flag to set discoverability. The second parameter is the duration in seconds for the device to remain discoverable.

```
SPPM>SetDiscoverable 1 0
Attempting to set Discoverability Mode: General.

Local Device Properties Changed.
Disc. Mode:   TRUE , 0x00000000
SPPM>DEVM_UpdateLocalDeviceProperties()
Success: 0.
```

6. Set the device connectability. The first parameter is the flag to set connectability. The second parameter is the duration in seconds for the device to remain connectable.

```
SPPM>SetConnectable 1 0
Attempting to set Connectability Mode: Connectable.
DEVM_UpdateLocalDeviceProperties() Success: 0.
SPPM> Local Device Properties Changed
```

7. Set the device pairability. The first parameter is the flag to set pairability. The second parameter is the duration in seconds for the device to remain pairable.

```
SPPM>SetPairable 1 0
Attempting to set Pairability Mode: Pairable.
DEVM_UpdateLocalDeviceProperties() Success: 0.
```

8. Register the app to receive authentication notifications. This command takes no parameters

```
SPPM>RegisterAuthentication
DEVM_RegisterAuthentication() Success: 4.
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9. Register a SPP server port. The first parameter is RFCOMM port handle to register. The second parameter is a set of flags to enable authorization, authentication or encryption.

```
SPPM>RegisterServerPort 5
SPPM_RegisterServerPort(5) Success. Port Handle: 2.
SPPM_RegisterServerPortServiceRecord Success. Record Handle: 65544
(0x00010008).
```

   The port handle will be used by the client

10. Open an SPP port on a specified remote device. The first parameter is the Bluetooth address of the remote device. The second parameter is the RFCOMM port to use. The third parameter is a flag to use authentication or encryption.

```
SPPM>OpenRemotePort 5CF37082FAE9 5
Remote Device Found.
BD_ADDR:      5CF37082FAE9
COD:           0x000000
Device Name:
Device Flags:  0x80000000
RSSI:          0
Friendly Name:
App. Info:   : 00000000
Paired State : FALSE
Connect State: FALSE
Encrypt State: FALSE
Sniff State  : FALSE
Serv. Known  : FALSE
SPPM>SPPM_OpenRemotePort(1) Success. Port Handle: 1.
SPPM>
Remote Device Properties Changed.
BD_ADDR:      5CF37082FAE9
Device Flags:  0x80400048
Connect State: TRUE
SPPM>
Remote Device Properties Changed.
BD_ADDR:      5CF37082FAE9
Device Name:   SS1BTPM Device
Device Flags:  0x80400049
```

11. Issue a response to a secure simple pairing request. The parameter sets whether the pairing is accepted.

```
SPPM>UserConfirmationResponse 1
DEVM_AuthenticationResponse(), User Confirmation Response Success.
SPPM>
Remote Device Properties Changed.
BD_ADDR:      5CF37082FAE9
Device Flags:  0x804000CD
Paired State : TRUE
SPPM>
Remote Device Properties Changed.
BD_ADDR:      5CF37082FAE9
Device Flags:  0x804000DD
Encrypt State: TRUE
SPPM>
Remote Port Open Status.
Port Handle: 4
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
Status:      Remote Port Open Successful
SPPM>
Port Status Changed.
Port Handle: 4
Port Status:
Mask:         0x00000003
BreakSignal:  0
BreakTimeout: 0
```

12. Write data to a remote SPP device. The first parameter is the port handle to write. The second parameter is the data to send. The third parameter is a timeout to try to write the data.

```
SPPM>WriteData 4 100
Success, sending data.
SPPM>Transmit thread started.
Send complete.
Transmit thread stopped.
```

13. The following command reads data that has been sent by the remote SPP device. The first parameter is the port handle to read. The second parameter is the amount of data to read. The third parameter is a timeout to try to read the data.

```
SPPM>ReadData 4 100
SPPM_ReadData(100) Success: 100 bytes read.
Data:0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0A 0x0B 0x0C
0x0D 0x0E 0x0F
0x10 0x11 0x12 0x13 0x14 0x15 0x16 0x17 0x18 0x19 0x1A 0x1B 0x1C 0x1D
0x1E 0x1F
0x20 0x21 0x22 0x23 0x24 0x25 0x26 0x27 0x28 0x29 0x2A 0x2B 0x2C 0x2D
0x2E 0x2F
0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x3A 0x3B 0x3C 0x3D
0x3E 0x3F
0x40 0x41 0x42 0x43 0x44 0x45 0x46 0x47 0x48 0x49 0x4A 0x4B 0x4C 0x4D
0x4E 0x4F
0x50 0x51 0x52 0x53 0x54 0x55 0x56 0x57 0x58 0x59 0x5A 0x5B 0x5C 0x5D
0x5E 0x5F 0x60 0x61 0x62 0x63
```

Device node name

Board	Kernel version	Device node name
IPQ5018	**4.4**	**/dev/ttyBT0**

NOTE: For more information regarding Bluetopia installation and build instructions for IPQ50xx platforms, see *Bluetopia installation and operation for IPQ50xx Bluetooth Application Note* (80-16052-31).

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.7.1 Audio TDM driver support

The preliminary base code for TDM support is available as given here:

1. LPASS TDM driver – API is exposed to perform PCM operations. For details, see *IPQ5018 LPASS TDM Configuration Application Note* (80-Y0631-11).
2. DMA Lite driver – Used by TDM driver.
3. Test PCM driver.
4. Test standalone PCM loopback (master mode).

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.8 Low memory profile – feature portfolio

6.8.1 List of features

16 MB flash – list of removed features

Removed in 16 MB base profile

- Boot
  - NAND, eMMC, USB
  - Secure Boot
  - ATF
- Kernel
  - USB, QDSS, USB-DIAG, EUD
  - NAND, eMMC, SCSI
  - Crypto – algorithms (unused), SoC crypto driver
  - Mini crash dump, Dynamic debug, KALL syms, Highmem, user space coredump, sysrq-trigger
  - Bluetooth
  - USB storage – SCSI
- OpenWrt
  - FTM
  - Web GUI, Wi-Fi/SON with webGUI (HYFI UI)
  - Pm-utils, Thermald
  - Wlan-fw upgrade
  - Env-Tools, Opkg tools – IPK support
  - SKB recycler
  - Sysupgrade, Failsafe
  - FTPd, Samba, USB storage
  - TR69
  - AISA tools, LTP, LKDTM, security - test, Perf, Rng tools
  - Fwauthtest
  - Stripped Busybox supported.
  - Gzip (Lzma supported)
- Wi-Fi Host:
  - QWRAP
  - ICM
  - WAPI

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- IBSS
- Vow_debug
- IBSS related
- ACFG
- Debug configs
- Agile DFS
- QCN90xx - full monitor

- HALPHY tools:
  - Qcmbr
  - Athtestcmd
- Wi-Fi SON
  - Debugging tools – breakpad, qca-son-mem-debug, qca-son-cli
  - HYD diagnostics service (soneventservice) and qca-wifison-ext-lib library
  - qca-mad application and DE service in HYD
- QCA-SSDK:
  - On QCA8337:
    - FDB table aging time
    - FDB learning limit
    - FDB static entry adding
    - Reserved FDB
    - IP routing
    - VLAN translation
    - QoS mapping between DSCP and COS
  - On IPQ50xx internal GEPHY:
    - Hibernation mode
    - CDT
    - MDIX
    - local/remote loopback
    - WOL
    - Debug PHY counters

16 MB flash – list of features to be added in debug profile

Features added in debug profile	Additional memory needed (compressed)
Dynamic debug	176 KB
USB/QDSS	500 KB (total)
USB	105 KB
Env-Tools, Opkg tools – IPK support	14 KB (uboot-envtools) 64 KB (opkg)
Diag	145 KB

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Features added in debug profile	Additional memory needed (compressed)
EUD	1.2 KB
KALL syms	249 KB
Sysrq-trigger	1.5 KB
FTM	13 KB
Wlan-fw upgrade	92 KB
Perf, Fwauthtest	1996 KB ,1 KB
Samba	920 KB
USB Storage (SCSI/ATA)	346 KB
Ethtool	89 KB
Tcpdump	439 KB
Breakpad	68 KB
SON Memory debug	14 KB
SON_CLI	14 KB

Wi-Fi stats tools added in debug profile	Additional memory needed
80211stats	6 KB
Athstats	15 KB
Athstatsclr	2 KB
apstats	18 KB
Pktlogconf	4 KB
Pktlogdump	4 KB
Wifistats	34 KB
Peerratestats	4 KB
Sigma_dut	220 KB
athdiag	30 KB
thermaltool	4 KB
exttool	60 KB
assocdenialnotify	3 KB
athkey	4 KB
radartool	6 KB
spectraltool	18 KB
athadhoc	6 KB
tx99tool	3 KB
acsdbgtool	4 KB
cfg_test_app	3 KB
qldtool	5 KB
wifitool	40 KB
wrapd	40 KB
Cnss-daemon	24 KB

6.8.2 NSS features – 256 MB DDR/16 MB flash retail profiles

No feature differentiation between 256 MB profile versus 16 MB flash profile

Feature	Supported platform
WAN	1G
LAN	1 x 1G

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Feature	Supported platform
NSS Support	Supported
Crypto support	Not Supported
IPsec	Not Supported
IPv4<->IPv6 transition (6rd/dslite/..)	Not Supported
VPN tunnels (GRE/L2TP/PPTP..)	Not Supported
Enterprise VPN tunnels (DTLS/CAPWAP/..)	Not Supported
Dual stack (IPv6+IPv4)	Supported
Link Aggregation	Not Supported
IPv6 stack support	Supported
QOS	Not Supported
WAN – PPPoE	Supported
Multicast Proxy	Not Supported
Multicast Snooping	Supported
SFE	Not Supported
# of accelerated connections (NSS)	1024
# of Linux connections	2000
Max SKB Recycler buffers	Disabled
SKB size	2048+232
Wi-Fi Offload to NSS	Supported (The only mode supported)
Jumbo mode	Not supported

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7 Features list

7.1 New features

For the list of new features, see Section 2.5 in *QCA_Networking_2021.SPF.11.5 Quick Start Guide (80-YA185-4)*. For a summarized description of the new features in this release, see *QCA_Networking_2021.SPF.11.5 Feature Overview* (80-19559-10).

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8 QDART

8.1 QDART reference documents

See these documents for the use of QDART_Connectivity [version 1.0.90] with this release:

- For 802.11ax chipsets, see:
  - *IPQ807x/IPQ807xA/IPQ817x/IPQ60xx FTM RF Test Application Note* (80-YB215-1)
  - QCN90xx FTM RF Test Application Note (80-Y9005-20)
- For legacy chipsets, see:
  - *QCA99xx QDART Connectivity User Guide* (80-Y8050-1)
  - *IPQ4018/IPQ4019/IPQ4028/IPQ4029 RF Test User Guide* (80-Y9700-3)
  - *Use QDART Connectivity to Test QCA99XX /QCA98XX/QCA95XX Application Note* (80-Y8050-46)

Note for using QDART_Connectivity with this release

- BDF conversion Python utilities (such as bin2txt.py and txt2bin.py) require Python version 2.7
- From QDART v87 and above, as part of implementing PBW < CBW support, rateBW 1 ‘RateBW_LegacyOFDM’ which was used for nonHT DUP transmission would perform ‘LegacyOFDM’ transmission in corresponding channel bonding frequency (cbstate value) in the channel's maximum BW. Enum 50 ‘RateBW_NON_HT_DUP40’, Enum 51 ‘RateBW_NON_HT_DUP80’ and Enum 52 ‘RateBW_NON_HT_DUP160’ would be used to perform nonHT DUP mode Transmission from the QDART version mentioned. i.e.
- List mode configuration:
  - Configuration to connect through QUTS is enabled.

    CONNECT=QUTS:192.168.1.1:TIMEOUT_SEC:60:TARGET_TYPE:1
  - WLAN_PHYRFMODE must be set to 7
  - QSEQ_CONNVFS_WLAN_SeqLoad SequenceCtl.xml:

BASE_DUT_PHY_ID	0	PHYA0
	1	PHYB
	2	PHYA1
BASE_DUT_Instance	0	WLAN0
	1	WLAN1
	2	WLAN2
BASE_DUT_TX_MeasOffsetMsec		
BASE_DUT_TX_MeasDurationMsec		
BASE_DUT_TX_SegmentDurationMsec		

BASE_DUT_PHY_ID	0	PHYA0
	1	PHYB
	2	PHYA1
BASE_DUT_TX_BSSID	Ex: 00:03:7F:44:55:71	
BASE_DUT_TX_TXSTATION	Ex: 00:03:7F:44:55:72	
BASE_DUT_TX_RXSTATION	Ex: 00:03:7F:44:55:73	
BASE_DUT_RX_MeasOffsetMsec		
BASE_DUT_RX_SegmentDurationMsec		
BASE_DUT_RX_NUMPACKETS		
BASE_DUT_RX_MODE	0	Promiscuous
	1	Filter
BASE_DUT_RX_BSSID: BASE_DUT_RX_STAADDR		
BASE_DUT_RX_STAADDR	Ex: 00:00:00:C0:FF:EE	
BASE_DUT_TX_LDPC	0	Disable LDPC
	1	Enable LDPC
BASE_DUT_TX_STBC	0	Disable STBC
	1	Enable STBC
BASE_DUT_TX_PAPRD	0	Disable DPD
	1	Enable DPD

8.2 QDART requirements for QCN90xx with IPQ807x/IPQ50xx/IPQ60xx and QN61xx with IPQ50xx

Contact equipment vendor Litepoint and Keysight (KS) for the 160 MHz updated software package.

6 GHz is recommended with Litepoint/Keysight (KS).

The 160-wide channel support requires using QTI WLAN SCPI interface.

This WLAN test equipment is supported exclusively with the QTI WLAN SCPI interfaces. Native commands for this test equipment are not supported after QDART-CONN 1.0.42.

- Anritsu 8870A
- LitePoint IQxel80/IQxel160
- NI PXI 5644/5645/5646

This build supports these functionalities:

- QDART connectivity components running on a Windows 10 PC
- Tx power calibration - OLPC support
- Instrument support: NI MIMO, LP MIMO, KS MIMO
- Single instrument: LP IQxel 80 or 160 (these are for 11ac chipsets)
- FTM support with 1x1 LP
- Board data file
- All chain mask combinations including 8x8
- Tx single tone at carrier frequency
- Tx99 support

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- DBDC support
- Link test (DUT to DUT)
- Tx power accuracy test
- Frequency accuracy test
- Tx mask test
- Target power versus Tx EVM sweep
- Rx waterfall sweep
- Rx gain and Noise Floor (NF) calibration
- Provide capability to get MAC from board data
- XTAL calibration: 2 GHz, 5 GHz, 6 GHz
- Backup/restore of calibration data
- Provide capability to get MAC from board data
- Include several Microsoft redistributable DLLs to eliminate installation difficulties that may be experienced on some PCs.
- For the PC station configuration, the Windows system must be set up to run in test mode before the driver can be installed.
- Instrument support: LP MIMO requires minimum two port supported instruments.

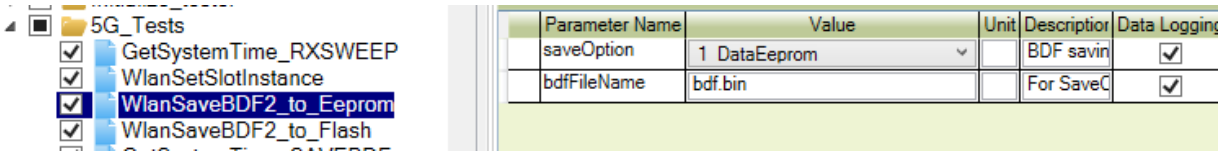
8.3 QDART bringup

```
rmmod monitor
rmmod wifi_3_0
rmmod qca_ol
insmod qca_ol hw_mode_id=1 testmode=1 cfg80211_config=1
insmod wifi_3_0
diag_socket_app -a 192.168.1.10 &
/etc/init.d/ftm start
/usr/sbin/ftm -n -dd &
```

For details, see *QCN90xx FTM RF Test Application Note* (80-Y9005-20).

8.4 EEPROM, Flash commit

This feature is supported from QDART-76 version. Refer to tis image to set up EEPROM and Flash commit:



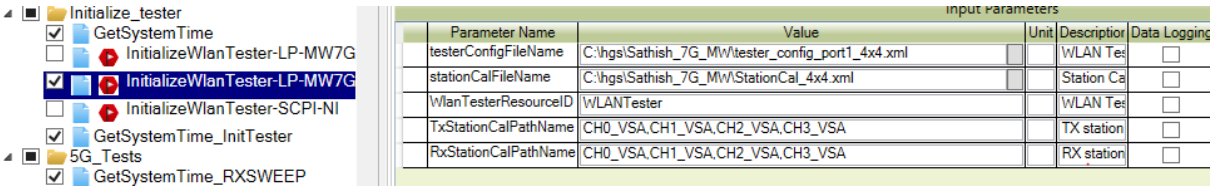
For flash commit, select saveOption 0.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.5 Managing BDF when EEPROM, Flash are not used

The below is in reference to QCN90xx.

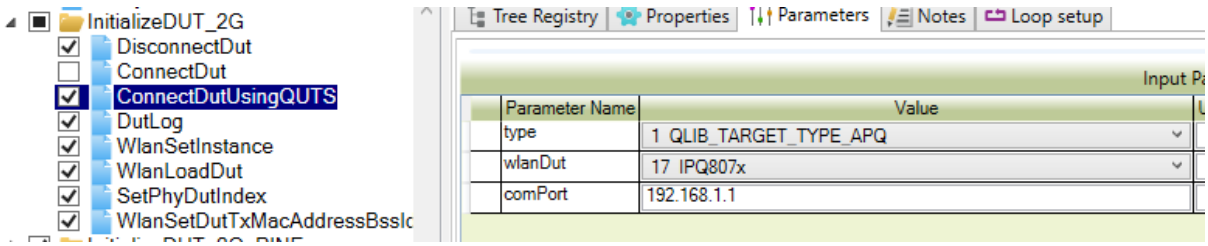
1. After board is booted, initialize the instrument using QSPR test tree, by selecting the node of InitializeWlanTester.



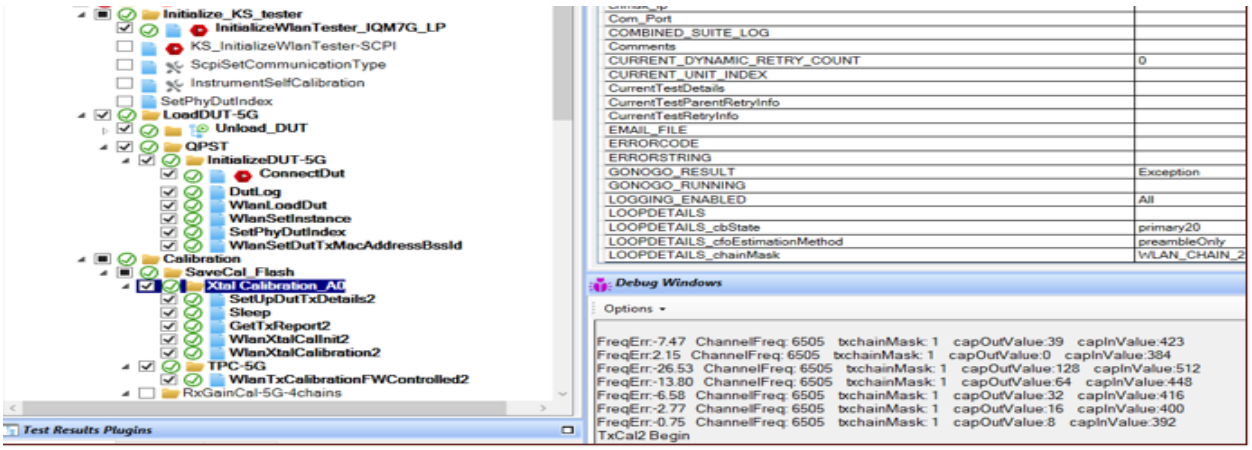
2. After initializing the instrument, load the DUT by using following ftm command.

```
diag_socket_app -a 192.168.1.9 &
/etc/init.d/ftm start
ftm -n &
ftm -n -dd &
# If any issue need debug, could add -dd parameter to output ftm debug
message
# ftm -n -dd &
```

NOTE: The server IP address needs to change accordingly.

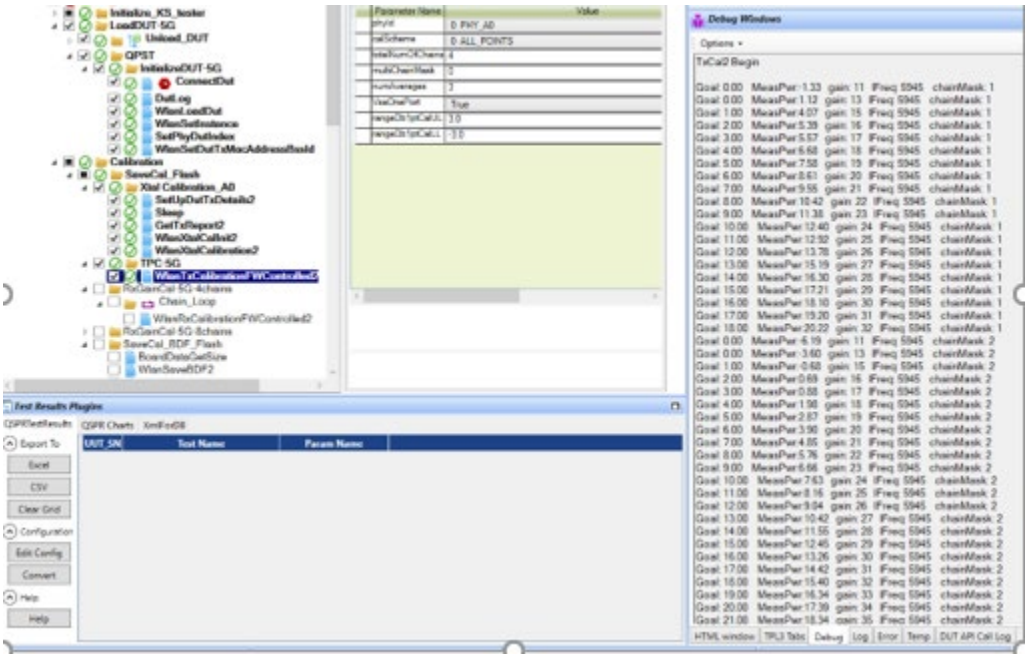


3. Run the XTAL_calibration node where board is calibrated to almost zero PPM.

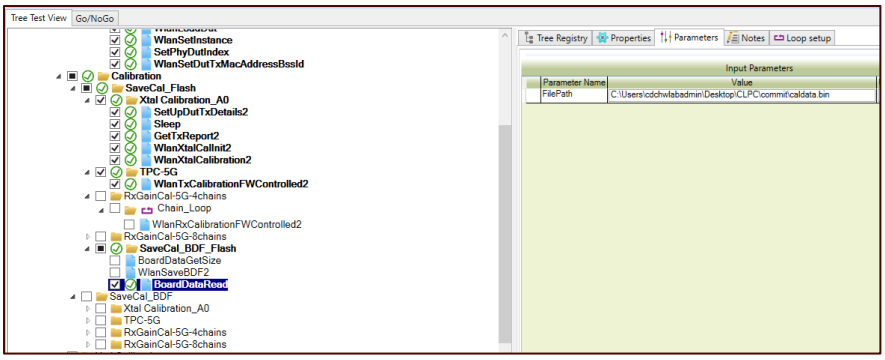


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4. After running, XTAL_calibration node, run WLANTxCalibrationFWControlled2 node and FW calibration will start.



5. After firmware calibration is complete, save caldataFile in the appropriate location:



6. Replace the bdf in /lib/firmware/qcn90xx folder with the specific band of RD:
   - Bdwlan.ba0 > 5 GHz
   - Bdwlan.ba1 > 2 GHz
   - Bdwlan.ba2 > 6 GHz
   - Bdwlan.ba3 > 5G_SBSFILTER_BASED
   - Bdwlan.ba4 > 6G_SBSFILTER_BASED

   Using this command:

```
cd /lib/firmware
rm -r /lib/firmware/qcn9000
cp IPQ8074/WIFI_FW/qcn9000/ qcn9000 -r
cd /lib/firmware/qcn9000
```

Replace BDF, then remove "f" in vi /etc/init.d/wifi_fw_mount in line (line 79).

7. Transfer the file:

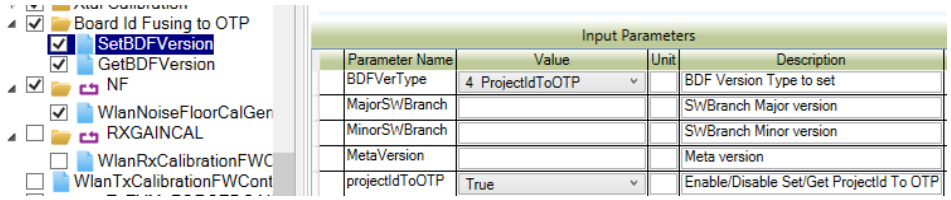
```
tftp -r backup_caldata.bin -g machineIP
```

1. Reboot the board and start the test.

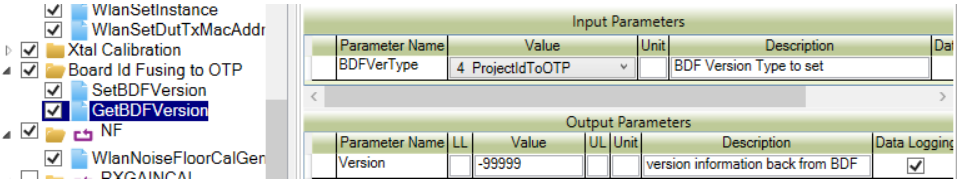
8.6 Fusing board ID to OTP in QCN61xx

1. If It is the first time in FTM to configure the board id. Use boot args from U-Boot to select the BDFs in case the DTS is not set correctly. By default, the DTS configuration takes care of this and no additional change is needed.
2. Fusing board ID in FTM
   a. QC.CTE.WlanDutWrapper.WlanDutWrapper.SetBDFVersion > write project id to OTP
   b. Select BDFVerType param to ProjectIdToOTP

      Select ProjectIdToOTP param to True



   c. QC.CTE.WlanDutWrapper.WlanDutWrapper.GetBDFVersion > get project id from OTP
   d. Select BDFVerType param to ProjectIdToOTP
   e. Project ID displays in Version output parameter



3. Mission mode: The right BDF is downloaded by the Host driver even without bootargs approach.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.7 XTAL CAL for QCN61xx

MP02.1 boards have no separate crystal in QCN61xx 5G card and it makes use of crystal from QCN61xx 6 GHz. Thus, XTAL calibration must be run only for QCN61xx 6 GHz. In case of MP03.3, XTAL calibration should be run in the QCN61xx 5 GHz card.

Below table provides the **WlanInstance** and **SlotID** (Saving the Caldata to flash) when all three radios are brought up in MP02.1 and MP03.3.

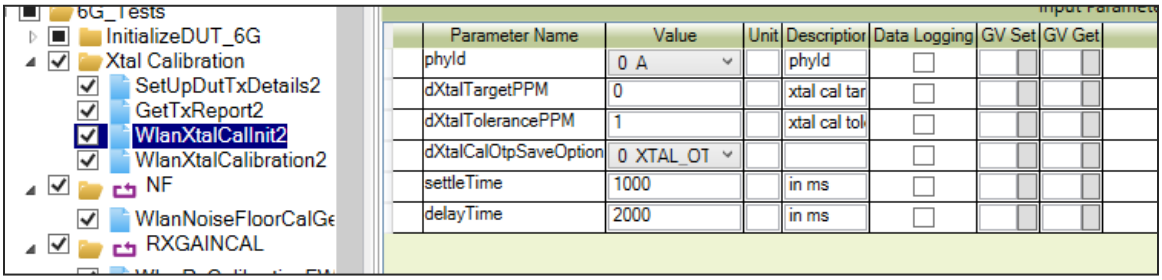
	MP02.1		MP03.3	
SlotID	0	IPQ5018 2 GHz	0	IPQ5018 2 GHz
	1	QCN61xx 6 GHz iPA	1	QCN61xx 5 GHz xPA
	2	QCN61xx 5 GHz iPA	2	QCN9074 6 GHz
WlanInstance	0	IPQ5018 2 GHz	0	IPQ5018 2G
	1	QCN61xx 6 GHz iPA	1	QCN61xx 5 GHz iPA
	2	QCN61xx 5 GHz iPA	2	QCN9074 6 GHz

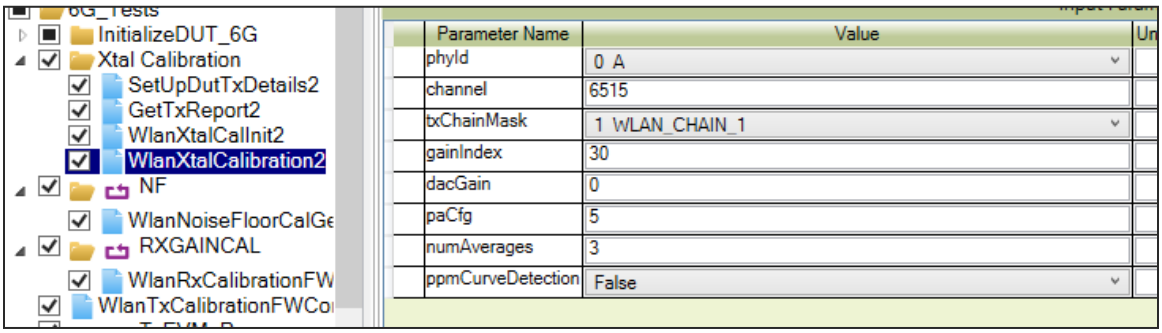
In case only QCN61xx cards (5 GHz + 6 GHz) are enabled in MP02.1 (IPQ5018 disabled):

	MP02.1	
SlotID	1	QCN61xx 6 GHz iPA
	2	QCN61xx 5 GHz iPA
WlanInstance	1	QCN61xx 6 GHz iPA
	2	QCN61xx 5 GHz iPA

XtalCal test nodes: WlanXtalCalInit2

phyId	Should always be 0
dXtalTargetPPM	Value to which FreqErr should converge (usually 0)
dXtalTolerancePPM	Maximum deviation from the target PPM, +/- value
dXtalCalOtpSaveOption	XTAL_OTP_DISABLE for now, because OTP is not enabled





MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.8 Enable DPD in the default BDF for FTM mode

8.8.1 Modify the BDF

1. Select the required board data file (BDF) txt for the IPQ5018 or QCN61xx BDF path.
2. Change this field in the appropriate BDF txt file (common for IPQ5018 and QCN61xx):
   ```
   -uint8      DPD_CONFIG_EXTN.edpdConfigBand.edpdEnable        0
   +uint8      DPD_CONFIG_EXTN.edpdConfigBand.edpdEnable        1
   ```
3. For QCN61xx 5 GHz, also change this field to disable DPD:
   ```
   -uint8 DPD_CONFIG.dpdConfigBand.dpdEnable 1
   +uint8 DPD_CONFIG.dpdConfigBand.dpdEnable 0
   ```
4. For QCN61xx 6 GHz also change the below field to disable DPD
   ```
   -uint8 DPD_CONFIG.dpdConfigBand6g.dpdEnable 1
   +uint8 DPD_CONFIG.dpdConfigBand6g.dpdEnable 0
   ```
5. Convert the txt back to bin using the txt2bin converter. For example, txt2bin conversion: *python txt2bin.py bdwlan_b50.txt bdwlan.b50.*

8.8.2 Changes to load the modified BDF in DUT

1. Remove the existing BDF to replace in the DUT, for example:
   ```
   rm -rf /lib/firmware/qcn91xx/bdwlan.b50
   ```
2. Copy the modified BDF bin file from Section 3.4.1 to this path:
   /lib/firmware/qcn91xx/
3. Edit this file in DUT:
   /etc/init.d/wifi_fw_mount
4. Change this line (for IPQ5018, change the corresponding path for IPQ5018):
   ```
   cd qcn9100 && ln -sf /lib/firmware/$arch/WIFI_FW/qcn9100/*.* . && ln -sf
      /lib/firmware/$arch/WIFI_FW/q6_fw.* .
   ```
   To this:
   ```
   cd qcn9100 && ln -s /lib/firmware/$arch/WIFI_FW/qcn9100/*.* . && ln -s
      /lib/firmware/$arch/WIFI_FW/q6_fw.* .
   ```
5. Change this line
   ```
   cd $fwfolder/$arch && ln -sf /lib/firmware/$arch/WIFI_FW/*.* .
   ```
   To this:
   ```
   cd $fwfolder/$arch && ln -s /lib/firmware/$arch/WIFI_FW/*.* .
   ```
6. Save and exit the file.
7. Execute the **sync** command and reboot DUT to continue operation.

8.9 Verification test tree

Below are the recommended test tree combinations

Details of the changes:

- For all chipsets, we will go with below combination for Tx
  - Mode will be HE
  - Channels selected will Low, Mid, High channels (non-calibrated channels). For 160 (mid) and 80 (low/high), will be selected. For 2.4GHz, we will select a 40 (Mid)
  - MCS chosen will be MCS0 (to cover Mask) and MCS11 (to cover EVM)
  - Target power of MCS0 and MCS11 will be chosen as the power level
  - Chain chosen will be max of the chip. (Customer can add a switch in order to do combined chain mask Tx as they use single VXT). Default test tree will be supporting individual chainmasks.
  - Depending on low/high band support, channels to be selected appropriately.
- For all chipsets, we will go with below combination for Rx
  - Mode will be HE
  - 5G/6G: For 20MHz low/High and 80MHz mid channel. 2.4G : 20 MHz mid channel
  - MCS chosen will be MCS0 and MCS11
  - Chains chosen will be max of the chip
  - It is recommended to sweep from +6 to +2 powerlevels from the KPI ensuring 10% PER criteria is met for those powerlevels.

<table>
<tr><th rowspan="2">Sl. No</th><th rowspan="2">Chipset</th><th rowspan="2">Band</th><th rowspan="2">RDP</th><th rowspan="2">Description (if any)</th><th rowspan="2">Mode</th><th colspan="2">BW : Channels</th><th rowspan="2">MCS</th><th rowspan="2">Target Power</th><th rowspan="2">Rx Start/Stop power level with Sensitivity</th><th rowspan="2">Chain</th><th rowspan="2">BDF</th></tr>
<tr><th>TX</th><th>RX</th></tr>
<tr><td rowspan="2">1</td><td rowspan="2">QCN90xx</td><td rowspan="2">6G</td><td rowspan="2">RDP 420</td><td rowspan="2"></td><td rowspan="2">HE</td><td rowspan="2">80 : 6225,6945
160 : 6655</td><td rowspan="2">20 : 6195, 6975
80 : 6515</td><td rowspan="2">0,11</td><td rowspan="2">22,16</td><td rowspan="2"><table><tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr><tr><td>20</td><td>MCS0</td><td>-90</td><td>-94</td><td>-96</td></tr></table></td><td>4/3/2</td><td>bdwlan.ba4/ bb4/ba9</td></tr>
<tr><td>4/3/2</td><td>bdwlan.ba4/ bb4/ba9</td></tr>
<tr><td></td><td></td><td>6G</td><td>RDP 419</td><td></td><td>HE</td><td>80 : 6225,6945
160 : 6655</td><td>20 : 6195, 6975
80 : 6515</td><td>0,11</td><td>22,16</td><td></td><td></td><td></td></tr>
</table>

Sl. No	Chipset	Band	RDP	Description (if any)	Mode	BW : Channels		MCS	Target Power	Rx Start/Stop power level with Sensitivity					Chain	BDF
						TX	RX									
		6G	RDP 412/ RDP 414		HE	**80 :** 6225,6945 **160 :** 6655	**20 :** 6195, 6975 **80 :** 6515	0,11	22,16		MCS11	-59	-63	**-65**	4/3/2	bdwlan.ba4/ bb4/ba9
										80	MCS0	-83	-87	**-89**		
											MCS11	-53	-57	**-59**		
		5G	RDP 412/ RDP 413		HE	**80 :** 5210,5775 **160 :** 5570	**20 :** 5180, 5745 **80 :** 5530	0,11	22,19	BW	Rate	Start Power	Stop Power	**Sensitivity**	4/3/2	bdwlan.ba3/ bb3/ba8
										20	MCS0	-91	-95	**-97**		
											MCS11	-61	-65	**-67**		
										80	MCS0	-85	-89	**-91**		
											MCS11	-55	-59	**-61**		
		2G	RDP 413/ RDP 414		HE	**40 :** 2442	**20 :** 2437	0,11	24,21	BW	Rate	Start Power	Stop Power	**Sensitivity**	4/3/2	bdwlan.ba1/ bb1/ba6
										20	MCS0	-90	-94	**-96**		
											MCS11	-60	-64	**-66**		
		Scan Radio 2x2 - 2G			HE	**40 :** 2442	**20 :** 2437	0,11	18,13	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.baa
										20	MCS0	-90	-94	**-96**		
											MCS11	-59	-63	**-65**		

<table>
<tr><th>Sl. No</th><th>Chipset</th><th>Band</th><th>RDP</th><th>Description (if any)</th><th>Mode</th><th colspan="2">BW : Channels</th><th>MCS</th><th>Target Power</th><th>Rx Start/Stop power level with Sensitivity</th><th>Chain</th><th>BDF</th></tr>
<tr><th></th><th></th><th></th><th></th><th></th><th></th><th>TX</th><th>RX</th><th></th><th></th><th></th><th></th><th></th></tr>
<tr><td></td><td></td><td>Scan Radio 2x2 - 5G</td><td></td><td></td><td>HE</td><td>80 : 5210,5775
160 : 5570</td><td>20 : 5180, 5745
80 : 5530</td><td>0,11</td><td>18,9</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-88</td><td>-92</td><td>-94</td></tr>
<tr><td>MCS11</td><td>-58</td><td>-62</td><td>-64</td></tr>
<tr><td rowspan="2">80</td><td>MCS0</td><td>-82</td><td>-86</td><td>-88</td></tr>
<tr><td>MCS11</td><td>-52</td><td>-56</td><td>-58</td></tr>
</table>
</td><td>2</td><td>bdwlan.baa</td></tr>
<tr><td></td><td></td><td>Scan Radio 2x2 - 6G</td><td></td><td></td><td>HE</td><td>80 : 6225,6945
160 : 6655</td><td>20 : 6195, 6975
80 : 6515</td><td>0,11</td><td>18,9</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-88</td><td>-92</td><td>-94</td></tr>
<tr><td>MCS11</td><td>-58</td><td>-62</td><td>-64</td></tr>
<tr><td rowspan="2">80</td><td>MCS0</td><td>-82</td><td>-86</td><td>-88</td></tr>
<tr><td>MCS11</td><td>-52</td><td>-56</td><td>-58</td></tr>
</table>
</td><td>2</td><td>bdwlan.baa</td></tr>
<tr><td></td><td></td><td>Wide Band Radio 5G-4x4</td><td></td><td></td><td>HE</td><td>80 : 5210,,5775
160 : 5570</td><td>20 : 5180, 5745
80 : 5530</td><td>0,11</td><td>18,14</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-88</td><td>-92</td><td>-94</td></tr>
<tr><td>MCS11</td><td>-58</td><td>-62</td><td>-64</td></tr>
<tr><td rowspan="2">80</td><td>MCS0</td><td>-82</td><td>-86</td><td>-88</td></tr>
<tr><td>MCS11</td><td>-55</td><td>-59</td><td>-61</td></tr>
</table>
</td><td>4</td><td>bdwlan.bab</td></tr>
</table>

<table>
<tr><th rowspan="2">Sl. No</th><th rowspan="2">Chipset</th><th rowspan="2">Band</th><th rowspan="2">RDP</th><th rowspan="2">Description (if any)</th><th rowspan="2">Mode</th><th colspan="2">BW : Channels</th><th rowspan="2">MCS</th><th rowspan="2">Target Power</th><th rowspan="2">Rx Start/Stop power level with Sensitivity</th><th rowspan="2">Chain</th><th rowspan="2">BDF</th></tr>
<tr><th>TX</th><th>RX</th></tr>
<tr><td></td><td></td><td>Wide Band Radio 6G-4x4</td><td></td><td></td><td>HE</td><td>80 : 6225,6945
160 : 6655</td><td>20 : 6195, 6975
80 : 6515</td><td>0,11</td><td>18,14</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-88</td><td>-92</td><td>-94</td></tr>
<tr><td>MCS11</td><td>-58</td><td>-62</td><td>-64</td></tr>
<tr><td rowspan="2">80</td><td>MCS0</td><td>-82</td><td>-86</td><td>-88</td></tr>
<tr><td>MCS11</td><td>-52</td><td>-56</td><td>-58</td></tr>
</table>
</td><td>4</td><td>bdwlan.bab</td></tr>
<tr><td>2</td><td>IPQ6018</td><td>2G</td><td>CP01 XFEM, CP04</td><td></td><td>HE</td><td>40 : 2442</td><td>20 : 2437</td><td>0,11</td><td>24,21</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-86</td><td>-90</td><td>-92</td></tr>
<tr><td>MCS11</td><td>-56</td><td>-60</td><td>-62</td></tr>
</table>
</td><td>2</td><td>bdwlan.b10/ b11/b12/b40</td></tr>
<tr><td></td><td></td><td>5G</td><td>CP01 XFEM, CP04</td><td></td><td>HE</td><td>80 : 5210,5530,5775</td><td>20 : 5180, 5745
80 : 5530</td><td>0,11</td><td>24,19</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-84</td><td>-88</td><td>-90</td></tr>
<tr><td>MCS11</td><td>-54</td><td>-58</td><td>-60</td></tr>
<tr><td rowspan="2">80</td><td>MCS0</td><td>-79</td><td>-83</td><td>-85</td></tr>
<tr><td>MCS11</td><td>-49</td><td>-53</td><td>-55</td></tr>
</table>
</td><td>2</td><td>bdwlan.b10/ b11/b12/b14/ b15/b16/b17/ b40</td></tr>
<tr><td></td><td></td><td>2G</td><td>CP02.3/CP02.1</td><td>DBS</td><td>HE</td><td>40 : 2442</td><td>20 : 2437</td><td>0,11</td><td>18,13</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-88</td><td>-92</td><td>-94</td></tr>
<tr><td>MCS11</td><td>-58</td><td>-62</td><td>-64</td></tr>
</table>
</td><td>2</td><td>bdwlan.b21/b20</td></tr>
</table>

Sl. No	Chipset	Band	RDP	Description (if any)	Mode	BW : Channels		MCS	Target Power	Rx Start/Stop power level with Sensitivity					Chain	BDF
						TX	RX									
		5G	CP02.3	DBS	HE	80 : 5210,5530,5775		0,11	18,14	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.b21
										20	MCS0	-86	-90	**-92**		
											MCS11	-56	-60	**-62**		
										80	MCS0	-80	-84	**-86**		
											MCS11	-50	-54	**-56**		
		5G-LB	CP02.1	SBS	HE	80 :5210		0,11	15,10	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.b20/b25
		5G-HB	CP02.1	SBS	HE	80: 5530,5775		0,11	15,10	20	MCS0	-84	-88	**-90**	2	bdwlan.b20/b25
											MCS11	-54	-58	**-60**		
										80	MCS0	-80	-84	**-86**		
											MCS11	-50	-54	**-56**		
		2G	CP03-IPA		HE	40 : 2442	**20 :** 2437	0,11	19,11	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.b30
										20	MCS0	-83	-87	**-89**		
											MCS11	-53	-57	**-59**		
		5G	CP03-XFEM		HE	80 : 5210,5530,5775		0,11	20,15	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.b30
										20	MCS0	-84	-88	**-90**		
											MCS11	-54	-58	**-60**		
										80	MCS0	-78	-82	**-84**		
											MCS11	-48	-52	**-54**		

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Sl. No	Chipset	Band	RDP	Description (if any)	Mode	BW : Channels		MCS	Target Power	Rx Start/Stop power level with Sensitivity					Chain	BDF
						TX	RX									
3	IPQ807x	2G	HK01/OAK01/ OAK02/OAK03		HE	**40 :** 2442	**20 :** 2437	0,11	24.21	BW	Rate	Start Power	Stop Power	**Sensitivity**	4/2	bdwlan.b210/ b211/b212/b213
										20	MCS0	-88	-92	**-94**		
											MCS11	-58	-62	**-64**		
		5G	HK01	DBS	HE	**80 :** 5210,5775 **80+80:** 5530,5610	**20 :** 5180, 5745 **80 :** 5530	0,11	24,19	BW	Rate	Start Power	Stop Power	**Sensitivity**	8	bdwlan.b210
		5G-LB	HK01-OAK01/ OAK02/OAK03	SBS-LB	HE	**80 :** 5210	**20 :** 5180 **80 :** 5290	0,11	24,17	20	MCS0	-87	-91	**-93**	4/2	bdwlan.b211/ b212/b213
		5G-HB	HK01-OAK01/ OAK02/OAK03	SBS-LB	HE	**80 :** 5530,5775	**20 :** 5500, 5825 **80 :** 5610	0,11	24,13		MCS11	-58	-62	**-64**	4/2	bdwlan.b211/ b212/b213
										80	MCS0	-82	-86	**-88**		
											MCS11	-52	-56	**-58**		
		2G	HK09/AC01/ AC02/AC03		HE	**40 :** 2442	**20 :** 2437	0,11	24,19	BW	Rate	Start Power	Stop Power	**Sensitivity**	4/2	bdwlan.b290/ b291/b292/ b293
										20	MCS0	-88	-92	**-94**		
											MCS11	-58	-62	**-64**		
		5G	HK09/AC01/ AC02		HE	**80 :** 5210,5775 **80+80:** 5530,5610	**20 :** 5180, 5745 **80 :** 5530	0,11	24,13	BW	Rate	Start Power	Stop Power	**Sensitivity**	4/2	bdwlan.b290/ b291/b292
		5G	AC03	5G-HB	HE	**80 :** 5530,5775	**20 :** 5500, 5825 **80 :** 5610	0,11	24,19	20	MCS0	-87	-91	**-93**	4	bdwlan.b293
											MCS11	-58	-62	**-64**		
										80	MCS0	-82	-86	**-88**		
											MCS11	-52	-56	**-58**		

Sl. No	Chipset	Band	RDP	Description (if any)	Mode	BW : Channels		MCS	Target Power	Rx Start/Stop power level with Sensitivity					Chain	BDF
						TX	RX									
4	IPQ5018	2G	RDP 404	MP3.1 XFEM	HE	**40 :** 2442	**20 :** 2437	0,11	24,21	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.b24
										20	MCS0	-90	-94	**-96**		
											MCS11	-60	-64	**-66**		
			RDP 403	MP2.1-IPA	HE	**40 :** 2442	**20 :** 2437	0,11	22,15	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.b10
										20	MCS0	-88	-92	**-94**		
											MCS11	-58	-62	**-64**		
			RDP 415/ RDP 432	MP3.3 -IPA	HE	**40 :** 2442	**20 :** 2437	0,11	22,15	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.b23
										20	MCS0	-88	-92	**-94**		
											MCS11	-58	-62	**-64**		
5	QCN61xx	5G	RDP 403	IPA	HE	**80 :** 5210,,5775 **160 :** 5570	**20 :** 5180, 5745 **80 :** 5530	0,11	19,12	BW	Rate	Start Power	Stop Power	**Sensitivity**	2	bdwlan.b50
										20	MCS0	-87	-91	**-93**		
											MCS11	-55	-59	**-61**		
										80	MCS0	-81	-85	**-87**		
											MCS11	-50	-54	**-56**		

<table>
<tr><th>Sl. No</th><th>Chipset</th><th>Band</th><th>RDP</th><th>Description (if any)</th><th>Mode</th><th colspan="2">BW : Channels</th><th>MCS</th><th>Target Power</th><th>Rx Start/Stop power level with Sensitivity</th><th>Chain</th><th>BDF</th></tr>
<tr><th></th><th></th><th></th><th></th><th></th><th></th><th>TX</th><th>RX</th><th></th><th></th><th></th><th></th><th></th></tr>
<tr><td></td><td></td><td>5G</td><td>RDP 415, RDP 432</td><td>XPA</td><td>HE</td><td>80 : 5210,5775
160 : 5570</td><td>20 : 5180, 5745
80 : 5530</td><td>0,11</td><td>24,18</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-89</td><td>-93</td><td>-95</td></tr>
<tr><td>MCS11</td><td>-58</td><td>-62</td><td>-64</td></tr>
<tr><td rowspan="2">80</td><td>MCS0</td><td>-83</td><td>-87</td><td>-89</td></tr>
<tr><td>MCS11</td><td>-52</td><td>-56</td><td>-58</td></tr>
</table>
</td><td>2</td><td>bdwlan.b60</td></tr>
<tr><td></td><td></td><td>6G</td><td>RDP 403</td><td>IPA</td><td>HE</td><td>80 : 6225,6945
160 : 6655</td><td>20 : 6195, 6975
80 : 6515</td><td>0,11</td><td>18,8</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-86</td><td>-90</td><td>-92</td></tr>
<tr><td>MCS11</td><td>-55</td><td>-59</td><td>-61</td></tr>
<tr><td rowspan="2">80</td><td>MCS0</td><td>-80</td><td>-84</td><td>-86</td></tr>
<tr><td>MCS11</td><td>-49</td><td>-53</td><td>-55</td></tr>
</table>
</td><td>2</td><td>bdwlan.b90</td></tr>
<tr><td></td><td></td><td>6G</td><td>RDP 432</td><td>XPA</td><td>HE</td><td>80 : 6225,6945
160 : 6655</td><td>20 : 6195, 6975
80 : 6515</td><td>0,11</td><td>23,18</td><td>
<table>
<tr><th>BW</th><th>Rate</th><th>Start Power</th><th>Stop Power</th><th>Sensitivity</th></tr>
<tr><td rowspan="2">20</td><td>MCS0</td><td>-89</td><td>-93</td><td>-95</td></tr>
<tr><td>MCS11</td><td>-57</td><td>-61</td><td>-63</td></tr>
<tr><td rowspan="2">80</td><td>MCS0</td><td>-83</td><td>-87</td><td>-89</td></tr>
<tr><td>MCS11</td><td>-51</td><td>-55</td><td>-58</td></tr>
</table>
</td><td>2</td><td>bdwlan.bb0</td></tr>
</table>

8.10 FTM calibration and verification KPI

Below defined are the KPI time taken for calibration and verification procedure. Below numbers are taken from the last 11.5 release.

8.10.1 Factors impacting KPI

After Tx transmission from DUT, QDART requests equipment to measure the signal and also, The equipment measures and send the response to DUT by using TLV exchanges, these TLVs exchanges timing are not in DUT's control.

Below are the factors that can impact the KPI timing:

- HW configurations of the test machine (where QDART running)
- Load on the test machine, i.e. CPU load, RAM, and multiple devices connected to it
- Equipment type i.e., vendor specific equipment
- SW version running on the equipment side
- QDART version
- Plugins enabled on QDART
- QDART test tree configurations and settings
- QPST or QUTS software running on the test machine
- QCA tools vs customer tools

NOTE: Time taken for calibration/verification can be changed from setup to setup due to the above factors.

NOTE: The recommended test tree configuration is provided in section 8.9.

NOTE: Run to run variation of below KPI can be 5 to 10%

KPI data capturing the time taken (in seconds) for the below cases:

- Calibration FW time
- Verification Tx time
- Verification Rx time
- Verficiation FW only Tx time – this is calculated from the difference between request TLV received and response TLV sent out from the firmware
- Verficiation FW only Rx time – this is calculated from the difference between request TLV received and response TLV sent out from the firmware

RDPs covered

- RDP386 (IPQ8074 2 GHz/5 GHz 4x4)
- RDP413 (QCN90xx 2 GHz 4x4, 5 GHz 4x4)
- RDP420 (IPQ6018 2 GHz/5 GHz 2x2, QCN90xx 6 GHz 4x4)
- RDP403 (IPQ5018 2 GHz 2x2, QCN61xx 5 GHz/6 GHz 2x2)

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.10.2 RDP386

8.10.2.1 Test equipment details:

- Instrument ID= LitePoint, LitePoint,IQXEL-M16W7G,IQ140CA1794, 1.19.0
- SCPI Firmware Rev.= QTI_WLAN_SCPI_revision_2018-08-04
- System Ver.= 1.93
- WLAN Ver.= 2.2
- Instrument Ver.= 1.19.0, 2.4.0

8.10.2.2 Calibration time

Band	Cal type	Combination used	
		Total time W/ CalDB (in Secs)	Total time W/O CalDB (in Secs)
5 GHz	FPC + other cals	501	509
	OPC + other cals	25	27
2 Ghz	FPC + other cals	180	181
	OPC + other cals	8	9

8.10.2.3 Calibration FW time:

Band	Cal type	Combination used			
		Total time W/ CalDB (in Secs)		Total time W/ O CalDB (in Secs)	
		Tx	Rx	Tx	Rx
5 GHz	FPC + other cals	12.64	0.301	7.82	3.912
	OPC + other cals	0.694	0.391	1.978	3.903
2 GHz	FPC + other cals	3.521	0.683	2.341	1.102
	OPC + other cals	0.457	0.612	0.695	1.124

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.10.2.4 Verification Tx time:

Below table indicates the overall time for Tx verification KPI.

Band	Mode	BW	W/ CalDB W/O DPD (in secs)	W/O CalDB W/O DPD (in secs)	W/ DPD W/ CalDB (in secs)	W/ DPD W/O calDB (in secs)
Tx						
5 GHz	Regular FTM	80	21	24	28	31
	List Mode	80	5	23	8	26
2 GHz	Regular FTM	40	10	11	10	12
	List Mode	40	3	10	4	13

8.10.2.5 Verification Rx time:

Below table indicates the overall time for Rx verification KPI.

Band	Mode	BW	W/ CalDB (in secs)	W/O CalDB (in secs)
Rx				
5 GHz	Regular FTM	20 & 80	90	95
	List Mode	20 & 80	35	58
2 GHz	Regular FTM	20	32	35
	List Mode	20	15	21

8.10.2.6 Verification FW only Tx time

Below table indicates the indicates FW only time for Tx verification KPI.

Band	Mode	BW	W/ CalDB W/O DPD (in secs)	W/O CalDB W/O DPD (in secs)	W/ DPD W/ CalDB (in secs)	W/ DPD W/O calDB (in secs)
Tx						
5 GHz	Regular FTM	80	0.376	0.959	0.378	0.963
	List Mode	80	4.96	22.72	7.92	25.44
2 GHz	Regular FTM	40	0.149	0.23	0.153	0.238
	List Mode	40	2.36	9.82	3.8	12.64

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.10.2.7 Verification FW only Rx time:

Below table indicates the indicates FW only time for Rx verification KPI.

Band	Mode	BW	W/ CalDB (in secs)	W/O CalDB (in secs)
Rx				
5 GHz	Regular FTM	20 & 80	0.125	0.595
	List Mode	20 & 80	23.4	56.84
2 GHz	Regular FTM	20	0.062	0.141
	List Mode	20	7.8	19.2

8.10.3 RDP413

8.10.3.1 Test Equipment details:

- Instrument ID= LitePoint, LitePoint,IQXEL-M16W7G,IQ140CA1794, 1.19.0
- SCPI Firmware Rev.= QTI_WLAN_SCPI_revision_2018-08-04
- System Ver.= 1.93
- WLAN Ver.= 2.2
- Instrument Ver.= 1.19.0, 2.4.0

8.10.3.2 Calibration KPI

Band	Cal type	Total time W/ CalDB (in Secs)	Total W/O CalDB(in Secs)
2 GHz	FPC + other cals	221	215
	OPC + other cals	9	13
5 GHz	FPC + other cals	504	496
	OPC + other cals	13	20

8.10.3.3 Calibration FW time:

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Band	Cal type	Combination used			
		Total time W/ CalDB (in Secs)		Total time W/ O CalDB (in Secs)	
		Tx	Rx	Tx	Rx
2 GHz	FPC + other cals	9.81	0.27	8.31	1.73
	OPC + other cals	0.43	0.28	1.74	1.61
5 GHz	FPC + other cals	27.32	0.47	18.12	7.67
	OPC + other cals	0.77	0.42	3.27	7.54

8.10.3.4 Verification Tx time

Band	Mode	BW	W/ CalDB W/O DPD (in secs)	W/O CalDB W/O DPD (in secs)	W/ DPD W/ CalDB (in secs)	W/ DPD W/O calDB (in secs)
Tx						
2 GHz	Regular FTM	40	11	15	12	13
	List Mode	40	3	13	5	13
5 GHz	Regular FTM	80 & 160	38	57	39	41
	List Mode	80 & 160	8	40	12	41

8.10.3.5 Verification Rx time

Band	Mode	BW	W/ CalDB (in secs)	W/O CalDB (in secs)
Rx				
2 GHz	Regular FTM	20	34	36
	List Mode	20	16	20
5 GHz	Regular FTM	20 & 80	111	134
	List Mode	20 & 80	48	73

8.10.3.6 Verification FW only Tx time

Band	Mode	BW	W/ CalDB W/O DPD (in secs)	W/O CalDB W/O DPD (in secs)	W/ DPD W/ CalDB (in secs)	W/ DPD W/O calDB (in secs)
Tx						
2 GHz	Regular FTM	40	0.32	0.80	0.33	0.82
	List Mode	40	2.16	11.2	3.12	12.64
5 GHz	Regular FTM	80 & 160	0.90	2.20	0.88	2.28
	List Mode	80 & 160	6.72	31.12	9.84	39.6

8.10.3.7 Verification FW only Rx time

Band	Mode	BW	W/ CalDB (in secs)	W/O CalDB (in secs)
Rx				
2 GHz	Regular FTM	20	0.04	1.12
	List Mode	20	7.2	18.01
5 GHz	Regular FTM	20 & 80	0.90	3.10
	List Mode	20 & 80	21.88	54.01

8.10.4 RDP420

8.10.4.1 Test Equipment details:

- Instrument ID= LitePoint, LitePoint,IQXEL-M16W7G,IQ140CA1794, 1.19.0
- SCPI Firmware Rev.= QTI_WLAN_SCPI_revision_2018-08-04
- System Ver.= 1.93
- WLAN Ver.= 2.2
- Instrument Ver.= 1.19.0, 2.4.0

8.10.4.2 Calibration

Band	Cal type	Total time W/ CalDB (in Secs)	Total time W/O CalDB (in Secs)
5 GHz	FPC + other cals	211	211
	OPC + other cals	15	16

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Band	Cal type	Total time W/ CalDB (in Secs)	Total time W/O CalDB (in Secs)
2 GHz	FPC + other cals	88	89
	OPC + other cals	4	5
6 GHz	FPC + other cals	487	484
	OPC + other cals	26	37

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.10.4.3 Calibration FW time:

Band	Cal type	Total time W/ CalDB (in Secs)		Total time W/ O CalDB (in Secs)	
		Tx	Rx	Tx	Rx
5 GHz	FPC + other cals	2.916	0.015	1.508	0.47
	OPC + other cals	0.231	0.03	0.641	0.468
2 GHz	FPC + other cals	0.724	0.039	0.474	0.373
	OPC + other cals	0.157	0.015	0.297	0.376
6 GHz	FPC + other cals	9.63	0.32	7.75	7.07
	OPC + other cals	0.765	0.301	3.294	7.37

8.10.4.4 Verification Tx overall time:

Band	Mode	BW	W/ CalDB W/O DPD (in secs)	W/O CalDB W/O DPD (in secs)	W/ DPD W/ CalDB (in secs)	W/ DPD W/O calDB (in secs)
Tx						
5 GHz	Regular FTM	80	20	24	29	30
	List Mode	80	3.72	17.92	10.22	18.1
2 GHz	Regular FTM	40	7	8	9.8	11
	List Mode	40	1.24	5.75	1.75	6.45
6 GHz	Regular FTM	80	42	47	44	43
	List Mode	80	7.27	36.38	10.47	45

8.10.4.5 Verification Rx overall time:

Band	Mode	BW	W CalDB (in secs)	W/O CalDB (in secs)
Rx				
5 GHz	Regular FTM	20 & 80	26	22
	List Mode	20 & 80	23	39
2 GHz	Regular FTM	20	9	10
	List Mode	20	7.63	7.56
6 GHz	Regular FTM	20 & 80	72	73
	List Mode	20 & 80	36.8	78.89

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.10.4.6 Verification FW only Tx time:

Band	Mode	BW	W/ CalDB W/O DPD (in secs)	W/O CalDB W/O DPD (in secs)	W/ DPD W/ CalDB (in secs)	W/ DPD W/O calDB (in secs)
Tx						
5 GHz	Regular FTM	80	0.142	0.438	0.118	0.451
	List Mode	80	3.36	17.04	10.12	18.96
2 GHz	Regular FTM	40	0.031	0.109	0.054	0.101
	List Mode	40	1.12	5.68	1.9	6.32
6 GHz	Regular FTM	80	0.86	2.155	0.784	2.177
	List Mode	80	6.16	32.94	9.02	40.39

8.10.4.7 Verification FW only Rx time:

Band	Mode	BW	W CalDB (in secs)	W/O CalDB (in secs)
Rx				
5 GHz	Regular FTM	20 & 80	0.157	0.204
	List Mode	20 & 80	10.2	25.5
2 GHz	Regular FTM	20	0.015	0.074
	List Mode	20	3.3	8.25
6 GHz	Regular FTM	20 & 80	0.409	1.67
	List Mode	20 & 80	21.23	53.19

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.10.5 RDP403

8.10.5.1 Test Equipment details:

- Instrument ID= LitePoint, LitePoint,IQXEL-M16W7G,IQ140CA1794, 1.19.0
- SCPI Firmware Rev.= QTI_WLAN_SCPI_revision_2018-08-04
- System Ver.= 1.93
- WLAN Ver.= 2.2
- Instrument Ver.= 1.19.0, 2.4.0

8.10.5.2 Calibration Time:

Band	Cal type	Combination
		Total W/O CalDB (in Secs)
2 GHz	FPC + other cals	103
	OPC + other cals	6
5 GHz	FPC + other cals	287
	OPC + other cals	25
6 GHz	FPC + other cals	490
	OPC + other cals	28

8.10.5.3 Calibration FW only time

Band	Cal type		Total W/O CalDB (in Secs)
2 GHz	FPC + other cals	TX	1.815
		RX	0.839
	OPC + other cals	TX	0.752
		RX	0.836
5 GHz	FPC + other cals	TX	12.436
		RX	3.983
	OPC + other cals	TX	6.48
		RX	3.93
6 GHz	FPC + other cals	TX	17.594
		RX	3.578
	OPC + other cals	TX	6.453
		RX	3.63

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.10.5.4 Verification Tx time:

Band	Mode	BW	W/O CalDB W/O DPD (in secs)	W/ DPD W/O calDB (in secs)
Tx				
2 GHz	Regular FTM	40	10	7
	List Mode	40	12	10
5 GHz	Regular FTM	80 and 160	27	27
	List Mode	80 and 160	22	46
6 GHz	Regular FTM	80 and 160	28	30
	List Mode	80 and 160	22	47

8.10.5.5 Verification Rx time:

Band	Mode	BW	W CalDB (in secs)	W/O CalDB (in secs)
Rx				
2 GHz	Regular FTM	20	NA	13
	List Mode	20	NA	13
5 GHz	Regular FTM	20 and 80	NA	39
	List Mode	20 and 80	NA	41
6 GHz	Regular FTM	20 and 80	NA	44
	List Mode	20 and 80	NA	41

8.10.5.6 Verification FW only Tx time:

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Band	Mode	BW	W/O CalDB W/O DPD (in secs)	W/ DPD W/O calDB (in secs)
Tx				
2 GHz	Regular FTM	40	0.315	0.289
	List Mode	40	3.6	8.5
5 GHz	Regular FTM	80 & 160	2.26	2.14
	List Mode	80 & 160	19.6	46.1
6 GHz	Regular FTM	80 & 160	2.016	2.033
	List Mode	80 & 160	19.6	46.1

8.10.5.7 Verification FW only Rx time:

Band	Mode	BW	W/O CalDB (in secs)
Rx			
2 GHz	Regular FTM	20	0.261
	List Mode	20	7.2
5 GHz	Regular FTM	20 & 80	1.44
	List Mode	20 & 80	28.8
6 GHz	Regular FTM	20 & 80	1.239
	List Mode	20 & 80	28.8

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9 BDF updates

This section shows a high-level delta for the BDF between 11.4 CS and 11.5 CSU1 For a description of the new fields or a list of changes, see the files as listed.

9.1 BDF updates (IPQ8074)

- For a description of the new fields, see **BDF_QCA8074_Description.xlsx**
- For a list of changes, see **NvData_QCA8074.xlsm**

Changes	NvSection
Radar detection threshold configuration	COMMON_BDF_HEADER
Support for additional offchannel based bias in RTT	RTT_DELAY_TABLES
Regulatory Data base updated to v36	REGULATORY_DB_SECTION

9.2 BDF updates (IPQ5018/QCN61xx)

- For a description of the new fields, see **BDF_IPQ5018_QCN6122_Description.xlsx**
- For a list of changes, see **NvData_IPQ5018_QCN6122.xlsm**

Changes	NvSection
Added TxGainLutSel for TxGain LUT selection field in 2G/5G/6G	TPC_DATA
Added RxGainLutSel for RxGain LUT selection field in 2G/5G/6G	BDF_ENABLE_DISABLE_FLAGS
Enable 5935 channel flag	COMMON_BDF_HEADER
Radar detection threshold configuration	COMMON_BDF_HEADER
Added fields for DPD Post Scaling values per GLUT	DPD_CONFIG_EXTN
Added field regDbVersion in Regulatory section. Regulatory Data base updated to v36	REGULATORY_DB_SECTION
Added new field to enable/disable channel 7115	COMMON_BDF_HEADER
RSSI Accuracy Tuning based on BypassXlna mode	TWO_G_FIVE_G_BAND_SPECIFIC

Changes	NvSection
Added fields for customer board data revision version major and minor	COMMON_BDF_HEADER
Added SP client CTL offsets	CTL_TABLES_6G
Added fields for Spruce PowerSave Feature	TPC_DATA

9.2.1 BDF support (IPQ5018)

For information on IPQ5018 BDF details, see the following documents:

- FTM RF Test For 802.11ax WLAN AP Chipsets User Guide (80-YB215-1)
- Board Data File Utility for 802.11ax WLAN AP Chipsets Application Note (80-YB215-4).

9.3 BDF updates (IPQ6018)

- For a description of the new fields, see **BDF_IPQ6018_Description.xlsx**
- For a list of changes, see **NvData_IPQ6018.xlsm**

Changes	NvSection
Radar detection threshold configuration	COMMON_BDF_HEADER
Support for additional offchannel based bias in RTT	RTT_DELAY_TABLES
Regulatory Data base updated to v36	REGULATORY_DB_SECTION

9.4 BDF updates (QCN90xx)

- For a description of the new fields, see **BDF_QCN90_Description.xlsx**
- For a list of changes, see **NvData_QCN90xx.xlsm**

Changes	NvSection
Radar detection threshold configuration	COMMON_BDF_HEADER
Enable 5935 channel flag	COMMON_BDF_HEADER
Enable/disable 7115 channel	COMMON_BDF_HEADER
Added SP client CTL offsets	CTL_TABLES_6G
RSSI Accuracy Tuning based on BypassXlna mode	TWO_G_FIVE_G_BAND_SPECIFIC

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10 Known issues

10.1 Functional, performance, and other issues for SPF11.5 CSU1

Sl. No.	Known issue	Impact	Target
1.	RDP413 maximum client triple play traffic - observe memory leak	**Impact: High** Memory leak is observed during triple play traffic in VW setup with maximum client enable in DBDC mode.	IPQ8074
2.	QCN61xx IPA specific: AP to do bandwidth drop to 40 Mbps from 160 Mbps by skipping 80 Mbps to pass FCC high noise floor test.	**Impact : Medium** FCC-6 GHz high noise floor test might fail for QCN61xx IPA as APUT is not dropping the bandwidth to 40 Mbps from 160 Mbps.	IPQ5018
3.	Observed consecutive ping drops on 6E clients (HSP and Intel AX210) during auditorium test run (25+ in 15 minutes run)	**Impact : High** Customer might observe random instances of 3+ consecutive ping loss with >36+ 6E clients but no impact to video streaming/data transfer in a multi-client real world environment.	IPQ8074

10.2 Stability issues for SPF11.5 CSU1

Sl. No.	Known issue	Impact	Target
1.	Q6 Crash : ar_wal_tqm.c:6094 Assertion 0 failed.	**Impact : High** Customer might observe Q6 crash while connecting 240 clients on 2.4 GHz/240 clients on 5 GHz/128 clients on 6 GHz and running UDP/TCP traffics.	IPQ8074
2.	APSS Crash : Host Asserted : FileName = dp_rx_err.c FunctionName = dp_rx_err_exception at LineNo: 2076.	**Impact : High** Customer might observe Q6 crash while connecting 250 clients on 2.4 GHz / 250 clients on 5 GHz / 128 clients on 6 GHz and running UDP/TCP traffics.	IPQ8074

Sl. No.	Known issue	Impact	Target
3.	Q6 Crash : wal_soc_dev_hw.c:803 Assertion !is_assert failed	**Impact : High** Customer might observe Q6 crash while connecting 250 clients on 2.4 GHz / 250 clients on 5 GHz / 256 clients on 6 GHz and running UDP/TCP traffics.	IPQ8074
4.	APSS Crash : Kernel Panic : ModuleOrFile = wifi_3_0 PC = dp_tx_comp_handler+0x46c/0x23b0 LR = dp_tx_comp_handler+0x180/0x23b0	**Impact : High** Customer might observe Q6 crash while connecting 250 clients on 2.4 GHz / 250 clients on 5 GHz / 128 clients on 6 GHz and running UDP/TCP traffics	IPQ8074
5.	Q6 Crash : wal_soc_dev_hw.c:585 Assertion !is_assert failed	**Impact : High** Customer might observe Q6 crash in an endurance scenario with 10 Clients on 2.4 GHz, 30 Clients on 5 GHz and 80 clients on 6 GHz running IxChariot and real world traffic.	IPQ8074
6.	Q6 Crash : whal_reset.c:1567 Assertion (wait < wait_timeout) failed	**Impact : High** Customer might observe Q6 crash in endurance scenario with 10 Clients on 2.4 GHz, 30 Clients on 5 GHz and 60 clients on 6 GHz running IxChariot and real world traffic.	IPQ8074
7.	Q6 Crash - UCode PhyA PDMEM Bin Corrupted - Region : VECTORS_AT_0	**Impact : High** Customer might observe Q6 crash in an endurance scenario with 10 clients on 2.4 GHz, 30 Clients on 5 GHz and 80 Clients on 6 GHz running IxChariot and real world traffic.	IPQ8074
8.	Q6 Crash : ar_wal_rx.c:3014 ExIPC: Exception received tid=1b func: rx_process_recv_status.	**Impact: High** Crash observed when AP is in idle state for more than 3 hours with 20+clients associated.	IPQ8074
9.	Q6 Crash : wal_soc_dev_hw.c:544 Assertion 0 failed.	**Impact : High** Customer might observe Q6 crash while connecting 250 clients on 2.4 GHz / 250 clients on 5 GHz / 256 clients on 6 GHz and running UDP/TCP traffics	IPQ8074

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Sl. No.	Known issue	Impact	Target
10.	Q6 Crash : ar_wal_tx_de.c:113 Assertion 0 failed.	**Impact : High** Customer might observe Q6 crash while connecting 250 clients on 2.4 GHz / 250 clients on 5 GHz / 256 clients on 6 GHz and running UDP/TCP traffics	IPQ8074
11.	APSS Crash : Kernel Panic : ModuleOrFile = umac PC = ieee80211_mlme_recv_csa+0x8/0x1b0 LR = ieee80211_synced_disconnect_sta+0x1c/0x28	**Impact : High** Customer might hit kernel panic in a mesh network while running high volume of IxAttack traffic from WAN(CAP) to LAN(RE) over wireless.	IPQ5018
12.	Q6 Crash : Error cause: ERR_FATAL_EXIT, Application called blast_fatal_exit(). - ar_wal_tx_de.c : wal_tx_de_complete_to_fw()	**Impact : High** Customer might observe Q6 crash in an endurance scenario with 10 clients on 2.4 GHz, 30 Clients on 5 GHz and 80 Clients on 6 GHz running IxChariot and real world traffic.	IPQ5018
13.	NSS Crash - Core 0 PC : _db_tables_free()	**Impact : High** When connected to 192 clients, customer might observe NSS crash doing Channel change on 2.4 GHz/5 GHz and aruba SBS DBS in parallel.	IPQ8074

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

11 Fixed Issues

S. No.	Title	Impact	Target
1	DPD status shows in progress via configuration command after disabling in BDF.	Medium	IPQ6018
2	Observed video stream buffering and ping latency with IOT stations while running IOT dashboard with RTS enabled.	High	IPQ8074
3	+- 50% throughput deviation from average throughput is observed for more than 50% of clients allocated within the same range for TCP DL (Short range: 4HST + 4 Intel+ 1GS20, middle range:4HST + 4 Intel+2GS20+3 iphone11, Far range: 2 HST + 2 Intel+1GS10)Â	High	IPQ8074
4	VxLANoIPSec bidirectional UDP IPv4 traffic with 1400 frame size dropped after 12hrs to 15hrs in transport mode. Issue is observed in normal IPSec traffic with transport mode (DUT to DUT traffic).	High	IPQ8074
5	Observed 10-35% UL MU-MIMO throughput degradation with frequent BSRP feature enabled when more number of users connected to the AP other than the active users under test (1492 vs 2097).	High	IPQ8074
6	APSS Crash : Kernel Panic : ModuleOrFile = qca_ol PC = 0x7000000 LR = htc_ctrl_msg_cmpl+0x2bc/0xfc0	High	IPQ5018
7	Q6 Crash : NOC_error.c:474 NOCError: FATAL ERROR	High	IPQ8074
8	Intermittent HYD hang is observed on the RE(SBS) while running endurance traffic tests.	Medium	IPQ8074
9	RDP435 - Long run multiclient roaming - observe NSS peer auth update message failed with error and wlan: [0:E:NSS] [nss-wifili]: peer NULL - Null nss context.	High	IPQ5018
10	Q6 Crash : whal_reset.c:1556 Assertion (wait < wait_timeout) failed.	High	IPQ8074
11	Q6 Crash : whal_xmit.c:22735 ExIPC: Exception recieved tid=67 func: whal_tx_send_selfgen_trigger_info_init.	High	IPQ8074
12	Q6 Crash : Error cause: ERR_FATAL_EXIT, application called blast_fatal_exit(). - osal_patched.c : QuRT_exception_raise_nonfatal___ROM()	High	IPQ5018

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

S. No.	Title	Impact	Target
13	Q6 Crash : wal_soc_dev_hw.c:803 Assertion is_assert failed.	High	IPQ8074
14	Enabling HIF_PRIO_MGR in IPQ8074 platform	High	IPQ8074
15	Q6 Crash : wal_rate.c:394 :Excep :0 Exception detected PName:wlan1 TNa	High	IPQ5018
16	6 to 9% in UDP and 1 % in TCP TP drop observed from KPI commit in 5G/6GHE160 mode (5 GHz UDP KPI 1975, observed 1860 Mbps. TCP KPI 1825 Mbps , observed 1810 Mbps).(6 GHz UDP KPI 1927 Mbps, observed 1780 Mbps to 1834 Mbps).	Medium	IPQ5018
17	Enabling HIF_PRIO_MGR in IPQ6018 platform	High	IPQ6018
18	Observed continuous Wi-Fi restart during MAPr2 ethernet onboarding-regression.	High	IPQ5018
19	APSS crash : Kernel Panic : ModuleOrFile = umac PC = IEEE80211_DPRINTF+0x4c/0x178 LR = ieee80211_mlme_recv_csa+0x3c/0x	High	IPQ8074
20	Q6 Crash : phyrf_reset.c:1139 assertion 0 failed	High	IPQ8074
21	APSS Crash : Kernel Panic : ModuleOrFile = umac PC = _raw_spin_lock+0x14/0x44 LR	High	IPQ8074
22	Observed intermittent high ping latency/failure with IOT stations associated to AP 5 GHz radio [Pass Meta : 272;Fail Meta : 273]	High	IPQ8074
23	After STA sending OMI with channel width = 2 (BW=80 MHz) ,UL MU Disable= 0, Tx NSTS = 1 (actual Tx NSTS=2). APUT is not responding with BlockAck and observing zero throughput while running UL Traffic.	High	IPQ5018
24	Call trace is observed while configuring Qwrap test case on 6 GHz radio.	High	IPQ8074

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

12 Limitations

12.1 Limitations for SPF11.5

- STP counter for VLAN tag does increment in IPQ807x (HW limitation)
- Proper sequence (Set NSS-> Set mode > Set MCS) must be followed to avoid WDS STA connection failure with WPA3 SAE, while changing the MCS/NSS on the run time
- Mode 5 is not supported with WLAN IGMP conversion feature
- Tx capture support with full configuration (simultaneous Tx capture on multiple radios with multiple clients) will need enough memory headroom. Recommendation is to use a platform with at least 2 GB DDR (estimates are based on QTI reference platform).
- Beacon and probe response frames do not retain the RNR IE. User must clear the existing setting and reapply the setting to make it work.
- The driver always goes back to the SBS mode on reset. If dynamic type is 2 (fast mode switch)
- Tx multicast enhancement between 6 and 0 for a few times will lead to multicast traffic failure
- In MAP R2, the Ezmesh daemon restarts association process after enabling global preference flag.
- Interoperability between quad radio board with other radio access points is not supported for Ezmesh
- Due to SFM changed setting, impacted 10% lower throughput for TCP DL-MU-MIMO from the committed KPI.
- AP CLI displays 6 Mbps power only for BW20 rates even if CTL values for other OFDM rates are different (lower).
- CTL file name penalty is set to 0 to have all NSS /chain masks listed out separately instead of being combined.
- Due to Rx sensitivity issue (HW limitation), ~100 Mbps lower throughput performance with MBP,AC68 and AC88 clients compared to Asus AX88U
- In mid to far, IPQ6018 throughput will be lower by 50% with Client ASUS PCIE AC88U when compared to ASUS RT AX AP (third party AP behavior).
- IPQ8074 against Mi10 (client limitation-One of the Mi10 HW is bad) has low performance in mid-range compared to competitor AP
- IPQ5018(16-bit DDR/Ubi32 Clock) performance is lower (10-100 Mbps) compared to IPQ6018 platform for small packet size (64B/88B)
- IPQ5018(16-bit DDR/Ubi32 Clock) performance is lower (80-150 Mbps) compared to QCN90xx platform for small packet size (64B/88B) due to architecture limitation.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- LG-G8 phone is doing 1x1 data traffic with QCA AP in short to mid-range which will have impact on uplink performance compared to competitor AP (issue specific to Android OS version).
- STP feature is not supported in IPQ501x and IPQ401x due to single GMAC support. RSTP to be used instead.
- When OVPN module is enabled, ECM restart does not work.
- Bandwidth reduction feature is not supported in legacy platforms in the non-ETSI domain..
- Applicable for IPQ4019 and IPQ8064.
- Radar is detected at center frequency during CAC unless injected at other offset (IPQ6018 HW limitation).
- After interCAC, during channel width switch support to and from 160 MHz, Traffic would fail.
- Applicable only for IPQ8074
- High CPU utilization (15% increase than expected on IPQ5018 (Dual Core) limitations) is observed only with SON configuration, WDS mode no impact.
- Intel-AX210
  - Observing AX210 client behavior that client does not move to target band by default even though it accepts BTM response(status code:0).Impacting Active Upgrade / Idle Upgrade in SON
  - Delayed Beacon report being sent - Impacting Active Offload / IdleOffload in SON
  - Intel-AX210 is unable to associate with AP with auto channel due to Intel Station not supporting 6 GHz Mandatory Certification Security requirement of SAE_PWE=1 (H2E) and it supports only SAE_PWE=0. Station side issue resolved with driver update
  - Observed Intel AX210 is frequently disconnecting and reconnecting back with AP Non-Tx Vap's on Spruce
- Customer might observe delay in AP-steering with OP-7T station on WPA3 Security
- GS21 Limitation
  - GS21 requires OOB configuration to scan QCA AP. GS21 Ultra is unable to discover 6 GHz APs without the aid of OOB advertisement.
  - Galaxy S21 requires Tx VAP to must secured (sae_pwe = 1) on QCA AP to discover 6 GHz VAP's.
- When QCN61xx operates as an infrastructure STA in 160Mhz bandwidth mode, if OFDMA test run with pri20 at upper80 band, low throughput could be observed due to trigger failures.
- In QWRAP mode, when operating as infrastructure STA, UL-OFDMA and UL-MUMIMO features are not supported in any of the chipsets.
- Multiple beacon rates cannot be set to two different VAP's in the same radio.
- First time direct ethernet onboarding with MAPr2 configuration will result in multiple wifi restart in agent if Controller and Agent are configured with different HT modes, Onboarding agent via wifi to ethernet will resolve this issue
- Limitation with 6 GHz Samsung S21/Fold3 client issue for SON
  - Observing BTM steer fails with nonsuccess code
  - Frequent disconnect is seen when Force BTM steer from 6 GHz to 5 GHz
- Ping loss (~30%) observed for non-11k/v Legacy client connected in 2 GHz radio of Root AP – only with AR9580 Legacy Atheros client

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- SON configuration - Preferred Band should be configured the same value across all nodes in the Network, as the nodes do not communicate the same across all nodes causing unexpected behavior and stability issues
- EasyMesh - R2 Agent with Vlan Id will not have connectivity to R1 Agent with no vlan-backhaul
- With Intel AX200 client, HE80 MU2+2+2 UDP Mixed AC (VI+BE) throughput of BE(1030) & VI (1037) are almost same due to client limitation.
- delayed kickout event for some peers when simulated peers > 100
- 18% degrade might be seen in UDP BIDI with 25clients in smaller frame size less than 1024Bytes due to FW limitation (Multi destination SIFS bursting FR) in IPQ8074
- 5% deviation will be seen in all TCP/UDP traffics from KPI in 6 GHz HE160 on RDP420 due to platform limitation
- MU MIMO Gain KPI not met by (1.34 Vs 1.5) in QCN90xx 2 GHz/5 GHz with 100(1x1) veriwave clients. This is a veriwave limitation issue has been raised on Ixia for tracking the same.
- 12% throughput deviation in 36 users 6 GHz 160 MHz UDP DLOFDMA against the peak KPI commit.
- Tx BFee features not supported by Maple, that is the reason for weak performance for UL compared to the competition.
- CH6_HT40+ setting, which is spur channel. With HT40- configuration Maple performance on par with the competition.
- LP-BAR frame is seen with setting BAR count = 0
- 6 GHz SEM marginally fails for MCS 0 and MCS 7 for Spruce
- SEM marginally fails for 11n HT40 for RDP415 Spruce 2 GHz
- In SAE+WPS, if both key and sae_password are configured in uci wireless file, they should be the same else VAP should not come up
- tx_rx_stats are not supported
- FHSS false detection on Maple 2 GHz if external antennas as not keep far from AP (-40dBm)
- IPQ6018+Pericom switch does not support on LM profile build
- wifi down is required after stopping SON daemon and before wifi detect cmd.
- In raw mode with PMF configuration, client disconnect and reconnect is expected during GTK rekey interval change.
- AP VAP does not come up when not setting htmode HT80 in wireless file with channel – 149
- Throughput regression of 6Mbps is observed in UDP DL traffic for 25 veriwave client.
- If RxSOP is configured beyond the maximum signal strength, then higher deviation/early fall in sweep can be observed
- Valid STA got disconnected after injecting auth request with FTIE in WPA2-PSK or WPA3
- EXTAP 5 GHz radio fails to associate with Root AP with vap_diff_mode enable
- In WLAN config list Tx rate remains same at 6Mbps and RX Rate at 0Mbps for IOT stations.
- Due to Ixia tool limitation (MLR/DF Ixia issues), DBDC veriwave multiclient test may inconsistently fail with respect to Delay Factor and MLR.
- Crash is expected in Maple radio with monitor mode enable in stress condition.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- 2 GHz 16 User UDP BW40 throughput on CH1 HT40+ and CH6 HT40+ (440Mbp) is 100 to 200Mbps lower than CH11 HT40 due to Known limitation from IPQ8074 board layout.
- ~25% throughput drop will be seen while running UDP UL/BIDI with TxAMSDU disabled in Ixia VW multiclient due to limitation of the reduction in small packet performance.
- If a non-Tx profile is preceded by a partial-profile in a particular MBSSID element, then Intel STA fails to scan Non Tx profile when EMA is enable
- Backpressure handling for Spectral scan is not implemented
- AFC Repeater cannot move to SP power unless it receive its own AFC response.
- System will crash if Beacon rate is changed from, HE to Legacy in Non PSC channel where Non HT Duplicate Ack is enable.
- Enablement of 160 MHz traffic with interCAC feature requires option **advertise_ap_maxcap 1** to be added in addition to interCAC command. Applicable only for IPQ8074 platform.
- Any change in AFC config file needs to restart the AFC app so that it will retrieve a new AFC Token to be able to get a valid response from the dynamically updated AFC URL
- ~20% drop in DL MU peak Throughput and ~2% drop in MU gain with respect to KPI commit in HE 160MHz on both 5 GHz and 6 GHz radio will be observed while using two Intel210 client (STA limitation)
- Throughput difference of app. 100Mbps seen in TCP UL with MBP, PAL5 clients compared to competitor AP. This is known limitation in IPQ8074 compared to competitor.
- 30% low Throughput with ASUS AC68, AC88, MBP, PAL2 as client's in 36 Orientation & SNB RvR test with respect to Competitive AP - Its due to the RX sensitivity limitation
- From Mid to Far Range observed > 30% throughput delta in 2 GHz TCP_DL|UL RvR test with 11ax_MBP client with respect to. competitor AP - Tx power higher 2dB in lower modulation, so mid - far range delta is expected
- UL SU throughput is less than 100Mbps when running TCP/UDP traffic with more than 16 Intel AX210 clients (Forced SU case) due to Intel client limitation.
- ~20-65% low UDP UL throughput observed in 5 GHz-L radio compared to 5 GHz-H radio in SBS board due to Limitation in IPQ8074 architecture for lower Frame size
- Due to improvement made to address the overall KPI, ~50mbps TCP DL regression is seen with 25 veriwave client.
- In carrier Vow WDS video traffic test, we observed Up to ~50 Frame loss randomly in QCN90xx.
- In carrier Vow WDS video traffic test, we observed video glitch/hang in IPQ6018.
- AFC enabled (SP mode) TCP DL and UL RVR is under performing (up to ~2%) than the LPI in few attenuation points in the short range due to SP limitation (Small throughput drop expected due to RTS/CTS retries in SP where the powers are higher)
- SP mode has higher target power than LPI mode. The higher target power, the worse EVM will be which may cause some PER. Thus, it is normal that SP mode may have a 2 to 4% DL / 5 to 7% UL peak throughput (in small attenuation range) than LPI mode.
- RVR falls early by 1 to 2db at knee point in 5 GHz RDP403 DL/UL/BiDi and this is based on known limitation of HDL 2.7 improvements.
- Due to Ixia veriwave Client limitation, ~20-110Mbps UDP BIDI throughput fluctuation seen while sending lower frame size [64,88,128] traffic to single/multi veriwave clients.
- 20% deviation low throughput in 2 GHz TCP_UL RvR test with AX58BT & AX3000 Client with respect to competitor AP. Known HW limitation

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- For AFC feature with current release, the following limitations will be observed
  a. User Cannot trigger Off Channel Tx/scan to LPI channel in SP mode.
  b. Repeater Sta on disconnect from Root AP, will reset to LPI Power mode
  c. Repeater Sta fails to remain connected to LPI channel using CSA
  d. Repeater Sta, not able to set SP power before starting the connection
  e. AP cannot upgrade the BW after downgrading due to AFC response using ACS.
- With AFC enabled we will observe leakage in adjacent channels when we duplicate beacons in the entire 160MHz with bandwidth.
- Memory utilization spikes observed with ECM enabled and next_hop not configured as NSS 5-Tuple lookup; need to ensure next_hop is configured to NSS 5-Tuple Lookup with ECM enablement for all NSS offload enabled configurations.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

13 Workaround to pass WFA certification

802.11ax Cert Test Cases ID	Comments
4.36.1_24 GHz/5 GHz	Customer might face orientation/throughput related issues. to address this, WAR has been introduced in FW for all platforms. If customer faces low throughput issue with Marvell station, please recommend to reboot the Marvell test bed station and to reduce the AP Tx power to 5. Once the AP VAP is up, use the command “iwconfig ath0 txpower 5” and continue the test
4.37.1	Customer might face orientation/throughput related issues. to address this, WAR has been introduced in FW for all platforms. If customer faces low throughput issue with Marvell station, please recommend to reboot the Marvell test bed station. Recommended RSSI is -40 to -50 dB
4.40.2	Customer might observe basic triggers frames not having UL MCS 7. Also, some may observe that IPQ6018 /Infenion testbed station having issue in association in mixed NSS. We have reported this issue to WFA. As a manual step, we shall connect IPQ6018 station using the below manual commands. In APUT (IPQ5018 /QCN61xx), once the AP VAP is up please configure the AP Tx power to 10 or 5 dB - iwconfig ath0 txpower 10 also configured fixed MCS7 in APUT cfg80211tool ath0 he_ul_mcs 7 and continue the test IPQ6018 /Infineon Commands: (under wpa_cli) remove_net 0 add_net set_net 0 ssid "HE-4.40.2" set_network 0 key_mgmt WPA-PSK set_network 0 pairwise CCMP set_network 0 psk "12345678" save_config enable_net 0 reconnect
4.40.4	Sometimes Marvell station might get hang while running UL OFDMA traffic due to unicast trigger to Marvell STA. Please ensure to have good RSSI in the range of -35 to -40 dB.
4.41.1	Sometimes Marvell station might get hang while running UL OFDMA traffic due to unicast trigger to Marvell STA. Please ensure to have good RSSI in the range of -35 to -40 dB.
4.60.1	Sometimes Marvell station might get hang while running UL OFDMA traffic due to unicast trigger to Marvell STA. Please ensure to have good RSSI in the range of -35 to -40 db.
General waiver from WFA	Broadcom 75 test bed station may get crashed while association. This is randomly seen across 11ax test cases. WFA has shared waiver for such issue and alternate testbed station not involved in that particular cases can be replaced instead of broadcom 75.